

ICPC/CCPC Algo Templates

许居尚

August 13, 2026

Contents

1	树图与网络流	2
1.1	树	2
1.1.1	先中后序建树	2
1.1.2	LCA	3
1.1.3	树上算法	4
1.1.4	dsu on tree	6
1.1.5	最近公共祖先 LCA	8
1.1.6	树上启发式合并 (DSU on tree)	11
1.1.7	树上路径交	13
1.1.8	树的直径	13
1.1.9	树论大封装 (直径 + 重心 + 中心)	14
1.1.10	点分治 / 树的重心	15
1.2	图论	17
1.2.1	最短路	17
1.2.2	kruscal	21
1.2.3	强连通分量缩点	22
1.2.4	差分约束	25
1.2.5	拓扑	27
1.2.6	2-Sat	28
1.2.7	单源最短路径 (SSSP 问题)	30
1.2.8	多源汇最短路 (APSP 问题)	32
1.2.9	差分约束	33
1.2.10	常见例题	33
1.2.11	常见概念	43
1.2.12	常见结论	43
1.2.13	平面图最短路 (对偶图)	44
1.2.14	最小生成树 (MST 问题)	45
1.2.15	最短路径树 (SPT 问题)	47
1.2.16	最长路 (topsort+DP 算法)	48
1.2.17	欧拉路径/欧拉回路 Hierholzers	49
1.2.18	缩点 (Tarjan 算法)	51
1.2.19	链式前向星建图与搜索	55
1.3	匹配	57
1.3.1	二分图匹配	57
1.3.2	一般图最大匹配 (带花树算法)	58
1.3.3	一般图最大权匹配 (带权带花树算法)	60

1.3.4	二分图最大匹配	65
1.3.5	二分图最大权匹配 (二分图完美匹配)	67
1.3.6	二分图最大独立点集 (Konig 定理)	70
1.3.7	染色法判定二分图 (dfs 算法)	70
1.4	网络流	70
1.4.1	网络流	70
1.4.2	无源汇点的最小割问题 Stoer-Wagner	72
1.4.3	最大流	73
1.4.4	最小割	75
1.4.5	最小割树 Gomory-Hu Tree	75
1.4.6	网络流 最大流 预流推进 HLPP	77
1.4.7	费用流	79
2	数学	81
2.1	数论	81
2.1.1	数论 frequently use	81
2.1.2	灵神数学模板	82
2.1.3	线性求逆元	85
2.1.4	唯一分解	86
2.1.5	exgcd 相关	88
2.1.6	线筛——枚举所有数的质数倍	90
2.1.7	试除法求所有约数	90
2.1.8	欧拉定理 (单次求 phi)	91
2.1.9	milller rabin	91
2.1.10	杜教筛	92
2.1.11	gcd	94
2.1.12	Min25 筛	94
2.1.13	同余方程组、拓展中国剩余定理 exCRT	95
2.1.14	基础算法 最大公约数 'gcd' 位运算加速	96
2.1.15	容斥原理	96
2.1.16	常见例题	97
2.1.17	常见数列	99
2.1.18	常见结论	100
2.1.19	康拓展开	107
2.1.20	快速幂	109
2.1.21	扩展欧几里得 exgcd	109
2.1.22	数论 取模运算类 蒙哥马利模乘	110
2.1.23	数论 质因数分解 Pollard-Rho	111
2.1.24	数论 质数判定 Miller-Rabin	112
2.1.25	数论 质数判定 预分类讨论加速	113
2.1.26	整除 (数论) 分块	113
2.1.27	最大公约数 'gcd'	113
2.1.28	欧拉函数	114
2.1.29	求解连续按位异或	115
2.1.30	求解连续数字的正约数集合——倍数法	115
2.1.31	离散对数 bsgs 与 exbsgs	116
2.1.32	莫比乌斯函数/反演	117
2.1.33	裴蜀定理	118
2.1.34	质因子分解	119

2.1.35	质数判定	120
2.1.36	逆元	121
2.1.37	高斯消元求解线性方程组	122
2.2	组合数学	123
2.2.1	组合数学——lucas	123
2.2.2	GospersHack: 与个数同阶, 即组合数	124
2.2.3	组合数	124
2.2.4	n 中任意 m 个数的全排列	124
2.2.5	n 个数的子集	125
2.2.6	n 中任意 m 个数的组合	126
2.2.7	组合数	127
2.3	多项式	129
2.3.1	FFT	129
2.3.2	多项式	131
2.3.3	Berlekamp-Massey 算法 (杜教筛)	138
2.3.4	Linear-Recurrence 算法	139
2.3.5	多项式封装	139
2.3.6	常用结论	143
2.3.7	快速傅里叶变换 FFT	144
2.3.8	快速数论变换 NTT	146
2.3.9	拉格朗日插值	147
2.3.10	离散傅里叶变换 dft 与其逆变换 idft	148
2.4	矩阵	149
2.4.1	矩阵模板	149
2.4.2	矩阵快速幂 (推荐使用数论-矩阵)	150
2.4.3	位运算——线性基	151
2.4.4	矩阵加速	152
2.4.5	矩阵四则运算	153
2.4.6	矩阵快速幂	154
2.5	博弈论	155
2.5.1	Anti-Nim 游戏 (反 Nim 游戏)	155
2.5.2	Anti-SG 游戏 (反 SG 游戏)	155
2.5.3	Every-SG 游戏	155
2.5.4	Lasker' s-Nim 游戏 (Multi-SG 游戏)	156
2.5.5	Moore' s Nim 游戏 (Nim-K 游戏)	156
2.5.6	Nim 博弈	156
2.5.7	Nim 游戏具体取法	157
2.5.8	SG 游戏 (有向图游戏)	157
2.5.9	威佐夫博弈	158
2.5.10	巴什博弈	158
2.5.11	扩展巴什博弈	159
2.5.12	斐波那契博弈	159
2.5.13	无向图删边游戏 (Fusion Principle 定理)	159
2.5.14	树上删边游戏	160
2.5.15	阶梯 - Nim 博弈	160
2.6	动态规划	161
2.6.1	打家劫舍	161
2.6.2	最长上升子序列 + 最长公共递增子序列 (LCIS) +LCS	161

2.6.3	划分型 dp	165
2.6.4	背包	167
2.6.5	数位 dp	170
2.6.6	01 背包	176
2.6.7	二维费用的背包	176
2.6.8	分组背包	177
2.6.9	多重背包	177
2.6.10	完全背包	178
2.6.11	常用例题	179
2.6.12	数位 DP	181
2.6.13	有依赖的背包	182
2.6.14	混合背包	182
2.6.15	状压 DP	183
2.6.16	背包问题求具体方案	184
2.6.17	背包问题求方案数	185
3	数据结构科技	186
3.1	基础	186
3.1.1	并查集	186
3.1.2	单调栈, 单调队列	186
3.1.3	树状数组 + 求中位数	189
3.1.4	树状数组: 逆序对	190
3.1.5	二维数点离线 (树状数组)	191
3.1.6	ST 表	192
3.1.7	对顶堆	194
3.1.8	归并排序逆序对	195
3.1.9	ST 表	196
3.1.10	二维树状数组	197
3.1.11	坐标压缩与离散化	199
3.1.12	基于状压的线性 RMQ 算法	200
3.1.13	并查集 (全功能)	201
3.1.14	树状数组	202
3.2	线段树	205
3.2.1	线段树板子 (jiangly)	205
3.2.2	线段树: 维护最大子段和	221
3.2.3	线段树	222
3.2.4	轻重链剖分/树链剖分	230
3.3	平衡树	233
3.3.1	Treap: 数堆	233
3.3.2	pbds 扩展库实现平衡二叉树	236
3.3.3	vector 模拟实现平衡二叉树	236
3.3.4	珂朵莉树 (OD Tree)	237
3.4	莫队	238
3.4.1	Mo 算法	238
3.4.2	带修改的莫队 (带时间维度的莫队)	240
3.4.3	普通莫队	241
3.5	高阶	242
3.5.1	高精度: 重载运算符	242
3.5.2	KD Tree	245

3.5.3	主席树（可持久化线段树）	248
3.5.4	分数运算类	248
3.5.5	取模运算类	249
3.5.6	大整数类（高精度计算）	252
3.5.7	小波矩阵树：高效静态区间第 K 大查询	258
3.5.8	常见例题	259
3.5.9	常见结论	261
3.5.10	线性基（高斯消元法）	261
4	杂项	263
4.1	基础算法	263
4.1.1	OJ 测试	263
4.1.2	Python 常用语法	264
4.1.3	cout 输出流控制	266
4.1.4	int128 输入输出流控制	266
4.1.5	头文件	267
4.1.6	实数域三分	267
4.1.7	实数域二分	268
4.1.8	对拍版子	268
4.1.9	常用函数	268
4.1.10	快读	269
4.1.11	手工哈希	271
4.1.12	整数域三分	271
4.1.13	整数域二分	271
4.1.14	编译器设置	272
4.1.15	读取一行数字，个数未知	273
4.1.16	随机数生成与样例构造	273
4.2	STL	274
4.2.1	容器与成员函数	274
4.2.2	库函数	279
4.2.3	程序标准化	282
4.3	杂类	283
4.3.1	蒙特卡洛模拟概率问题	283
4.3.2	区间哈希 1	284
4.3.3	区间哈希 2	285
4.3.4	N*M 数独字典序最小方案	287
4.3.5	单调队列	287
4.3.6	幻方	287
4.3.7	德州扑克	288
4.3.8	日期换算（基姆拉尔森公式）	292
4.3.9	最长严格/非严格递增子序列 (LIS)	292
4.3.10	浮点数比较	293
4.3.11	物品装箱	294
4.3.12	约瑟夫问题	295
4.3.13	统计区间不同数字的数量（离线查询）	295
4.3.14	网格路径计数	296
4.3.15	选数（DFS 解）	297
4.3.16	选数（位运算状压）	297
4.3.17	阿达马矩阵 (Hadamard matrix)	297

4.3.18	高精度快速幂	298
4.3.19	高精度进制转换	300
4.4	字符串	300
4.4.1	Manacher	300
4.4.2	kmp	301
4.4.3	拓展 kmp/z 函数	302
4.4.4	ac 自动机	303
4.4.5	最小表示法	305
4.4.6	trie	306
4.4.7	字符串哈希	311
4.4.8	AC 自动机	312
4.4.9	Z 函数 (扩展 KMP)	313
4.4.10	后缀数组 SA	313
4.4.11	后缀自动机 SAM	315
4.4.12	回文自动机 PAM (回文树)	315
4.4.13	子串与子序列	316
4.4.14	子序列自动机	316
4.4.15	字典树 trie	318
4.4.16	字符串哈希	319
4.4.17	字符串模式匹配算法 KMP	321
4.4.18	最长公共子序列 LCS	321
4.4.19	马拉车	322
4.5	计算几何	323
4.5.1	计算几何	323
4.5.2	三维几何必要初始化	335
4.5.3	三维点线面相关	336
4.5.4	三维角度与弧度	340
4.5.5	二维凸包	340
4.5.6	常用例题	344
4.5.7	常用结论	349
4.5.8	平面三角形相关 (浮点数处理)	350
4.5.9	平面几何必要初始化	351
4.5.10	平面圆相关 (浮点数处理)	354
4.5.11	平面多边形	357
4.5.12	平面点线相关	360
4.5.13	平面直线方程转换	363
4.5.14	平面角度与弧度	365
4.5.15	库实数类实现 (双精度)	366
4.5.16	空间多边形	366
4.6	frequently use	368
4.6.1	pbds	368
4.6.2	重载小于号	371
4.6.3	优先队列	372
4.6.4	set	372
4.6.5	map	372
4.6.6	others	373
4.6.7	二分	375
4.6.8	print 相关: 快读 and int128 读写等	375

4.6.9	all you need	376
4.7	搜索	381
4.7.1	二维数组的一维化	381
4.7.2	八数码问题	381
4.7.3	剪枝函数: 八皇后	383
4.7.4	迭代加深搜索 IDDFS	384

1 树图与网络流

1.1 树

1.1.1 先中后序建树

```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  const int N = 35;
7  vector<int> pre, in;
8  vector<int> g[N];
9
10 // 重建二叉树, 返回当前子树的根节点
11 int build(int root, int start, int end) {
12     if (start > end) return -1; // 空节点
13     int u = pre[root]; // 当前根节点
14     int i = start;
15     while (i <= end && in[i] != u) i++; // 在中序中找到根节点位置
16     g[u].push_back(build(root + 1, start, i - 1)); // 左子树
17     g[u].push_back(build(root + (i - start) + 1, i + 1, end)); // 右子树
18     return u;
19 }
20
21 int main() {
22     int n;
23     cin >> n;
24     pre.resize(n), in.resize(n);
25     for (int i = 0; i < n; i++) cin >> pre[i];
26     for (int i = 0; i < n; i++) cin >> in[i];
27
28     int root = build(0, 0, n - 1); // 先序的第一个节点是根
29
30     // BFS 层序遍历
31     queue<int> q;
32     q.push(root);
33     vector<int> ans;
34     while (!q.empty()) {
35         int u = q.front();
36         q.pop();
37         ans.push_back(u);
38         for (int v : g[u]) {
39             if (v != -1) q.push(v); // -1 表示空节点
40         }
41     }
42
43     // 输出结果
44     for (int i = 0; i < ans.size(); i++) {
45         if (i) cout << " ";
46         cout << ans[i];
47     }
```



```
48     return 0;
49 }
```

1.1.2 LCA

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  typedef long long ll;
4
5  const int N=2e5+9;
6  int n;
7  vector<int> g[N];
8  int dep[N];
9  int fa[N][22];
10
11 void dfs(int x,int p){
12     dep[x]=dep[p]+1; //深度
13     fa[x][0]=p; //我的父亲是。。父亲!
14     for(int i=1;i<=20;i++){ //从 1 开始
15         fa[x][i]=fa[fa[x][i-1]][i-1];
16     }
17     for(auto &y:g[x]){
18         if(y==p)continue;
19         dfs(y,x);
20     }
21 }
22
23 int lca(int x,int y){
24     if(dep[x]<dep[y])swap(x,y); //使x的深度更深
25     for(int i=20;i>=0;i--){ //x上跳, 由大到小尝试
26         //x能跳就跳
27         if( dep[fa[x][i]]>=dep[y] ){
28             x=fa[x][i];
29         }
30     }
31     if(x==y)return x;
32     for(int i=20;i>=0;i--){
33         //x和y再一起跳一轮
34         if(fa[x][i]!=fa[y][i]){
35             x=fa[x][i],y=fa[y][i];
36         }
37     }
38     return fa[x][0]; //返回x的爸爸 (或y)
39 }
40
41 int main(){
42     ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
43     cin>>n;
44     int u,v;
45     for(int i=1;i<=n-1;i++){
46         cin>>u>>v;
47         g[u].push_back(v);
```

```

48     g[v].push_back(u);
49 }
50 dfs(1,0);
51 int m;cin>>m;
52 int x,y;
53 while(m--){
54     cin>>x>>y;
55     cout<<lca(x,y)<<endl;
56 }
57 return 0;
58 }

```

1.1.3 树上算法

```

1  // -----
2  // 树的重心 (1-2个)
3  // -----
4  #include <bits/stdc++.h>
5  using namespace std;
6
7  const int MAXN = 100000 + 5;
8
9  int n; // 节点数
10 vector<int> g[MAXN]; // 邻接表
11 int sz[MAXN]; // 子树大小
12 int maxPart[MAXN]; // 去掉该节点后“最大连通块”的大小
13 vector<int> centroids;
14
15 // dfs 求出每个节点的子树大小 sz[u], 并计算 maxPart[u]
16 void dfs(int u, int p) {
17     sz[u] = 1;
18     maxPart[u] = 0;
19     for (int v : g[u]) {
20         if (v == p) continue;
21         dfs(v, u);
22         sz[u] += sz[v];
23         maxPart[u] = max(maxPart[u], sz[v]);
24     }
25     // “剩余的部分”大小
26     maxPart[u] = max(maxPart[u], n - sz[u]);
27 }
28
29 int main() {
30     ios::sync_with_stdio(false);
31     cin.tie(nullptr);
32
33     // 读入 n, 和 n-1 条边
34     cin >> n;
35     for (int i = 1; i <= n; i++) {
36         g[i].clear();
37     }
38     for (int i = 0, u, v; i < n-1; i++) {

```

```

39     cin >> u >> v;
40     g[u].push_back(v);
41     g[v].push_back(u);
42 }
43
44 // 从任意节点 (例如 1) 开始 DFS
45 dfs(1, 0);
46
47 // 找最小的 maxPart 值
48 int best = n;
49 for (int i = 1; i <= n; i++) {
50     best = min(best, maxPart[i]);
51 }
52 // 收集所有重心
53 for (int i = 1; i <= n; i++) {
54     if (maxPart[i] == best) {
55         centroids.push_back(i);
56     }
57 }
58
59 // 输出: 重心个数 和 节点编号
60 cout << centroids.size() << "\n";
61 for (int u : centroids) {
62     cout << u << " ";
63 }
64 cout << "\n";
65
66 return 0;
67 }
68
69 // -----
70 // dfs序
71 // -----
72 #include <bits/stdc++.h>
73 using namespace std;
74 using ll = long long;
75
76 const int MAXN = 100000;
77 vector<int> g[MAXN + 1];
78 int tin[MAXN + 1], tout[MAXN + 1], timer = 0;
79
80 // 深度优先遍历; p 表示父节点
81 void dfs(int u, int p) {
82     tin[u] = ++timer; // 记录进入时间
83     for (int v : g[u]) {
84         if (v == p) continue; // 不回到父节点
85         dfs(v, u);
86     }
87     tout[u] = timer; // 记录子树结束时间
88 }
89
90 int main() {
91     ios::sync_with_stdio(false);

```

```

92     cin.tie(nullptr);
93
94     int n, root = 1;
95     cin >> n;
96     for (int i = 1; i < n; i++) {
97         int u, v;
98         cin >> u >> v;
99         g[u].push_back(v);
100        g[v].push_back(u);
101    }
102
103    dfs(root, 0);
104
105    // 示例: 输出所有节点的 tin/tout
106    for (int u = 1; u <= n; u++) {
107        cout << "Node " << u
108             << ": tin=" << tin[u]
109             << ", tout=" << tout[u] << "\n";
110    }
111    return 0;
112 }

```

1.1.4 dsu on tree

```

1  #include <bits/stdc++.h> //dsu on tree 解决离线子树问题
2  using namespace std; //例题: 树上数颜色
3  const int mod=1e9+7, N=2e5+8;
4  int n, m, cnt, k, l, r, a[N];
5  vector<int> G[N];
6  int hc[N], sz[N], col[N], L[N], R[N], totdfn, node[N], ans[N];
7  void add(int x, int delta) { // 添加入目前集合
8      if (col[a[x]] == 0) cnt++;
9      col[a[x]]++;
10 }
11 void del(int x) { // 移出集合
12     col[a[x]]--;
13     if (col[a[x]] == 0) cnt--;
14 }
15 int getans() { // 计算当前子树答案
16     return cnt;
17 }
18 void dfs0(int x, int fa) {
19     L[x] = ++totdfn; // x 的进入 dfs 的时机
20     node[totdfn] = x;
21     sz[x] = 1;
22     hc[x] = 0;
23     int maxx = 0;
24     for (int i : G[x]) {
25         if (i != fa) {
26             dfs0(i, x);
27             sz[x] += sz[i]; // 统计当前节点大小
28             if (maxx < sz[i]) { // 找重孩子

```

```

29         hc[x]=i;
30         maxx=sz[i];
31     }
32 }
33 }
34 R[x]=totdfn;//x出dfs的时机
35 }
36 void dfs1(int x,int fa,bool keep){
37     for (int v:G[x]) { //先完成轻孩子的遍历且不保留数据
38         if (v!=fa&&v!=hc[x]) {
39             dfs1(v,x,0);
40         }
41     }
42
43     // 处理重儿子 (保留数据)
44     if(hc[x]){
45         dfs1(hc[x],x,1);
46     }
47     // 添加当前节点
48     add(x,1);
49     // 添加所有轻儿子的子树节点 (利用DFS序的连续区间)
50     for (int v:G[x]) {
51         if (v!=fa&&v!=hc[x]) {
52             for (int i=L[v];i<=R[v];i++) {
53                 add(node[i],1); // dfn[i]是DFS序i对应的节点
54             }
55         }
56     }
57
58     // 记录当前节点的答案
59     ans[x] = getans();
60
61     // 若不保留数据, 则删除当前子树的所有节点
62     if (!keep) {
63         for (int i = L[x]; i <= R[x]; i++) {
64             del(node[i]);
65         }
66     }
67 }
68 void solve(){
69     cin>>n;
70     for(int i=1;i<n;i++){
71         int u,v;
72         cin>>u>>v;
73         G[u].push_back(v);
74         G[v].push_back(u);
75     }
76     for(int i=1;i<=n;i++){
77         cin>>a[i];
78     }
79     dfs0(1,0);
80     dfs1(1,0,0);
81     cin>>m;

```

```

82     while(m--){
83         int t;cin>>t;
84         cout<<ans[t]<<"\n";
85     }
86 }
87 signed main(){
88     close
89     solve();
90
91 }

```

1.1.5 最近公共祖先 LCA

树链剖分解法

预处理时间复杂度 $\mathcal{O}(N)$ ；单次查询 $\mathcal{O}(\log N)$ ，常数较小。

树上倍增解法

预处理时间复杂度 $\mathcal{O}(N \log N)$ ；单次查询 $\mathcal{O}(\log N)$ ，但是常数比树链剖分解法更大。

封装一：基础封装，针对无权图。

封装二：扩展封装，针对有权图，支持“倍增查询两点路径上的最大边权”功能。

```

1  struct HLD {
2      int n, idx;
3      vector<vector<int>> ver;
4      vector<int> siz, dep;
5      vector<int> top, son, parent;
6
7      HLD(int n) {
8          this->n = n;
9          ver.resize(n + 1);
10         siz.resize(n + 1);
11         dep.resize(n + 1);
12
13         top.resize(n + 1);
14         son.resize(n + 1);
15         parent.resize(n + 1);
16     }
17     void add(int x, int y) { // 建立双向边
18         ver[x].push_back(y);
19         ver[y].push_back(x);
20     }
21     void dfs1(int x) {
22         siz[x] = 1;
23         dep[x] = dep[parent[x]] + 1;
24         for (auto y : ver[x]) {
25             if (y == parent[x]) continue;
26             parent[y] = x;
27             dfs1(y);
28             siz[x] += siz[y];
29             if (siz[y] > siz[son[x]]) {
30                 son[x] = y;
31             }

```

```

32     }
33 }
34 void dfs2(int x, int up) {
35     top[x] = up;
36     if (son[x]) dfs2(son[x], up);
37     for (auto y : ver[x]) {
38         if (y == parent[x] || y == son[x]) continue;
39         dfs2(y, y);
40     }
41 }
42 int lca(int x, int y) {
43     while (top[x] != top[y]) {
44         if (dep[top[x]] > dep[top[y]]) {
45             x = parent[top[x]];
46         } else {
47             y = parent[top[y]];
48         }
49     }
50     return dep[x] < dep[y] ? x : y;
51 }
52 int clac(int x, int y) { // 查询两点间距离
53     return dep[x] + dep[y] - 2 * dep[lca(x, y)];
54 }
55 void work(int root = 1) { // 在此初始化
56     dfs1(root);
57     dfs2(root, root);
58 }
59 };
60
61 struct Tree {
62     int n;
63     vector<vector<int>> ver, val;
64     vector<int> lg, dep;
65     Tree(int n) {
66         this->n = n;
67         ver.resize(n + 1);
68         val.resize(n + 1, vector<int>(30));
69         lg.resize(n + 1);
70         dep.resize(n + 1);
71         for (int i = 1; i <= n; i++) { //预处理 log
72             lg[i] = lg[i - 1] + (1 << lg[i - 1] == i);
73         }
74     }
75     void add(int x, int y) { // 建立双向边
76         ver[x].push_back(y);
77         ver[y].push_back(x);
78     }
79     void dfs(int x, int fa) {
80         val[x][0] = fa; // 储存 x 的父节点
81         dep[x] = dep[fa] + 1;
82         for (int i = 1; i <= lg[dep[x]]; i++) {
83             val[x][i] = val[val[x][i - 1]][i - 1];
84         }

```

```

85     for (auto y : ver[x]) {
86         if (y == fa) continue;
87         dfs(y, x);
88     }
89 }
90 int lca(int x, int y) {
91     if (dep[x] < dep[y]) swap(x, y);
92     while (dep[x] > dep[y]) {
93         x = val[x][lg[dep[x]] - dep[y] - 1];
94     }
95     if (x == y) return x;
96     for (int k = lg[dep[x]] - 1; k >= 0; k--) {
97         if (val[x][k] == val[y][k]) continue;
98         x = val[x][k];
99         y = val[y][k];
100    }
101    return val[x][0];
102 }
103 int clac(int x, int y) { // 倍增查询两点间距离
104     return dep[x] + dep[y] - 2 * dep[lca(x, y)];
105 }
106 void work(int root = 1) { // 在此初始化
107     dfs(root, 0);
108 }
109 };
110
111 struct Tree {
112     int n;
113     vector<vector<int>> val, Max;
114     vector<vector<pair<int, int>>> ver;
115     vector<int> lg, dep;
116     Tree(int n) {
117         this->n = n;
118         ver.resize(n + 1);
119         val.resize(n + 1, vector<int>(30));
120         Max.resize(n + 1, vector<int>(30));
121         lg.resize(n + 1);
122         dep.resize(n + 1);
123         for (int i = 1; i <= n; i++) { //预处理 log
124             lg[i] = lg[i - 1] + (1 << lg[i - 1] == i);
125         }
126     }
127     void add(int x, int y, int w) { // 建立双向边
128         ver[x].push_back({y, w});
129         ver[y].push_back({x, w});
130     }
131     void dfs(int x, int fa) {
132         val[x][0] = fa;
133         dep[x] = dep[fa] + 1;
134         for (int i = 1; i <= lg[dep[x]]; i++) {
135             val[x][i] = val[val[x][i - 1]][i - 1];
136             Max[x][i] = max(Max[x][i - 1], Max[val[x][i - 1]][i - 1]);
137         }

```



```

138     for (auto [y, w] : ver[x]) {
139         if (y == fa) continue;
140         Max[y][0] = w;
141         dfs(y, x);
142     }
143 }
144 int lca(int x, int y) {
145     if (dep[x] < dep[y]) swap(x, y);
146     while (dep[x] > dep[y]) {
147         x = val[x][lg[dep[x]] - dep[y] - 1];
148     }
149     if (x == y) return x;
150     for (int k = lg[dep[x]] - 1; k >= 0; k--) {
151         if (val[x][k] == val[y][k]) continue;
152         x = val[x][k];
153         y = val[y][k];
154     }
155     return val[x][0];
156 }
157 int clac(int x, int y) { // 倍增查询两点间距离
158     return dep[x] + dep[y] - 2 * dep[lca(x, y)];
159 }
160 int query(int x, int y) { // 倍增查询两点路径上的最大边权 (带权图)
161     auto get = [&](int x, int y) -> int {
162         int ans = 0;
163         if (x == y) return ans;
164         for (int i = lg[dep[x]]; i >= 0; i--) {
165             if (dep[val[x][i]] > dep[y]) {
166                 ans = max(ans, Max[x][i]);
167                 x = val[x][i];
168             }
169         }
170         ans = max(ans, Max[x][0]);
171         return ans;
172     };
173     int fa = lca(x, y);
174     return max(get(x, fa), get(y, fa));
175 }
176 void work(int root = 1) { // 在此初始化
177     dfs(root, 0);
178 }
179 };

```

1.1.6 树上启发式合并 (DSU on tree)

$\mathcal{O}(N \log N)$ 。

图论

```

1 struct HLD {
2     vector<vector<int>>> e;
3     vector<int> siz, son, cnt;
4     vector<LL> ans;

```

```
5   LL sum, Max;
6   int hson;
7   HLD(int n) {
8       e.resize(n + 1);
9       siz.resize(n + 1);
10      son.resize(n + 1);
11      ans.resize(n + 1);
12      cnt.resize(n + 1);
13      hson = 0;
14      sum = 0;
15      Max = 0;
16  }
17  void add(int u, int v) {
18      e[u].push_back(v);
19      e[v].push_back(u);
20  }
21  void dfs1(int u, int fa) {
22      siz[u] = 1;
23      for (auto v : e[u]) {
24          if (v == fa) continue;
25          dfs1(v, u);
26          siz[u] += siz[v];
27          if (siz[v] > siz[son[u]]) son[u] = v;
28      }
29  }
30  void calc(int u, int fa, int val) {
31      cnt[color[u]] += val;
32      if (cnt[color[u]] > Max) {
33          Max = cnt[color[u]];
34          sum = color[u];
35      } else if (cnt[color[u]] == Max) {
36          sum += color[u];
37      }
38      for (auto v : e[u]) {
39          if (v == fa || v == hson) continue;
40          calc(v, u, val);
41      }
42  }
43  void dfs2(int u, int fa, int opt) {
44      for (auto v : e[u]) {
45          if (v == fa || v == son[u]) continue;
46          dfs2(v, u, 0);
47      }
48      if (son[u]) {
49          dfs2(son[u], u, 1);
50          hson = son[u]; //记录重链编号, 计算的时候跳过
51      }
52      calc(u, fa, 1);
53      hson = 0; //消除的时候所有儿子都清除
54      ans[u] = sum;
55      if (!opt) {
56          calc(u, fa, -1);
57          sum = 0;
```

```

58         Max = 0;
59     }
60 }
61 };

```

1.1.7 树上路径交

计算两条路径的交点数量，直接载入任意 LCA 封装即可。

```

1  int intersection(int x, int y, int X, int Y) {
2      vector<int> t = {lca(x, X), lca(x, Y), lca(y, X), lca(y, Y)};
3      sort(t.begin(), t.end());
4      int r = lca(x, y), R = lca(X, Y);
5      if (dep[t[0]] < min(dep[r], dep[R]) || dep[t[2]] < max(dep[r], dep[R])) {
6          return 0;
7      }
8      return 1 + clac(t[2], t[3]);
9  }

```

1.1.8 树的直径

```

1  struct Tree {
2      int n;
3      vector<vector<int>> ver;
4      Tree(int n) {
5          this->n = n;
6          ver.resize(n + 1);
7      }
8      void add(int x, int y) {
9          ver[x].push_back(y);
10         ver[y].push_back(x);
11     }
12     int getlen(int root) { // 获取x所在树的直径
13         map<int, int> dep; // map用于优化输入为森林时的深度计算，亦可用vector
14         function<void(int, int)> dfs = [&](int x, int fa) -> void {
15             for (auto y : ver[x]) {
16                 if (y == fa) continue;
17                 dep[y] = dep[x] + 1;
18                 dfs(y, x);
19             }
20             if (dep[x] > dep[root]) {
21                 root = x;
22             }
23         };
24         dfs(root, 0);
25         int st = root; // 记录直径端点
26
27         dep.clear();
28         dfs(root, 0);
29         int ed = root; // 记录直径另一端点
30
31         return dep[root];

```

```

32     }
33 };

```

1.1.9 树论大封装 (直径 + 重心 + 中心)

```

1  struct Tree {
2      int n;
3      vector<vector<pair<int, int>>> e;
4      vector<int> dep, parent, maxdep, d1, d2, s1, s2, up;
5      Tree(int n) {
6          this->n = n;
7          e.resize(n + 1);
8          dep.resize(n + 1);
9          parent.resize(n + 1);
10         maxdep.resize(n + 1);
11         d1.resize(n + 1);
12         d2.resize(n + 1);
13         s1.resize(n + 1);
14         s2.resize(n + 1);
15         up.resize(n + 1);
16     }
17     void add(int u, int v, int w) {
18         e[u].push_back({w, v});
19         e[v].push_back({w, u});
20     }
21     void dfs(int u, int fa) {
22         maxdep[u] = dep[u];
23         for (auto [w, v] : e[u]) {
24             if (v == fa) continue;
25             dep[v] = dep[u] + 1;
26             parent[v] = u;
27             dfs(v, u);
28             maxdep[u] = max(maxdep[u], maxdep[v]);
29         }
30     }
31
32     void dfs1(int u, int fa) {
33         for (auto [w, v] : e[u]) {
34             if (v == fa) continue;
35             dfs1(v, u);
36             int x = d1[v] + w;
37             if (x > d1[u]) {
38                 d2[u] = d1[u], s2[u] = s1[u];
39                 d1[u] = x, s1[u] = v;
40             } else if (x > d2[u]) {
41                 d2[u] = x, s2[u] = v;
42             }
43         }
44     }
45     void dfs2(int u, int fa) {
46         for (auto [w, v] : e[u]) {
47             if (v == fa) continue;

```

```

48         if (s1[u] == v) {
49             up[v] = max(up[u], d2[u]) + w;
50         } else {
51             up[v] = max(up[u], d1[u]) + w;
52         }
53         dfs2(v, u);
54     }
55 }
56
57 int radius, center, diam;
58 void getCenter() {
59     center = 1; //中心
60     for (int i = 1; i <= n; i++) {
61         if (max(d1[i], up[i]) < max(d1[center], up[center])) {
62             center = i;
63         }
64     }
65     radius = max(d1[center], up[center]); //距离最远点的距离的最小值
66     diam = d1[center] + up[center] + 1; //直径
67 }
68
69 int rem; //删除重心后剩余连通块体积的最小值
70 int cog; //重心
71 vector<bool> vis;
72 void getCog() {
73     vis.resize(n);
74     rem = INT_MAX;
75     cog = 1;
76     dfsCog(1);
77 }
78 int dfsCog(int u) {
79     vis[u] = true;
80     int s = 1, res = 0;
81     for (auto [w, v] : e[u]) {
82         if (vis[v]) continue;
83         int t = dfsCog(v);
84         res = max(res, t);
85         s += t;
86     }
87     res = max(res, n - s);
88     if (res < rem) {
89         rem = res;
90         cog = u;
91     }
92     return s;
93 }
94 };

```

1.1.10 点分治 / 树的重心

重心的定义：删除树上的某一个点，会得到若干棵子树；删除某点后，得到的最大子树最小，这个点称为重心。我们假设某个点是重心，记录此时最大子树的最小值，遍历完所有点后取最大

值即可。

重心的性质：重心最多可能会有两个，且此时两个重心相邻。

点分治的一般过程是：取重心为新树的根，随后使用 `dfs` 处理当前这棵树，灵活运用 `child` 和 `pre` 两个数组分别计算通过根节点、不通过根节点的路径信息，根据需要进行答案的更新；再对子树分治，寻找子树的重心，……。时间复杂度降至 $\mathcal{O}(N \log N)$ 。

```

1  int root = 0, MaxTree = 1e18; //分别代表重心下标、最大子树大小
2  vector<int> vis(n + 1), siz(n + 1);
3  auto get = [&](auto self, int x, int fa, int n) -> void { // 获取树的重心
4      siz[x] = 1;
5      int val = 0;
6      for (auto [y, w] : ver[x]) {
7          if (y == fa || vis[y]) continue;
8          self(self, y, x, n);
9          siz[x] += siz[y];
10         val = max(val, siz[y]);
11     }
12     val = max(val, n - siz[x]);
13     if (val < MaxTree) {
14         MaxTree = val;
15         root = x;
16     }
17 };
18
19 auto clac = [&](int x) -> void { // 以 x 为新的根，维护询问
20     set<int> pre = {0}; // 记录到根节点 x 距离为 i 的路径是否存在
21     vector<int> dis(n + 1);
22     for (auto [y, w] : ver[x]) {
23         if (vis[y]) continue;
24         vector<int> child; // 记录 x 的子树节点的深度信息
25         auto dfs = [&](auto self, int x, int fa) -> void {
26             child.push_back(dis[x]);
27             for (auto [y, w] : ver[x]) {
28                 if (y == fa || vis[y]) continue;
29                 dis[y] = dis[x] + w;
30                 self(self, y, x);
31             }
32         };
33         dis[y] = w;
34         dfs(dfs, y, x);
35
36         for (auto it : child) {
37             for (int i = 1; i <= m; i++) { // 根据询问更新值
38                 if (q[i] < it || !pre.count(q[i] - it)) continue;
39                 ans[i] = 1;
40             }
41         }
42         pre.insert(child.begin(), child.end());
43     }
44 };
45

```

```

46 auto dfz = [&](auto self, int x, int fa) -> void { // 点分治
47     vis[x] = 1; // 标记已经被更新过的旧重心, 确保只对子树分治
48     clac(x);
49     for (auto [y, w] : ver[x]) {
50         if (y == fa || vis[y]) continue;
51         MaxTree = 1e18;
52         get(get, y, x, siz[y]);
53         self(self, root, x);
54     }
55 };
56
57 get(get, 1, 0, n);
58 dfz(dfz, root, 0);

```

1.2 图论

1.2.1 最短路

```

1 // -----
2 // bfs (本题对每个节点求奇数最短路和偶数最短路)
3 // -----
4 #include <bits/stdc++.h>
5 using namespace std;
6 using pii = pair<int,int>;
7 const int INF = 0x3f3f3f3f;
8
9 int main(){
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     int n, m, Q;
14     cin >> n >> m >> Q;
15     vector<vector<int>>> g(n+1);
16     for(int i = 0, u, v; i < m; i++){
17         cin >> u >> v;
18         g[u].push_back(v);
19         g[v].push_back(u);
20     }
21     // dist[u][p]: 最短步行到 u 且步数奇偶为 p 的长度
22     vector<array<int,2>> dist(n+1, {INF, INF});
23     queue<pii> q;
24     dist[1][0] = 0;
25     q.emplace(1, 0);
26
27     while(!q.empty()){
28         auto [u, p] = q.front(); q.pop();
29         int nd = dist[u][p] + 1, np = p ^ 1;
30         for(int v : g[u]){
31             if(dist[v][np] > nd){
32                 dist[v][np] = nd;
33                 q.emplace(v, np);
34             }

```

```

35     }
36 }
37
38 while(Q--){
39     int a; long long L;
40     cin >> a >> L;
41     int p = L & 1;
42     cout << (dist[a][p] <= L ? "Yes\n" : "No\n");
43 }
44 return 0;
45 }
46
47 // -----
48 // dijk
49 // -----
50 void solve() {
51     int n , m , s , t; cin>>n>>m>>s>>t;
52     vector<vector< pii > > g(n+1);
53     while(m--){
54         int u , v , w;
55         cin>>u>>v>>w;
56         g[u].push_back({v,w});
57         g[v].push_back({u,w}); // 插入双向边
58     }
59
60     vector<int> dist(n+1,INT_MAX);
61     auto dijk = [&](){
62         // 最小堆
63         priority_queue<pii,vector<pii>,greater<pii> > pq;
64         pq.push({0,s});
65         dist[s] = 0;
66
67         while(pq.size()){
68             auto jm = pq.top(); pq.pop();
69             int d = jm.first;
70             int u = jm.second;
71             if (d > dist[u]) continue; // 已经有更优解，则跳过
72             // if(u==t) break;
73             for(auto &e:g[u]){
74                 int v = e.first;
75                 int w = e.second;
76                 if(dist[v]>d+w){
77                     dist[v] = d+w;
78                     pq.push({dist[v],v});
79                 }
80             }
81         }
82     };
83     dijk();
84     cout<< (dist[t]==INT_MAX?-1:dist[t]);
85 }
86
87 // -----

```



```
88 // spfa 负权
89 // -----
90 #include <bits/stdc++.h>
91 using namespace std;
92 using ll = long long;
93 const ll INF = LLONG_MAX;
94
95 struct Edge {
96     int to, w;
97 };
98
99 vector<vector<Edge>> g;
100 vector<ll> dist;
101 vector<bool> inq;
102 vector<int> cnt; // 松弛计数, 用于负环检测
103
104 // 返回 true 表示无负环, false 表示存在负环
105 bool spfa(int s, int n) {
106     queue<int> q;
107     dist.assign(n+1, INF);
108     inq.assign(n+1, false);
109     cnt.assign(n+1, 0);
110
111     dist[s] = 0;
112     q.push(s);
113     inq[s] = true;
114
115     while (!q.empty()) {
116         int u = q.front(); q.pop();
117         inq[u] = false;
118
119         for (auto &e : g[u]) {
120             int v = e.to, w = e.w;
121             if (dist[u] != INF && dist[u] + w < dist[v]) {
122                 dist[v] = dist[u] + w;
123                 cnt[v] = cnt[u] + 1;
124                 if (cnt[v] >= n)
125                     return false; // 负环检测
126                 if (!inq[v]) {
127                     inq[v] = true;
128                     q.push(v);
129                 }
130             }
131         }
132     }
133     return true;
134 }
135
136 int main() {
137     ios::sync_with_stdio(false);
138     cin.tie(nullptr);
139
140     int n, m;
```

```

141     cin >> n >> m; // n 点数, m 边数
142     g.assign(n+1, {});
143
144     for (int i = 0; i < m; i++) {
145         int u, v, w;
146         cin >> u >> v >> w;
147         g[u].push_back({v, w});
148         //g[v].push_back({u, w}); 双向的话
149     }
150
151     int s;
152     cin >> s; // 源点
153
154     if (spfa(s, n)) {
155         for (int i = 1; i <= n; i++) {
156             if (dist[i] == INF)
157                 cout << "INF ";
158             else
159                 cout << dist[i] << ' ';
160         }
161     } else {
162         cout << "Negative cycle detected";
163     }
164     return 0;
165 }
166 // -----
167 // 0-1 bfs
168 // -----
169 void zeroOneBFS() {
170     fill(dis, dis + (k + 2) * n + 1, INF);
171     deque<int> dq;
172     dis[1] = 0;
173     dq.push_back(1);
174
175     while (!dq.empty()) {
176         int u = dq.front();
177         dq.pop_front();
178         for (auto [v, w] : e[u]) {
179             if (dis[v] > dis[u] + w) {
180                 dis[v] = dis[u] + w;
181                 if (w == 0)
182                     dq.push_front(v);
183                 else
184                     dq.push_back(v);
185             }
186         }
187     }
188 }
189
190 // -----
191 // 多源: floyd
192 // -----
193 const int N=500;

```

```

194 const ll inf=2e18;
195 ll d[N][N];
196
197 int main(){
198     cin>>n>>m>>q;
199     for(int i=1;i<=n;i++)
200         for(int j=1;j<=n;j++)
201             d[i][j]=inf; //dist初始为inf
202
203     while(m--){
204         ll u,v,w;
205         cin>>u>>v>>w;
206         d[u][v]=d[v][u]=min(w,d[u][v]); //1.有重边取最小
207     }
208     for(int i=1;i<=n;i++)d[i][i]=0; //2.对角线为0
209
210     for(int k=1;k<=n;k++){
211         for(int i=1;i<=n;i++){
212             for(int j=1;j<=n;j++){
213                 //if(d[i][k]!=INF and d[k][j]!=INF)
214                 //3.不需要原因: inf=2e18
215                 d[i][j]=min(d[i][j],d[i][k]+d[k][j]);
216             }
217         }
218     }
219     while(q--){
220         int st,ed;
221         cin>>st>>ed;
222         cout<<d[st][ed]<<endl;
223     }
224     return 0;
225 }

```

1.2.2 kruscal

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  //最小生成树
4  const int N = 1e5 + 10;
5  int fa[N];
6
7  int find(int x) {
8      return fa[x] == x ? x : fa[x] = find(fa[x]);
9  }
10
11 struct Edge {
12     int u, v, w;
13     bool operator < (const Edge &e) const {
14         return w < e.w;
15     }
16 };
17

```

```

18 int main() {
19     int n, m;
20     cin >> n >> m;
21     vector<Edge> edges(m);
22     for (int i = 0; i < m; ++i)
23         cin >> edges[i].u >> edges[i].v >> edges[i].w;
24
25     sort(edges.begin(), edges.end());
26     for (int i = 1; i <= n; ++i) fa[i] = i;
27
28     int mst = 0, cnt = 0;
29     for (auto e : edges) {
30         int fu = find(e.u), fv = find(e.v);
31         if (fu != fv) {
32             fa[fu] = fv;
33             mst += e.w;
34             if (++cnt == n - 1) break;
35         }
36     }
37
38     if (cnt == n - 1) cout << mst << '\n';
39     else cout << "no spanning tree\n";
40 }

```

1.2.3 强连通分量缩点

```

1 //刻录光盘2: tarjan + 缩点
2 1.是在有N个营员的夏令营场景中，每个营员会说明自己愿意把夏令营资料拷贝给其他哪些营员。比如A愿
   意把资料拷贝给
3 B, B又愿意拷贝给C，那么只要A获得了资料，B和C也能获得。我们需要计算出最少要刻录多少张光盘，
   才能让所有营员
4 都能得到资料。
5 2.是假如组委会只提供1张光盘，并且随机给一个营员，为了保证不管最开始这张光盘给了哪个营员，其
   他所有营
6 员最后都能拿到资料，我们要算出最少需要增加多少条新的允许拷贝资料的关系。
7
8 对于第一问，缩点之后，统计入度为0的强连通分量的数量，这个数量就是最少要刻录的光盘数。
9 对于第二问，同样先进行缩点，分别统计入度为0和出度为0的强连通分量的数量，取这两个数
10 量中的最大值，就是最少需要增加的关系条数。
11 不过有一种特殊情况，如果整个图只有一个强连通分量，那么答案就是0。
12
13 #include <bits/stdc++.h>
14 using namespace std;
15 const int N = 1e4+5, M = 5e4+5;
16 int n, m;
17 bool ins[N];
18 stack<int> stk;
19 int dfn[N], low[N];
20 int num;
21 vector<int> scc[N], g[N];
22 int cnt;
23 int c[N]; // c[i]: i所属强连通分量编号

```

```
24
25 void tarjan(int u) {
26     dfn[u] = low[u] = ++num;
27     stk.push(u);
28     ins[u] = true;
29
30     for (int v : g[u]) {
31         if (!dfn[v]) {
32             tarjan(v);
33             low[u] = min(low[u], low[v]);
34         }
35         else if (ins[v]) {
36             low[u] = min(low[u], low[v]);
37         }
38     }
39
40     if (dfn[u] == low[u]) {
41         cnt++;
42         int v;
43         do {
44             v = stk.top();
45             stk.pop();
46             ins[v] = false;
47             c[v] = cnt;
48             scc[cnt].push_back(v);
49         } while(u != v);
50     }
51 }
52
53 int main() {
54     cin >> n;
55     for (int i = 1; i <= n; i++) {
56         int t;
57         while (cin >> t, t) {
58             g[i].push_back(t);
59         }
60     }
61     for (int i = 1; i <= n; i++) {
62         if (!dfn[i]) {
63             tarjan(i);
64         }
65     }
66
67     vector<int> gn[N];
68     int ind[N] = {0}, out[N] = {0};
69     for (int u = 1; u <= n; u++) {
70         for (auto v : g[u]) {
71             if (c[u] != c[v]) {
72                 gn[c[u]].push_back(c[v]);
73                 ind[c[v]]++;
74                 out[c[u]]++;
75             }
76         }
77     }
```

```

77     }
78     if (cnt == 1) {
79         cout << 1 << endl;
80         cout << 0 << endl;
81         return 0;
82     }
83     int in = 0, ou = 0;
84     for (int i = 1; i <= cnt; i++) {
85         if (ind[i] == 0) in++;
86         if (out[i] == 0) ou++;
87     }
88     cout << in << endl;
89     cout << max(in, ou) << endl;
90     return 0;
91 }
92
93 // -----
94 // 求割点
95 // 附: x->y是桥: low[y]>dfn[x]
96 // -----
97 #include <iostream>
98 #include <vector>
99 #include <sstream>
100 #include <algorithm>
101 using namespace std;
102 int n, t;
103 vector<int> g[105];
104 int d[105], low[105];
105 bool a[105];
106 void dfs(int u, int p) {
107     d[u] = low[u] = ++t;
108     int c = 0;
109     for (auto v : g[u]) {
110         if (!d[v]) {
111             c++;
112             dfs(v, u);
113             low[u] = min(low[u], low[v]);
114             if (p != -1 && low[v] >= d[u]) //对于非root结点
115                 a[u] = true;
116         } else if (v != p)
117             low[u] = min(low[u], d[v]);
118     }
119     if (p == -1 && c > 1) //对于root结点
120         a[u] = true;
121 }
122 int main(){
123     while(cin >> n){
124         if(n == 0)
125             break;
126         for(int i = 1; i <= n; i++){
127             g[i].clear();
128             d[i] = 0;
129             low[i] = 0;

```

```

130         a[i] = false;
131     }
132     int u, v;
133     string s;
134     getline(cin, s);
135     while(getline(cin, s)){
136         if(s == "0")
137             break;
138         istringstream iss(s);
139         iss >> u;
140         while(iss >> v && v){
141             g[u].push_back(v);
142             g[v].push_back(u);
143         }
144     }
145     t = 0;
146     for(int i = 1; i <= n; i++){
147         if(!d[i])
148             dfs(i, -1);
149     }
150     int cnt = 0;
151     for(int i = 1; i <= n; i++)
152         if(a[i])cnt++;
153     cout << cnt << "\n"; //输出个数
154 }
155 return 0;
156 }

```

1.2.4 差分约束

```

1  /*
2  * 差分约束系统求解:
3  *
4  * 解决的问题:
5  * 给定 N 个变量  $x_1 \cdots x_N$  和 M 条形如
6  *  $x_i - x_j \leq w$ 
7  * 的不等式约束,
8  * 判断这组约束是否有可行解,
9  * 若存在, 则求出一组每个变量尽可能小的可行值。
10 *
11 * 转化思路:
12 * 1. 将约束  $x_i - x_j \leq w$  转为有向图中一条从节点 j 指向节点 i、权重为 w 的边。
13 * 2. 添加超级源点 0, 并对每个 i 加入边  $0 \rightarrow i$ , 权重为 0, 保证所有节点可达。
14 * 3. 运行最短路算法 (Bellman-Ford 或 SPFA) :
15 * - 若检测到负权环, 则说明约束无解;
16 * - 否则, 源点到 i 的最短距离即为  $x_i$  的最小可行值。
17 */
18 #include <bits/stdc++.h>
19 using namespace std;
20 using ll = long long;
21 const ll INF = (1LL<<60);
22

```

```

23 struct Edge { int v; ll w; };
24 int main(){
25     ios::sync_with_stdio(false);
26     cin.tie(NULL);
27
28     int N, M;
29     // N = 变量个数, 编号 1..N; M = 不等式条数
30     cin >> N >> M;
31
32     vector<vector<Edge>> g(N+1);
33     int u, v;
34     ll w;
35     while(M--){
36         // 读取约束  $x_i - x_j \leq w$ 
37         cin >> u >> v >> w;
38         // 转化为边  $j \rightarrow i$  权重  $w$ 
39         g[v].push_back({u, w});
40     }
41     // 超级源点  $0 \rightarrow i$  权  $0$ 
42     for(int i=1; i<=N; i++){
43         g[0].push_back({i, 0});
44     }
45
46     vector<ll> dist(N+1, INF);
47     vector<int> inq(N+1, 0);
48     vector<int> cnt(N+1, 0);
49     queue<int> q;
50
51     // SPFA
52     dist[0] = 0;
53     q.push(0);
54     inq[0] = 1;
55
56     bool hasNegCycle = false;
57     while(!q.empty() && !hasNegCycle){
58         int x = q.front(); q.pop();
59         inq[x] = 0;
60         for(auto &e : g[x]){
61             if(dist[e.v] > dist[x] + e.w){
62                 dist[e.v] = dist[x] + e.w;
63                 if(!inq[e.v]){
64                     q.push(e.v);
65                     inq[e.v] = 1;
66                     if(++cnt[e.v] > N+1){
67                         hasNegCycle = true;
68                         break;
69                     }
70                 }
71             }
72         }
73     }
74
75     if(hasNegCycle){

```



```

76     cout << "No solution: negative cycle detected\n";
77 } else {
78     // dist[i] 即为 x_i 的最小可行值
79     for(int i=1;i<=N;i++){
80         // 若需要, 可根据题意输出或处理 dist[i]
81         cout << dist[i] << (i==N?'\n':' ');
82     }
83 }
84
85 return 0;
86 }

```

1.2.5 拓扑

```

1  #include<iostream>
2  #include<queue>
3  #include<vector>
4  using namespace std;
5  typedef long long ll;
6
7  const int N=2e5+10;
8  int n;
9  int ind[N];
10 queue<int> q;
11 vector<int> son[N];
12 typedef pair<int,int> pii;
13 vector<pii> wu;
14 vector<int> topo(N);
15 int idx;
16
17 int main(){
18     std::ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
19     int n;
20     cin>>n;
21     int m,k;
22     cin>>m>>k;
23     for(int i=0;i<m;i++){
24         int x,y;
25         cin>>x>>y;
26         wu.push_back({x,y});
27     }
28
29     for(int i=1;i<=k;i++){
30         int u,v;
31         cin>>u>>v;
32         son[u].push_back(v);
33         ind[v]++;
34     }
35     // for(int i=1;i<=n;i++){
36     // do{
37     //     cin>>x;
38     //     if(x==0)break;

```

```

39         // son[i].push_back(x);
40         // ind[x]++;
41         // }while(x!=0);
42     // }
43     for(int i=1;i<=n;i++){
44         if(!ind[i]) q.push(i);
45     }
46
47     while(q.size()){
48         int x=q.front(); q.pop();
49         topo[x]=idx++;
50         for( auto y:son[x]){
51             ind[y]--;
52             if(ind[y]==0){
53 //         cout<<y;
54                 q.push(y);
55             }
56         }
57     }
58     if(idx<n){
59         cout<<-1;
60         return 0;
61     }
62     for(int i=0;i<m;i++){
63         if( topo[wu[i].first] <= topo[wu[i].second] ){
64             cout<<wu[i].first<<" "<<wu[i].second<<endl;
65         }
66         else{
67             cout<<wu[i].second<<" "<<wu[i].first<<endl;
68         }
69     }
70
71     return 0;
72 }

```

1.2.6 2-Sat

基础封装

基于 tarjan 缩点，时间复杂度为 $\mathcal{O}(N + M)$ 。注意下标从 0 开始，答案输出为字典序最小的一个可行解。

答案不唯一时不输出

在运行后针对每一个点进行一次 dfs，时间复杂度为 $\mathcal{O}(N^2)$ ，当且仅当答案唯一时才输出，否则输出 ? 替代。

```

1 struct TwoSat {
2     int n;
3     vector<vector<int>>> e;
4     vector<bool> ans;
5     TwoSat(int n) : n(n), e(2 * n), ans(n) {}
6     void add(int u, bool f, int v, bool g) {
7         e[2 * u + !f].push_back(2 * v + g);

```

```

8      e[2 * v + !g].push_back(2 * u + f);
9  }
10  bool work() {
11      vector<int> id(2 * n, -1), dfn(2 * n, -1), low(2 * n, -1);
12      vector<int> stk;
13      int now = 0, cnt = 0;
14      auto tarjan = [&](auto self, int u) -> void {
15          stk.push_back(u);
16          dfn[u] = low[u] = now++;
17          for (auto v : e[u]) {
18              if (dfn[v] == -1) {
19                  self(self, v);
20                  low[u] = min(low[u], low[v]);
21              } else if (id[v] == -1) {
22                  low[u] = min(low[u], dfn[v]);
23              }
24          }
25          if (dfn[u] == low[u]) {
26              int v;
27              do {
28                  v = stk.back();
29                  stk.pop_back();
30                  id[v] = cnt;
31              } while (v != u);
32              ++cnt;
33          }
34      };
35      for (int i = 0; i < 2 * n; ++i) {
36          if (dfn[i] == -1) {
37              tarjan(tarjan, i);
38          }
39      }
40      for (int i = 0; i < n; ++i) {
41          if (id[2 * i] == id[2 * i + 1]) return false;
42          ans[i] = id[2 * i] > id[2 * i + 1];
43      }
44      return true;
45  }
46  vector<bool> answer() {
47      return ans;
48  }
49  };
50
51  // 结构体中增加
52  int check(int x, int y) {
53      vector<int> vis(2 * n);
54      auto dfs = [&](auto self, int x) -> void {
55          vis[x] = 1;
56          for (auto y : e[x]) {
57              if (vis[y]) continue;
58              self(self, y);
59          }
60      };

```

```

61     dfs(dfs, x);
62     return vis[y];
63 }
64 // 主函数中增加
65 for (int i = 0; i < n; i++) {
66     if (sat.check(2 * i, 2 * i + 1)) {
67         cout << 1 << " ";
68     } else if (sat.check(2 * i + 1, 2 * i)) {
69         cout << 0 << " ";
70     } else {
71         cout << "?" << " ";
72     }
73 }

```

1.2.7 单源最短路径 (SSSP 问题)

(正权稀疏图) 动态数组存图 + Dijkstra 算法

使用优先队列优化, 以 $O(M \log N)$ 的复杂度计算。

(负权图、判负环) Bellman-ford 算法

使用结构体存边 (该算法无需存图), 以 $O(NM)$ 的复杂度计算。

(负权图) SPFA 算法

以 $O(KM)$ 的复杂度计算, 其中 K 虽然为常数, 但是可以通过特殊的构造退化接近 N , 需要注意被卡。

```

1  vector<int> dis(n + 1, 1E18);
2  auto djikstra = [&](int s = 1) -> void {
3      using PII = pair<int, int>;
4      priority_queue<PII, vector<PII>, greater<PII>> q;
5      q.emplace(0, s);
6      dis[s] = 0;
7      vector<int> vis(n + 1);
8      while (!q.empty()) {
9          int x = q.top().second;
10         q.pop();
11         if (vis[x]) continue;
12         vis[x] = 1;
13         for (auto [y, w] : ver[x]) {
14             if (dis[y] > dis[x] + w) {
15                 dis[y] = dis[x] + w;
16                 q.emplace(dis[y], y);
17             }
18         }
19     }
20 };
21
22 int n, m, s;
23 cin >> n >> m >> s;
24
25 vector<tuple<int, int, i64>> ver(m + 1);
26 for (int i = 1; i <= m; ++i) {

```

```

27     int x, y;
28     i64 w;
29     cin >> x >> y >> w;
30     ver[i] = {x, y, w};
31 }
32
33 vector<i64> dis(n + 1, inf), chk(n + 1);
34 dis[s] = 0;
35 for (int i = 1; i <= 2 * n; ++i) { // 双倍松弛, 获取负环信息
36     vector<i64> backup = dis;
37     for (int j = 1; j <= m; ++j) {
38         auto [x, y, w] = ver[j];
39         chk[y] |= (i > n && backup[x] + w < dis[y]);
40         dis[y] = min(dis[y], backup[x] + w);
41     }
42 }1
43
44 for (int i = 1; i <= n; ++i) {
45     if (i == s) {
46         cout << 0 << " ";
47     } else if (dis[i] >= inf / 2) {
48         cout << "no ";
49     } else if (chk[i]) {
50         cout << "inf ";
51     } else {
52         cout << dis[i] << " ";
53     }
54 }
55
56 const int N = 1e5 + 7, M = 1e6 + 7;
57 int n, m;
58 int ver[M], ne[M], h[N], edge[M], tot;
59 int d[N], v[N];
60
61 void add(int x, int y, int w) {
62     ver[++ tot] = y, ne[tot] = h[x], h[x] = tot;
63     edge[tot] = w;
64 }
65 void spfa() {
66     ms(d, 0x3f); d[1] = 0;
67     queue<int> q; q.push(1);
68     v[1] = 1;
69     while(!q.empty()) {
70         int x = q.front(); q.pop(); v[x] = 0;
71         for (int i = h[x]; i; i = ne[i]) {
72             int y = ver[i];
73             if(d[y] > d[x] + edge[i]) {
74                 d[y] = d[x] + edge[i];
75                 if(v[y] == 0) q.push(y), v[y] = 1;
76             }
77         }
78     }
79 }

```

```

80 int main() {
81     cin >> n >> m;
82     for (int i = 1; i <= m; ++ i) {
83         int x, y, w; cin >> x >> y >> w;
84         add(x, y, w);
85     }
86     spfa();
87     for (int i = 1; i <= n; ++ i) {
88         if (d[i] == INF) cout << "N" << endl;
89         else cout << d[n] << endl;
90     }
91 }

```

1.2.8 多源汇最短路 (APSP 问题)

使用邻接矩阵存图，可以处理负权边，以 $O(N^3)$ 的复杂度计算。注意，这里建立的是单向边，计算双向边需要额外加边。

```

1  const int N = 210;
2  int n, m, d[N][N];
3
4  void floyd() {
5      for (int k = 1; k <= n; k ++ )
6          for (int i = 1; i <= n; i ++ )
7              for (int j = 1; j <= n; j ++ )
8                  d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
9  }
10 int main() {
11     cin >> n >> m;
12     for (int i = 1; i <= n; i ++ )
13         for (int j = 1; j <= n; j ++ )
14             if (i == j) d[i][j] = 0;
15             else d[i][j] = INF;
16     while (m --) {
17         int x, y, w; cin >> x >> y >> w;
18         d[x][y] = min(d[x][y], w);
19     }
20     floyd();
21     for (int i = 1; i <= n; ++ i) {
22         for (int j = 1; j <= n; ++ j) {
23             if (d[i][j] > INF / 2) cout << "N" << endl;
24             else cout << d[i][j] << endl;
25         }
26     }
27 }

```

1.2.9 差分约束

给出一组包含 m 个不等式, 有 n 个未知数的形如:

$$\begin{cases} u_1 - v_1 \leq w_1 \\ u_2 - v_2 \leq w_2 \\ \dots \\ u_m - v_m \leq w_m \end{cases}$$

的不等式组, 求任意一组满足这个不等式组的解。SPFA 解, $\mathcal{O}(nm)$ 。参考

```

1 signed main() {
2     int n, m;
3     cin >> n >> m;
4
5     vector<array<int, 3>> e(m + 1);
6     for (int i = 1; i <= m; i++) {
7         int u, v, w;
8         cin >> u >> v >> w;
9         e[i] = {v, u, w};
10    }
11
12    vector<int> d(n + 1, 1E9);
13    d[1] = 0;
14    for (int i = 1; i < n; i++) {
15        for (int j = 1; j <= m; j++) {
16            auto [u, v, w] = e[j];
17            d[v] = min(d[v], d[u] + w);
18        }
19    }
20    for (int i = 1; i <= m; i++) {
21        auto [u, v, w] = e[i];
22        if (d[v] > d[u] + w) {
23            cout << "NO\n";
24            return 0;
25        }
26    }
27    for (int i = 1; i <= n; i++) {
28        cout << d[i] << " \n"[i == n];
29    }
30    return 0;
31 }

```

1.2.10 常见例题

杂

题意: 给出一棵节点数为 $2n$ 的树, 要求将点分割为 n 个点对, 使得点对的点之间的距离和最大。

可以转化为边上问题: 对于每一条边, 其被利用的次数即为 $\min\{\text{其左边的点的数量}, \text{其右边的点的数量}\}$, 使用树形 dp 计算一遍即可。如下图样例, 答案为 10。

题意: 以哪些点为起点可以无限的在有向图上绕

概括一下这些点可以发现，一类是环上的点，另一类是可以到达环的点。建反图跑一遍 topsort 板子，根据容斥，未被移除的点都是答案 See。

题意：添加最少的边，使得有向图变成一个 SCC

将原图的 SCC 缩点，统计缩点后的新图上入度为 0 和出度为 0 的点的数量 cnt_{in} cnt_{out} ，答案即为 $\max(cnt_{in}, cnt_{out})$ 。过程大致是先将一个出度为 0 的点和一个入度为 0 的点相连，剩下的点随便连 See。

题意：添加最少的边，使得无向图变成一个 E-DCC

将原图的 E-DCC 缩点，统计缩点后的新图上入度为 1 的点（叶子结点）的数量 cnt ，答案即为 $\lceil \frac{cnt}{2} \rceil$ 。过程大致是每次找两个叶子结点（但是还有一些条件限制）相连，若最后余下一个点随便连 See。

题意：在树上找到一个最大的连通块，使得这个联通内点权和边权之和最大，输出这个值，数据中存在负数的情况。

使用 dfs 即可解决。

Prüfer 序列：凯莱公式

题意：给定 n 个顶点，可以构建出多少棵标记树？

$n \leq 4$ 时的样例如上，通项公式为 n^{n-2} 。

Prüfer 序列

一个 n 个点 m 条边的带标号无向图有 k 个连通块。我们希望添加 $k-1$ 条边使得整个图连通，求方案数量 See。

设 s_i 表示每个连通块的数量，通项公式为 $n^{k-2} \cdot \prod_{i=1}^k s_i$ ，当 $k < 2$ 时答案为 1。

单源最短/次短路计数

输出任意一个三元环

原题：给出一张有向完全图，输出任意一个三元环上的全部元素 See。使用 dfs，复杂度 $\mathcal{O}(N+M)$ ，可以扩展到非完全图和无向图。

带权最小环大小与计数

原题：给出一张有向带权图，求解图上最小环的长度、有多少个这样的最小环 See。使用 floyd，复杂度为 $\mathcal{O}(N^3)$ ，可以扩展到无向图。

最小环大小

原题：给出一张无向图，求解图上最小环的长度、有多少个这样的最小环 See。使用 floyd，可以扩展到有向图。

使用 bfs，复杂度为 $\mathcal{O}(N^2)$ 。

本质不同简单环计数

原题：给出一张无向图，输出简单环的数量 See。注意这里环套环需要分别多次统计，下图答案应当为 7。使用状压 dp，复杂度为 $\mathcal{O}(M \cdot 2^N)$ ，可以扩展到有向图。

输出任意一个非二元简单环

原题：给出一张无向图，不含自环与重边，输出任意一个简单环的大小以及其上面的全部元素 See 。注意输出的环的大小是随机的，**不等价于最小环**。

由于不含重边与自环，所以环的大小至少为 3，使用 dfs 处理出 dfs 序，复杂度为 $\mathcal{O}(N + M)$ ，可以扩展到无向图；如果有向图中二元环也允许计入答案，则需要删除下方标注行。

有向图环计数

原题：给出一张有向图，输出环的数量。注意这里环套环仅需要计算一次，数据包括二元环和自环，下图例应当输出 3 个环。使用 dfs 染色法，复杂度为 $\mathcal{O}(N + M)$ 。

有向图简单环检查、输出

NowCoder ，带自环重边、不连通。

无向图简单环检查、输出

NowCoder ，带自环重边、不连通。

判定带环图是否是平面图

原题：给定一个环以一些额外边，对于每一条额外边判定其位于环外还是环内，使得任意两条无重合顶点的额外边都不相交（即这张图构成平面图） See1, See2 。

使用 2-sat。考虑全部边都位于环内，那么“一条边完全包含另一条边”、“两条边完全没有交集”这两种情况都不会相交，可以直接跳过这两种情况的讨论。

网络流

```

1 vector<int> val(n + 1, 1);
2 int ans = 0;
3 function<void(int, int)> dfs = [&](int x, int fa) {
4     for (auto y : ver[x]) {
5         if (y == fa) continue;
6         dfs(y, x);
7         val[x] += val[y];
8         ans += min(val[y], k - val[y]);
9     }
10 };
11 dfs(1, 0);
12 cout << ans << endl;
13
14 LL n, point[N];
15 LL ver[N], head[N], nex[N], tot; bool v[N];
16 map<pair<LL, LL>, LL> edge;
17 // void add(LL x, LL y) {}
18 void dfs(LL x) {
19     for (LL i = head[x]; i; i = nex[i]) {
20         LL y = ver[i];
21         if (v[y]) continue;
22         v[y] = true; dfs(y); v[y] = false;
23     }
24     for (LL i = head[x]; i; i = nex[i]) {
25         LL y = ver[i];
26         if (v[y]) continue;
27         point[x] += max(point[y] + edge[{x, y}], 0LL);
28     }

```

```

29 }
30 void Solve() {
31     cin >> n;
32     FOR(i, 1, n) cin >> point[i];
33     FOR(i, 2, n) {
34         LL x, y, w; cin >> x >> y >> w;
35         edge[{x, y}] = edge[{y, x}] = w;
36         add(x, y), add(y, x);
37     }
38     v[1] = true; dfs(1); LL ans = -MAX18;
39     FOR(i, 1, n) ans = max(ans, point[i]);
40     cout << ans << endl;
41 }
42
43 const int N = 2e5 + 7, M = 1e6 + 7;
44 int n, m, s, e; int d[N][2], v[N][2]; // 0 代表最短路, 1 代表次短路
45 Z num[N][2];
46
47 void Clear() {
48     for (int i = 1; i <= n; ++ i) h[i] = edge[i] = 0;
49     tot = 0;
50     for (int i = 1; i <= n; ++ i) num[i][0] = num[i][1] = v[i][0] = v[i][1] = 0;
51     for (int i = 1; i <= n; ++ i) d[i][0] = d[i][1] = INF;
52 }
53
54 int ver[M], ne[M], h[N], edge[M], tot;
55 void add(int x, int y, int w) {
56     ver[++ tot] = y, ne[tot] = h[x], h[x] = tot;
57     edge[tot] = w;
58 }
59
60 void dji() {
61     priority_queue<PIII, vector<PIII>, greater<PIII> > q; q.push({0, s, 0});
62     num[s][0] = 1; d[s][0] = 0;
63     while (!q.empty()) {
64         auto [dis, x, type] = q.top(); q.pop();
65         if (v[x][type]) continue; v[x][type] = 1;
66         for (int i = h[x]; i; i = ne[i]) {
67             int y = ver[i], w = dis + edge[i];
68             if (d[y][0] > w) {
69                 d[y][1] = d[y][0], num[y][1] = num[y][0];
70                 // 如果找到新的最短路, 将原有的最短路数据转化为次短路
71                 q.push({d[y][1], y, 1});
72                 d[y][0] = w, num[y][0] = num[x][type];
73                 q.push({d[y][0], y, 0});
74             }
75             else if (d[y][0] == w) num[y][0] += num[x][type];
76             else if (d[y][1] > w) {
77                 d[y][1] = w, num[y][1] = num[x][type];
78                 q.push({d[y][1], y, 1});
79             }
80             else if (d[y][1] == w) num[y][1] += num[x][type];
81         }
82     }

```

```

82     }
83 }
84 void Solve() {
85     cin >> n >> m >> s >> e;
86     Clear(); //多组样例务必完全清空
87     for (int i = 1; i <= m; ++i) {
88         int x, y, w; cin >> x >> y; w = 1;
89         add(x, y, w), add(y, x, w);
90     }
91     dji();
92     Z ans = num[e][0];
93     if (d[e][1] == d[e][0] + 1) {
94         ans += num[e][1]; // 只有在次短路满足条件时才计算 (距离恰好比最短路大1)
95     }
96     cout << ans.val() << endl;
97 }
98
99 int n;
100 cin >> n;
101 vector<vector<int>> a(n + 1, vector<int>(n + 1));
102 for (int i = 1; i <= n; ++i) {
103     for (int j = 1; j <= n; ++j) {
104         char x;
105         cin >> x;
106         if (x == '1') a[i][j] = 1;
107     }
108 }
109
110 vector<int> vis(n + 1);
111 function<void(int, int)> dfs = [&](int x, int fa) {
112     vis[x] = 1;
113     for (int y = 1; y <= n; ++y) {
114         if (a[x][y] == 0) continue;
115         if (a[y][fa] == 1) {
116             cout << fa << " " << x << " " << y;
117             exit(0);
118         }
119         if (!vis[y]) dfs(y, x); // 这一步的if判断很关键
120     }
121 };
122 for (int i = 1; i <= n; ++i) {
123     if (!vis[i]) dfs(i, -1);
124 }
125 cout << -1;
126
127 LL Min = 1e18, ans = 0;
128 for (int k = 1; k <= n; k++) {
129     for (int i = 1; i <= n; i++) {
130         for (int j = 1; j <= n; j++) {
131             if (dis[i][j] > dis[i][k] + dis[k][j]) {
132                 dis[i][j] = dis[i][k] + dis[k][j];
133                 cnt[i][j] = cnt[i][k] * cnt[k][j] % mod;
134             } else if (dis[i][j] == dis[i][k] + dis[k][j]) {

```

```

135         cnt[i][j] = (cnt[i][j] + cnt[i][k] * cnt[k][j] % mod) % mod;
136     }
137 }
138 }
139 for (int i = 1; i < k; i++) {
140     if (a[k][i]) {
141         if (a[k][i] + dis[i][k] < Min) {
142             Min = a[k][i] + dis[i][k];
143             ans = cnt[i][k];
144         } else if (a[k][i] + dis[i][k] == Min) {
145             ans = (ans + cnt[i][k]) % mod;
146         }
147     }
148 }
149 }
150
151 int flody(int n) {
152     for (int i = 1; i <= n; ++ i) {
153         for (int j = 1; j <= n; ++ j) {
154             val[i][j] = dis[i][j]; // 记录最初的边权值
155         }
156     }
157     int ans = 0x3f3f3f3f;
158     for (int k = 1; k <= n; ++ k) {
159         for (int i = 1; i < k; ++ i) { // 注意这里是没有等于号的
160             for (int j = 1; j < i; ++ j) {
161                 ans = min(ans, dis[i][j] + val[i][k] + val[k][j]);
162             }
163         }
164         for (int i = 1; i <= n; ++ i) { // 往下是标准的flody
165             for (int j = 1; j <= n; ++ j) {
166                 dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
167             }
168         }
169     }
170     return ans;
171 }
172
173 auto bfs = [&] (int s) {
174     queue<int> q; q.push(s);
175     dis[s] = 0;
176     fa[s] = -1;
177     while (q.size()) {
178         auto x = q.front(); q.pop();
179         for (auto y : ver[x]) {
180             if (y == fa[x]) continue;
181             if (dis[y] == -1) {
182                 dis[y] = dis[x] + 1;
183                 fa[y] = x;
184                 q.push(y);
185             }
186             else ans = min(ans, dis[x] + dis[y] + 1);
187         }
188     }
189 }

```

```

188     }
189 };
190 for (int i = 1; i <= n; ++ i) {
191     fill(dis + 1, dis + 1 + n, -1);
192     bfs(i);
193 }
194 cout << ans;
195
196 int n, m;
197 cin >> n >> m;
198 vector<vector<int>> G(n);
199 for (int i = 0; i < m; i++) {
200     int u, v;
201     cin >> u >> v;
202     u--, v--;
203     G[u].push_back(v);
204     G[v].push_back(u);
205 }
206 vector<vector<LL>> dp(1 << n, vector<LL>(n));
207 for (int i = 0; i < n; i++) dp[1 << i][i] = 1;
208 LL ans = 0;
209 for (int st = 1; st < (1 << n); st++) {
210     for (int u = 0; u < n; u++) {
211         if (!dp[st][u]) continue;
212         int start = st & -st;
213         for (auto v : G[u]) {
214             if ((1 << v) < start) continue;
215             if ((1 << v) & st) {
216                 if ((1 << v) == start) {
217                     ans += dp[st][u];
218                 }
219             } else {
220                 dp[st | (1 << v)][v] += dp[st][u];
221             }
222         }
223     }
224 }
225 cout << (ans - m) / 2 << "\n";
226
227 vector<int> dis(n + 1, -1), fa(n + 1);
228 auto dfs = [&](auto self, int x) -> void {
229     for (auto y : ver[x]) {
230         if (y == fa[x]) continue; // 二元环需删去该行
231         if (dis[y] == -1) {
232             dis[y] = dis[x] + 1;
233             fa[y] = x;
234             self(self, y);
235         } else if (dis[y] < dis[x]) {
236             cout << dis[x] - dis[y] + 1 << endl;
237             int pre = x;
238             cout << pre << " ";
239             while (pre != y) {
240                 pre = fa[pre];

```

```
241         cout << pre << " ";
242     }
243     cout << endl;
244     exit(0);
245 }
246 }
247 };
248 for (int i = 1; i <= n; i++) {
249     if (dis[i] == -1) {
250         dis[i] = 0;
251         dfs(dfs, 1);
252     }
253 }
254
255 int ans = 0;
256 vector<int> vis(n + 1);
257 auto dfs = [&](auto self, int x) -> void {
258     vis[x] = 1;
259     for (auto y : ver[x]) {
260         if (vis[y] == 0) {
261             self(self, y);
262         } else if (vis[y] == 1) {
263             ans++;
264         }
265     }
266     vis[x] = 2;
267 };
268 for (int i = 1; i <= n; i++) {
269     if (!vis[i]) {
270         dfs(dfs, i);
271     }
272 }
273 cout << ans << endl;
274
275 signed main() {
276     int n, m;
277     cin >> n >> m;
278
279     vector<vector<array<int, 2>>> ver(n + 1);
280     for (int i = 1; i <= m; i++) {
281         int x, y;
282         cin >> x >> y;
283         ver[x].push_back({y, i});
284     }
285
286     vector<int> vis(n + 1);
287     vector<array<int, 2>> fa(n + 1);
288     auto dfs = [&](auto self, int x, int from) -> void {
289         vis[x] = 1;
290         for (auto [y, id] : ver[x]) {
291             if (id == from) continue;
292             if (!vis[y]) {
293                 fa[y] = {x, id};
```

```

294         self(self, y, id);
295     } else if (vis[y] == 1) {
296         vector<int> V = {y}, E = {id};
297         for (int pre = x; pre != y; pre = fa[pre][0]) {
298             V.push_back(pre);
299             E.push_back(fa[pre][1]);
300         }
301
302         int l = V.size();
303         cout << l << "\n";
304         reverse(V.begin(), V.end());
305         reverse(E.begin(), E.end());
306         rotate(E.begin(), E.begin() + 1, E.end());
307         for (int i = 0; i < l; i++) {
308             cout << V[i] << " \n"[i == l - 1];
309         }
310         for (int i = 0; i < l; i++) {
311             cout << E[i] << " \n"[i == l - 1];
312         }
313         exit(0);
314     }
315 }
316 vis[x] = 2;
317 };
318 for (int i = 1; i <= n; i++) {
319     if (!vis[i]) {
320         dfs(dfs, i, -1);
321     }
322 }
323
324 cout << -1 << "\n";
325 }
326
327 signed main() {
328     int n, m;
329     cin >> n >> m;
330
331     vector<vector<array<int, 2>>> ver(n + 1);
332     for (int i = 1; i <= m; i++) {
333         int x, y;
334         cin >> x >> y;
335         ver[x].push_back({y, i});
336         ver[y].push_back({x, i});
337     }
338
339     vector<int> vis(n + 1);
340     vector<array<int, 2>> fa(n + 1);
341     auto dfs = [&](auto self, int x, int from) -> void {
342         vis[x] = 1;
343         for (auto [y, id] : ver[x]) {
344             if (id == from) continue;
345             if (!vis[y]) {
346                 fa[y] = {x, id};

```

```

347         self(self, y, id);
348     } else if (vis[y] == 1) {
349         vector<int> ans1 = {y}, ans2 = {id};
350         for (int pre = x; pre != y; pre = fa[pre][0]) {
351             ans1.push_back(pre);
352             ans2.push_back(fa[pre][1]);
353         }
354
355         int l = ans1.size();
356         cout << l << "\n";
357         for (int i = 0; i < l; i++) {
358             cout << ans1[i] << " \n"[i == l - 1];
359         }
360         for (int i = 0; i < l; i++) {
361             cout << ans2[i] << " \n"[i == l - 1];
362         }
363         exit(0);
364     }
365 }
366 vis[x] = 2;
367 };
368 for (int i = 1; i <= n; i++) {
369     if (!vis[i]) {
370         dfs(dfs, i, -1);
371     }
372 }
373
374 cout << -1 << "\n";
375 }
376
377 signed main() {
378     int n, m;
379     cin >> n >> m;
380     vector<pair<int, int>> in(m);
381     for (int i = 0, x, y; i < m; i++) {
382         cin >> x >> y;
383         in[i] = minmax(x, y);
384     }
385     TwoSat sat(m);
386     for (int i = 0; i < m; i++) {
387         auto [s, e] = in[i];
388         for (int j = i + 1; j < m; j++) {
389             auto [S, E] = in[j];
390             if (s < S && S < e && e < E || S < s && s < E && E < e) {
391                 sat.add(i, 0, j, 0);
392                 sat.add(i, 1, j, 1);
393             }
394         }
395     }
396     if (!sat.work()) {
397         cout << "Impossible\n";
398         return 0;
399     }

```



```

400     auto ans = sat.answer();
401     for (auto it : ans) {
402         cout << (it ? "out" : "in") << " ";
403     }
404 }

```

1.2.11 常见概念

```

1  ### 常见概念
2
3  > oriented graph: 有向图
4  >
5  > bidirectional edges: 双向边
6
7  平面图: 若能将无向图  $G=(V,E)$  画在平面上使得任意两条无重合顶点的边不相交, 则称  $G$  是平面图。
8
9  无向正权图上某一点的偏心距: 记为  $\text{ecc}(u) = \max \big\{ \text{dist}(u, v) \big\}$ , 即以这个点为源, 到其他点的**所有最短路的最大值**。如下图  $A$  点,  $\text{ecc}(A)$  即为  $12$ 。
10
11 图的直径: 定义为  $d = \max \big\{ \text{ecc}(u) \big\}$ , 即**最大的偏心距**, 亦可以简化为图中最远的一对点的距离。
12
13 图的中心: 定义为  $\text{arg}=\min \big\{ \text{ecc}(u) \big\}$ , 即**偏心距最小的点**。如下图, 图的中心即为  $B$  点。
14
15 图的绝对中心: 可以定义在边上的图的中心。
16
17 图的半径: 图的半径不同于圆的半径, 其不等于直径的一半 (但对于绝对中心定义上的直径而言是一半)。定义为  $r = \min \big\{ \text{ecc}(u) \big\}$ , 即**中心的偏心距**。计算方式: 使用全源最短路, 计算出所有点的偏心距, 再加以计算。
18
19 

```

1.2.12 常见结论

```

1  ### 常见结论
2
3  1. 要在有向图上求一个最大点集, 使得任意两个点  $(i,j)$  之间至少存在一条路径 (可以从  $i$  到  $j$ , 也可以反过来, 这两种有一个就行), **即求解最长路** ;
4  2. 要求出连通图上的任意一棵生成树, 只需要跑一遍 **bfs** ;
5  3. 给出一棵树, 要求添加尽可能多的边, 使得其是二分图: 对树进行二分染色, 显然, 相同颜色的点之间连边不会破坏二分图的性质, 故可添加的最多的边数即为  $\text{cnt}_{\text{Black}} \cdot \text{cnt}_{\text{White}} - (n-1)$  ;
6  4. 当一棵树可以被黑白染色时, 所有染黑节点的度之和等于所有染白节点的度之和;
7  5. 在竞赛图中, 入度小的点, 必定能到达出度小 (入度大) 的点 [See](https://codeforces.com/contest/1498/problem/E) 。
8  6. 在竞赛图中, 将所有点按入度从小到大排序, 随后依次遍历, 若对于某一点  $i$  满足前  $i$  个点的入度之和恰好等于  $\left\lfloor \frac{n \cdot (n+1)}{2} \right\rfloor$ , 那么对于上一次满足这一条件的点  $p$ ,  $p+1$  到  $i$  点构成一个新的强连通分量 [See](https://codeforces.com/contest/1498/problem/E) 。

```

- 9 > 举例说明, 设满足上方条件的点为 $p_1, p_2 \ (p_1+1 < p_2)$, 那么点 p_1 到 p_1 构成一个强连通分量、点 p_1+1 到 p_2 构成一个强连通分量。
- 10 7. 选择图中最少数量的边删除, 使得图不连通, 即求最小割; 如果是删除点, 那么拆点后求最小割 [See](<https://www.luogu.com.cn/problem/P1345>)。
- 11 8. 如果一张图是**平面图**, 那么其边数一定小于等于 $3n-6$ [See](P3209)。
- 12 9. 若一张有向完全图存在环, 则一定存在三元环。
- 13 10. 竞赛图三元环计数: [See](<https://ac.nowcoder.com/acm/contest/84244/F>)。
- 14 11. 有向图判是否存在环直接用 topsort; 无向图判是否存在环直接用 dsu, 也可以使用 topsort, 条件变为 $\text{deg}[i] \leq 1$ 时入队。

1.2.13 平面图最短路 (对偶图)

对于矩阵图, 建立对偶图的过程如下 (注释部分为建立原图), 其中数据的给出顺序依次为: 各 $n(n+1)$ 个数字分别代表从左向右、从上向下、从右向左、从下向上的边。

```

1 for (int i = 1; i <= n + 1; i++) {
2     for (int j = 1, w; j <= n; j++) {
3         cin >> w;
4         int pre = Hash(i - 1, j), now = Hash(i, j);
5         if (i == 1) {
6             add(s, now, w);
7         } else if (i == n + 1) {
8             add(pre, t, w);
9         } else {
10            add(pre, now, w);
11        }
12        // flow.add(Hash(i, j), Hash(i, j + 1), w);
13    }
14 }
15 for (int i = 1; i <= n; i++) {
16     for (int j = 1, w; j <= n + 1; j++) {
17         cin >> w;
18         int now = Hash(i, j), net = Hash(i, j - 1);
19         if (j == 1) {
20             add(now, t, w);
21         } else if (j == n + 1) {
22             add(s, net, w);
23         } else {
24             add(now, net, w);
25        }
26        // flow.add(Hash(i, j), Hash(i + 1, j), w);
27    }
28 }
29 for (int i = 1; i <= n + 1; i++) {
30     for (int j = 1, w; j <= n; j++) {
31         cin >> w;
32         int now = Hash(i, j), net = Hash(i - 1, j);
33         if (i == 1) {
34             add(now, s, w);
35         } else if (i == n + 1) {
36             add(t, net, w);
37         } else {

```

```

38         add(now, net, w);
39     }
40     // flow.add(Hash(i, j), Hash(i, j - 1), w);
41 }
42 }
43 for (int i = 1; i <= n; i++) {
44     for (int j = 1, w; j <= n + 1; j++) {
45         cin >> w;
46         int pre = Hash(i, j - 1), now = Hash(i, j);
47         if (j == 1) {
48             add(t, now, w);
49         } else if (j == n + 1) {
50             add(pre, s, w);
51         } else {
52             add(pre, now, w);
53         }
54         // flow.add(Hash(i, j), Hash(i - 1, j), w);
55     }
56 }

```

1.2.14 最小生成树 (MST 问题)

(稀疏图) Prim 算法

使用邻接矩阵存图，以 $\mathcal{O}(N^2 + M)$ 的复杂度计算，思想与 `dijkstra` 基本一致。

(稠密图) Kruskal 算法

平均时间复杂度为 $\mathcal{O}(M \log M)$ ，简化了并查集。

```

1  const int N = 550, INF = 0x3f3f3f3f;
2  int n, m, g[N][N];
3  int d[N], v[N];
4  int prim() {
5      ms(d, 0x3f); //这里的d表示到“最小生成树集合”的距离
6      int ans = 0;
7      for (int i = 0; i < n; ++ i) { //遍历 n 轮
8          int t = -1;
9          for (int j = 1; j <= n; ++ j)
10             if (v[j] == 0 && (t == -1 || d[j] < d[t])) //如果这个点不在集合内且当前距离集
                合最近
11                 t = j;
12             v[t] = 1; //将t加入“最小生成树集合”
13             if (i && d[t] == INF) return INF; //如果发现不连通，直接返回
14             if (i) ans += d[t];
15             for (int j = 1; j <= n; ++ j) d[j] = min(d[j], g[t][j]); //用t更新其他点到集合
                的距离
16         }
17         return ans;
18     }
19 int main() {
20     ms(g, 0x3f); cin >> n >> m;
21     while (m -- ) {
22         int x, y, w; cin >> x >> y >> w;

```

```

23     g[x][y] = g[y][x] = min(g[x][y], w);
24 }
25 int t = prim();
26 if (t == INF) cout << "impossible" << endl;
27 else cout << t << endl;
28 } //22.03.19已测试
29
30 struct DSU {
31     vector<int> fa;
32     DSU(int n) : fa(n + 1) {
33         iota(fa.begin(), fa.end(), 0);
34     }
35     int get(int x) {
36         while (x != fa[x]) {
37             x = fa[x] = fa[fa[x]];
38         }
39         return x;
40     }
41     bool merge(int x, int y) { // 设x是y的祖先
42         x = get(x), y = get(y);
43         if (x == y) return false;
44         fa[y] = x;
45         return true;
46     }
47     bool same(int x, int y) {
48         return get(x) == get(y);
49     }
50 };
51 struct Tree {
52     using TII = tuple<int, int, int>;
53     int n;
54     priority_queue<TII, vector<TII>, greater<TII>> ver;
55
56     Tree(int n) {
57         this->n = n;
58     }
59     void add(int x, int y, int w) {
60         ver.emplace(w, x, y); // 注意顺序
61     }
62     int kruskal() {
63         DSU dsu(n);
64         int ans = 0, cnt = 0;
65         while (ver.size()) {
66             auto [w, x, y] = ver.top();
67             ver.pop();
68             if (dsu.same(x, y)) continue;
69             dsu.merge(x, y);
70             ans += w;
71             cnt++;
72         }
73         assert(cnt < n - 1); // 输入有误, 建树失败
74         return ans;
75     }

```

76 };

1.2.15 最短路径树 (SPT 问题)

定义：在一张无向带权联通图中，有这样一棵**生成树**：满足从根节点到任意点的路径都为原图中根到任意点的最短路径。性质：记根节点 $Root$ 到某一结点 x 的最短距离 $dis_{Root,x}$ ，在 SPT 上这两点之间的距离为 $len_{Root,x}$ ——则两者长度相等。

该算法与最小生成树无关，基于最短路 **Dijkstra** 算法完成（但多了个等于号）。下方代码实现的功能为：读入图后，输出以 1 为根的 SPT 所使用的各条边的编号、边权和。

```

1 map<pair<int, int>, int> id;
2 namespace G {
3     vector<pair<int, int> > ver[N];
4     map<pair<int, int>, int> edge;
5     int v[N], d[N], pre[N], vis[N];
6     int ans = 0;
7
8     void add(int x, int y, int w) {
9         ver[x].push_back({y, w});
10        edge[{x, y}] = edge[{y, x}] = w;
11    }
12    void djikstra(int s) { // ! 注意, 该 djikstra 并非原版, 多加了一个等于号
13        priority_queue<PII, vector<PII>, greater<PII> > q; q.push({0, s});
14        memset(d, 0x3f, sizeof d); d[s] = 0;
15        while (!q.empty()) {
16            int x = q.top().second; q.pop();
17            if (v[x]) continue; v[x] = 1;
18            for (auto [y, w] : ver[x]) {
19                if (d[y] >= d[x] + w) { // ! 注意, SPT 这里修改为>=号
20                    d[y] = d[x] + w;
21                    pre[y] = x; // 记录前驱结点
22                    q.push({d[y], y});
23                }
24            }
25        }
26    }
27    void dfs(int x) {
28        vis[x] = 1;
29        for (auto [y, w] : ver[x]) {
30            if (vis[y]) continue;
31            if (pre[y] == x) {
32                cout << id[{x, y}] << " "; // 输出SPT所使用的边编号
33                ans += edge[{x, y}];
34                dfs(y);
35            }
36        }
37    }
38    void solve(int n) {
39        djikstra(1); // 以 1 为根
40        dfs(1); // 以 1 为根

```

```

41     cout << endl << ans; // 输出SPT的边权和
42 }
43 }
44 bool Solve() {
45     int n, m; cin >> n >> m;
46     for (int i = 1; i <= m; ++ i) {
47         int x, y, w; cin >> x >> y >> w;
48         G::add(x, y, w), G::add(y, x, w);
49         id[{x, y}] = id[{y, x}] = i;
50     }
51     G::solve(n);
52     return 0;
53 }

```

1.2.16 最长路 (topsort+DP 算法)

计算一张 DAG 中的最长路径，在执行前可能需要使用 tarjan 重构一张正确的 DAG，复杂度 $\mathcal{O}(N + M)$ 。

```

1 struct DAG {
2     int n;
3     vector<vector<pair<int, int>>> ver;
4     vector<int> deg, dis;
5     DAG(int n) : n(n) {
6         ver.resize(n + 1);
7         deg.resize(n + 1);
8         dis.assign(n + 1, -1E18);
9     }
10    void add(int x, int y, int w) {
11        ver[x].push_back({y, w});
12        ++deg[y];
13    }
14    int topsort(int s, int t) {
15        queue<int> q;
16        for (int i = 1; i <= n; i++) {
17            if (deg[i] == 0) {
18                q.push(i);
19            }
20        }
21        dis[s] = 0;
22        while (!q.empty()) {
23            int x = q.front();
24            q.pop();
25            for (auto [y, w] : ver[x]) {
26                dis[y] = max(dis[y], dis[x] + w);
27                --deg[y];
28                if (deg[y] == 0) {
29                    q.push(y);
30                }
31            }
32        }
33        return dis[t];

```

```

34     }
35 };
36
37 signed main() {
38     int n, m;
39     cin >> n >> m;
40     DAG dag(n);
41     for (int i = 1; i <= m; i++) {
42         int x, y, w;
43         cin >> x >> y >> w;
44         dag.add(x, y, w);
45     }
46
47     int s, t;
48     cin >> s >> t;
49     cout << dag.topsort(s, t) << "\n";
50 }

```

1.2.17 欧拉路径/欧拉回路 Hierholzers

欧拉路径：一笔画完图中全部边，画的顺序就是一个可行解；当起点终点相同时称欧拉回路。

有向图欧拉路径存在判定

有向图欧拉路径存在：¹ 恰有一个点出度比入度多 1（为起点）；² 恰有一个点入度比出度多 1（为终点）；³ 恰有 $N - 2$ 个点入度均等于出度。如果是欧拉回路，则上方起点与终点的条件不存在，全部点均要满足最后一个条件。

无向图欧拉路径存在判定

无向图欧拉路径存在：¹ 恰有两个点度数为奇数（为起点与终点）；² 恰有 $N - 2$ 个点度数为偶数。

有向图欧拉路径求解（字典序最小）

无向图欧拉路径求解

```

1 signed main() {
2     int n, m;
3     cin >> n >> m;
4
5     DSU dsu(n + 1); // 如果保证连通，则不需要 DSU
6     vector<unordered_multiset<int>> ver(n + 1); // 如果对于字典序有要求，则不能使用
7         unordered
8     vector<int> degI(n + 1), degO(n + 1);
9     for (int i = 1; i <= m; i++) {
10         int x, y;
11         cin >> x >> y;
12         ver[x].insert(y);
13         degI[y]++;
14         degO[x]++;
15         dsu.merge(x, y); // 直接当无向图
16     }
17 }

```

```

16     int s = 1, t = 1, cnt = 0;
17     for (int i = 1; i <= n; i++) {
18         if (degI[i] == degO[i]) {
19             cnt++;
20         } else if (degI[i] + 1 == degO[i]) {
21             s = i;
22         } else if (degI[i] == degO[i] + 1) {
23             t = i;
24         }
25     }
26     if (dsu.size(1) != n || (cnt != n - 2 && cnt != n)) {
27         cout << "No\n";
28     } else {
29         cout << "Yes\n";
30     }
31 }
32
33 signed main() {
34     int n, m;
35     cin >> n >> m;
36
37     DSU dsu(n + 1); // 如果保证连通, 则不需要 DSU
38     vector<unordered_multiset<int>> ver(n + 1); // 如果对于字典序有要求, 则不能使用
39         unordered
40     vector<int> deg(n + 1);
41     for (int i = 1; i <= m; i++) {
42         int x, y;
43         cin >> x >> y;
44         ver[x].insert(y);
45         ver[y].insert(x);
46         deg[y]++;
47         deg[x]++;
48         dsu.merge(x, y); // 直接当无向图
49     }
50     int s = -1, t = -1, cnt = 0;
51     for (int i = 1; i <= n; i++) {
52         if (deg[i] % 2 == 0) {
53             cnt++;
54         } else if (s == -1) {
55             s = i;
56         } else {
57             t = i;
58         }
59     }
60     if (dsu.size(1) != n || (cnt != n - 2 && cnt != n)) {
61         cout << "No\n";
62     } else {
63         cout << "Yes\n";
64     }
65
66     vector<int> ans;
67     auto dfs = [&](auto self, int x) -> void {

```



```

68     while (ver[x].size()) {
69         int net = *ver[x].begin();
70         ver[x].erase(ver[x].begin());
71         self(self, net);
72     }
73     ans.push_back(x);
74 };
75 dfs(dfs, s);
76 reverse(ans.begin(), ans.end());
77 for (auto it : ans) {
78     cout << it << " ";
79 }
80
81 auto dfs = [&](auto self, int x) -> void {
82     while (ver[x].size()) {
83         int net = *ver[x].begin();
84         ver[x].erase(ver[x].find(net));
85         ver[net].erase(ver[net].find(x));
86         cout << x << " " << net << endl;
87         self(self, net);
88     }
89 };
90 dfs(dfs, s);

```

1.2.18 缩点 (Tarjan 算法)

(有向图) 强连通分量缩点

强连通分量缩点后的图称为 SCC。以 $\mathcal{O}(N + M)$ 的复杂度完成上述全部操作。

性质：缩点后的图拥有拓扑序 $color_{cnt}, color_{cnt-1}, \dots, 1$ ，可以不需再另跑一遍 `topsort`；缩点后的图是一张有向无环图 (DAG、拓扑图)。

(无向图) 割边缩点

割边缩点后的图称为边双连通图 (E-DCC)，该模板可以在 $\mathcal{O}(N + M)$ 复杂度内求解图中全部割边、划分边双（颜色相同的点位于同一个边双连通分量中）。

割边（桥）：将某边 e 删去后，原图分成两个以上不相连的子图，称 e 为图的割边。

边双连通：在一张连通的无向图中，对于两个点 u 和 v ，删去任何一条边（只能删去一条）它们依旧连通，则称 u 和 v 边双连通。一个图如果不存在割边，则它是一个边双连通图。性质补充：对于一个边双，删去任意边后依旧联通；对于边双中的任意两点，一定存在两条不相交的路径连接这两个点（路径上可以有公共点，但是没有公共边）。

(无向图) 割点缩点

割点缩点后的图称为点双连通图 (V-DCC)，该模板可以在 $\mathcal{O}(N + M)$ 复杂度内求解图中全部割点、划分点双（颜色相同的点位于同一个点双连通分量中）。

割点（割顶）：将与某点 i 连接的所有边删去后，原图分成两个以上不相连的子图，称 i 为图的割点。点双连通：在一张连通的无向图中，对于两个点 u 和 v ，删去任

何一个点（只能删去一个，且不能删 u 和 v 自己）它们依旧连通，则称 u 和 v 边双连通。如果一个图不存在割点，那么它是一个点双连通图。性质补充：每一个割点至少属于两个点双。

```

1 struct SCC {
2     int n, now, cnt;
3     vector<vector<int>> ver;
4     vector<int> dfn, low, col, S;
5
6     SCC(int n) : n(n), ver(n + 1), low(n + 1) {
7         dfn.resize(n + 1, -1);
8         col.resize(n + 1, -1);
9         now = cnt = 0;
10    }
11    void add(int x, int y) {
12        ver[x].push_back(y);
13    }
14    void tarjan(int x) {
15        dfn[x] = low[x] = now++;
16        S.push_back(x);
17        for (auto y : ver[x]) {
18            if (dfn[y] == -1) {
19                tarjan(y);
20                low[x] = min(low[x], low[y]);
21            } else if (col[y] == -1) {
22                low[x] = min(low[x], dfn[y]);
23            }
24        }
25        if (dfn[x] == low[x]) {
26            int pre;
27            cnt++;
28            do {
29                pre = S.back();
30                col[pre] = cnt;
31                S.pop_back();
32            } while (pre != x);
33        }
34    }
35    auto work() { // [cnt 新图的顶点数量]
36        for (int i = 1; i <= n; i++) { // 避免图不连通
37            if (dfn[i] == -1) {
38                tarjan(i);
39            }
40        }
41
42        vector<int> siz(cnt + 1); // siz 每个 scc 中点的数量
43        vector<vector<int>> adj(cnt + 1);
44        for (int i = 1; i <= n; i++) {
45            siz[col[i]]++;
46            for (auto j : ver[i]) {
47                int x = col[i], y = col[j];
48                if (x != y) {

```

```

49         adj[x].push_back(y);
50     }
51 }
52 }
53 return {cnt, adj, col, siz};
54 }
55 };
56
57 struct EDCC {
58     int n, m, now, cnt;
59     vector<vector<array<int, 2>>> ver;
60     vector<int> dfn, low, col, S;
61     set<array<int, 2>> bridge, direct; // 如果不需要, 删除这一部分可以得到一些时间上的优化
62
63     EDCC(int n) : n(n), low(n + 1), ver(n + 1), dfn(n + 1), col(n + 1) {
64         m = now = cnt = 0;
65     }
66     void add(int x, int y) { // 和 scc 相比多了一条连边
67         ver[x].push_back({y, m});
68         ver[y].push_back({x, m++});
69     }
70     void tarjan(int x, int fa) {
71         dfn[x] = low[x] = ++now;
72         S.push_back(x);
73         for (auto &[y, id] : ver[x]) {
74             if (!dfn[y]) {
75                 direct.insert({x, y});
76                 tarjan(y, id);
77                 low[x] = min(low[x], low[y]);
78                 if (dfn[x] < low[y]) {
79                     bridge.insert({x, y});
80                 }
81             } else if (id != fa && dfn[y] < dfn[x]) {
82                 direct.insert({x, y});
83                 low[x] = min(low[x], dfn[y]);
84             }
85         }
86         if (dfn[x] == low[x]) {
87             int pre;
88             cnt++;
89             do {
90                 pre = S.back();
91                 col[pre] = cnt;
92                 S.pop_back();
93             } while (pre != x);
94         }
95     }
96     auto work() {
97         for (int i = 1; i <= n; i++) { // 避免图不连通
98             if (!dfn[i]) {
99                 tarjan(i, 0);
100             }

```

```

101     }
102     /**
103      * @param cnt 新图的顶点数量, adj 新图, col 旧图节点对应的新图节点
104      * @param siz 旧图每一个边双中点的数量
105      * @param bridge 全部割边, direct 非割边定向
106      */
107     vector<int> siz(cnt + 1);
108     vector<vector<int>> adj(cnt + 1);
109     for (int i = 1; i <= n; i++) {
110         siz[col[i]]++;
111         for (auto &[j, id] : ver[i]) {
112             int x = col[i], y = col[j];
113             if (x != y) {
114                 adj[x].push_back(y);
115             }
116         }
117     }
118     return tuple{cnt, adj, col, siz};
119 }
120 };
121
122 struct V_DCC {
123     int n;
124     vector<vector<int>> ver, col;
125     vector<int> dfn, low, S;
126     int now, cnt;
127     vector<bool> point; // 记录是否为割点
128
129     V_DCC(int n) : n(n) {
130         ver.resize(n + 1);
131         dfn.resize(n + 1);
132         low.resize(n + 1);
133         col.resize(2 * n + 1);
134         point.resize(n + 1);
135         S.clear();
136         cnt = now = 0;
137     }
138     void add(int x, int y) {
139         if (x == y) return; // 手动去除重边
140         ver[x].push_back(y);
141         ver[y].push_back(x);
142     }
143     void tarjan(int x, int root) {
144         low[x] = dfn[x] = ++now;
145         S.push_back(x);
146         if (x == root && !ver[x].size()) { // 特判孤立点
147             ++cnt;
148             col[cnt].push_back(x);
149             return;
150         }
151
152         int flag = 0;
153         for (auto y : ver[x]) {

```

```

154         if (!dfn[y]) {
155             tarjan(y, root);
156             low[x] = min(low[x], low[y]);
157             if (dfn[x] <= low[y]) {
158                 flag++;
159                 if (x != root || flag > 1) {
160                     point[x] = true; // 标记为割点
161                 }
162                 int pre = 0;
163                 cnt++;
164                 do {
165                     pre = S.back();
166                     col[cnt].push_back(pre);
167                     S.pop_back();
168                 } while (pre != y);
169                 col[cnt].push_back(x);
170             }
171         } else {
172             low[x] = min(low[x], dfn[y]);
173         }
174     }
175 }
176 pair<int, vector<vector<int>>> rebuild() { // 【新图的顶点数量，新图】
177     work();
178     vector<vector<int>> adj(cnt + 1);
179     for (int i = 1; i <= cnt; i++) {
180         if (!col[i].size()) { // 注意，孤立点也是 V-DCC
181             continue;
182         }
183         for (auto j : col[i]) {
184             if (point[j]) { // 如果 j 是割点
185                 adj[i].push_back(point[j]);
186                 adj[point[j]].push_back(i);
187             }
188         }
189     }
190     return {cnt, adj};
191 }
192 void work() {
193     for (int i = 1; i <= n; ++i) { // 避免图不连通
194         if (!dfn[i]) {
195             tarjan(i, i);
196         }
197     }
198 }
199 };

```

1.2.19 链式前向星建图与搜索

很少使用这种建图法。**dfs**：标准复杂度为 $\mathcal{O}(N + M)$ 。节点子节点的数量包含它自己（至少为 1），深度从 0 开始（根节点深度为 0）。**bfs**：深度从 1 开始（根节点深度为 1）。**topsort**：有向无环图（包括非联通）才拥有完整的拓扑序列（故该算法也可用于判断图中是否存在环）。

每次找到入度为 0 的点并将其放入待查找队列。

```

1 namespace Graph {
2     const int N = 1e5 + 7;
3     const int M = 1e6 + 7;
4     int tot, h[N], ver[M], ne[M];
5     int deg[N], vis[M];
6
7     void clear(int n) {
8         tot = 0; //多组样例清空
9         for (int i = 1; i <= n; ++i) {
10             h[i] = 0;
11             deg[i] = vis[i] = 0;
12         }
13     }
14     void add(int x, int y) {
15         ver[++tot] = y, ne[tot] = h[x], h[x] = tot;
16         ++deg[y];
17     }
18     void dfs(int x) {
19         a.push_back(x); // DFS序
20         siz[x] = vis[x] = 1;
21         for (int i = h[x]; i; i = ne[i]) {
22             int y = ver[i];
23             if (vis[y]) continue;
24             dis[y] = dis[x] + 1;
25             dfs(y);
26             siz[x] += siz[y];
27         }
28         a.push_back(x);
29     }
30     void bfs(int s) {
31         queue<int> q;
32         q.push(s);
33         dis[s] = 1;
34         while (!q.empty()) {
35             int x = q.front();
36             q.pop();
37             for (int i = h[x]; i; i = ne[i]) {
38                 int y = ver[i];
39                 if (dis[y]) continue;
40                 d[y] = d[x] + 1;
41                 q.push(y);
42             }
43         }
44     }
45     bool topsort() {
46         queue<int> q;
47         vector<int> ans;
48         for (int i = 1; i <= n; ++i)
49             if (deg[i] == 0) q.push(i);
50         while (!q.empty()) {
51             int x = q.front();

```

```

52     q.pop();
53     ans.push_back(x);
54     for (int i = h[x]; i; i = ne[i]) {
55         int y = ver[i];
56         --deg[y];
57         if (deg[y] == 0) q.push(y);
58     }
59 }
60 return ans.size() == n; //判断是否存在拓扑排序
61 }
62 } // namespace Graph

```

1.3 匹配

1.3.1 二分图匹配

```

1  #include <iostream>
2  #include <vector>
3  #include <cstring>
4  using namespace std;
5
6  const int N = 505;
7  vector<int> g[N]; // 邻接表
8  int match[N]; // 右侧点匹配的左侧点编号
9  bool vis[N]; // 标记右侧点是否访问过
10
11 // 尝试从左侧点u匹配
12 bool dfs(int u) {
13     for (int v : g[u]) {
14         if (!vis[v]) {
15             vis[v] = true;
16             if (match[v] == 0 || dfs(match[v])) {
17                 match[v] = u;
18                 return true;
19             }
20         }
21     }
22     return false;
23 }
24
25 int main() {
26     int n, m, e;
27     cin >> n >> m >> e; // 左侧点数, 右侧点数, 边数
28
29     while (e--) {
30         int u, v;
31         cin >> u >> v;
32         g[u].push_back(v);
33     }
34
35     int res = 0;
36     for (int u = 1; u <= n; ++u) {

```

```

37     memset(vis, 0, sizeof(vis));
38     if (dfs(u)) res++;
39 }
40
41 cout << res << endl; // 最大匹配数
42 return 0;
43 }

```

1.3.2 一般图最大匹配（带花树算法）

与二分图匹配的差别在于图中可能存在奇环，时间复杂度与边的数量无关，为 $\mathcal{O}(N^3)$ 。下方模板编号从 0 开始，例题为 UOJ #79. 一般图最大匹配。

```

1 struct Graph {
2     int n;
3     vector<vector<int>>> e;
4     Graph(int n) : n(n), e(n) {}
5     void add(int u, int v) {
6         e[u].push_back(v);
7         e[v].push_back(u);
8     }
9     pair<int, vector<int>> work() {
10         vector<int> match(n, -1), vis(n), link(n), f(n), dep(n);
11         auto find = [&](int u) {
12             while (f[u] != u) u = f[u] = f[f[u]];
13             return u;
14         };
15         auto lca = [&](int u, int v) {
16             u = find(u), v = find(v);
17             while (u != v) {
18                 if (dep[u] < dep[v]) swap(u, v);
19                 u = find(link[match[u]]);
20             }
21             return u;
22         };
23         queue<int> q;
24         auto blossom = [&](int u, int v, int p) {
25             while (find(u) != p) {
26                 link[u] = v;
27                 v = match[u];
28                 if (vis[v] == 0) {
29                     vis[v] = 1;
30                     q.push(v);
31                 }
32                 f[u] = f[v] = p;
33                 u = link[v];
34             }
35         };
36         auto augment = [&](int u) {
37             while (!q.empty()) q.pop();
38             iota(f.begin(), f.end(), 0);
39             fill(vis.begin(), vis.end(), -1);

```



```

40     q.push(u);
41     vis[u] = 1;
42     dep[u] = 0;
43     while (!q.empty()) {
44         int u = q.front();
45         q.pop();
46         for (auto v : e[u]) {
47             if (vis[v] == -1) {
48                 vis[v] = 0;
49                 link[v] = u;
50                 dep[v] = dep[u] + 1;
51                 if (match[v] == -1) {
52                     for (int x = v, y = u, temp; y != -1;
53                         x = temp, y = x == -1 ? -1 : link[x]) {
54                         temp = match[y];
55                         match[x] = y;
56                         match[y] = x;
57                     }
58                     return;
59                 }
60                 vis[match[v]] = 1;
61                 dep[match[v]] = dep[u] + 2;
62                 q.push(match[v]);
63             } else if (vis[v] == 1 && find(v) != find(u)) {
64                 int p = lca(u, v);
65                 blossom(u, v, p);
66                 blossom(v, u, p);
67             }
68         }
69     }
70 };
71 auto greedy = [&]() {
72     for (int u = 0; u < n; ++u) {
73         if (match[u] != -1) continue;
74         for (auto v : e[u]) {
75             if (match[v] == -1) {
76                 match[u] = v;
77                 match[v] = u;
78                 break;
79             }
80         }
81     }
82 };
83 greedy();
84 for (int u = 0; u < n; u++) {
85     if (match[u] == -1) {
86         augment(u);
87     }
88 }
89 int ans = 0;
90 for (int u = 0; u < n; u++) {
91     if (match[u] != -1) {
92         ans++;

```

```

93     }
94     }
95     return {ans / 2, match};
96 }
97 };
98
99 signed main() {
100     int n, m;
101     cin >> n >> m;
102
103     Graph graph(n);
104     for (int i = 1; i <= m; i++) {
105         int x, y;
106         cin >> x >> y;
107         graph.add(x - 1, y - 1);
108     }
109     auto [ans, match] = graph.work();
110     cout << ans << endl;
111     for (auto it : match) {
112         cout << it + 1 << " ";
113     }
114 }

```

1.3.3 一般图最大权匹配 (带权带花树算法)

下方模板编号从 1 开始, 复杂度为 $\mathcal{O}(N^3)$ 。

```

1 namespace Graph {
2     const int N = 403 * 2; //两倍点数
3     typedef int T; //权值大小
4     const T inf = numeric_limits<int>::max() >> 1;
5     struct Q { int u, v; T w; } e[N][N];
6     T lab[N];
7     int n, m = 0, id, h, t, lk[N], sl[N], st[N], f[N], b[N][N], s[N], ed[N], q[N];
8     vector<int> p[N];
9     #define dvd(x) (lab[x.u] + lab[x.v] - e[x.u][x.v].w * 2)
10    #define FOR(i, b) for (int i = 1; i <= (int)(b); i++)
11    #define ALL(x) (x).begin(), (x).end()
12    #define ms(x, i) memset(x + 1, i, m * sizeof x[0])
13    void upd(int u, int v) {
14        if (!sl[v] || dvd(e[u][v]) < dvd(e[sl[v]][v])) {
15            sl[v] = u;
16        }
17    }
18    void ss(int v) {
19        sl[v] = 0;
20        FOR(u, n) {
21            if (e[u][v].w > 0 && st[u] != v && !s[st[u]]) {
22                upd(u, v);
23            }
24        }
25    }
26    void ins(int u) {

```

```

27     if (u <= n) { q[++t] = u; }
28     else {
29         for (int v : p[u]) ins(v);
30     }
31 }
32 void mdf(int u, int w) {
33     st[u] = w;
34     if (u > n) {
35         for (int v : p[u]) mdf(v, w);
36     }
37 }
38 int gr(int u, int v) {
39     v = find(ALL(p[u]), v) - p[u].begin();
40     if (v & 1) {
41         reverse(1 + ALL(p[u]));
42         return (int)p[u].size() - v;
43     }
44     return v;
45 }
46 void stm(int u, int v) {
47     lk[u] = e[u][v].v;
48     if (u <= n) return;
49     Q w = e[u][v];
50     int x = b[u][w.u], y = gr(u, x);
51     for (int i = 0; i < y; i++) {
52         stm(p[u][i], p[u][i ^ 1]);
53     }
54     stm(x, v);
55     rotate(p[u].begin(), y + ALL(p[u]));
56 }
57 void aug(int u, int v) {
58     int w = st[lk[u]];
59     stm(u, v);
60     if (!w) return;
61     stm(w, st[f[w]]), aug(st[f[w]], w);
62 }
63 int lca(int u, int v) {
64     for (++id; u | v; swap(u, v)) {
65         if (!u) continue;
66         if (ed[u] == id) return u;
67         ed[u] = id;
68         if (u = st[lk[u]]) u = st[f[u]];
69     }
70     return 0;
71 }
72 void add(int u, int a, int v) {
73     int x = n + 1, i, j;
74     while (x <= m && st[x]) ++x;
75     if (x > m) ++m;
76     lab[x] = s[x] = st[x] = 0;
77     lk[x] = lk[a];
78     p[x].clear();
79     p[x].push_back(a);

```

```

80     for (i = u; i != a; i = st[f[j]]) {
81         p[x].push_back(i);
82         p[x].push_back(j = st[lk[i]]);
83         ins(j);
84     }
85     reverse(1 + ALL(p[x]));
86     for (i = v; i != a; i = st[f[j]]) { // 复制, 只需改循环
87         p[x].push_back(i);
88         p[x].push_back(j = st[lk[i]]);
89         ins(j);
90     }
91     mdf(x, x);
92     FOR(i, m) {
93         e[x][i].w = e[i][x].w = 0;
94     }
95     memset(b[x] + 1, 0, n * sizeof b[0][0]);
96     for (int u : p[x]) {
97         FOR(v, m) {
98             if (!e[x][v].w || dvd(e[u][v]) < dvd(e[x][v])) {
99                 e[x][v] = e[u][v], e[v][x] = e[v][u];
100             }
101         }
102         FOR(v, n) {
103             if (b[u][v]) { b[x][v] = u; }
104         }
105     }
106     ss(x);
107 }
108 void ex(int u) {
109     for (int x : p[u]) mdf(x, x);
110     int a = b[u][e[u][f[u]].u], r = gr(u, a);
111     for (int i = 0; i < r; i += 2) {
112         int x = p[u][i], y = p[u][i + 1];
113         f[x] = e[y][x].u;
114         s[x] = 1;
115         s[y] = sl[x] = 0;
116         ss(y), ins(y);
117     }
118     s[a] = 1, f[a] = f[u];
119     for (int i = r + 1; i < p[u].size(); i++) {
120         s[p[u][i]] = -1;
121         ss(p[u][i]);
122     }
123     st[u] = 0;
124 }
125 bool on(const Q &e) {
126     int u = st[e.u], v = st[e.v];
127     if (s[v] == -1) {
128         f[v] = e.u, s[v] = 1;
129         int a = st[lk[v]];
130         sl[v] = sl[a] = s[a] = 0;
131         ins(a);
132     } else if (!s[v]) {

```

```

133     int a = lca(u, v);
134     if (!a) {
135         return aug(u, v), aug(v, u), 1;
136     } else {
137         add(u, a, v);
138     }
139 }
140 return 0;
141 }
142 bool bfs() {
143     ms(s, -1), ms(sl, 0);
144     h = 1, t = 0;
145     FOR(i, m) {
146         if (st[i] == i && !lk[i]) {
147             f[i] = s[i] = 0;
148             ins(i);
149         }
150     }
151     if (h > t) return 0;
152     while (1) {
153         while (h <= t) {
154             int u = q[h++];
155             if (s[st[u]] == 1) continue;
156             FOR(v, n) {
157                 if (e[u][v].w > 0 && st[u] != st[v]) {
158                     if (dvd(e[u][v])) upd(u, st[v]);
159                     else if (on(e[u][v])) return 1;
160                 }
161             }
162         }
163         T x = inf;
164         for (int i = n + 1; i <= m; i++) {
165             if (st[i] == i && s[i] == 1) {
166                 x = min(x, lab[i] >> 1);
167             }
168         }
169         FOR(i, m) {
170             if (st[i] == i && sl[i] && s[i] != 1) {
171                 x = min(x, dvd(e[sl[i]][i]) >> s[i] + 1);
172             }
173         }
174         FOR(i, n) {
175             if (~s[st[i]]) {
176                 if ((lab[i] += (s[st[i]] * 2 - 1) * x) <= 0) return 0;
177             }
178         }
179         for (int i = n + 1; i <= m; i++) {
180             if (st[i] == i && ~s[st[i]]) {
181                 lab[i] += (2 - s[st[i]] * 4) * x;
182             }
183         }
184         h = 1, t = 0;
185         FOR(i, m) {

```

```

186         if (st[i] == i && sl[i] && st[sl[i]] != i && !dvd(e[sl[i]][i]) && on(e[
            sl[i]][i])) {
187             return 1;
188         }
189     }
190     for (int i = n + 1; i <= m; i++) {
191         if (st[i] == i && s[i] == 1 && !lab[i]) ex(i);
192     }
193 }
194 return 0;
195 }
196 template<typename TT> i64 work(int N, const vector<tuple<int, int, TT>> &edges)
    {
197     ms(ed, 0), ms(lk, 0);
198     n = m = N; id = 0;
199     iota(st + 1, st + n + 1, 1);
200     T wm = 0; i64 r = 0;
201     FOR(i, n) FOR(j, n) {
202         e[i][j] = {i, j, 0};
203     }
204     for (auto [u, v, w] : edges) {
205         wm = max(wm, e[v][u].w = e[u][v].w = max(e[u][v].w, (T)w));
206     }
207     FOR(i, n) { p[i].clear(); }
208     FOR(i, n) FOR(j, n) {
209         b[i][j] = i * (i == j);
210     }
211     fill_n(lab + 1, n, wm);
212     while (bfs()) {};
213     FOR(i, n) if (lk[i]) {
214         r += e[i][lk[i]].w;
215     }
216     return r / 2;
217 }
218 auto match() {
219     vector<array<int, 2>> ans;
220     FOR(i, n) if (lk[i]) {
221         ans.push_back({i, lk[i]});
222     }
223     return ans;
224 }
225 } // namespace Graph
226 using Graph::work, Graph::match;
227
228 signed main() {
229     int n, m;
230     cin >> n >> m;
231     vector<tuple<int, int, i64>> ver(m);
232     for (auto &[u, v, w] : ver) {
233         cin >> u >> v >> w;
234     }
235     cout << work(n, ver) << "\n";
236     auto ans = match();

```

237 }

1.3.4 二分图最大匹配

二分图：一个图能被分为左右两部分，任何一条边的两个端点都不在同一部分中。匹配（独立边集）：一个边的集合，这些边没有公共顶点。二分图最大匹配即找到边的数量最多的那个匹配。一般我们规定，左半部包含 n_1 个点（编号 $1 - n_1$ ），右半部包含 n_2 个点（编号 $1 - n_2$ ），保证任意一条边的两个端点都不可能都在同一部分中。

匈牙利算法（KM 算法）解

$O(NM)$ 。

HopcroftKarp 算法（基于最大流）解

该算法基于最大流，常数极小，且引入随机化，几乎卡不掉。最坏时间复杂度为 $O(\sqrt{NM})$ ，经测试，在 N, M 均为 2×10^5 的情况下能在 60ms 内跑完。

```

1 signed main() {
2     int n1, n2, m;
3     cin >> n1 >> n2 >> m;
4
5     vector<vector<int>> ver(n1 + 1);
6     for (int i = 1; i <= m; ++i) {
7         int x, y;
8         cin >> x >> y;
9         ver[x].push_back(y); //只需要建立单向边
10    }
11
12    int ans = 0;
13    vector<int> match(n2 + 1);
14    for (int i = 1; i <= n1; ++i) {
15        vector<int> vis(n2 + 1);
16        auto dfs = [&](auto self, int x) -> bool {
17            for (auto y : ver[x]) {
18                if (vis[y]) continue;
19                vis[y] = 1;
20                if (!match[y] || self(self, match[y])) {
21                    match[y] = x;
22                    return true;
23                }
24            }
25            return false;
26        };
27        if (dfs(dfs, i)) {
28            ans++;
29        }
30    }
31    cout << ans << endl;
32 }
33
34 struct HopcroftKarp {
35     int n, m;

```

```

36     vector<array<int, 2>> ver;
37     vector<int> l, r;
38
39     HopcroftKarp(int n, int m) : n(n), m(m) { // 左右半部
40         l.assign(n, -1);
41         r.assign(m, -1);
42     }
43     void add(int x, int y) {
44         x--, y--; // 这个板子是 0-idx 的
45         ver.push_back({x, y});
46     }
47     int work() {
48         vector<int> adj(ver.size());
49
50         mt19937 rgen(chrono::steady_clock::now().time_since_epoch().count());
51         shuffle(ver.begin(), ver.end(), rgen); // 随机化防卡
52
53         vector<int> deg(n + 1);
54         for (auto &[u, v] : ver) {
55             deg[u]++;
56         }
57         for (int i = 1; i <= n; i++) {
58             deg[i] += deg[i - 1];
59         }
60         for (auto &[u, v] : ver) {
61             adj[--deg[u]] = v;
62         }
63
64         int ans = 0;
65         vector<int> a, p, q(n);
66         while (true) {
67             a.assign(n, -1), p.assign(n, -1);
68
69             int t = 0;
70             for (int i = 0; i < n; i++) {
71                 if (l[i] == -1) {
72                     q[t++] = a[i] = p[i] = i;
73                 }
74             }
75
76             bool match = false;
77             for (int i = 0; i < t; i++) {
78                 int x = q[i];
79                 if (~l[a[x]]) continue;
80
81                 for (int j = deg[x]; j < deg[x + 1]; j++) {
82                     int y = adj[j];
83                     if (r[y] == -1) {
84                         while (~y) {
85                             r[y] = x;
86                             swap(l[x], y);
87                             x = p[x];
88                         }

```



```

89         match = true;
90         ++ans;
91         break;
92     }
93     if (p[r[y]] == -1) {
94         q[t++] = y = r[y];
95         p[y] = x;
96         a[y] = a[x];
97     }
98     }
99     }
100     if (!match) break;
101 }
102 return ans;
103 }
104 vector<array<int, 2>> answer() {
105     vector<array<int, 2>> ans;
106     for (int i = 0; i < n; i++) {
107         if (~l[i]) {
108             ans.push_back({i, l[i]});
109         }
110     }
111     return ans;
112 }
113 };
114
115 signed main() {
116     int n1, n2, m;
117     cin >> n1 >> n2 >> m;
118     HopcroftKarp flow(n1, n2);
119     while (m--) {
120         int x, y;
121         cin >> x >> y;
122         flow.add(x, y);
123     }
124
125     cout << flow.work() << "\n";
126
127     auto match = flow.answer();
128     for (auto [u, v] : match) {
129         cout << u << " " << v << "\n";
130     }
131 }

```

1.3.5 二分图最大权匹配 (二分图完美匹配)

定义：找到边权和最大的那个匹配。一般我们规定，左半部包含 n_1 个点（编号 $1 - n_1$ ），右半部包含 n_2 个点（编号 $1 - n_2$ ）。

使用匈牙利算法（KM 算法）解，时间复杂度为 $\mathcal{O}(N^3)$ 。下方模板用于求解最大权值、且可以输出其中一种可行方案，例题为 UOJ #80. 二分图最大权匹配。

```

1 struct MaxCostMatch {
2     vector<int> ans1, ansr, pre;
3     vector<int> lx, ly;
4     vector<vector<int>> ver;
5     int n;
6
7     MaxCostMatch(int n) : n(n) {
8         ver.resize(n + 1, vector<int>(n + 1));
9         ans1.resize(n + 1, -1);
10        ansr.resize(n + 1, -1);
11        lx.resize(n + 1);
12        ly.resize(n + 1, -1E18);
13        pre.resize(n + 1);
14    }
15    void add(int x, int y, int w) {
16        ver[x][y] = w;
17    }
18    void bfs(int x) {
19        vector<bool> visl(n + 1), visr(n + 1);
20        vector<int> slack(n + 1, 1E18);
21        queue<int> q;
22        function<bool(int)> check = [&](int x) {
23            visr[x] = 1;
24            if (~ansr[x]) {
25                q.push(ansr[x]);
26                visl[ansr[x]] = 1;
27                return false;
28            }
29            while (~x) {
30                ansr[x] = pre[x];
31                swap(x, ans1[pre[x]]);
32            }
33            return true;
34        };
35        q.push(x);
36        visl[x] = 1;
37        while (1) {
38            while (!q.empty()) {
39                int x = q.front();
40                q.pop();
41                for (int y = 1; y <= n; ++y) {
42                    if (visr[y]) continue;
43                    int del = lx[x] + ly[y] - ver[x][y];
44                    if (del < slack[y]) {
45                        pre[y] = x;
46                        slack[y] = del;
47                        if (!slack[y] && check(y)) return;
48                    }
49                }
50            }
51            int val = 1E18;
52            for (int i = 1; i <= n; ++i) {

```

```

53         if (!visr[i]) {
54             val = min(val, slack[i]);
55         }
56     }
57     for (int i = 1; i <= n; ++i) {
58         if (visl[i]) lx[i] -= val;
59         if (visr[i]) {
60             ly[i] += val;
61         } else {
62             slack[i] -= val;
63         }
64     }
65     for (int i = 1; i <= n; ++i) {
66         if (!visr[i] && !slack[i] && check(i)) {
67             return;
68         }
69     }
70 }
71 }
72 int work() {
73     for (int i = 1; i <= n; ++i) {
74         for (int j = 1; j <= n; ++j) {
75             ly[i] = max(ly[i], ver[j][i]);
76         }
77     }
78     for (int i = 1; i <= n; ++i) bfs(i);
79     int res = 0;
80     for (int i = 1; i <= n; ++i) {
81         res += ver[i][ansl[i]];
82     }
83     return res;
84 }
85 void getMatch(int x, int y) { // 获取方案 (0代表无匹配)
86     for (int i = 1; i <= x; ++i) {
87         cout << (ver[i][ansl[i]] ? ansl[i] : 0) << " ";
88     }
89     cout << endl;
90     for (int i = 1; i <= y; ++i) {
91         cout << (ver[i][ansr[i]] ? ansr[i] : 0) << " ";
92     }
93     cout << endl;
94 }
95 };
96
97 signed main() {
98     int n1, n2, m;
99     cin >> n1 >> n2 >> m;
100
101     MaxCostMatch match(max(n1, n2));
102     for (int i = 1; i <= m; i++) {
103         int x, y, w;
104         cin >> x >> y >> w;
105         match.add(x, y, w);

```

```

106     }
107     cout << match.work() << '\n';
108     match.getMatch(n1, n2);
109 }

```

1.3.6 二分图最大独立点集 (Konig 定理)

给出一张二分图，要求选择一些点使得它们两两没有边直接连接。最小点覆盖等价于最大匹配数，转换为最小割模板，答案即为总点数减去最大流得到的值。

```

1 cout << n - flow.work(s, t) << endl;

```

1.3.7 染色法判定二分图 (dfs 算法)

判断一张图能否被二分染色。

```

1 vector<int> vis(n + 1);
2 auto dfs = [&](auto self, int x, int type) -> void {
3     vis[x] = type;
4     for (auto y : ver[x]) {
5         if (vis[y] == type) {
6             cout << "NO\n";
7             exit(0);
8         }
9         if (vis[y]) continue;
10        self(self, y, 3 - type);
11    }
12 };
13 for (int i = 1; i <= n; ++i) {
14     if (vis[i]) {
15         dfs(dfs, i, 1);
16     }
17 }
18 cout << "Yes\n";

```

1.4 网络流

1.4.1 网络流

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int maxn=50005;
4 const int maxm=500005;
5 const int inf=0x3f3f3f3f;
6 struct Node{
7     int to,val,next;
8 }q[maxm<<1];
9 int head[maxn],cnt=0,dep[maxn],cur[maxn],vis[maxn];
10 int sp,ep,maxflow;
11 void init(){
12     memset(head,-1,sizeof(head));

```

```
13     cnt=2,maxflow=0;
14 }
15 void addedge(int from,int to,int val){
16     q[cnt].to=to;
17     q[cnt].val=val;
18     q[cnt].next=head[from];
19     head[from]=cnt++;
20 }
21 void add_edge(int from,int to,int val){
22     addedge(from,to,val);
23     addedge(to,from,0);
24 }
25 bool bfs(int n){
26     for(int i=0;i<=n;i++){
27         cur[i]=head[i],dep[i]=0x3f3f3f3f;
28         vis[i]=0;
29     }
30     dep[sp]=0;
31     queue<int>que;
32     que.push(sp);
33     while(!que.empty()){
34         int x=que.front();
35         que.pop();
36         vis[x]=0;
37         for(int i=head[x];i!=-1;i=q[i].next){
38             int to=q[i].to;
39             if(dep[to]>dep[x]+1&&q[i].val){
40                 dep[to]=dep[x]+1;
41                 if(!vis[to]){
42                     que.push(to);
43                     vis[to]=1;
44                 }
45             }
46         }
47     }
48     if(dep[ep]!=inf) return true;
49     else return false;
50 }
51 int dfs(int x,int flow){
52     int rlow=0;
53     if(x==ep){
54         maxflow+=flow;
55         return flow;
56     }
57     int used=0;
58     for(int i=cur[x];i!=-1;i=q[i].next){
59         cur[x]=i;
60         int to=q[i].to;
61         if(q[i].val&&dep[to]==dep[x]+1){
62             if(rlow=dfs(to,min(flow-used,q[i].val))){
63                 used+=rlow;
64                 q[i].val-=rlow;
65                 q[i^1].val+=rlow;
```

```

66         if(used==flow) break;
67     }
68 }
69 }
70 return used;
71 }
72 int dinic(int n){
73     while(bfs(n)){
74         dfs(sp,inf);
75     }
76     return maxflow;
77 }
78 int main()
79 {
80     int n,m;
81     scanf("%d%d%d",&n,&m,&sp,&ep);
82     register int i;
83     int u,v,val;
84     init();
85     for(i=1;i<=m;i++){
86         scanf("%d%d%d",&u,&v,&val);
87         add_edge(u,v,val);
88     }
89     printf("%d",dinic(n));
90     return 0;
91 }

```

1.4.2 无源汇点的最小割问题 Stoer–Wagner

也称为全局最小割。定义补充（与《网络流》中的定义不同）：**割**：是一个边集，去掉其中所有边能使一张网络流图不再连通（即分成两个子图）。

通过**递归**的方式来解决**无向正权图**上的全局最小割问题，算法复杂度 $\mathcal{O}(VE + V^2 \log V)$ ，一般可近似看作 $\mathcal{O}(V^3)$ 。

```

1  signed main() {
2      int n, m;
3      cin >> n >> m;
4
5      DSU dsu(n); // 这里引入DSU判断图是否联通，如题目有保证，则不需要此步骤
6      vector<vector<int>> edge(n + 1, vector<int>(n + 1));
7      for (int i = 1; i <= m; i++) {
8          int x, y, w;
9          cin >> x >> y >> w;
10         dsu.merge(x, y);
11         edge[x][y] += w;
12         edge[y][x] += w;
13     }
14
15     if (dsu.Poi(1) != n || m < n - 1) { // 图不联通
16         cout << 0 << endl;
17         return 0;

```

```

18     }
19
20     int MinCut = INF, S = 1, T = 1; // 虚拟源汇点
21     vector<int> bin(n + 1);
22     auto contract = [&]() -> int { // 求解S到T的最小割, 定义为 cut of phase
23         vector<int> dis(n + 1), vis(n + 1);
24         int Min = 0;
25         for (int i = 1; i <= n; i++) {
26             int k = -1, maxc = -1;
27             for (int j = 1; j <= n; j++) {
28                 if (!bin[j] && !vis[j] && dis[j] > maxc) {
29                     k = j;
30                     maxc = dis[j];
31                 }
32             }
33             if (k == -1) return Min;
34             S = T, T = k, Min = maxc;
35             vis[k] = 1;
36             for (int j = 1; j <= n; j++) {
37                 if (!bin[j] && !vis[j]) {
38                     dis[j] += edge[k][j];
39                 }
40             }
41         }
42         return Min;
43     };
44     for (int i = 1; i < n; i++) { // 这里取不到等号
45         int val = contract();
46         bin[T] = 1;
47         MinCut = min(MinCut, val);
48         if (!MinCut) {
49             cout << 0 << endl;
50             return 0;
51         }
52         for (int j = 1; j <= n; j++) {
53             if (!bin[j]) {
54                 edge[S][j] += edge[j][T];
55                 edge[j][S] += edge[j][T];
56             }
57         }
58     }
59     cout << MinCut << endl;
60 }

```

1.4.3 最大流

使用 Dinic 算法, 理论最坏复杂度为 $\mathcal{O}(N^2M)$, 例题范围: $N = 1200$, $m = 5 \times 10^3$ 。一般步骤: BFS 建立分层图, 无回溯 DFS 寻找所有可行的增广路径。封装: 求从点 S 到点 T 的最大流。预流推进见 == 常数优化章节 ==。

```

1 template<typename T> struct Flow_ {
2     const int n;

```

```

3   const T inf = numeric_limits<T>::max();
4   struct Edge {
5       int to;
6       T w;
7       Edge(int to, T w) : to(to), w(w) {}
8   };
9   vector<Edge> ver;
10  vector<vector<int>>> h;
11  vector<int> cur, d;
12
13  Flow_(int n) : n(n + 1), h(n + 1) {}
14  void add(int u, int v, T c) {
15      h[u].push_back(ver.size());
16      ver.emplace_back(v, c);
17      h[v].push_back(ver.size());
18      ver.emplace_back(u, 0);
19  }
20  bool bfs(int s, int t) {
21      d.assign(n, -1);
22      d[s] = 0;
23      queue<int> q;
24      q.push(s);
25      while (!q.empty()) {
26          auto x = q.front();
27          q.pop();
28          for (auto it : h[x]) {
29              auto [y, w] = ver[it];
30              if (w && d[y] == -1) {
31                  d[y] = d[x] + 1;
32                  if (y == t) return true;
33                  q.push(y);
34              }
35          }
36      }
37      return false;
38  }
39  T dfs(int u, int t, T f) {
40      if (u == t) return f;
41      auto r = f;
42      for (int &i = cur[u]; i < h[u].size(); i++) {
43          auto j = h[u][i];
44          auto &[v, c] = ver[j];
45          auto &[u, rc] = ver[j ^ 1];
46          if (c && d[v] == d[u] + 1) {
47              auto a = dfs(v, t, std::min(r, c));
48              c -= a;
49              rc += a;
50              r -= a;
51              if (!r) return f;
52          }
53      }
54      return f - r;
55  }

```



```

56     T work(int s, int t) {
57         T ans = 0;
58         while (bfs(s, t)) {
59             cur.assign(n, 0);
60             ans += dfs(s, t, inf);
61         }
62         return ans;
63     }
64 };
65 using Flow = Flow_<int>;

```

1.4.4 最小割

基础模型：构筑二分图，左半部 n 个点代表盈利项目，右半部 m 个点代表材料成本，收益为盈利之和减去成本之和，求最大收益。

建图：建立源点 S 向左半部连边，建立汇点 T 向右半部连边，如果某个项目需要某个材料，则新增一条容量 $+\infty$ 的跨部边。

割边：放弃某个项目则断开 S 至该项目的边，购买某个原料则断开该原料至 T 的边，最终的图一定不存在从 S 到 T 的路径，此时我们得到二分图的一个 $S-T$ 割。此时最小割即为求解最大流，边权之和减去最大流即为最大收益。

```

1  signed main() {
2      int n, m;
3      cin >> n >> m;
4
5      int S = n + m + 1, T = n + m + 2;
6      Flow flow(T);
7      for (int i = 1; i <= n; i++) {
8          int w;
9          cin >> w;
10         flow.add(S, i, w);
11     }
12
13     int sum = 0;
14     for (int i = 1; i <= m; i++) {
15         int x, y, w;
16         cin >> x >> y >> w;
17         flow.add(x, n + i, 1E18);
18         flow.add(y, n + i, 1E18);
19         flow.add(n + i, T, w);
20         sum += w;
21     }
22     cout << sum - flow.work(S, T) << endl;
23 }

```

1.4.5 最小割树 Gomory-Hu Tree

无向连通图抽象出的一棵树，满足任意两点间的距离是他们的最小割。一共需要跑 n 轮最小割，总复杂度 $\mathcal{O}(N^3M)$ ，预处理最小割树上任意两点的距离 $\mathcal{O}(N^2)$ 。

过程：分治 n 轮，每一轮在图上随机选点，跑一轮最小割后连接树边；这一网络的残留网络会将剩余的点分为两组，根据分组分治。

```

1 void reset() { // struct需要额外封装退流
2     for (int i = 0; i < ver.size(); i += 2) {
3         ver[i].w += ver[i ^ 1].w;
4         ver[i ^ 1].w = 0;
5     }
6 }
7
8 signed main() { // Gomory-Hu Tree
9     int n, m;
10    cin >> n >> m;
11
12    Flow<int> flow(n);
13    for (int i = 1; i <= m; i++) {
14        int u, v, w;
15        cin >> u >> v >> w;
16        flow.add(u, v, w);
17        flow.add(v, u, w);
18    }
19
20    vector<int> vis(n + 1), fa(n + 1);
21    vector ans(n + 1, vector<int>(n + 1, 1E9)); // N^2 枚举出全部答案
22    vector<vector<pair<int, int>>> adj(n + 1);
23    for (int i = 1; i <= n; i++) { // 分治 n 轮
24        int s = 0; // 本质是在树上随机选点、跑最小割后连边
25        for (; s <= n; s++) {
26            if (fa[s] != s) break;
27        }
28        int t = fa[s];
29
30        int ans = flow.work(s, t); // 残留网络将点集分为两组，分治
31        adj[s].push_back({t, ans});
32        adj[t].push_back({s, ans});
33
34        vis.assign(n + 1, 0);
35        auto dfs = [&](auto dfs, int u) -> void {
36            vis[u] = 1;
37            for (auto it : flow.h[u]) {
38                auto [v, c] = flow.ver[it];
39                if (c && !vis[v]) {
40                    dfs(dfs, v);
41                }
42            }
43        };
44        dfs(dfs, s);
45        for (int j = 0; j <= n; j++) {
46            if (vis[j] && fa[j] == t) {
47                fa[j] = s;
48            }
49        }
50    }

```

```

51
52     for (int i = 0; i <= n; i++) {
53         auto dfs = [&](auto dfs, int u, int fa, int c) -> void {
54             ans[i][u] = c;
55             for (auto [v, w] : adj[u]) {
56                 if (v == fa) continue;
57                 dfs(dfs, v, u, min(c, w));
58             }
59         };
60         dfs(dfs, i, -1, 1E9);
61     }
62
63     int q;
64     cin >> q;
65     while (q--) {
66         int u, v;
67         cin >> u >> v;
68         cout << ans[u][v] << "\n"; // 预处理答案数组
69     }
70 }

```

1.4.6 网络流 | 最大流 | 预流推进 HLPP

理论最坏复杂度为 $\mathcal{O}(N^2\sqrt{M})$ ，例题范围： $N = 1200$, $m = 1.2 \times 10^5$ 。

```

1  template <typename T> struct PushRelabel {
2      const int inf = 0x3f3f3f3f;
3      const T INF = 0x3f3f3f3f3f3f3f3f;
4      struct Edge {
5          int to, cap, flow, anti;
6          Edge(int v = 0, int w = 0, int id = 0) : to(v), cap(w), flow(0), anti(id) {}
7      };
8      vector<vector<Edge>> e;
9      vector<vector<int>> gap;
10     vector<T> ex; // 超额流
11     vector<bool> ingap;
12     vector<int> h;
13     int n, gobalcnt, maxH = 0;
14     T maxflow = 0;
15
16     PushRelabel(int n) : n(n), e(n + 1), ex(n + 1), gap(n + 1) {}
17     void addedge(int u, int v, int w) {
18         e[u].push_back({v, w, (int)e[v].size()});
19         e[v].push_back({u, 0, (int)e[u].size() - 1});
20     }
21     void PushEdge(int u, Edge &edge) {
22         int v = edge.to, d = min(ex[u], 1LL * edge.cap - edge.flow);
23         ex[u] -= d;
24         ex[v] += d;
25         edge.flow += d;
26         e[v][edge.anti].flow -= d;
27         if (h[v] != inf && d > 0 && ex[v] == d && !ingap[v]) {
28             ++gobalcnt;

```

```

29         gap[h[v]].push_back(v);
30         ingap[v] = 1;
31     }
32 }
33 void PushPoint(int u) {
34     for (auto k = e[u].begin(); k != e[u].end(); k++) {
35         if (h[k->to] + 1 == h[u] && k->cap > k->flow) {
36             PushEdge(u, *k);
37             if (!ex[u]) break;
38         }
39     }
40     if (!ex[u]) return;
41     if (gap[h[u]].empty()) {
42         for (int i = h[u] + 1; i <= min(maxH, n); i++) {
43             for (auto v : gap[i]) {
44                 ingap[v] = 0;
45             }
46             gap[i].clear();
47         }
48     }
49     h[u] = inf;
50     for (auto [to, cap, flow, anti] : e[u]) {
51         if (cap > flow) {
52             h[u] = min(h[u], h[to] + 1);
53         }
54     }
55     if (h[u] >= n) return;
56     maxH = max(maxH, h[u]);
57     if (!ingap[u]) {
58         gap[h[u]].push_back(u);
59         ingap[u] = 1;
60     }
61 }
62 void init(int t, bool f = 1) {
63     ingap.assign(n + 1, 0);
64     for (int i = 1; i <= maxH; i++) {
65         gap[i].clear();
66     }
67     gobalcnt = 0, maxH = 0;
68     queue<int> q;
69     h.assign(n + 1, inf);
70     h[t] = 0, q.push(t);
71     while (q.size()) {
72         int u = q.front();
73         q.pop(), maxH = h[u];
74         for (auto &[v, cap, flow, anti] : e[u]) {
75             if (h[v] == inf && e[v][anti].cap > e[v][anti].flow) {
76                 h[v] = h[u] + 1;
77                 q.push(v);
78                 if (f) {
79                     gap[h[v]].push_back(v);
80                     ingap[v] = 1;
81                 }

```

```

82         }
83     }
84 }
85 }
86 T work(int s, int t) {
87     init(t, 0);
88     if (h[s] == inf) return maxflow;
89     h[s] = n;
90     ex[s] = INF;
91     ex[t] = -INF;
92     for (auto k = e[s].begin(); k != e[s].end(); k++) {
93         PushEdge(s, *k);
94     }
95     while (maxH > 0) {
96         if (gap[maxH].empty()) {
97             maxH--;
98             continue;
99         }
100         int u = gap[maxH].back();
101         gap[maxH].pop_back();
102         ingap[u] = 0;
103         PushPoint(u);
104         if (gobalcnt >= 10 * n) {
105             init(t);
106         }
107     }
108     ex[s] -= INF;
109     ex[t] += INF;
110     return maxflow = ex[t];
111 }
112 };

```

1.4.7 费用流

给定一个带费用的网络，规定 (u, v) 间的费用为 $f(u, v) \times w(u, v)$ ，求解该网络中总花费最小的最大流称之为**最小费用最大流**。总时间复杂度为 $\mathcal{O}(NMf)$ ，其中 f 代表最大流。

数论

```

1 struct MinCostFlow {
2     using LL = long long;
3     using PII = pair<LL, int>;
4     const LL INF = numeric_limits<LL>::max();
5     struct Edge {
6         int v, c, f;
7         Edge(int v, int c, int f) : v(v), c(c), f(f) {}
8     };
9     const int n;
10    vector<Edge> e;
11    vector<vector<int>>> g;
12    vector<LL> h, dis;
13    vector<int> pre;
14 }

```

```

15 MinCostFlow(int n) : n(n), g(n) {}
16 void add(int u, int v, int c, int f) { // c 流量, f 费用
17     // if (f < 0) {
18     //     g[u].push_back(e.size());
19     //     e.emplace_back(v, 0, f);
20     //     g[v].push_back(e.size());
21     //     e.emplace_back(u, c, -f);
22     // } else {
23     g[u].push_back(e.size());
24     e.emplace_back(v, c, f);
25     g[v].push_back(e.size());
26     e.emplace_back(u, 0, -f);
27     // }
28 }
29 bool dijkstra(int s, int t) {
30     dis.assign(n, INF);
31     pre.assign(n, -1);
32     priority_queue<PII, vector<PII>, greater<PII>> que;
33     dis[s] = 0;
34     que.emplace(0, s);
35     while (!que.empty()) {
36         auto [d, u] = que.top();
37         que.pop();
38         if (dis[u] < d) continue;
39         for (int i : g[u]) {
40             auto [v, c, f] = e[i];
41             if (c > 0 && dis[v] > d + h[u] - h[v] + f) {
42                 dis[v] = d + h[u] - h[v] + f;
43                 pre[v] = i;
44                 que.emplace(dis[v], v);
45             }
46         }
47     }
48     return dis[t] != INF;
49 }
50 pair<int, LL> flow(int s, int t) {
51     int flow = 0;
52     LL cost = 0;
53     h.assign(n, 0);
54     while (dijkstra(s, t)) {
55         for (int i = 0; i < n; ++i) h[i] += dis[i];
56         int aug = numeric_limits<int>::max();
57         for (int i = t; i != s; i = e[pre[i] ^ 1].v) aug = min(aug, e[pre[i]].c);
58         for (int i = t; i != s; i = e[pre[i] ^ 1].v) {
59             e[pre[i]].c -= aug;
60             e[pre[i] ^ 1].c += aug;
61         }
62         flow += aug;
63         cost += LL(aug) * h[t];
64     }
65     return {flow, cost};
66 }
67 };

```

2 数学

2.1 数论

2.1.1 数论 frequently use

```

1 // -----
2 // 不要高精。。。可以发现，a和b只有对mod的余数才会有用。直接一路滚过去取模即可。
3 // -----
4 int m = 19260817;
5 int x=0,y=0;
6 for(int i=0;i<a1.size();i++)x=((x*10)%m+a1[i]-'0')%m;
7 for(int i=0;i<b1.size();i++)y=((y*10)%m+b1[i]-'0')%m;
8
9 // -----
10 // 数论分块
11 // -----
12 for (ll l=2,r; l<=n; l=r+1){
13     ll t = n / l; r = n / t;
14     //do sth
15 }
16
17 // -----
18 // 简短预处理spf、mu
19 // -----
20 const int N = 1000000;
21 vector<int> spf(N+1);
22 spf[1] = 1;
23 for(int i = 2; i <= N; i++){if(spf[i] == 0){
24     for(int j = i; j <= N; j += i)if(spf[j] == 0)
25         spf[j] = i;
26 }
27
28 mu[1] = 1;
29 for(int i = 1; i <= N; ++i) {
30     for(int j = i + i; j <= N; j += i) {
31         mu[j] -= mu[i];
32     }
33 }
34
35 // -----
36 // 枚因子倍数形式  $O(N \log N)$  卷积
37 // -----
38 //枚因子形式
39 for(int d = 1;d<=n;d++){
40     for(int j=1;j<=n/d;j++){
41         h[d*j]+=f[d]*g[j]
42     }
43 }
44 //枚因子形式：同上面完全等价，常数略优
45 int N = Amax;
46 vector<ll> h(N+1), f(N+1), g(N+1);
47 for (int d = 1; d <= N; ++d) {

```

```

48     for (int m = d; m <= N; m += d) { 按因子 d 枚举它的倍数 m
49         h[m] += f[d] * g[m/d];
50     }
51 }
52
53
54 //倍数形式
55 vector<int> h(Amax+1, 0);
56 for(int d = 1; d <= Amax; d++){
57     for(int j = d; j <= Amax; j += d){
58         h[d]+=f[j/d]*g[j];
59     }
60 }

```

2.1.2 灵神数学模板

```

1  # 数学算法核心内容与C++模板代码
2  ## 一、数论
3  ### 1.1 判断质数
4  - 功能：判断一个整数是否为质数（1不是质数）。
5  - 时间复杂度：O(sqrt(n))。
6  ```cpp
7  bool is_prime(int n) {
8      for (int i = 2; i * i <= n; ++i) {
9          if (n % i == 0) {
10             return false;
11         }
12     }
13     return n >= 2;
14 }
15 ```
16
17 ---
18
19 ### 1.2 预处理质数（埃氏筛）
20 - 功能：预处理指定范围内（MX）的所有质数，标记每个数是否为质数并存储质数列表。
21 - 时间复杂度：O(MX * log log MX)。
22 ```cpp
23 const int MX = 1000001;
24 bool is_prime[MX];
25 vector<int> primes;
26
27 void sieve() {
28     memset(is_prime, false, sizeof(is_prime));
29     is_prime[0] = is_prime[1] = false;
30     for (int i = 2; i < MX; ++i) {
31         is_prime[i] = true;
32     }
33     for (int i = 2; i < MX; ++i) {
34         if (is_prime[i]) {
35             primes.push_back(i);
36             if ((long long)i * i < MX) {

```



```

37         for (int j = i * i; j < MX; j += i) {
38             is_prime[j] = false;
39         }
40     }
41 }
42 }
43 }
44 ```
45
46 ---
47
48 ### 1.3 质因数分解
49 #### 模板一：预处理每个数的所有不同质因子
50 - 功能：预处理指定范围内 (MX) 每个数的所有不同质因子，原理同埃氏筛。
51 ```cpp
52 const int MX = 1000001;
53 vector<int> prime_factors[MX];
54
55 void preprocess_prime_factors() {
56     for (int i = 2; i < MX; ++i) {
57         if (prime_factors[i].empty()) { // i是质数
58             for (int j = i; j < MX; j += i) {
59                 prime_factors[j].push_back(i);
60             }
61         }
62     }
63 }
64 ```
65
66 #### 模板二：通过最小质因子 (LPF) 快速分解
67 - 功能：先预处理每个数的最小质因子，再通过该数组快速分解任意数的质因子（含每个质因子的个数）。
68 - 分解时间复杂度： $O(\log x)$ 。
69 ```cpp
70 const int MX = 1000001;
71 int lpf[MX];
72
73 void preprocess_lpf() {
74     memset(lpf, 0, sizeof(lpf));
75     for (int i = 2; i < MX; ++i) {
76         if (lpf[i] == 0) { // i是质数
77             for (int j = i; j < MX; j += i) {
78                 if (lpf[j] == 0) {
79                     lpf[j] = i;
80                 }
81             }
82         }
83     }
84 }
85
86 vector<pair<int, int>> prime_factorization(int x) {
87     vector<pair<int, int>> res;
88     while (x > 1) {

```

```

89     int p = lpf[x];
90     int e = 1;
91     x /= p;
92     while (x % p == 0) {
93         e++;
94         x /= p;
95     }
96     res.emplace_back(p, e);
97 }
98 return res;
99 }
100 ...
101
102 ---
103
104 ### 1.5 因子预处理
105 - 功能：预处理指定范围内 (MX) 每个数的所有因子。
106 ```cpp
107 const int MX = 1000001; // 根据题目调整
108 vector<int> divisors[MX];
109
110 void preprocess_divisors() {
111     for (int i = 1; i < MX; ++i) {
112         for (int j = i; j < MX; j += i) {
113             divisors[j].push_back(i);
114         }
115     }
116 }
117 ...
118
119 ---
120
121 ### 1.6 最大公约数 (GCD) 与最小公倍数 (LCM)
122 - 功能：计算两个整数的最大公约数和最小公倍数，LCM采用先除后乘避免溢出。
123 ```cpp
124 long long gcd(long long a, long long b) {
125     while (a != 0) {
126         long long tmp = a;
127         a = b % a;
128         b = tmp;
129     }
130     return b;
131 }
132
133 long long lcm(long long a, long long b) {
134     return a / gcd(a, b) * b;
135 }
136 ...
137
138 ---
139
140 ## 七、杂项
141 ### 7.1 回文数生成

```

```

142 - 功能：按从小到大的顺序生成所有回文数，包括奇数长度和偶数长度。
143 ```cpp
144 #include <iostream>
145 #include <string>
146 #include <iterator>
147 using namespace std;
148
149 struct PalindromeGenerator {
150     int base;
151     PalindromeGenerator() : base(1) {}
152
153     int operator()() {
154         int res;
155         // 生成奇数长度回文数
156         string s = to_string(base);
157         string rev_s = s;
158         reverse(rev_s.begin(), rev_s.end());
159         string odd_pali = s + rev_s.substr(1);
160         res = stoi(odd_pali);
161
162         // 检查是否需要切换到下一个base（生成完当前base的奇数和偶数长度后）
163         static bool generated_even = false;
164         if (generated_even) {
165             base *= 10;
166             generated_even = false;
167         } else {
168             generated_even = true;
169         }
170
171         // 若当前生成的是偶数长度，覆盖res
172         if (generated_even) {
173             string even_pali = s + rev_s;
174             res = stoi(even_pali);
175         }
176
177         return res;
178     }
179 };
180
181 // 使用示例：
182 // PalindromeGenerator gen;
183 // for (int x = gen(); x <= 1e9; x = gen()) {
184 //     cout << x << endl;
185 // }
186 ```
187
188 ---
189
190 要不要我帮你整理一份**按难度分级的C++模板调用示例**，包含每个算法的典型使用场景和输入输出样例？

```

2.1.3 线性求逆元

```

1  vector<int> inv(n+10);
2  inv[1] = 1;
3  for(int i=2;i<=n;i++){ //从2开始, 不然全0
4      inv[i]=(1ll)(p-p/i)*inv[p%i]%p;
5  }

```

2.1.4 唯一分解

```

1  // -----
2  // 试除法 sqrt(n) n≤1e12
3  // -----
4  #include <bits/stdc++.h>
5  using namespace std;
6
7  // 单次试除分解, 返回 (质因子, 指数) 列表
8  vector<pair<long long,int>> factor_trial(long long n) {
9      vector<pair<long long,int>> res;
10     for (long long p = 2; p * p <= n; ++p) {
11         if (n % p == 0) {
12             int cnt = 0;
13             while (n % p == 0) {
14                 n /= p;
15                 ++cnt;
16             }
17             res.emplace_back(p, cnt);
18         }
19     }
20     if (n > 1) res.emplace_back(n, 1);
21     return res;
22 }
23
24 int main() {
25     long long n;
26     cin >> n;
27     auto fac = factor_trial(n);
28     for (auto [p, c] : fac) {
29         cout << p << (c>1?("^(c):"):"") << " ";
30     }
31     cout << "\n";
32     return 0;
33 }
34 // -----
35 // 常数优化 快6倍
36 // -----
37 vector<pair<long long,int>> factor6k1(long long n) {
38     vector<pair<long long,int>> res;
39     // 2 的幂
40     if (n % 2 == 0) {
41         int c = 0;
42         while (n % 2 == 0) { n /= 2; ++c; }
43         res.emplace_back(2, c);
44     }

```

```

45 // 3 的幂
46 if (n % 3 == 0) {
47     int c = 0;
48     while (n % 3 == 0) { n /= 3; ++c; }
49     res.emplace_back(3, c);
50 }
51 // 6k±1 轮子
52 for (long long p = 5; p * p <= n; p += 6) {
53     // 检查 6k-1
54     if (n % p == 0) {
55         int c = 0;
56         while (n % p == 0) { n /= p; ++c; }
57         res.emplace_back(p, c);
58     }
59     // 检查 6k+1
60     if (n % (p + 2) == 0) {
61         int c = 0;
62         while (n % (p + 2) == 0) { n /= (p + 2); ++c; }
63         res.emplace_back(p + 2, c);
64     }
65 }
66 if (n > 1) res.emplace_back(n, 1);
67 return res;
68 }
69
70
71 // -----
72 // 多次分解且上界确定 n<=1e7: 先做SPF预处理, 之后每次O(logN)返回。
73 // -----
74 // 用 spf 数组快速分解
75 vector<pair<int,int>> factor_spf(int x) {
76     vector<pair<int,int>> res;
77     while (x > 1) {
78         int p = spf[x], cnt = 0;
79         while (x % p == 0) {
80             x /= p;
81             ++cnt;
82         }
83         res.emplace_back(p, cnt);
84     }
85     return res;
86 }
87
88 int main() {
89     init_spf();
90     int q;
91     cin >> q; // 查询次数
92     while (q--) {
93         int x;
94         cin >> x;
95         for (auto [p, c] : factor_spf(x)) {
96             cout << p << (c>1?("^\n"+to_string(c)):"") << " ";
97         }

```

```

98     cout << "\n";
99 }
100 return 0;
101 }
102
103 // -----
104 // 预处理区间唯一分解  $O(n)$  空间复杂度  $N\log N$ 
105 // -----
106 // 最大值, 可根据需要调整
107 const int N = 10000000;
108
109 int spf[N+1]; // spf[i] = i 的最小质因子
110 vector<vector<pair<int,int>>> factors(N+1);
111
112 void linear_sieve_with_factors() {
113     vector<int> primes;
114     spf[1] = 1;
115     factors[1] = {}; // 1 没有质因数
116
117     for (int i = 2; i <= N; ++i) {
118         if (spf[i] == 0) {
119             // i 是新质数
120             spf[i] = i;
121             primes.push_back(i);
122             factors[i].push_back({i, 1});
123         }
124         for (int p : primes) {
125             long long x = 1LL * i * p;
126             if (x > N) break;
127             spf[x] = p;
128             // 构造  $x = i * p$  的分解列表
129             // 先拷贝 i 的因子表
130             auto v = factors[i];
131             if (!v.empty() && v.back().first == p) {
132                 // p 已在末尾, 指数+1
133                 v.back().second++;
134             } else {
135                 // 新质因子 p
136                 v.push_back({p, 1});
137             }
138             factors[x] = move(v);
139
140             if (p == spf[i]) {
141                 // 保证每个合成数只被最小质因子“标定”一次
142                 break;
143             }
144         }
145     }
146 }

```

2.1.5 exgcd 相关

```

1  int exgcd(int a,int b,int &x,int &y){
2      if(b==0){x=1;y=0;return a;}
3      int g=exgcd(b,a%b,y,x);
4      y-=a/b*x;
5      return g;
6  }
7
8  // -----
9  // 求单个乘法逆元 O(log min(a, m))
10 // -----
11 int inv(int a, int m) {
12     int x, y;
13     if (exgcd(a, m, x, y) == 1) {
14         return (x % m + m) % m;
15     }
16     return 0; // 无解返回 0
17 }
18
19 // -----
20 // 二元一次不定方程
21 // -----
22 void sol() {
23     ll a,b,c,x0,y0;
24     cin>>a>>b>>c;
25     ll d = exgcd(a,b,x0,y0);
26     if(c%d!=0){ //不满足裴蜀定理, 直接输出-1
27         cout<<-1<<endl;
28         return;
29     }
30     ll x1 = x0*c/d , y1 = y0*c/d;
31     ll delta_x = b/d ,delta_y = a/d;
32
33     ll xmin = (x1%delta_x+delta_x)%delta_x;
34     if(xmin==0) xmin+=delta_x;
35     ll ymin = (y1%delta_y+delta_y)%delta_y;
36     if(ymin==0) ymin+=delta_y;
37
38     //// t* (b/d) = (x-xmin)
39     // cout<<delta_x<<" "<<delta_y<<"*"<<endl;
40     ll t1 = (x1-xmin)/delta_x;
41     ll ymax = y1+t1*delta_y;
42     if(ymax>0){
43         ll t2 = (y1-ymin)/delta_y;
44         ll xmax = x1+t2*delta_x;
45         ll num = (xmax-xmin)/delta_x+1;
46         cout<<num<<" "<<xmin<<" "<<ymin<<" "<<xmax<<" "<<ymax<<endl;
47     }
48     else{ //无一对正整数解
49         cout<<xmin<<" "<<ymin<<endl;
50     }
51 }
52
53 void sol() {

```

```

54     ll n , m , a , b;
55     cin>>n>>m>>a>>b;
56     // 求解  $n+ax == m+by$ 
57     //  $ax+by == m-n$  (这里y求出来的符号就相反了,但保持了 $a>0, b>0$ 就很nice)
58     ll c = m-n;
59     ll x , y;
60     ll g = exgcd(a,b,x,y);
61     if(c%g!=0){cout<<"Impossible"; return;}
62
63     ll x0 = x*c/g ; ll y0 = y*c/g;
64     ll t = b/g;
65     x0 = (x0*t+t)%t;
66     y0 = (c-a*x0)/b;
67     while(y0>0){
68         x0+=t;
69         y0 = (c-a*x0)/b;
70     }
71     cout<<n+a*x0;
72 }

```

2.1.6 线筛——枚举所有数的质数倍

```

1  const int MAXN = ;
2  int prime[MAXN];
3  bool not_prime[MAXN];
4
5  // 生成质数表
6  void get_prime(int n) {
7      int cnt = 0;
8      for (int i = 2; i <= n; i++) {
9          if (!not_prime[i]) prime[cnt++] = i;
10         for (int j = 0; j < cnt && prime[j] <= n / i; j++) {
11             not_prime[prime[j] * i] = true;
12             if (i % prime[j] == 0) break;
13         }
14     }
15 }

```

2.1.7 试除法求所有约数

```

1  vector<int> get_divisors(int x)
2  {
3      vector<int> res;
4      for (int i = 1; i <= x / i; i++)
5          if (x % i == 0)
6              {
7                  res.push_back(i);
8                  if (i != x / i) res.push_back(x / i);
9              }
10     sort(res.begin(), res.end());
11     return res;

```



```
12 }
```

2.1.8 欧拉定理 (单次求 phi)

```
1 // -----
2 // 不用预处理, 单次  $O(\sqrt{n})$  求phi : 跟随唯一分解求
3 // -----
4 int euler_phi(int n) {
5     int m = (int)sqrt(n+0.5);
6     int ans = n;
7     for(int i = 2; i <= m; i++)
8         if(n % i == 0) {
9             ans = ans / i * (i-1); // 先除后乘, 避免类型溢出
10            while(n % i == 0) n /= i;
11        }
12    if(n > 1) ans = ans / n * (n-1); // 最后还有素因子
13    return ans;
14 }
```

2.1.9 miller rabin

```
1 #include<bits/stdc++.h>
2 #define ll long long
3 #define i128 __int128
4 using namespace std;
5 ll n;
6 int primes[19]={2,3,5,7,11,13,17,19,23,29,31,37};
7 ll poww(ll a,ll b,ll c){
8     ll ans=1;i128 base=a;
9     while(b){
10         if(b&1)ans=ans*base%c;
11         base=base*base%c;
12         b>>=1;
13     }
14     return ans;
15 }
16 bool miller_rabin(ll n){
17     for(int i=0;i<=8;i++){
18         if(primes[i]==n)return 1;
19         else if(primes[i]>n)return 0;
20         ll t=poww(primes[i],n-1,n),x=n-1;
21         if(t!=1)return 0;
22         while(t==1&&x%2==0){
23             x/=2;
24             t=poww(primes[i],x,n);
25             if(t!=1&&t!=n-1)return 0;
26         }
27     }
28     return 1;
29 }
30 int main(){
```

```

31 ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
32 while(cin>>n){
33     if(miller_rabin(n))puts("Y");else puts("N");
34 }
35 return 0;
36 }

```

2.1.10 杜教筛

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  using ll = long long;
4
5  namespace Prime {
6      vector<int> spf; vector<int> prime;
7      vector<int> mu; vector<int> phi;
8      void initp(int n) {
9          spf.resize(n + 1, 0);
10         mu.resize(n + 1, 0);
11         phi.resize(n + 1, 0);
12         mu[1] = 1; phi[1] = 1; //1
13         for (int i = 2; i <= n; ++i) {
14             if (!spf[i]) {
15                 spf[i] = i;
16                 prime.push_back(i);
17                 mu[i] = -1; phi[i] = i - 1; //2
18             }
19             for (int j = 0; j < (int)prime.size() && prime[j] <= spf[i] && i * prime[j]
20                 ] <= n; ++j) {
21                 int x = i * prime[j];
22                 spf[x] = prime[j];
23                 if (i % prime[j] == 0) {
24                     mu[x] = 0; phi[x] = phi[i] * prime[j]; //3
25                     break;
26                 } else {
27                     mu[x] = -mu[i]; phi[x] = phi[i] * (prime[j] - 1); //4
28                 }
29             }
30         }
31     }
32 using namespace Prime;
33
34 // —— 杜教筛大 n 部分 ——
35 const int MAXN = 5000000;
36 vector<ll> sumMu, sumPhi;
37 unordered_map<ll, ll> cacheMu, cachePhi;
38
39 // 获取前缀和  $S_{\mu}(n) = \sum_{i=1}^n \mu(i)$ 
40 ll getMu(ll n) {
41     if (n <= MAXN) return sumMu[n];
42     auto it = cacheMu.find(n);

```

```

43     if (it != cacheMu.end()) return it->second;
44
45     ll ans = 1;
46     for (ll l = 2, r; l <= n; l = r + 1) {
47         ll t = n / l;
48         r = n / t;
49         ans -= (r - l + 1) * getMu(t);
50     }
51     return cacheMu[n] = ans;
52 }
53
54 // 获取前缀和  $S_{\phi}(n) = \sum_{i=1}^n \phi(i)$ 
55 ll getPhi(ll n) {
56     if (n <= MAXN) return sumPhi[n];
57     auto it = cachePhi.find(n);
58     if (it != cachePhi.end()) return it->second;
59
60     ll ans = n * (n + 1) / 2;
61     for (ll l = 2, r; l <= n; l = r + 1) {
62         ll t = n / l;
63         r = n / t;
64         ans -= (r - l + 1) * getPhi(t);
65     }
66     return cachePhi[n] = ans;
67 }
68
69 int main() {
70     ios::sync_with_stdio(false);
71     cin.tie(nullptr);
72
73     // 1) 线性筛计算 mu, phi 及它们的前缀和
74     Prime::initp(MAXN);
75     sumMu.resize(MAXN + 1); sumPhi.resize(MAXN + 1);
76     sumMu[0] = 0; sumPhi[0] = 0;
77     for (int i = 1; i <= MAXN; ++i) {
78         sumMu[i] = sumMu[i-1] + mu[i];
79         sumPhi[i] = sumPhi[i-1] + phi[i];
80     }
81
82     // 2) 预分配哈希表大小 (可选, 减少重哈希开销)
83     cacheMu.reserve(1 << 20);
84     cachePhi.reserve(1 << 20);
85
86     int q;
87     cin >> q;
88     while (q--) {
89         ll n; cin >> n;
90         ll ansMu = getMu(n);
91         cout << getPhi(n) << " " << getMu(n) << "\n";
92     }
93     return 0;
94 }

```

2.1.11 gcd

```

1
2 inline int gcd(int a,int b) {
3     return b>0 ? gcd(b,a%b):a;
4 }

```

2.1.12 Min25 筛

求解 $1 - N$ 的质数和, 其中 $N \leq 10^{10}$ 。

```

1 namespace min25{
2     const int N = 1000000 + 10;
3     int prime[N], id1[N], id2[N], flag[N], ncnt, m;
4     LL g[N], sum[N], a[N], T;
5     LL n;
6     LL mod;
7     inline LL ps(LL n,LL k) {LL r=1;for(;;k>>=1){if(k&1)r=r*n%mod;n=n*n%mod;}return
8         r;}
9     void finit(){ // 最开始清0
10         memset(g, 0, sizeof(g));
11         memset(a, 0, sizeof(a));
12         memset(sum, 0, sizeof(sum));
13         memset(prime, 0, sizeof(prime));
14         memset(id1, 0, sizeof(id1));
15         memset(id2, 0, sizeof(id2));
16         memset(flag, 0, sizeof(flag));
17         ncnt = m = 0;
18     }
19     int ID(LL x) {
20         return x <= T ? id1[x] : id2[n / x];
21     }
22     LL calc(LL x) {
23         return x * (x + 1) / 2 - 1;
24     }
25
26     LL init(LL x) {
27         T = sqrt(x + 0.5);
28         for (int i = 2; i <= T; i++) {
29             if (!flag[i]) prime[++ncnt] = i, sum[ncnt] = sum[ncnt - 1] + i;
30             for (int j = 1; j <= ncnt && i * prime[j] <= T; j++) {
31                 flag[i * prime[j]] = 1;
32                 if (i % prime[j] == 0) break;
33             }
34         }
35         for (LL l = 1; l <= x; l = x / (x / l) + 1) {
36             a[++m] = x / l;
37             if (a[m] <= T) id1[a[m]] = m; else id2[x / a[m]] = m;
38             g[m] = calc(a[m]);
39         }
40         for (int i = 1; i <= ncnt; i++)
41             for (int j = 1; j <= m && (LL) prime[i] * prime[i] <= a[j]; j++)

```

```

42         g[j] = g[j] - (LL) prime[i] * (g[ID(a[j]) / prime[i]] - sum[i - 1]);
43     }
44     LL solve(LL x) {
45         if (x <= 1) return x;
46         return n = x, init(n), g[ID(n)];
47     }
48 }
49
50 using namespace min25;
51
52 int main() {
53     // while (1) {
54     int tt;
55     scanf("%d",&tt);
56     while(tt--){
57         finit();
58         scanf("%lld%lld", &n, &mod);
59         LL ans = (n + 3) % mod * n % mod * ps(2, mod - 2) % mod + solve(n + 1) - 4;
60         // cout << solve(n) << endl;
61         // ans = (ans + mod) % mod;
62         ans = (ans + mod) % mod;
63         printf("%lld\n", ans);
64     }
65
66     // }
67 }

```

2.1.13 同余方程组、拓展中国剩余定理 excrt

公式: $x \equiv b_i \pmod{a_i}$, 即 $(x - b_i) \mid a_i$ 。

```

1  int n; LL ai[maxn], bi[maxn];
2  inline int mypow(int n, int k, int p) {
3      int r = 1;
4      for (; k >>= 1, n = n * n % p)
5          if (k & 1) r = r * n % p;
6      return r;
7  }
8  LL exgcd(LL a, LL b, LL &x, LL &y) {
9      if (b == 0) { x = 1, y = 0; return a; }
10     LL gcd = exgcd(b, a % b, x, y), tp = x;
11     x = y, y = tp - a / b * y;
12     return gcd;
13 }
14 LL excrt() {
15     LL x, y, k;
16     LL M = bi[1], ans = ai[1];
17     for (int i = 2; i <= n; ++i) {
18         LL a = M, b = bi[i], c = (ai[i] - ans % b + b) % b;
19         LL gcd = exgcd(a, b, x, y), bg = b / gcd;
20         if (c % gcd != 0) return -1;
21         x = mul(x, c / gcd, bg);
22         ans += x * M;

```

```

23     M *= bg;
24     ans = (ans % M + M) % M;
25 }
26 return (ans % M + M) % M;
27 }
28 int main() {
29     cin >> n;
30     for (int i = 1; i <= n; ++ i) cin >> bi[i] >> ai[i];
31     cout << excrt() << endl;
32     return 0;
33 }

```

2.1.14 基础算法 | 最大公约数 ‘gcd’ | 位运算加速

略快于内置函数。

```

1 LL gcd(LL a, LL b) {
2     #define tz __builtin_ctzll
3     if (!a || !b) return a | b;
4     int t = tz(a | b);
5     a >>= tz(a);
6     while (b) {
7         b >>= tz(b);
8         if (a > b) swap(a, b);
9         b -= a;
10    }
11    return a << t;
12    #undef tz
13 }

```

2.1.15 容斥原理

定义: $|S_1 \cup S_2 \cup S_3 \cup \dots \cup S_n| = \sum_{i=1}^N |S_i| - \sum_{i,j=1}^N |S_i \cap S_j| + \sum_{i,j,k=1}^N |S_i \cap S_j \cap S_k| - \dots$

例题: 给定一个整数 n 和 m 个不同的质数 $p_1, p_2, \dots, p_m$, 请你求出 $1 \sim n$ 中能被 $p_1, p_2, \dots, p_m$ 中的至少一个数整除的整数有多少个。

二进制枚举解

dfs 解

```

1 int main(){
2     ios::sync_with_stdio(false);cin.tie(0);
3     LL n, m;
4     cin >> n >> m;
5     vector <LL> p(m);
6     for (int i = 0; i < m; i ++ )
7         cin >> p[i];
8     LL ans = 0;
9     for (int i = 1; i < (1 << m); i ++ ){
10         LL t = 1, cnt = 0;
11         for (int j = 0; j < m; j ++ ){
12             if (i >> j & 1){

```

```

13         cnt ++ ;
14         t *= p[j];
15         if (t > n){
16             t = -1;
17             break;
18         }
19     }
20 }
21 if (t != -1){
22     if (cnt & 1) ans += n / t;
23     else ans -= n / t;
24 }
25 }
26 cout << ans << "\n";
27 return 0;
28 }
29
30 int main(){
31     ios::sync_with_stdio(false);cin.tie(0);
32     LL n, m;
33     cin >> n >> m;
34     vector <LL> p(m);
35     for (int i = 0; i < m; i ++ )
36         cin >> p[i];
37     LL ans = 0;
38     function<void(LL, LL, LL)> dfs = [&](LL x, LL s, LL odd){
39         if (x == m){
40             if (s == 1) return;
41             ans += odd * (n / s);
42             return;
43         }
44         dfs(x + 1, s, odd);
45         if (s <= n / p[x]) dfs(x + 1, s * p[x], -odd);
46     };
47     dfs(0, 1, -1);
48     cout << ans << "\n";
49     return 0;
50 }

```

2.1.16 常见例题

题意：将 1 至 N 的每个数字分组，使得每一组的数字之和均为质数。输出每一个数字所在的组别，且要求分出的组数最少 See 。

考察哥德巴赫猜想，记全部数字之和为 S ，分类讨论如下：

- 为 S 质数时，只需要分入同一组；
- 当 S 为偶数时，由猜想可知一定能分成两个质数，可以证明其中较小的那个一定小于 N ，暴力枚举分组；
- 当 $S - 2$ 为质数时，特殊判断出答案；

- 其余情况一定能被分成三组，其中 3 单独成组， $S - 3$ 后成为偶数，重复讨论二的过程即可。

题意：给定一个长度为 n 的数组，定义这个数组是 *BAD* 的，当且仅当可以把数组分成两个子序列，这两个子序列的元素之和相等。现在你需要删除**最少**的元素，使得删除后的数组不是 *BAD* 的。

最少删除一个元素——如果原数组存在奇数，则直接删除这个奇数即可；反之，我们发现，对数列同除以一个数不影响计算，故我们只需要找到最大的满足 $2^k \mid a_i$ 成立的 2^k ，随后将全部的 a_i 变为 $\frac{a_i}{2^k}$ ，此时一定有一个奇数（换句话说，我们可以对原数列的每一个元素不断的除以 2 直到出现奇数为止），删除这个奇数即可 See。

题意：设当前有一个数字为 x ，减去、加上最少的数字使得其能被 k 整除。

最少减去 $x \bmod k$ 这个很好想；最少加上 $\left(\left\lceil \frac{x}{k} \right\rceil * k\right) \bmod k$ 也比较好想，但是更简便的方法为加上 $k - x \bmod k$ ，这个式子等价于前面这一坨。

题意：给定一个整数 n ，用恰好 k 个 2 的幂次数之和表示它。例如： $n = 9, k = 4$ ，答案为 $1 + 2 + 2 + 4$ 。

结论 1: k 合法当且仅当 `__builtin_popcountll(n) <= k && k <= n`，显然。

结论 2: $2^{k+1} = 2 \cdot 2^k$ ，所以我们可以将二进制位看作是数组，然后从高位向低位推，一个高位等于两个低位，直到数组之和恰好等于 k ，随后依次输出即可。举例说明， $\{1, 0, 0, 1\} \rightarrow \{0, 2, 0, 1\} \rightarrow \{0, 1, 2, 1\}$ ，即答案为 0 个 2^3 、1 个 2^2 、.....。

题意： n 个取值在 $[0, k)$ 之间的数之和为 m 的方案数

答案为 $\sum_{i=0}^n -1^i \cdot \binom{n}{i} \cdot \binom{m-i \cdot k+n-1}{n-1}$ See1 See2。

¹ 先考虑没有 k 的限制，那么即球盒模型： m 个球放入 n 个盒子，球同、盒子不同、能空。使用隔板法得到公式： $C(m+n-1, n-1)$ ；² 下面加上取值范围后进一步考虑：假设现在 n 个数之和为 $m-k$ ，运用上述隔板法可得公式： $C(m-k+n-1, n-1)$ ；³ 随后，选择任意一个数字，将其加上 k ，这样，这个数字一定不满足条件，选法为： $C(n, 1)$ ；⁴ 此时，至少有一个数字是不满足条件的，按照一般流程，到这里， $C(m+n-1, n-1) - C(n, 1) * C(m-k+n-1, n-1)$ 即是答案；但是，这样的操作会导致重复的部分，所以这里要使用容斥原理将重复部分去除（关于为什么会重复，试比较概率论中的加法公式）。

几何

```
1 signed main() {
2     int n, k;
3     cin >> n >> k;
4
5     int cnt = __builtin_popcountll(n);
6
7     if (k < cnt || n < k) {
8         cout << "NO\n";
9     }
```



```

9      return 0;
10    }
11    cout << "YES\n";
12
13    vector<int> num;
14    while (n) {
15        num.push_back(n % 2);
16        n /= 2;
17    }
18
19    for (int i = num.size() - 1; i > 0; i--) {
20        int p = min(k - cnt, num[i]);
21        num[i] -= p;
22        num[i - 1] += 2 * p;
23        cnt += p;
24    }
25
26    for (int i = 0; i < num.size(); i++) {
27        for (int j = 1; j <= num[i]; j++) {
28            cout << (1LL << i) << " ";
29        }
30    }
31 }
32
33 Z clac(int n, int k, int m) {
34     Z ans = 0;
35     ans += C(n, i) * C(m - i * k + n - 1, n - 1) * pow(-1, i);
36 }
37 return ans;
38 }

```

2.1.17 常见数列

调和级数

满足调和级数 $\mathcal{O}\left(\frac{N}{1} + \frac{N}{2} + \frac{N}{3} + \cdots + \frac{N}{N}\right)$, 可以用 $\approx N \ln N$ 来拟合, 但是会略小, 误差量级在 10% 左右。本地可以在 500ms 内完成 10^8 量级的预处理计算。

N 的量级	1	2	3	4	5	6	7	8
累加和	27	482	7' 069	93 '668	1' 166 '750	13 '970' 034	162 '725' 364	1 '857' 511 '568

下方示例为求解 1 到 N 中各个数字的因数值。

素数密度与分布

N 的量级	1	2	3	4	5	6	7	8	9
素数数量	4	25	168	1 '229	9' 592	78 '498	664' 579	5 '761' 455	50 '847' 534

除此之外, 对于任意两个相邻的素数 $p_1, p_2 \leq 10^9$, 有 $|p_1 - p_2| < 300$ 成立, 更具体的说, 最大的差值为 282。

因数最多数字与其因数数量

N 的量级	1	2	3	4	5	6
因数最多数字的因数数量	4	25	32	64	128	240
因数最多的数字	-	-	-	7560, 9240	83160, 98280	720720, 831600, 942480, 982800, 997920

```

1  const int N = 1E5;
2  vector<vector<int>> dic(N + 1);
3  for (int i = 1; i <= N; i++) {
4      for (int j = i; j <= N; j += i) {
5          dic[j].push_back(i);
6      }
7  }

```

2.1.18 常见结论

球盒模型

参考链接。给定 n 个小球 m 个盒子。

- 球同，盒不同、不能空

隔板法： N 个小球即一共 $N - 1$ 个空，分成 M 堆即 $M - 1$ 个隔板，答案为 $\binom{n-1}{m-1}$ 。

- 球同，盒不同、能空

隔板法：多出 $M - 1$ 个虚空球，答案为 $\binom{m-1+n}{n}$ 。

- 球同，盒同、能空

$\frac{1}{(1-x)(1-x^2)\dots(1-x^m)}$ 的 x^n 项的系数。动态规划，答案为

$$dp[i][j] = \begin{cases} dp[i][j-1] + dp[i-j][j] & i \geq j \\ dp[i][j-1] & i < j \\ 1 & j == 1 \parallel i \leq 1 \end{cases}$$

- 球同，盒同、不能空

$\frac{x^m}{(1-x)(1-x^2)\dots(1-x^m)}$ 的 x^n 项的系数。动态规划，答案为

$$dp[n][m] = \begin{cases} dp[n-m][m] & n \geq m \\ 0 & n < m \end{cases}$$

- 球不同，盒同、不能空

第二类斯特林数 $\text{Stirling2}(n, m)$, 答案为

$$dp[n][m] = \begin{cases} m * dp[n-1][m] + dp[n-1][m-1] & 1 \leq m < n \\ 1 & 0 \leq n == m \\ 0 & m == 0 \ 1 \leq n \end{cases}$$

- 球不同, 盒同、能空

第二类斯特林数之和 $\sum_{i=1}^m \text{Stirling2}(n, m)$, 答案为 $\sum_{i=0}^m dp[n][i]$ 。

- 球不同, 盒不同、不能空

第二类斯特林数乘上 m 的阶乘 $m! \cdot \text{Stirling2}(n, m)$, 答案为 $dp[n][m] * m!$ 。

- 球不同, 盒不同、能空

答案为 m^n 。

麦乐鸡定理

给定两个互质的数 n, m , 定义 $x = a * n + b * m \ a \geq 0, b \geq 0$, 当 $x > n * m - n - m$ 时, 该式子恒成立。

抽屉原理 (鸽巢原理)

将 $n + 1$ 个物体, 划分为 n 组, 那么有至少一组有两个 (或以上) 的物体。

哥德巴赫猜想

任何一个大于 5 的整数都可写成三个质数之和; 任何一个大于 2 的偶数都可写成两个素数之和。

除法、取模运算的本质

有公式: $x \div i = \left\lfloor \frac{x}{i} \right\rfloor + x - i \cdot \left\lfloor \frac{x}{i} \right\rfloor$, $x \bmod i = x - i \cdot \left\lfloor \frac{x}{i} \right\rfloor$ 。

与、或、异或

运算	运算符、数学符号表示	解释
与	&、and	同 1 出 1
或	‘	、 or‘
异或	^、 $\oplus$ 、xor	不同出 1

一些结论:

对于给定的 X 和序列 $[a_1, a_2, \dots, a_n]$, 有: $X = (X \& a_1) \text{or} (X \& a_2) \text{or} \dots \text{or} (X \& a_n)$ 。

原理是 *and* 意味着取交集, *or* 意味着取子集。来源 - 牛客小白月赛 49C

调和级数近似公式

欧拉函数常见性质

- $1 - n$ 中与 n 互质的数之和为 $n * \varphi(n) / 2$ 。

- 若 a, b 互质, 则 $\varphi(a * b) = \varphi(a) * \varphi(b)$ 。实际上, 所有满足这一条件的函数统称为积性函数。
- 若 f 是积性函数, 且有 $n = \prod_{i=1}^m p_i^{c_i}$, 那么 $f(n) = \prod_{i=1}^m f(p_i^{c_i})$ 。
- 若 p 为质数, 且满足 $p \mid n$,
- $p^2 \mid n$, 那么 $\varphi(n) = \varphi(n/p) * p$ 。
- $p^2 \nmid n$, 那么 $\varphi(n) = \varphi(n/p) * (p-1)$ 。
- $\sum_{d \mid n} \varphi(d) = n$ 。

> 如 $n = 10$, 则 $d = 10/5/2/1$, 那么 $10 = \varphi(10) + \varphi(5) + \varphi(2) + \varphi(1)$ 。

- $\sum_{i=1}^n \gcd(i, n) = \sum_{d \mid n} \left\lfloor \frac{n}{d} \right\rfloor \varphi(d)$ (欧拉反演)。

组合数学常见性质

- $k * C_n^k = n * C_{n-1}^{k-1}$;
- $C_k^m * C_m^k = C_m^n * C_{m-n}^{m-k}$;
- $C_n^k + C_n^{k+1} = C_{n+1}^{k+1}$;
- $\sum_{i=0}^n C_n^i = 2^n$;
- $\sum_{k=0}^n (-1)^k * C_n^k = 0$ 。
- 二项式反演:
$$\begin{cases} f_n = \sum_{i=0}^n \binom{n}{i} g_i \Leftrightarrow g_n = \sum_{i=0}^n (-1)^{n-i} \binom{n}{i} f_i \\ f_k = \sum_{i=k}^n \binom{i}{k} g_i \Leftrightarrow g_k = \sum_{i=k}^n (-1)^{i-k} \binom{i}{k} f_i \end{cases};$$
- $\sum_{i=1}^n i \binom{n}{i} = n * 2^{n-1}$;
- $\sum_{i=1}^n i^2 \binom{n}{i} = n * (n+1) * 2^{n-2}$;
- $\sum_{i=1}^n \frac{1}{i} \binom{n}{i} = \sum_{i=1}^n \frac{1}{i}$;
- $\sum_{i=0}^n \binom{n}{i}^2 = \binom{2n}{n}$;
- 拉格朗日恒等式: $\sum_{i=1}^n \sum_{j=i+1}^n (a_i b_j - a_j b_i)^2 = (\sum_{i=1}^n a_i)^2 (\sum_{i=1}^n b_i)^2 - (\sum_{i=1}^n a_i b_i)^2$ 。

范德蒙德卷积公式

在数量为 $n+m$ 的堆中选 k 个元素, 和分别在数量为 n, m 的堆中选 $i, k-i$ 个元素的方案数是相同的, 即 $\sum_{i=0}^k \binom{n}{i} \binom{m}{k-i} = \binom{n+m}{k}$;

变体:

- $\sum_{i=0}^k C_{i+n}^i = C_{k+n+1}^k$;
- $\sum_{i=0}^k C_n^i * C_m^i = \sum_{i=0}^k C_n^i * C_m^{m-i} = C_{n+m}^n$ 。

卡特兰数

是一类奇特的组合数，前几项为 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862 。如遇到以下问题，则直接套用即可。

- **【括号匹配问题】** n 个左括号和 n 个右括号组成的合法括号序列的数量，为 Cat_n 。
- **【进出栈问题】** $1, 2, \dots, n$ 经过一个栈，形成的合法出栈序列的数量，为 Cat_n 。
- **【二叉树生成问题】** n 个节点构成的不同二叉树的数量，为 Cat_n 。
- **【路径数量问题】** 在平面直角坐标系上，每一步只能**向上或向右**走，从 $(0, 0)$ 走到 (n, n) ，并且除两个端点外不接触直线 $y = x$ 的路线数量，为 $2Cat_{n-1}$ 。

计算公式： $Cat_n = \frac{C_{2n}^n}{n+1}$, $C_n = \frac{C_{n-1} * (4n-2)}{n+1}$ 。

狄利克雷卷积

$$\sum_{d|n} \varphi(d) = n, \quad \sum_{d|n} \mu(d) \frac{n}{d} = \varphi(n)。$$

斐波那契数列

$$\text{通项公式: } F_n = \frac{1}{\sqrt{5}} * \left[\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right]。$$

直接结论：

- 卡西尼性质： $F_{n-1} * F_{n+1} - F_n^2 = (-1)^n$ ；
- $F_n^2 + F_{n+1}^2 = F_{2n+1}$ ；
- $F_{n+1}^2 - F_{n-1}^2 = F_{2n}$ （由上一条写两遍相减得到）；
- 若存在序列 $a_0 = 1, a_n = a_{n-1} + a_{n-3} + a_{n-5} + \dots (n \geq 1)$ 则 $a_n = F_n (n \geq 1)$ ；
- 齐肯多夫定理：任何正整数都可以表示成若干个不连续的斐波那契数（ F_2 开始）可以用贪心实现。

求和公式结论：

- 奇数项求和： $F_1 + F_3 + F_5 + \dots + F_{2n-1} = F_{2n}$ ；
- 偶数项求和： $F_2 + F_4 + F_6 + \dots + F_{2n} = F_{2n+1} - 1$ ；
- 平方和： $F_1^2 + F_2^2 + F_3^2 + \dots + F_n^2 = F_n * F_{n+1}$ ；
- $F_1 + 2F_2 + 3F_3 + \dots + nF_n = nF_{n+2} - F_{n+3} + 2$ ；
- $-F_1 + F_2 - F_3 + \dots + (-1)^n F_n = (-1)^n (F_{n+1} - F_n) + 1$ ；
- $F_{2n-2m-2}(F_{2n} + F_{2n+2}) = F_{2m+2} + F_{4n-2m}$ 。

数论结论：

- $F_a \mid F_b \Leftrightarrow a \mid b$;
- $\gcd(F_a, F_b) = F_{\gcd(a,b)}$;
- 当 p 为 $5k \pm 1$ 型素数时,
$$\begin{cases} F_{p-1} \equiv 0 \pmod{p} \\ F_p \equiv 1 \pmod{p} \\ F_{p+1} \equiv 1 \pmod{p} \end{cases} ;$$
- 当 p 为 $5k \pm 2$ 型素数时,
$$\begin{cases} F_{p-1} \equiv 1 \pmod{p} \\ F_p \equiv -1 \pmod{p} \\ F_{p+1} \equiv 0 \pmod{p} \end{cases} ;$$
- $F(n) \% m$ 的周期 $\leq 6m$ ($m = 2 \times 5^k$ 时取到等号);
- 既是斐波那契数又是平方数的有且仅有 1, 144。

杂

- 负数取模得到的是负数, 如果要用 0/1 判断的话请取绝对值;
- 辗转相除法原式为 $\gcd(x, y) = \gcd(x, y - x)$, 推广到 N 项为 $\gcd(a_1, a_2, \dots, a_N) = \gcd(a_1, a_2 - a_1, \dots, a_N - a_{N-1})$,
- 该推论在“四则运算后 gcd”这类题中有特殊意义, 如求解 $\gcd(a_1 + X, a_2 + X, \dots, a_N + X)$ 时 See;
- 以下式子成立: $\gcd(a, m) = \gcd(a + x, m) \Leftrightarrow \gcd(a, m) = \gcd(x, m)$ 。求解上式满足条件的 x 的数量即为求比 $\frac{m}{\gcd(a, m)}$ 小且与其互质的数的个数, 即用欧拉函数求解 $\varphi\left(\frac{m}{\gcd(a, m)}\right)$ 。
- 已知序列 a , 定义集合 $S = \{a_i \cdot a_j \mid i < j\}$, 现在要求解 $\gcd(S)$, 即为求解 $\gcd(a_j, \gcd(a_i \mid i < j))$, 换句话说, 即为求解后缀 gcd。
- 连续四个数互质的情况如下, 当 n 为奇数时, $n, n-1, n-2$ 一定互质; 而当 n 为偶数时,
$$\begin{cases} n, n-1, n-3 \text{互质} & \gcd(n, n-3) = 1 \text{时} \\ n-1, n-2, n-3 \text{互质} & \gcd(n, n-3) \neq 1 \text{时} \end{cases} \text{ See};$$
- 由 $a \bmod b = (b + a) \bmod b = (2 \cdot b + a) \bmod b = \dots = (K \cdot b + a) \bmod b$ 可以推广得到 $(a \bmod b) \bmod c = ((K \cdot bc + a) \bmod b) \bmod c$, 由此可以得到一个 bc 的答案周期 See;
- 对于长度为 $2 \cdot N$ 的数列 a , 将其任意均分为两个长度为 N 的数列 p, q , 随后对 p 非递减排序、对 q 非递增排序, 定义 $f(p, q) = \sum_{i=1}^n |p_i - q_i|$, 那么答案为 a 数列前 N 大的数之和减去前 N 小的数之和 See。
- 令 $\begin{cases} X = a + b \\ Y = a \oplus b \end{cases}$, 如果该式子有解, 那么存在前提条件 $\begin{cases} X \geq Y \\ X, Y \text{同奇偶} \end{cases}$; 进一步, 此时最小的 a 的取值为 $\frac{X - Y}{2}$ See。

然而, 上方方程并不总是有解的, 只有当变量增加到三个时, 才**一定有解**, 即: **在保证上方前提条件成立的情况下**, 求解 $\begin{cases} X = a + b + c \\ Y = a \oplus b \oplus c \end{cases}$, 则一定存在一组解 $\{\frac{X-Y}{2}, \frac{X+Y}{2}, Y\}$ See。

- 已知序列 p 是由序列 a_1 、序列 a_2 、……、序列 a_n 合并而成, 且合并过程中各序列内元素相对顺序不变, 记 $T(p)$ 是 p 序列的最大前缀和, 则 $T(p) = \sum_{i=1}^n T(a_i)$ See。
- $x + y = x|y + x \& y$, 对于两个数字 x 和 y , 如果将 x 变为 $x|y$, 同时将 y 变为 $x \& y$, 那么在本质上即将 x 二进制模式下的全部 1 移动到了 y 的对应的位置上 See。
- 一个正整数 x 异或、加上另一个正整数 y 后奇偶性不发生变化: $a + b \equiv a \oplus b \pmod{2}$ See。

```

1 i64 mypow(i64 n, i64 k) { // 复杂度是 log N
2     i64 r = 1;
3     for (; k >>= 1, n *= n) {
4         if (k & 1) r *= n;
5     }
6     return r;
7 }
8
9 vector<vector<i64>> comb;
10 void YangHuiTriangle(int n = 60) {
11     comb.resize(n + 1, vector<i64>(n + 1));
12     comb[0][0] = 1;
13     for (int i = 1; i <= n; i++) {
14         comb[i][0] = 1;
15         for (int j = 1; j <= n; j++) {
16             comb[i][j] = comb[i - 1][j] + comb[i - 1][j - 1];
17         }
18     }
19 }
20
21 vector<vector<i64>> S;
22 void Stirling2(int n = 15) {
23     S.resize(n + 1, vector<i64>(n + 1));
24     S[1][1] = 1;
25     for (int i = 2; i <= 15; i++) {
26         for (int j = 1; j <= i; j++) {
27             S[i][j] = S[i - 1][j - 1] + S[i - 1][j] * j;
28         }
29     }
30 }
31
32 vector<vector<i64>> dp;
33 void GeneratingFunction(int n = 15) {
34     dp.resize(n + 1, vector<i64>(n + 1));
35     for (int i = 0; i <= n; i++) {
36         dp[i][1] = 1;
37         for (int j = 2; j <= n; j++) {
38             dp[i][j] = dp[i][j - 1];
39             if (i >= j) dp[i][j] += dp[i - j][j];

```

```

40     }
41 }
42 }
43
44 vector<i64> fac;
45 void Fac(int n = 30) {
46     fac.resize(n + 1);
47     fac[0] = 1;
48     for (int i = 1; i <= n; i++) {
49         fac[i] = fac[i - 1] * i;
50     }
51 }
52
53 i64 A(int n, int m) {
54     if (n < 0 || m < 0 || n < m) return 0;
55     return fac[n] / fac[n - m];
56 }
57
58 i64 C(int n, int m) {
59     if (n < 0 || m < 0 || n < m) return 0;
60     return comb[n][m];
61 }
62
63 signed main() {
64     int Task = 1;
65     for (cin >> Task; Task; Task--) {
66         int op, n, m;
67         cin >> op >> n >> m;
68
69         i64 ans = -1;
70         if (op == 1) { // 球同, 盒同、能空
71             ans = dp[n][m];
72         } else if (op == 2) { // 球同, 盒同、至多放一个
73             ans = (n <= m);
74         } else if (op == 3) { // 球同, 盒同、至少放一个
75             ans = (n < m ? 0 : dp[n - m][m]);
76         } else if (op == 4) { // 球同, 盒不同、能空
77             ans = C(m - 1 + n, n);
78         } else if (op == 5) { // 球同, 盒不同、至多放一个
79             ans = C(m, n);
80         } else if (op == 6) { // 球同, 盒不同、至少放一个
81             ans = C(n - 1, m - 1);
82         } else if (op == 7) { // 球不同, 盒同、能空
83             ans = accumulate(S[n].begin() + 1, S[n].begin() + m + 1, 0LL);
84         } else if (op == 8) { // 球不同, 盒同、至多放一个
85             ans = (n <= m);
86         } else if (op == 9) { // 球不同, 盒同、至少放一个
87             ans = S[n][m];
88         } else if (op == 10) { // 球不同, 盒不同、能空
89             ans = mypow(m, n);
90         } else if (op == 11) { // 球不同, 盒不同、至多放一个
91             ans = A(m, n);
92         } else if (op == 12) { // 球不同, 盒不同、至少放一个

```



```

93         ans = fac[m] * S[n][m];
94     }
95     cout << ans << "\n";
96 }
97 }
98
99 log(n) + 0.5772156649 + 1.0 / (2 * n)

```

2.1.19 康拓展开

正向展开普通解法

将一个字典序排列转换成序号。例如：12345->1, 12354->2。

正向展开树状数组解

给定一个全排列，求出它是 $1 \sim n$ 所有全排列的第几个，答案对 998244353 取模。

答案就是 $\sum_{i=1}^n res_{a_i}(n-i)!$ 。 res_x 表示剩下的比 x 小的数字的数量，通过树状数组处理。

逆向还原

```

1  int f[20];
2  void jie_cheng(int n) { // 打出1-n的阶乘表
3      f[0] = f[1] = 1; // 0的阶乘为1
4      for (int i = 2; i <= n; i++) f[i] = f[i - 1] * i;
5  }
6  string str;
7  int kangtuo() {
8      int ans = 1; // 注意，因为 12345 是算作0开始计算的，最后结果要把12345看作是第一个
9      int len = str.length();
10     for (int i = 0; i < len; i++) {
11         int tmp = 0; // 用来计数的
12         // 计算str[i]是第几大的数，或者说计算有几个比他小的数
13         for (int j = i + 1; j < len; j++)
14             if (str[i] > str[j]) tmp++;
15         ans += tmp * f[len - i - 1];
16     }
17     return ans;
18 }
19 int main() {
20     jie_cheng(10);
21     string str = "52413";
22     cout << kangtuo() << endl;
23 }
24
25 #include <bits/stdc++.h>
26 using namespace std;
27 #define LL long long
28 const int mod = 998244353, N = 1e6 + 10;
29 LL fact[N];
30 struct fwt{
31     LL n;
32     vector <LL> a;
33     fwt(LL n) : n(n), a(n + 1) {}
34     LL sum(LL x){

```

```

35     LL res = 0;
36     for (; x; x -= x & -x)
37         res += a[x];
38     return res;
39 }
40 void add(LL x, LL k){
41     for (; x <= n; x += x & -x)
42         a[x] += k;
43 }
44 LL query(LL x, LL y){
45     return sum(y) - sum(x - 1);
46 }
47 };
48 int main(){
49     ios::sync_with_stdio(false); cin.tie(0);
50     LL n;
51     cin >> n;
52     fwt a(n);
53     fact[0] = 1;
54     for (int i = 1; i <= n; i++){
55         fact[i] = fact[i - 1] * i % mod;
56         a.add(i, 1);
57     }
58     LL ans = 0;
59     for (int i = 1; i <= n; i++){
60         LL x;
61         cin >> x;
62         ans = (ans + a.query(1, x - 1) * fact[n - i] % mod) % mod;
63         a.add(x, -1);
64     }
65     cout << (ans + 1) % mod << "\n";
66     return 0;
67 }
68
69 string str;
70 int kangtuo(){
71     int ans = 1; //注意, 因为 12345 是算作0开始计算的, 最后结果要把12345看作是第一个
72     int len = str.length();
73     for(int i = 0; i < len; i++){
74         int tmp = 0; //用来计数的
75         for(int j = i + 1; j < len; j++){
76             if(str[i] > str[j]) tmp++;
77             //计算str[i]是第几大的数, 或者说计算有几个比他小的数
78         }
79         ans += tmp * f[len - i - 1];
80     }
81     return ans;
82 }
83 int main(){
84     jie_cheng(10);
85     string str = "52413";
86     cout<<kangtuo()<<endl;
87 }

```

2.1.20 快速幂

常规

长整型 with 防爆乘法

```

1 i64 pow(i64 a, int b, int m) { // 复杂度是 log N
2     i64 r = 1 % m; /**! 这里的取模容易遗漏 */
3     for (; b >>= 1, a = a * a % m) {
4         if (b & 1) r = r * a % m;
5     }
6     return r;
7 }
8
9 i64 mul(i64 a, i64 b, i64 m) { // 由于不取模, 运行速度非常快
10    a %= m, b %= m; /**! 这里的取模容易遗漏 */
11    i64 r = a * b - m * i64(1.L / m * a * b);
12    return r - m * (r >= m) + m * (r < 0);
13 }
14
15 i64 mul(i64 a, i64 b, i64 m) { // 比较慢
16    return (__int128)a * b % m;
17 }
18
19 i64 pow(i64 a, i64 b, i64 m) {
20    i64 res = 1 % m; /**! 这里的取模容易遗漏 */
21    for (; b >>= 1, a = mul(a, a, m)) { // 配合下方手写乘法
22        if (b & 1) res = mul(res, a, m);
23    }
24    return res;
25 }

```

2.1.21 扩展欧几里得 exgcd

求解形如 $a \cdot x + b \cdot y = \gcd(a, b)$ 的不定方程的任意一组解。

例题：求解二元一次不定方程 $A \cdot x + B \cdot y = C$ 。

```

1 int exgcd(int a, int b, int &x, int &y) {
2     if (!b) {
3         x = 1, y = 0;
4         return a;
5     }
6     int d = exgcd(b, a % b, y, x);
7     y -= a / b * x;
8     return d;
9 }
10
11 auto clac = [&](int a, int b, int c) {
12     int u = 1, v = 1;
13     if (a < 0) { // 负数特判, 但是没用经过例题测试
14         a = -a;
15         u = -1;
16     }

```

```

17     if (b < 0) {
18         b = -b;
19         v = -1;
20     }
21
22     int x, y, d = exgcd(a, b, x, y), ans;
23     if (c % d != 0) { // 无整数解
24         cout << -1 << "\n";
25         return;
26     }
27     a /= d, b /= d, c /= d;
28     x *= c, y *= c; // 得到可行解
29
30     ans = (x % b + b - 1) % b + 1;
31     auto [A, B] = pair{u * ans, v * (c - ans * a) / b}; // x最小正整数 特解
32
33     ans = (y % a + a - 1) % a + 1;
34     auto [C, D] = pair{u * (c - ans * b) / a, v * ans}; // y最小正整数 特解
35
36     int num = (C - A) / b + 1; // xy均为正整数 的 解的组数
37 };

```

2.1.22 数论 | 取模运算类 | 蒙哥马利模乘

```

1 using u64 = uint64_t;
2 using u128 = __uint128_t;
3
4 struct Montgomery {
5     u64 m, m2, im, l1, l2;
6     Montgomery() {}
7     Montgomery(u64 m) : m(m) {
8         l1 = -(u64)m % m, l2 = -(u128)m % m;
9         m2 = m << 1, im = m;
10        for (int i = 0; i < 5; i++) im *= 2 - m * im;
11    }
12    inline u64 operator()(i64 a, i64 b) const {
13        u128 c = (u128)a * b;
14        return u64(c >> 64) + m - u64((u64)c * im * (u128)m >> 64);
15    }
16    inline u64 reduce(u64 a) const {
17        a = m - u64(a * im * (u128)m >> 64);
18        return a >= m ? a - m : a;
19    }
20    inline u64 trans(i64 a) const {
21        return (*this)(a, l2);
22    }
23
24    inline u64 add(i64 a, i64 b) const {
25        u64 c = trans(a) + trans(b);
26        if (c >= m2) c -= m2;
27        return reduce(c);
28    }

```

```

29 inline u64 sub(i64 a, i64 b) const {
30     u64 c = trans(a) - trans(b);
31     if (c >= m2) c += m2;
32     return reduce(c);
33 }
34 inline u64 mul(i64 a, i64 b) const {
35     return reduce((*this)(trans(a), trans(b)));
36 }
37 inline u64 div(i64 a, i64 b) const {
38     a = trans(a), b = trans(b);
39     u64 n = m - 2, inv = l1;
40     for (; n; n >>= 1, b = (*this)(b, b))
41         if (n & 1) inv = (*this)(inv, b);
42     return reduce((*this)(a, inv));
43 }
44 u64 pow(u64 a, u64 n) {
45     u64 r = l1;
46     a = trans(a);
47     for (; n; n >>= 1, a = (*this)(a, a))
48         if (n & 1) r = (*this)(r, a);
49     return reduce(r);
50 }
51 };

```

2.1.23 数论 | 质因数分解 | Pollard-Rho

```

1 struct Montgomery {} M(10); // 注意预赋值
2 bool isprime(i64 n) {}
3
4 i64 rho(i64 n) {
5     if (!(n & 1)) return 2;
6     i64 x = 0, y = 0, prod = 1;
7     auto f = [&](i64 x) -> i64 {
8         return M.mul(x, x) + 5; // 这里的种子能被 hack , 如果是在线比赛, 请务必 rand 生成
9     };
10    for (int t = 30, z = 0; t % 64 || gcd(prod, n) == 1; ++t) {
11        if (x == y) x = ++z, y = f(x);
12        if (i64 q = M.mul(prod, x + n - y)) prod = q;
13        x = f(x), y = f(f(y));
14    }
15    return gcd(prod, n);
16 }
17
18 vector<i64> factorize(i64 x) {
19     vector<i64> res;
20     auto f = [&](auto f, i64 x) {
21         if (x == 1) return;
22         M = Montgomery(x); // 重设模数
23         if (isprime(x)) return res.push_back(x);
24         i64 y = rho(x);
25         f(f, y), f(f, x / y);
26     };

```

```

27     f(f, x), sort(res.begin(), res.end());
28     return res;
29 }

```

2.1.24 数论 | 质数判定 | Miller-Rabin

借助蒙哥马利模乘加速取模运算。

```

1  using u64 = uint64_t;
2  using u128 = __uint128_t;
3
4  struct Montgomery {
5      u64 m, m2, im, l1, l2;
6      Montgomery() {}
7      Montgomery(u64 m) : m(m) {
8          l1 = -(u64)m % m, l2 = -(u128)m % m;
9          m2 = m << 1, im = m;
10         for (int i = 0; i < 5; i++) {
11             im *= 2 - m * im;
12         }
13     }
14     inline u64 operator()(i64 a, i64 b) const {
15         u128 c = (u128)a * b;
16         return u64(c >> 64) + m - u64((u64)c * im * (u128)m >> 64);
17     }
18     inline u64 reduce(u64 a) const {
19         a = m - u64(a * im * (u128)m >> 64);
20         return a >= m ? a - m : a;
21     }
22     inline u64 trans(i64 a) const {
23         return (*this)(a, l2);
24     }
25
26     inline u64 mul(i64 a, i64 b) const {
27         u64 r = (*this)(trans(a), trans(b));
28         return reduce(r);
29     }
30     u64 pow(u64 a, u64 n) {
31         u64 r = l1;
32         a = trans(a);
33         for (; n; n >>= 1, a = (*this)(a, a)) {
34             if (n & 1) r = (*this)(r, a);
35         }
36         return reduce(r);
37     }
38 };
39
40 bool isprime(i64 n) {
41     if (n < 2 || n % 6 % 4 != 1) {
42         return (n | 1) == 3;
43     }
44     u64 s = __builtin_ctzll(n - 1), d = n >> s;
45     Montgomery M(n);

```

```

46     for (i64 a : {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) {
47         u64 p = M.pow(a, d), i = s;
48         while (p != 1 && p != n - 1 && a % n && i--) {
49             p = M.mul(p, p);
50         }
51         if (p != n - 1 && i != s) return false;
52     }
53     return true;
54 }

```

2.1.25 数论 | 质数判定 | 预分类讨论加速

常数优化, 达到 $\mathcal{O}(\frac{\sqrt{N}}{3})$ 。

```

1 bool is_prime(int n) {
2     if (n < 2) return false;
3     if (n == 2 || n == 3) return true;
4     if (n % 6 != 1 && n % 6 != 5) return false;
5     for (int i = 5, j = n / i; i <= j; i += 6) {
6         if (n % i == 0 || n % (i + 2) == 0) {
7             return false;
8         }
9     }
10    return true;
11 }

```

2.1.26 整除 (数论) 分块

$\left\lfloor \frac{n}{l} \right\rfloor = \left\lfloor \frac{n}{l+1} \right\rfloor = \dots = \left\lfloor \frac{n}{r} \right\rfloor \iff \left\lfloor \frac{n}{l} \right\rfloor \leq \frac{n}{r} < \left\lfloor \frac{n}{l} \right\rfloor + 1$, 根据不等式左侧, 得到 $r \leq \left\lfloor \frac{n}{\left\lfloor \frac{n}{l} \right\rfloor} \right\rfloor$ 。

```

1 void solve() {
2     LL n; cin >> n;
3     LL ans = 0;
4     for (LL i = 1, j; i <= n; i = j + 1) {
5         j = n / (n / i);
6         ans += (LL)(j - i + 1) * (n / i);
7     }
8     cout << ans << "\n";
9 }
10 int main() {
11     int T; cin >> T;
12     while (T--) solve();
13 }

```

2.1.27 最大公约数 ‘gcd’

速度不如内置函数! 以 $\mathcal{O}(\log(a+b))$ 的复杂度求解最大公约数。与内置函数 `__gcd` 功能基本相同 (支持 $a, b \leq 0$)。有使用位运算的 `==` 常数优化版本 `==`。

```

1 inline int mygcd(int a, int b) { return b ? gcd(b, a % b) : a; }

```

2.1.28 欧拉函数

直接求解单个数的欧拉函数

1 到 N 中与 N 互质数的个数称为欧拉函数，记作 $\varphi(N)$ 。求解欧拉函数的过程即为分解质因数的过程，复杂度 $\mathcal{O}(\sqrt{n})$ 。

求解 1 到 N 所有数的欧拉函数

利用上述第四条性质，我们可以快速递推出 $2 - N$ 中每个数的欧拉函数，复杂度 $\mathcal{O}(N)$ ，而该算法即是线性筛的算法。

使用莫比乌斯反演求解欧拉函数

```

1  int phi(int n) { //求解 phi(n)
2      int ans = n;
3      for(int i = 2; i <= n / i; i++) { //注意，这里要写 n / i，以防止 int 型溢出风险和
          sqrt 超时风险
4          if(n % i == 0) {
5              ans = ans / i * (i - 1);
6              while(n % i == 0) n /= i;
7          }
8      }
9      if(n > 1) ans = ans / n * (n - 1); //特判 n 为质数的情况
10     return ans;
11 }
12
13 const int N = 1e5 + 7;
14 int v[N], prime[N], phi[N];
15 void euler(int n) {
16     ms(v, 0); //最小质因子
17     int m = 0; //质数数量
18     for (int i = 2; i <= n; ++ i) {
19         if (v[i] == 0) { // i 是质数
20             v[i] = i, prime[++ m] = i;
21             phi[i] = i - 1;
22         }
23         //为当前的数 i 乘上一个质因子
24         for (int j = 1; j <= m; ++ j) {
25             //如 i 有比 prime[j] 更小的质因子，或超出 n，停止
26             if(prime[j] > v[i] || prime[j] > n / i) break;
27             // prime[j] 是合数 i * prime[j] 的最小质因子
28             v[i * prime[j]] = prime[j];
29             phi[i * prime[j]] = phi[i] * (i % prime[j] ? prime[j] - 1 : prime[j]);
30         }
31     }
32 }
33 int main() {
34     int n; cin >> n; euler(n);
35     for (int i = 1; i <= n; ++ i) cout << phi[i] << endl;
36     return 0;
37 }
38
39 int phi[N];
40 vector<int> fac[N];

```



```

41 void get_eulers() {
42     for (int i = 1; i <= N - 10; i++) {
43         for (int j = i; j <= N - 10; j += i) {
44             fac[j].push_back(i);
45         }
46     }
47     phi[1] = 1;
48     for (int i = 2; i <= N - 10; i++) {
49         phi[i] = i;
50         for (auto j : fac[i]) {
51             if (j == i) continue;
52             phi[i] -= phi[j];
53         }
54     }
55 }

```

2.1.29 求解连续按位异或

以 $\mathcal{O}(1)$ 复杂度计算 $0 \oplus 1 \oplus \cdots \oplus n$ 。

```

1 unsigned xor_n(unsigned n) {
2     unsigned t = n & 3;
3     if (t & 1) return t / 2u ^ 1;
4     return t / 2u ^ n;
5 }
6
7 i64 xor_n(i64 n) {
8     if (n % 4 == 1) return 1;
9     else if (n % 4 == 2) return n + 1;
10    else if (n % 4 == 3) return 0;
11    else return n;
12 }

```

2.1.30 求解连续数字的正约数集合——倍数法

使用规律递推优化，时间复杂度为 $\mathcal{O}(N \log N)$ ，如果不需要详细的输出集合，则直接将 `vector` 换为普通数组即可（时间更快）。

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int N = 1e5 + 7;
4 vector<int> f[N];
5
6 void divide(int n) {
7     for (int i = 1; i <= n; ++ i)
8         for (int j = 1; j <= n / i; ++ j)
9             f[i * j].push_back(i);
10    for (int i = 1; i <= n; ++ i) {
11        for (auto it : f[i]) cout << it << " ";
12        cout << endl;
13    }
14 }

```

```

15 int main() {
16     int x; cin >> x; divide(x);
17     return 0;
18 }

```

2.1.31 离散对数 bsgs 与 exbsgs

以 $O(\sqrt{P})$ 的复杂度求解 $a^x \equiv b(\text{mod } P)$ 。其中标准 BSGS 算法不能计算 a 与 MOD 互质的情况，而 exbsgs 则可以。

```

1 namespace BSGS {
2     LL a, b, p;
3     map<LL, LL> f;
4     inline LL gcd(LL a, LL b) { return b > 0 ? gcd(b, a % b) : a; }
5     inline LL ps(LL n, LL k, int p) {
6         LL r = 1;
7         for (; k >= 1) {
8             if (k & 1) r = r * n % p;
9             n = n * n % p;
10        }
11        return r;
12    }
13    void exgcd(LL a, LL b, LL &x, LL &y) {
14        if (!b)
15            x = 1, y = 0;
16        else {
17            exgcd(b, a % b, x, y);
18            LL t = x;
19            x = y;
20            y = t - a / b * y;
21        }
22    }
23    LL inv(LL a, LL b) {
24        LL x, y;
25        exgcd(a, b, x, y);
26        return (x % b + b) % b;
27    }
28    LL bsgs(LL a, LL b, LL p) {
29        f.clear();
30        int m = ceil(sqrt(p));
31        b %= p;
32        for (int i = 1; i <= m; i++) {
33            b = b * a % p;
34            f[b] = i;
35        }
36        LL tmp = ps(a, m, p);
37        b = 1;
38        for (int i = 1; i <= m; i++) {
39            b = b * tmp % p;
40            if (f.count(b) > 0) return (i * m - f[b] + p) % p;
41        }
42        return -1;

```

```

43 }
44 LL exbsgs(LL a, LL b, LL p) {
45     if (b == 1 || p == 1) return 0;
46     LL g = gcd(a, p), k = 0, na = 1;
47     while (g > 1) {
48         if (b % g != 0) return -1;
49         k++;
50         b /= g;
51         p /= g;
52         na = na * (a / g) % p;
53         if (na == b) return k;
54         g = gcd(a, p);
55     }
56     LL f = bsgs(a, b * inv(na, p) % p, p);
57     if (f == -1) return -1;
58     return f + k;
59 }
60 } // namespace BSGS
61
62 using namespace BSGS;
63
64 int main() {
65     IOS;
66     cin >> p >> a >> b;
67     a %= p, b %= p;
68     LL ans = exbsgs(a, b, p);
69     if (ans == -1) cout << "no solution\n";
70     else cout << ans << "\n";
71     return 0;
72 }

```

2.1.32 莫比乌斯函数/反演

莫比乌斯函数定义: $\mu(n) = \begin{cases} 1 & n = 1 \\ (-1)^k & n = \prod_{i=1}^k p_i \text{ 且 } p_i \text{ 互质} \\ 0 & \text{else} \end{cases}$ 。

莫比乌斯函数性质: 对于任意正整数 n 满足 $\sum_{d|n} \mu(d) = \begin{cases} 1 & n = 1 \\ 0 & n \neq 1 \end{cases}$; $\sum_{d|n} \frac{\mu(d)}{d} = \frac{\varphi(n)}{n}$ 。

莫比乌斯反演定义: 定义: $F(n)$ 和 $f(n)$ 是定义在非负整数集合上的两个函数, 并且满足 $F(n) = \sum_{d|n} f(d)$, 可得 $f(n) = \sum_{d|n} \mu(d) F\left(\left\lfloor \frac{n}{d} \right\rfloor\right)$ 。

```

1 const int N = 5e4 + 10;
2 bool st[N];
3 int mu[N], prime[N], cnt, sum[N];
4 void getMu() {
5     mu[1] = 1;
6     for (int i = 2; i <= N - 10; i++) {

```

```

7     if (!st[i]) {
8         prime[++cnt] = i;
9         mu[i] = -1;
10    }
11    for (int j = 1; j <= cnt && i * prime[j] <= N - 10; j++) {
12        st[i * prime[j]] = true;
13        if (i % prime[j] == 0) {
14            mu[i * prime[j]] = 0;
15            break;
16        }
17        mu[i * prime[j]] = -mu[i];
18    }
19 }
20 for (int i = 1; i <= N - 10; i++) {
21     sum[i] = sum[i - 1] + mu[i];
22 }
23 }
24 void solve() {
25     int n, m, k; cin >> n >> m >> k;
26     n = n / k, m = m / k;
27     if (n < m) swap(n, m);
28     LL ans = 0;
29     for (int i = 1, j = 0; i <= m; i = j + 1) {
30         j = min(n / (n / i), m / (m / i));
31         ans += (LL)(sum[j] - sum[i - 1]) * (n / i) * (m / i);
32     }
33     cout << ans << "\n";
34 }
35 int main() {
36     getMu();
37     int T; cin >> T;
38     while (T--) solve();
39 }

```

2.1.33 裴蜀定理

$ax + by = c$ ($x \in \mathbb{Z}^*, y \in \mathbb{Z}^*$) 成立的充要条件是 $\gcd(a, b) \mid c$ ($\mathbb{Z}^*$ 表示正整数集)。

例题：给定一个序列 a ，找到一个序列 x ，使得 $\sum_{i=1}^n a_i x_i$ 最小。

```

1 LL n, a, ans;
2 LL gcd(LL a, LL b){
3     return b ? gcd(b, a % b) : a;
4 }
5 int main(){
6     cin >> n;
7     for (int i = 0; i < n; i++){
8         cin >> a;
9         if (a < 0) a = -a;
10        ans = gcd(ans, a);
11    }
12    cout << ans << "\n";
13    return 0;

```

14 }

2.1.34 质因子分解

筛法

使用欧拉筛（线性筛），预处理时间复杂度 $\mathcal{O}(N)$ ，单次查询 $\mathcal{O}(\text{Prime Numer})$ 。

Pollard-Rho

以单个因子 $\mathcal{O}(\log X)$ 的复杂度输出数字 X 的全部质因数，由于需要结合素数测试，总复杂度会略高一些。如果遇到超时的情况，可能需要考虑进一步优化，例如检查题目是否强制要求枚举全部质因数等等。有 == 常数优化版本 == 可以再快五倍。

```

1 vector<int> prime, minp, maxp;
2
3 void sieve(int n = 1e7) {
4     minp.resize(n + 1);
5     maxp.resize(n + 1);
6     for (int i = 2; i <= n; i++) {
7         if (!minp[i]) {
8             minp[i] = maxp[i] = i;
9             prime.push_back(i);
10        }
11        for (auto j : prime) {
12            if (j > minp[i] || j > n / i) break;
13            minp[i * j] = j;
14            maxp[i * j] = maxp[i];
15        }
16    }
17 }
18
19 vector<array<int, 2>> factorize(int n) {
20     vector<array<int, 2>> ans;
21     while (n > 1) {
22         int now = minp[n], cnt = 0;
23         while (n % now == 0) {
24             n /= now;
25             cnt++;
26         }
27         ans.push_back({now, cnt});
28     }
29     return ans;
30 }
31
32 i64 rho(i64 n) {
33     if (!(n & 1)) return 2;
34     i64 x = 0, y = 0, prod = 1;
35     auto f = [&](i64 x) -> i64 {
36         return mul(x, x, n) + 5; // 这里的种子为 1 时能被 hack, 取 5 到目前为止没有什么问
           题
37     };
38     for (int t = 30, z = 0; t % 64 || gcd(prod, n) == 1; ++t) {
39         if (x == y) x = ++z, y = f(x);

```

```

40     if (i64 q = mul(prod, x + n - y, n)) prod = q;
41     x = f(x), y = f(f(y));
42 }
43 return gcd(prod, n);
44 }
45
46 vector<i64> factorize(i64 x) {
47     vector<i64> res;
48     auto f = [&](auto f, i64 x) {
49         if (x == 1) return;
50         if (isprime(x)) return res.push_back(x);
51         i64 y = rho(x);
52         f(f, y), f(f, x / y);
53     };
54     f(f, x), sort(res.begin(), res.end());
55     return res;
56 }

```

2.1.35 质数判定

试除法

标准 $\mathcal{O}(\sqrt{N})$ ，有 == 常数优化版本 == 可达到 $\mathcal{O}(\frac{\sqrt{N}}{3})$ 。

筛法

使用欧拉筛（线性筛），预期时间复杂度为 $\mathcal{O}(N)$ 。

Miller-Rabin

随机化验证，非严谨计算的平均复杂度约为 $\mathcal{O}(3.5 \times \log X)$ 。对于某些强力质数，可能会退化至约 $\mathcal{O}(35 \times \log X)$ 。有 == 常数优化版本 == 可以再快五倍。

```

1 bool isprime(int n) {
2     if (n < 2) return false;
3     for (int i = 2; i <= n / i; i++) {
4         if (n % i == 0) return false;
5     }
6     return true;
7 }
8
9 vector<int> prime, minp;
10
11 void sieve(int n = 1e7) {
12     minp.resize(n + 1);
13     for (int i = 2; i <= n; i++) {
14         if (!minp[i]) {
15             minp[i] = i;
16             prime.push_back(i);
17         }
18         for (auto j : prime) {
19             if (j > minp[i] || j > n / i) break;
20             minp[i * j] = j;
21         }
22     }

```

```

23 }
24
25 bool isprime(int n) {
26     return minp[n] == n;
27 }
28
29 i64 mul(i64 a, i64 b, i64 m) { // 快速乘提速, 约四倍效果
30     i64 r = a * b - m * i64(1.L / m * a * b);
31     return r - m * (r >= m) + m * (r < 0);
32 }
33
34 i64 pow(i64 a, i64 b, i64 m) {
35     i64 res = 1 % m;
36     for (; b >>= 1, a = mul(a, a, m)) {
37         if (b & 1) res = mul(res, a, m);
38     }
39     return res;
40 }
41
42 bool isprime(i64 n) {
43     if (n < 2 || n % 6 % 4 != 1) {
44         return (n | 1) == 3;
45     }
46     i64 s = __builtin_ctzll(n - 1), d = n >> s;
47     for (i64 a : {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) {
48         i64 p = pow(a % n, d, n), i = s;
49         while (p != 1 && p != n - 1 && a % n && i--) {
50             p = mul(p, p, n);
51         }
52         if (p != n - 1 && i != s) return false;
53     }
54     return true;
55 }

```

2.1.36 逆元

费马小定理理解 (借助快速幂)

单次计算的复杂度即为快速幂的复杂度 $\mathcal{O}(\log X)$ 。限制: MOD 必须是质数, 且需要满足 x 与 MOD 互质。

扩展欧几里得解

此方法的 MOD 没有限制, 复杂度为 $\mathcal{O}(\log X)$, 但是比快速幂法常数大一些。

离线求解: 线性递推解

以 $\mathcal{O}(N)$ 的复杂度完成 $1 - N$ 中全部逆元的计算。

```

1 LL inv(LL x) { return mypow(x, mod - 2, mod); }
2
3 int x, y;
4 int exgcd(int a, int b, int &x, int &y) { //扩展欧几里得算法
5     if (b == 0) {
6         x = 1, y = 0;

```

```

7     return a; //到达递归边界开始向上一层返回
8 }
9 int r = exgcd(b, a % b, x, y);
10 int temp = y; //把x y变成上一层的
11 y = x - (a / b) * y;
12 x = temp;
13 return r; //得到a b的最大公因数
14 }
15 LL getInv(int a, int mod) { //求a在mod下的逆元, 不存在逆元返回-1
16     LL x, y, d = exgcd(a, mod, x, y);
17     return d == 1 ? (x % mod + mod) % mod : -1;
18 }
19
20 inv[1] = 1;
21 for (int i = 2; i <= n; i ++ )
22     inv[i] = (p - p / i) * inv[p % i] % p;

```

2.1.37 高斯消元求解线性方程组

题目大意：输入一个包含 N 个方程 N 个未知数的线性方程组，系数与常数均为实数（两位小数）。求解这个方程组。如果存在唯一解，则输出所有 N 个未知数的解，结果保留两位小数。如果无数解，则输出 X，如果无解，则输出 N。

```

1 const int N = 110;
2 const double eps = 1e-8;
3 LL n;
4 double a[N][N];
5 LL gauss(){
6     LL c, r;
7     for (c = 0, r = 0; c < n; c ++ ){
8         LL t = r;
9         for (int i = r; i < n; i ++ ) //找到绝对值最大的行
10             if (fabs(a[i][c]) > fabs(a[t][c]))
11                 t = i;
12         if (fabs(a[t][c]) < eps) continue;
13         for (int j = c; j < n + 1; j ++ ) swap(a[t][j], a[r][j]); //将绝对值最大的一行
14                                     换到最顶端
15         for (int j = n; j >= c; j -- ) a[r][j] /= a[r][c]; //将当前行首位变成 1
16         for (int i = r + 1; i < n; i ++ ) //将下面列消成 0
17             if (fabs(a[i][c]) > eps)
18                 for (int j = n; j >= c; j -- )
19                     a[i][j] -= a[r][j] * a[i][c];
20         r ++ ;
21     }
22     if (r < n){
23         for (int i = r; i < n; i ++ )
24             if (fabs(a[i][n]) > eps)
25                 return 2;
26         return 1;
27     }
28     for (int i = n - 1; i >= 0; i -- )
29         for (int j = i + 1; j < n; j ++ )

```



```

29     a[i][n] -= a[i][j] * a[j][n];
30     return 0;
31 }
32 int main(){
33     cin >> n;
34     for (int i = 0; i < n; i ++ )
35         for (int j = 0; j < n + 1; j ++ )
36             cin >> a[i][j];
37     LL t = gauss();
38     if (t == 0){
39         for (int i = 0; i < n; i ++ ){
40             if (fabs(a[i][n]) < eps) a[i][n] = abs(a[i][n]);
41             printf("%.21f\n", a[i][n]);
42         }
43     }
44     else if (t == 1) cout << "Infinite group solutions\n";
45     else cout << "No solution\n";
46     return 0;
47 }

```

2.2 组合数学

2.2.1 组合数学——lucas

```

1  #define int long long
2  int t,n,m,p,f[100005];
3  int pow(int x,int y,int p){ //快速幂
4      x%=p;
5      int ans=1;
6      for(int i=y;i>=1,x=x*x%p) if(i&1) ans=ans*x%p;
7      return ans;
8  }
9  int C(int n,int m,int p){ //求组合数
10     if(m>n) return 0;
11     return ((f[n]*pow(f[m],p-2,p))%p*pow(f[n-m],p-2,p)%p);
12 }
13 int lucas(int n,int m,int p){ //Lucas定理
14     if(!m) return 1;
15     return C(n%p,m%p,p)*lucas(n/p,m/p,p)%p;
16 }
17 signed main(){
18     scanf("%d",&t);
19     while(t--){
20         scanf("%d%d%d",&n,&m,&p);
21         f[0]=1;
22         for(int i=1;i<=p;i++)
23             f[i]=(f[i-1]*i)%p;
24         printf("%lld\n",lucas(n+m,m,p));
25     }
26 }

```

2.2.2 GspersHack: 与个数同阶, 即组合数

```

1 //这个算法复杂度与答案个数是同阶的
2 void GspersHack(int k, int n)
3 {
4     int cur = (1 << k) - 1;
5     int limit = (1 << n);
6     while (cur < limit)
7     {
8         // do something
9         int lb = cur & -cur;
10        int r = cur + lb;
11        cur = ((r ^ cur) >> __builtin_ctz(lb) + 2) | r;
12        // 或: cur = (((r ^ cur) >> 2) / lb) | r;
13    }
14 }
15 如果调用GspersHack(3, 5), 会依次遍历到以下二进制数:
16
17 00111
18 01011
19 01101

```

2.2.3 组合数

```

1 ll fac[N+1], ifac[N+1];
2
3 void init() {
4     fac[0] = 1;
5     for (int i = 1; i <= N; i++)
6         fac[i] = fac[i-1] * i % MOD;
7     ifac[N] = qpow(fac[N], MOD-2);
8     for (int i = N; i >= 1; i--)
9         ifac[i-1] = ifac[i] * i % MOD;
10 }
11
12 // 返回 C(n,k) mod MOD
13 ll C(int n, int k) {
14     if (k < 0 || k > n) return 0;
15     return fac[n] * ifac[k] % MOD * ifac[n-k] % MOD;
16 }

```

2.2.4 n 中任意 m 个数的全排列

```

1 #include <iostream>
2 using namespace std;
3
4 int n,m;
5 void perm(vector<int> &a,int dep){
6     if(dep==m){ //只有n-1层 所以深度为n时结束递归
7         for(int i=0;i<m;i++)
8             cout<<a[i]<<" ";

```

```

9         cout<<endl;
10     }
11     for(int i=dep;i<n;i++){//这层的深度的对应位置的数字固定，然后向下再递归
12
13         swap(a[i],a[dep]);
14
15         perm(a,dep+1);
16
17         swap(a[i],a[dep]); //恢复现场
18     }
19 }
20 int main(){
21     cin>>n>>m;
22     vector<int> a(n);
23     for(int i=0;i<n;i++)
24         cin>>a[i];
25     perm(a,0);
26     return 0;
27 }
28
29
30 //直接用stl的 next_permutation:
31 #include<iostream>
32 #include<algorithm>
33 using namespace std;
34
35 int main() {
36     int ans[4] = {3, 2, 1, 4};
37
38     // 对数组进行排序，以便生成所有排列
39     sort(ans, ans + 4);
40
41     // 使用 do-while 循环生成并输出所有排列
42     do {
43         for (int i = 0; i < 4; ++i) {
44             cout << ans[i] << " ";
45         }
46         cout << endl;
47     } while (next_permutation(ans, ans + 4));
48
49     return 0;
50 }

```

2.2.5 n 个数的子集

```

1 //转换成二进制
2 #include <iostream>
3
4 using namespace std;
5
6 vector<int> a(11111000);
7 void print_subset(int n){

```

```
8
9     for(int i=0;i<(1<<n);i++){
10         for(int j=0;j<n;j++){
11             if(i & (1<<j))
12                 cout<<a[j]<<" "; //cout<<j<<" ";
13         }
14         cout<<endl;
15     }
16 }
17
18 int main(){
19     int n;
20     cin>>n;
21     for(int i=0;i<n;i++)
22         cin>>a[i];
23     print_subset(n);
24     return 0;
25 }
```

2.2.6 n 中任意 m 个数的组合

```
1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  // 存储输入的n个数
7  vector<int> a(11111000);
8
9  // 打印出选取的m个数的组合
10 void print_subset(int n, int m) {
11     for (int i = 0; i < (1 << n); i++) {
12         // 计算二进制数i中1的个数
13         int count_ones = 0;
14         for (int j = 0; j < n; j++) {
15             if (i & (1 << j)) {
16                 count_ones++;
17             }
18         }
19
20         // 如果1的个数等于m, 则输出对应的组合
21         if (count_ones == m) {
22             for (int j = 0; j < n; j++) {
23                 if (i & (1 << j)) {
24                     cout << a[j] << " ";
25                 }
26             }
27             cout << endl;
28         }
29     }
30 }
31 }
```

```

32 int main() {
33     int n, m;
34     // 输入n和m
35     cin >> n >> m;
36
37     // 输入n个数
38     for (int i = 0; i < n; i++) {
39         cin >> a[i];
40     }
41
42     // 打印组合
43     print_subset(n, m);
44
45     return 0;
46 }

```

2.2.7 组合数

debug

提供一组测试数据: $\binom{132}{66} = 377'389'666'165'540'953'244'592'352'291'892'721'700$, 模数为 998244353 时为 241'200'029; $10^9 + 7$ 时为 598375978。

逆元 + 卢卡斯定理 (质数取模)

$\mathcal{O}(N)$, 模数必须为质数。

质因数分解

此法适用于: $1 < n, m, MOD < 10^7$ 的情况。

杨辉三角 (精确计算)

60 以内 long long 可解, 130 以内 __int128 可解。

```

1 struct Comb {
2     int n;
3     vector<Z> _fac, _inv;
4
5     Comb() : _fac{1}, _inv{0} {}
6     Comb(int n) : Comb() {
7         init(n);
8     }
9     void init(int m) {
10         if (m <= n) return;
11         _fac.resize(m + 1);
12         _inv.resize(m + 1);
13         for (int i = n + 1; i <= m; i++) {
14             _fac[i] = _fac[i - 1] * i;
15         }
16         _inv[m] = _fac[m].inv();
17         for (int i = m; i > n; i--) {
18             _inv[i - 1] = _inv[i] * i;
19         }
20         n = m;
21     }
22     Z fac(int x) {

```

```

23     if (x > n) init(x);
24     return _fac[x];
25 }
26 Z inv(int x) {
27     if (x > n) init(x);
28     return _inv[x];
29 }
30 Z C(int x, int y) {
31     if (x < 0 || y < 0 || x < y) return 0;
32     return fac(x) * inv(y) * inv(x - y);
33 }
34 Z P(int x, int y) {
35     if (x < 0 || y < 0 || x < y) return 0;
36     return fac(x) * inv(x - y);
37 }
38 } comb(1 << 21);
39
40 int n,m,p,b[1000005],prime[1000005],t,min_prime[1000005];
41 void euler_Prime(int n){//用欧拉筛求出1~n中每个数的最小质因数的编号是多少，保存在
    min_prime中
42     for(int i=2;i<=n;i++){
43         if(b[i]==0){
44             prime[++t]=i;
45             min_prime[i]=t;
46         }
47         for(int j=1;j<=t&&i*prime[j]<=n;j++){
48             b[prime[j]*i]=1;
49             min_prime[prime[j]*i]=j;
50             if(i%prime[j]==0) break;
51         }
52     }
53 }
54 long long c(int n,int m,int p){//计算C(n,m)%p的值
55     euler_Prime(n);
56     int a[t+5];//t代表1~n中质数的个数，a[i]代表编号为i的质数在答案中出现的次数
57     for(int i=1;i<=t;i++) a[i]=0;//注意清0，一开始是随机数
58     for(int i=n;i>=n-m+1;i--){//处理分子
59         int x=i;
60         while (x!=1){
61             a[min_prime[x]]++;//注意min_prime中保存的是这个数的最小质因数的编号 (1~t)
62             x/=prime[min_prime[x]];
63         }
64     }
65     for(int i=1;i<=m;i++){//处理分母
66         int x=i;
67         while (x!=1){
68             a[min_prime[x]]--;
69             x/=prime[min_prime[x]];
70         }
71     }
72     long long ans=1;
73     for(int i=1;i<=t;i++){//枚举质数的编号，看它出现了几次
74         while(a[i]>0){

```

```

75         ans=ans*prime[i]%p;
76         a[i]--;
77     }
78 }
79 return ans;
80 }
81 int main(){
82     cin>>n>>m;
83     m=min(m,n-m); //小优化
84     cout<<c(n,m,MOD);
85 }
86
87 vector C(n + 1, vector<int>(n + 1));
88 C[0][0] = 1;
89 for (int i = 1; i <= n; i++) {
90     C[i][0] = 1;
91     for (int j = 1; j <= n; j++) {
92         C[i][j] = C[i - 1][j] + C[i - 1][j - 1];
93     }
94 }
95 cout << C[n][m] << endl;

```

2.3 多项式

2.3.1 FFT

```

1  //快速傅里叶变换 (FFT)
2  /*
3  题目描述
4  给定一个  $n$  次多项式  $F(x)$ , 和一个  $m$  次多项式  $G(x)$ 。
5  请求出  $F(x)$  和  $G(x)$  的乘积。
6  输入格式
7  第一行两个整数  $n, m$ 。PS: 系数 0-9
8  接下来一行  $n+1$  个数字, 从低到高表示  $F(x)$  的系数。
9  接下来一行  $m+1$  个数字, 从低到高表示  $G(x)$  的系数。
10 输出格式
11 一行  $n+m+1$  个数字, 从低到高表示  $F(x) \cdot G(x)$  的系数。
12 输入输出样例
13 输入 #1复制
14 1 2
15 1 2
16 1 2 1
17 输出 #1复制
18 1 4 5 2
19 */
20 #include <bits/stdc++.h>
21 using namespace std;
22 class FFT {
23 public:
24     struct com {
25         double x, y;
26         com operator+(const com& t) const {

```

```

27         return {x + t.x, y + t.y};
28     }
29     com operator-(const com& t)const {
30         return {x - t.x, y - t.y};
31     }
32     com operator*(const com& t)const {
33         return {x*t.x - y * t.y, x*t.y + y * t.x};
34     }
35 };
36 static constexpr double PI = acos(-1);
37 vector<com>A, B;
38 int la, lb, mi;
39 vector<int>R;//R是辅助数组
40 FFT(int nn, int mm, int N) {
41     la = nn, lb = mm;
42     A.resize(N), B.resize(N), R.resize(N);
43 }
44 void update() { //把mi变成符合条件的2^n
45     for (mi = 1; mi <= la + lb; mi <= 1) ;
46 }
47 void fun(int op, int ab) { //ab=1对A操作,ab=2对B操作
48     vector<com>&C = (ab == 1 ? A : B);
49     for (int i = 0; i < mi; ++i)
50         R[i] = R[i / 2] / 2 + ((i & 1) ? mi / 2 : 0);
51     for (int i = 0; i < mi; ++i)
52         if (i < R[i]) swap(C[i], C[R[i]]);
53     for (int i = 2; i <= mi; i <= 1) {
54         com w1({cos(2 * PI / i), sin(2 * PI / i)*op});
55         for (int j = 0; j < mi; j += i) {
56             com wk({1, 0});
57             for (int k = j; k < j + i / 2; ++k) {
58                 com x = C[k], y = C[k + i / 2] * wk;
59                 C[k] = x + y;
60                 C[k + i / 2] = x - y;
61                 wk = wk * w1;
62             }
63         }
64     }
65 }
66
67 };
68 int main() {
69     ios::sync_with_stdio(false);
70     cin.tie(0);
71     int n, m;
72     cin >> n >> m;
73     FFT f(n, m, 4000005);
74     for (int i = 0; i <= n; i++) cin >> f.A[i].x;
75     for (int i = 0; i <= m; i++) cin >> f.B[i].x;
76     f.update();
77     f.fun(1, 1), f.fun(1, 2);
78     for (int i = 0; i < f.mi; i++)
79         f.A[i] = f.A[i] * f.B[i];

```



```

80     f.fun(-1, 1);
81     for (int i = 0; i <= f.la + f.lb; i++)
82         cout << (int)(f.A[i].x / f.mi + 0.5) << " ";
83 }

```

2.3.2 多项式

```

1  #include<bits/stdc++.h>
2
3  #define int long long
4
5  using namespace std;
6  int gcd(int a,int b){return b?gcd(b,a%b):a;}
7  constexpr int __lg(unsigned long long x) {if (x == 0) return -1; int res = -1; while
    (x){ res++; x >>= 1;} return res;}
8  typedef pair<int, int> pii;
9  #define pb emplace_back
10 #define all(x) x.begin(),x.end()
11 #define min_in(x) *min_element(all(x))
12 #define max_in(x) *max_element(all(x))
13 #define debug cout<<"*OK*"<<endl
14 using ll=long long;
15 using i128 = __int128_t;
16 using vt=vector<int>;
17 using ld=long double;
18
19 const int M = 998244353;
20 const int G = 3;
21
22 // -----NTT-----
23 int qp(int a, int e = M - 2) {
24     ll r = 1, x = a % M;
25     while (e > 0) {if (e & 1) r = r * x % M;x = x * x % M;e >>= 1;}
26     return (int)r;
27 }
28 void ntt(vector<int> &a, bool inv) {
29     int n = a.size();
30     static vector<int> rev, roots{0,1};
31     if ((int)rev.size() != n) {
32         int L = __builtin_ctz(n);
33         rev.assign(n,0);
34         for (int i = 0; i < n; i++) rev[i] = (rev[i>>1]>>1) | ((i&1)<<(L-1));
35     }
36     for (int i = 0; i < n; i++) if (i<rev[i]) swap(a[i], a[rev[i]]);
37
38     if ((int)roots.size() < n) {
39         int k = __builtin_ctz(roots.size());
40         roots.resize(n);
41         while ((1<<k) < n) {
42             int e = qp(G, (M-1) >> (k+1));
43             for (int i = 1<<(k-1); i < (1<<k); i++) {
44                 roots[2*i] = roots[i];

```

```

45         roots[2*i+1] = (ll)roots[i] * e % M;
46     }
47     k++;
48 }
49 }
50
51 for (int len = 1; len < n; len <= 1) {
52     for (int i = 0; i < n; i += 2*len) {
53         for (int j = 0; j < len; j++) {
54             int u = a[i+j];
55             int v = (ll)a[i+j+len] * roots[len+j] % M;
56             a[i+j] = u + v < M ? u + v : u + v - M;
57             a[i+j+len] = u - v >= 0 ? u - v : u - v + M;
58         }
59     }
60 }
61
62 if (inv) {
63     reverse(a.begin()+1, a.end());
64     int inv_n = qp(n);
65     for (int &x : a) x = (ll)x * inv_n % M;
66 }
67 }
68 vector<int> multiply(vector<int> a, vector<int> b) {
69     int sz = a.size() + b.size() - 1;
70     int n = 1; while (n < sz) n <= 1;
71     a.resize(n); b.resize(n);
72     ntt(a, false); ntt(b, false);
73     for (int i = 0; i < n; i++) a[i] = (ll)a[i] * b[i] % M;
74     ntt(a, true);
75     a.resize(sz);
76     return a;
77 }
78 // -----
79
80 // 多项式求逆: Newton 迭代 O(n log n)
81 vector<int> poly_inv(vector<int> f, int m) {
82     vector<int> g(1, qp(f[0])); // g0 = 1/f0
83     for (int len = 1; len < m; len <= 1) {
84         vector<int> f_cut(f.begin(), f.begin() + min((int)f.size(), 2*len));
85         auto tmp = multiply(multiply(g, g), f_cut);
86         tmp.resize(2*len);
87         g.resize(2*len);
88         for (int i = 0; i < 2*len; i++) {
89             g[i] = ((2LL*g[i] - tmp[i]) % M + M) % M;
90         }
91     }
92     g.resize(m);
93     return g;
94 }
95
96 // 多项式开方 sqrt(f), 要求 f[0]=1
97 vector<int> poly_sqrt(const vector<int>& f, int m) {

```

```

98     vector<int> g(1,1);
99     int inv2 = (M + 1) / 2;
100    for (int len = 1; len < m; len <= 1) {
101        auto g2 = multiply(g, g);
102        vector<int> sum(2*len);
103        for (int i = 0; i < 2*len; i++) {
104            int v = (i < (int)f.size() ? f[i] : 0);
105            sum[i] = (v + (i < (int)g2.size() ? g2[i] : 0)) % M;
106        }
107        auto inv_g = poly_inv(g, 2*len);
108        g = multiply(sum, inv_g);
109        for (int &x : g) x = (ll)x * inv2 % M;
110        g.resize(2*len);
111    }
112    g.resize(m);
113    return g;
114 }
115
116 // 多项式导数 derivative
117 vector<int> poly_d(const vector<int>& f) {
118     int n = f.size();
119     vector<int> g(max<int>(0,n-1));
120     for (int i = 1; i < n; i++) g[i-1] = (ll)f[i]*i % M;
121     return g;
122 }
123
124 // 多项式积分 integral
125 vector<int> poly_int(const vector<int>& f) {
126     int n = f.size();
127     vector<int> g(n+1);
128
129     // (可全局预处理)
130     static vector<int> inv{0,1};
131     {
132         while ((int)inv.size() <= n) {
133             int i = inv.size();
134             inv.push_back((ll)(M - M/i) * inv[M % i] % M);
135         }
136     }
137
138     for (int i = 0; i < n; i++) g[i+1] = (ll)f[i] * inv[i+1] % M;
139     return g;
140 }
141
142 // 多项式对数 ln(f), 要求 f[0]=1
143 vector<int> poly_ln(const vector<int>& f, int m) {
144     auto df = poly_d(f);
145     auto invf = poly_inv(f, m);
146     auto q = multiply(df, invf);
147     q.resize(m-1);
148     auto res = poly_int(q);
149     res.resize(m);
150     return res;

```

```

151 }
152
153 // 多项式指数 exp(f), 要求 f[0]=0
154 vector<int> poly_exp(const vector<int>& f, int m) {
155     vector<int> g(1,1);
156     for (int len = 1; len < m; len <= 1) {
157         auto g_ln = poly_ln(g, 2*len);
158         vector<int> diff(2*len);
159         for (int i = 0; i < 2*len; i++) {
160             diff[i] = ((i < (int)f.size() ? f[i] : 0) - g_ln[i] + M) % M;
161         }
162         diff[0] = (diff[0] + 1) % M;
163         g = multiply(g, diff);
164         g.resize(2*len);
165     }
166     g.resize(m);
167     return g;
168 }
169
170 // 多项式按常数幂次快速幂 f^k, 要求 f[0]=1 或处理常数项为0, O(m log m)
171 vector<int> poly_pow(const vector<int>& f, ll k, int m) {
172     auto ln_f = poly_ln(f, m);
173     for (int i = 0; i < m; i++) ln_f[i] = (ll)ln_f[i] * (k % (M-1)) % M;
174     return poly_exp(ln_f, m);
175 }
176
177 // 多项式常数项为 0, 可用 NTT + 二分幂快速求幂, 复杂度 O(m log k · log m) 会慢些
178 vector<int> poly_pow2(const vector<int>& f, ll k, int m) {
179     vector<int> a(f.begin(), f.end()); // a 保存当前基, 多项式 f 的前 m 项
180     if ((int)a.size() > m) a.resize(m);
181
182     vector<int> res(1, 1);
183
184     while (k > 0) {
185         if (k & 1) {
186             res = multiply(res, a);
187             if ((int)res.size() > m) res.resize(m);
188         }
189         a = multiply(a, a);
190         if ((int)a.size() > m) a.resize(m);
191         k >>= 1;
192     }
193     return res;
194 }
195
196 // 多项式除法与取模: a = b*q + r
197 pair<vector<int>, vector<int>> poly_divmod(vector<int> a, vector<int> b) {
198     int n = a.size(), m = b.size();
199     if (n < m) return {{0}, a};
200     // 反转并截断
201     vector<int> ra(a.rbegin(), a.rend()), rb(b.rbegin(), b.rend());
202     int k = n-m+1;
203     auto inv_rb = poly_inv(rb, k);

```

```

204     auto rq = multiply(ra, inv_rb);
205     rq.resize(k);
206     vector<int> q(rq.rbegin(), rq.rend());
207     // 计算余数 r = a - b*q
208     auto bq = multiply(b, q);
209     vector<int> r = a;
210     for (int i = 0; i < (int)bq.size(); i++)
211         r[i] = (r[i] - bq[i] + M) % M;
212     r.resize(m-1);
213     return {q, r};
214 }
215 // -----
216 namespace MATH{
217     vector<int> fac, ifac, inv;
218     int dv2(int x){return x&1? (x+M)>>1:x>>1;}
219     int C(int n, int m){
220         //assert(m<=n);
221         if(m<0 || m>n) return 0;
222         return 1ll*fac[n]*ifac[m]%M*ifac[n-m]%M;
223     }
224     int P(int n, int m){
225         return 1ll*fac[n]*ifac[n-m]%M;
226     }
227     int D(int n, int m){
228         if(n<0 || m<0) return 0;
229         if(!n) return 1;
230         if(!m) return 0;
231         return C(n+m-1, m-1);
232     }
233     int catalan(int n) { //1,1,2,5,14,42,132,429,1430,4862
234         return (C(2 * n, n) - C(2 * n, n + 1) + M) % M;
235     }
236     void init(int n){
237         int x;
238         fac.resize(n+2);
239         fac[0]=fac[1]=1;
240         ifac=inv=fac;
241         for(x=2;x<=n;++x){
242             fac[x]=1ll*x*fac[x-1]%M;
243             inv[x]=1l(M-M/x)*inv[M%x]%M;
244             ifac[x]=1ll*ifac[x-1]*inv[x]%M;
245         }
246     }
247 }
248
249 int T;
250 const int maxn = 2e5+10;
251 //int a[maxn];
252
253 void solve() {
254
255 }
256 signed main () {

```

```

257     ios::sync_with_stdio(false);
258     cin.tie(nullptr);
259     int T=1;
260     // cin>>T;
261     while(T--){ solve();}
262     return 0;
263 }
264
265
266
267
268 // -----
269 // 三模 NTT
270 // -----
271 const int M = 1e9+7;
272 using i128 = __int128;
273 int qp(int a, int e = M - 2) {
274     ll r = 1, x = a % M;
275     while (e > 0) {if (e & 1) r = r * x % M;x = x * x % M;e >>= 1;}
276     return (int)r;
277 }
278
279 const int M1 = 998244353, G1 = 3;
280 const int M2 = 1004535809, G2 = 3;
281 const int M3 = 469762049, G3 = 3;
282
283 template<int mod, int G>
284 struct NTT {
285     vector<int> rev, roots{0,1};
286     static ll qp(ll a, ll e) {
287         ll r = 1;
288         while (e) {if (e & 1) r = r * a % mod;a = a * a % mod;e >>= 1;}
289         return r;
290     }
291     void dft(vector<int>& a) {
292         int n = a.size();
293         if ((int)rev.size() != n) {
294             rev.assign(n, 0);
295             for (int i = 1; i < n; i++)rev[i] = (rev[i>>1] >> 1) | ((i&1) * (n>>1));
296         }
297         for (int i = 0; i < n; i++) if (i < rev[i]) swap(a[i], a[rev[i]]);
298         if ((int)roots.size() < n) {
299             int k = __builtin_ctz(roots.size());
300             roots.resize(n);
301             while ((1<<k) < n) {
302                 ll e = qp(G, (mod-1) >> (k+1));
303                 for (int i = 1<<(k-1); i < (1<<k); ++i) {
304                     roots[2*i] = roots[i];
305                     roots[2*i + 1] = (ll)roots[i] * e % mod;
306                 }
307                 ++k;
308             }
309         }

```

```

310     for (int len = 1; len < n; len <= 1) {
311         for (int i = 0; i < n; i += len<<1) {
312             for (int j = 0; j < len; ++j) {
313                 int u = a[i+j];
314                 int v = (ll)a[i+j+len] * roots[len+j] % mod;
315                 a[i+j] = u+v < mod ? u+v : u+v-mod;
316                 a[i+j+len] = u-v >= 0 ? u-v : u-v+mod;
317             }
318         }
319     }
320 }
321 void idft(vector<int>& a) {
322     int n = a.size();
323     reverse(a.begin()+1, a.end());
324     dft(a);
325     ll inv_n = qp(n, mod-2);
326     for (int i = 0; i < n; i++) a[i] = a[i] * inv_n % mod;
327 }
328 vector<int> conv(vector<int> a, vector<int> b) {
329     int need = (int)a.size() + (int)b.size() - 1;
330     int n = 1; while (n < need) n <= 1;
331     a.resize(n); b.resize(n);
332     dft(a); dft(b);
333     for (int i = 0; i < n; i++) a[i] = (ll)a[i] * b[i] % mod;
334     idft(a);
335     a.resize(need);
336     return a;
337 }
338 };
339 using N1 = NTT<M1,G1>; using N2 = NTT<M2,G2>; using N3 = NTT<M3,G3>;
340
341 // CRT 合并
342 vector<int> multiply(const vector<int>& A, const vector<int>& B, int mod = M) {
343     N1 n1; N2 n2; N3 n3;
344     auto c1 = n1.conv(A, B);
345     auto c2 = n2.conv(A, B);
346     auto c3 = n3.conv(A, B);
347     int sz = c1.size();
348     vector<int> C(sz);
349     ll inv12 = N2::qp(M1, M2-2);
350     ll inv123 = N3::qp((ll)M1 * M2 % M3, M3-2);
351     ll m12 = (ll)M1 * M2;
352     for (int i = 0; i < sz; ++i) {
353         ll x1 = c1[i];
354         ll x2 = ( ( (ll)c2[i] - x1 ) % M2 + M2 ) % M2;
355         x2 = x2 * inv12 % M2;
356         ll t = ( x1 + x2 * (ll)M1 ) % M3 + M3 ) % M3;
357         ll x3 = ( ( (ll)c3[i] - t ) % M3 + M3 ) % M3;
358         x3 = x3 * inv123 % M3;
359         i128 X = (i128)x1 + (i128)x2 * (i128)M1 + (i128)x3 * (i128)m12;
360         ll xr = (ll)(X % mod);
361         if (xr < 0) xr += mod;
362         C[i] = (int)xr;

```

```

363     }
364     return C;
365 }
366 // -----

```

2.3.3 Berlekamp-Massey 算法 (杜教筛)

求解数列的最短线性递推式, 最坏复杂度为 $\mathcal{O}(NM)$, 其中 N 为数列长度, M 为它的最短递推式的阶数。

```

1  template<int P = 998244353> Poly<P> berlekampMassey(const Poly<P> &s) {
2      Poly<P> c;
3      Poly<P> oldC;
4      int f = -1;
5      for (int i = 0; i < s.size(); i++) {
6          auto delta = s[i];
7          for (int j = 1; j <= c.size(); j++) {
8              delta -= c[j - 1] * s[i - j];
9          }
10         if (delta == 0) {
11             continue;
12         }
13         if (f == -1) {
14             c.resize(i + 1);
15             f = i;
16         } else {
17             auto d = oldC;
18             d *= -1;
19             d.insert(d.begin(), 1);
20             MInt<P> df1 = 0;
21             for (int j = 1; j <= d.size(); j++) {
22                 df1 += d[j - 1] * s[f + 1 - j];
23             }
24             assert(df1 != 0);
25             auto coef = delta / df1;
26             d *= coef;
27             Poly<P> zeros(i - f - 1);
28             zeros.insert(zeros.end(), d.begin(), d.end());
29             d = zeros;
30             auto temp = c;
31             c += d;
32             if (i - temp.size() > f - oldC.size()) {
33                 oldC = temp;
34                 f = i;
35             }
36         }
37     }
38     c *= -1;
39     c.insert(c.begin(), 1);
40     return c;
41 }

```


2.3.4 Linear-Recurrence 算法

```

1  template<int P = 998244353> MInt<P> linearRecurrence(Poly<P> p, Poly<P> q, i64 n) {
2      int m = q.size() - 1;
3      while (n > 0) {
4          auto newq = q;
5          for (int i = 1; i <= m; i += 2) {
6              newq[i] *= -1;
7          }
8          auto newp = p * newq;
9          newq = q * newq;
10         for (int i = 0; i < m; i++) {
11             p[i] = newp[i * 2 + n % 2];
12         }
13         for (int i = 0; i <= m; i++) {
14             q[i] = newq[i * 2];
15         }
16         n /= 2;
17     }
18     return p[0] / q[0];
19 }

```

2.3.5 多项式封装

```

1  template<int P = 998244353> struct Poly : public vector<MInt<P>> {
2      using Value = MInt<P>;
3
4      Poly() : vector<Value>() {}
5      explicit constexpr Poly(int n) : vector<Value>(n) {}
6
7      explicit constexpr Poly(const vector<Value> &a) : vector<Value>(a) {}
8      constexpr Poly(const initializer_list<Value> &a) : vector<Value>(a) {}
9
10     template<class InputIt, class = _RequireInputIter<InputIt>>
11     explicit constexpr Poly(InputIt first, InputIt last) : vector<Value>(first, last) {}
12
13     template<class F> explicit constexpr Poly(int n, F f) : vector<Value>(n) {
14         for (int i = 0; i < n; i++) {
15             (*this)[i] = f(i);
16         }
17     }
18
19     constexpr Poly shift(int k) const {
20         if (k >= 0) {
21             auto b = *this;
22             b.insert(b.begin(), k, 0);
23             return b;
24         } else if (this->size() <= -k) {
25             return Poly();
26         } else {
27             return Poly(this->begin() + (-k), this->end());
28         }
29     }
30 }

```

```

28     }
29 }
30 constexpr Poly trunc(int k) const {
31     Poly f = *this;
32     f.resize(k);
33     return f;
34 }
35 constexpr friend Poly operator+(const Poly &a, const Poly &b) {
36     Poly res(max(a.size(), b.size()));
37     for (int i = 0; i < a.size(); i++) {
38         res[i] += a[i];
39     }
40     for (int i = 0; i < b.size(); i++) {
41         res[i] += b[i];
42     }
43     return res;
44 }
45 constexpr friend Poly operator-(const Poly &a, const Poly &b) {
46     Poly res(max(a.size(), b.size()));
47     for (int i = 0; i < a.size(); i++) {
48         res[i] += a[i];
49     }
50     for (int i = 0; i < b.size(); i++) {
51         res[i] -= b[i];
52     }
53     return res;
54 }
55 constexpr friend Poly operator-(const Poly &a) {
56     vector<Value> res(a.size());
57     for (int i = 0; i < int(res.size()); i++) {
58         res[i] = -a[i];
59     }
60     return Poly(res);
61 }
62 constexpr friend Poly operator*(Poly a, Poly b) {
63     if (a.size() == 0 || b.size() == 0) {
64         return Poly();
65     }
66     if (a.size() < b.size()) {
67         swap(a, b);
68     }
69     int n = 1, tot = a.size() + b.size() - 1;
70     while (n < tot) {
71         n *= 2;
72     }
73     if (((P - 1) & (n - 1)) != 0 || b.size() < 128) {
74         Poly c(a.size() + b.size() - 1);
75         for (int i = 0; i < a.size(); i++) {
76             for (int j = 0; j < b.size(); j++) {
77                 c[i + j] += a[i] * b[j];
78             }
79         }
80         return c;

```

```

81     }
82     a.resize(n);
83     b.resize(n);
84     dft(a);
85     dft(b);
86     for (int i = 0; i < n; ++i) {
87         a[i] *= b[i];
88     }
89     idft(a);
90     a.resize(tot);
91     return a;
92 }
93 constexpr friend Poly operator*(Value a, Poly b) {
94     for (int i = 0; i < int(b.size()); i++) {
95         b[i] *= a;
96     }
97     return b;
98 }
99 constexpr friend Poly operator*(Poly a, Value b) {
100     for (int i = 0; i < int(a.size()); i++) {
101         a[i] *= b;
102     }
103     return a;
104 }
105 constexpr friend Poly operator/(Poly a, Value b) {
106     for (int i = 0; i < int(a.size()); i++) {
107         a[i] /= b;
108     }
109     return a;
110 }
111 constexpr Poly &operator+=(Poly b) {
112     return (*this) = (*this) + b;
113 }
114 constexpr Poly &operator-=(Poly b) {
115     return (*this) = (*this) - b;
116 }
117 constexpr Poly &operator*=(Poly b) {
118     return (*this) = (*this) * b;
119 }
120 constexpr Poly &operator*=(Value b) {
121     return (*this) = (*this) * b;
122 }
123 constexpr Poly &operator/=(Value b) {
124     return (*this) = (*this) / b;
125 }
126 constexpr Poly deriv() const {
127     if (this->empty()) {
128         return Poly();
129     }
130     Poly res(this->size() - 1);
131     for (int i = 0; i < this->size() - 1; ++i) {
132         res[i] = (i + 1) * (*this)[i + 1];
133     }

```

```

134     return res;
135 }
136 constexpr Poly integr() const {
137     Poly res(this->size() + 1);
138     for (int i = 0; i < this->size(); ++i) {
139         res[i + 1] = (*this)[i] / (i + 1);
140     }
141     return res;
142 }
143 constexpr Poly inv(int m) const {
144     Poly x((*this)[0].inv());
145     int k = 1;
146     while (k < m) {
147         k *= 2;
148         x = (x * (Poly{2} - trunc(k) * x)).trunc(k);
149     }
150     return x.trunc(m);
151 }
152 constexpr Poly log(int m) const {
153     return (deriv() * inv(m)).integr().trunc(m);
154 }
155 constexpr Poly exp(int m) const {
156     Poly x{1};
157     int k = 1;
158     while (k < m) {
159         k *= 2;
160         x = (x * (Poly{1} - x.log(k) + trunc(k))).trunc(k);
161     }
162     return x.trunc(m);
163 }
164 constexpr Poly pow(int k, int m) const {
165     int i = 0;
166     while (i < this->size() && (*this)[i] == 0) {
167         i++;
168     }
169     if (i == this->size() || 1LL * i * k >= m) {
170         return Poly(m);
171     }
172     Value v = (*this)[i];
173     auto f = shift(-i) * v.inv();
174     return (f.log(m - i * k) * k).exp(m - i * k).shift(i * k) * power(v, k);
175 }
176 constexpr Poly sqrt(int m) const {
177     Poly x{1};
178     int k = 1;
179     while (k < m) {
180         k *= 2;
181         x = (x + (trunc(k) * x.inv(k)).trunc(k)) * CInv<2, P>;
182     }
183     return x.trunc(m);
184 }
185 constexpr Poly mult(Poly b) const {
186     if (b.size() == 0) {

```

```

187         return Poly();
188     }
189     int n = b.size();
190     reverse(b.begin(), b.end());
191     return ((*this) * b).shift(-(n - 1));
192 }
193 constexpr vector<Value> eval(vector<Value> x) const {
194     if (this->size() == 0) {
195         return vector<Value>(x.size(), 0);
196     }
197     const int n = max(x.size(), this->size());
198     vector<Poly> q(4 * n);
199     vector<Value> ans(x.size());
200     x.resize(n);
201     function<void(int, int, int)> build = [&](int p, int l, int r) {
202         if (r - l == 1) {
203             q[p] = Poly{1, -x[l]};
204         } else {
205             int m = (l + r) / 2;
206             build(2 * p, l, m);
207             build(2 * p + 1, m, r);
208             q[p] = q[2 * p] * q[2 * p + 1];
209         }
210     };
211     build(1, 0, n);
212     function<void(int, int, int, const Poly &)> work = [&](int p, int l, int r,
213                                                         const Poly &num) {
214         if (r - l == 1) {
215             if (1 < int(ans.size())) {
216                 ans[l] = num[0];
217             }
218         } else {
219             int m = (l + r) / 2;
220             work(2 * p, l, m, num.mulT(q[2 * p + 1]).resize(m - 1));
221             work(2 * p + 1, m, r, num.mulT(q[2 * p]).resize(r - m));
222         }
223     };
224     work(1, 0, n, mulT(q[1].inv(n)));
225     return ans;
226 }
227 };

```

2.3.6 常用结论

```

1  ### 常用结论
2
3  #### 杂
4
5  - 求  $\displaystyle B_i = \sum_{k=i}^n C_k \cdot A_k$ , 即  $\displaystyle B_i = \frac{1}{i!} \sum_{k=i}^n \frac{1}{(k-i)!} \cdot k! A_k$ , 反转后卷积。
6  - NTT中,  $\omega_n = e^{-\frac{2\pi i}{n}}$ 。
7  - 遇到  $\displaystyle \sum_{i=0}^n n[i \% k = 0] f(i)$  可以转换为  $\displaystyle \sum_{i=0}^n$ 

```

```

n\frac 1 k\sum_{j=0}^{k-1}(\omega_k^i)^{jf(i)}$。(单位根卷积)
8 - 广义二项式定理  $(1+x)^\alpha=\sum_{i=0}^{\infty}\binom{\alpha}{i}x^i$ 。
9
10 ##### 普通生成函数 / OGF
11
12 - 普通生成函数:  $A(x)=a_0+a_1x+a_2x^2+\dots=\langle a_0,a_1,a_2,\dots\rangle$ ;
13 -  $1+x^k+x^{2k}+\dots=\frac{1}{1-x^k}$ ;
14 - 取对数后  $-\ln(1-x^k)=\sum_{i=1}^{\infty}\frac{1}{i}x^{ki}$  即  $\sum_{i=1}^{\infty}\frac{1}{i}x^i\otimes x^k$  (polymul_special);
15 -  $x+\frac{x^2}{2}+\frac{x^3}{3}+\dots=-\ln(1-x)$ ;
16 -  $1+x+x^2+\dots+x^{m-1}=\frac{1-x^m}{1-x}$ ;
17 -  $1+2x+3x^2+\dots=\frac{1}{(1-x)^2}$  (借用导数,  $nx^{n-1}=(x^n)'$ );
18 -  $C_m^0+C_m^1x+C_m^2x^2+\dots+C_m^mx^m=(1+x)^m$  (二项式定理);
19 -  $C_m^0+C_{m+1}^1x+C_{m+2}^2x^2+\dots=\frac{1}{(1-x)^{m+1}}$  (归纳法证明);
20 -  $\sum_{n=0}^{\infty}F_nx^n=\frac{(F_1-F_0)x+F_0}{1-x-x^2}$  (F 为斐波那契数列, 列方程  $G(x)=xG(x)+x^2G(x)+(F_1-F_0)x+F_0$ );
21 -  $\sum_{n=0}^{\infty}H_nx^n=\frac{1-\sqrt{1-4x}}{2x}$  (H 为卡特兰数);
22 - 前缀和  $\sum_{n=0}^{\infty}s_nx^n=\frac{1}{1-x}f(x)$ ;
23 - 五边形数定理:  $\prod_{i=1}^{\infty}(1-x^i)=\sum_{k=0}^{\infty}(-1)^kx^{\frac{1}{2}k(3k\pm 1)}$ 。
24
25 ##### 指数生成函数 / EGF
26
27 - 指数生成函数:  $A(x)=a_0+a_1x+a_2\frac{x^2}{2!}+a_3\frac{x^3}{3!}+\dots=\langle a_0,a_1,a_2,a_3,\dots\rangle$ ;
28 - 普通生成函数转换为指数生成函数: 系数乘以  $n!$ ;
29 -  $1+x+\frac{x^2}{2!}+\frac{x^3}{3!}+\dots=\exp x$ ;
30 - 长度为  $n$  的循环置换数为  $P(x)=-\ln(1-x)$ , 长度为  $n$  的置换数为  $\exp P(x)=\frac{1}{1-x}$  (注意是**指数**生成函数)
31 -  $n$  个点的生成树个数是  $P(x)=\sum_{n=1}^{\infty}n^{n-2}\frac{x^n}{n!}$ ,  $n$  个点的生成森林个数是  $\exp P(x)$ ;
32 -  $n$  个点的无向连通图个数是  $P(x)$ ,  $n$  个点的无向图个数是  $\exp P(x)=\sum_{n=0}^{\infty}2^{\frac{1}{2}n(n-1)}\frac{x^n}{n!}$ ;
33 - 长度为  $n$  的循环置换数是  $P(x)=-\ln(1-x)-x$ , 长度为  $n$  的错排数是  $\exp P(x)$ 。
34
35 <div style="page-break-after:always">/END/</div>

```

2.3.7 快速傅里叶变换 FFT

$\mathcal{O}(N \log N)$ 。

```

1 struct Polynomial {
2     constexpr static double PI = acos(-1);
3     struct Complex {
4         double x, y;
5         Complex(double _x = 0.0, double _y = 0.0) {
6             x = _x;
7             y = _y;
8         }
9         Complex operator-(const Complex &rhs) const {

```

```

10         return Complex(x - rhs.x, y - rhs.y);
11     }
12     Complex operator+(const Complex &rhs) const {
13         return Complex(x + rhs.x, y + rhs.y);
14     }
15     Complex operator*(const Complex &rhs) const {
16         return Complex(x * rhs.x - y * rhs.y, x * rhs.y + y * rhs.x);
17     }
18 };
19 vector<Complex> c;
20 Polynomial(vector<int> &a) {
21     int n = a.size();
22     c.resize(n);
23     for (int i = 0; i < n; i++) {
24         c[i] = Complex(a[i], 0);
25     }
26     fft(c, n, 1);
27 }
28 void change(vector<Complex> &a, int n) {
29     for (int i = 1, j = n / 2; i < n - 1; i++) {
30         if (i < j) swap(a[i], a[j]);
31         int k = n / 2;
32         while (j >= k) {
33             j -= k;
34             k /= 2;
35         }
36         if (j < k) j += k;
37     }
38 }
39 void fft(vector<Complex> &a, int n, int opt) {
40     change(a, n);
41     for (int h = 2; h <= n; h *= 2) {
42         Complex wn(cos(2 * PI / h), sin(opt * 2 * PI / h));
43         for (int j = 0; j < n; j += h) {
44             Complex w(1, 0);
45             for (int k = j; k < j + h / 2; k++) {
46                 Complex u = a[k];
47                 Complex t = w * a[k + h / 2];
48                 a[k] = u + t;
49                 a[k + h / 2] = u - t;
50                 w = w * wn;
51             }
52         }
53     }
54     if (opt == -1) {
55         for (int i = 0; i < n; i++) {
56             a[i].x /= n;
57         }
58     }
59 }
60 };

```

2.3.8 快速数论变换 NTT

$\mathcal{O}(N \log N)$ 。

```

1 struct Polynomial {
2     vector<Z> z;
3     vector<int> r;
4     Polynomial(vector<int> &a) {
5         int n = a.size();
6         z.resize(n);
7         r.resize(n);
8         for (int i = 0; i < n; i++) {
9             z[i] = a[i];
10            r[i] = (i & 1) * (n / 2) + r[i / 2] / 2;
11        }
12        ntt(z, n, 1);
13    }
14    LL power(LL a, int b) {
15        LL res = 1;
16        for (; b; b /= 2, a = a * a % mod) {
17            if (b % 2) {
18                res = res * a % mod;
19            }
20        }
21        return res;
22    }
23    void ntt(vector<Z> &a, int n, int opt) {
24        for (int i = 0; i < n; i++) {
25            if (r[i] < i) {
26                swap(a[i], a[r[i]]);
27            }
28        }
29        for (int k = 2; k <= n; k *= 2) {
30            Z gn = power(3, (mod - 1) / k);
31            for (int i = 0; i < n; i += k) {
32                Z g = 1;
33                for (int j = 0; j < k / 2; j++, g *= gn) {
34                    Z t = a[i + j + k / 2] * g;
35                    a[i + j + k / 2] = a[i + j] - t;
36                    a[i + j] = a[i + j] + t;
37                }
38            }
39        }
40        if (opt == -1) {
41            reverse(a.begin() + 1, a.end());
42            Z inv = power(n, mod - 2);
43            for (int i = 0; i < n; i++) {
44                a[i] *= inv;
45            }
46        }
47    }
48 };

```


2.3.9 拉格朗日插值

$n + 1$ 个点可以唯一确定一个最高为 n 次的多项式。普通情况: $f(k) = \sum_{i=1}^{n+1} y_i \prod_{i \neq j} \frac{k - x[j]}{x[i] - x[j]}$ 。

```

1 struct Lagrange {
2     int n;
3     vector<Z> x, y, fac, invfac;
4     Lagrange(int n) {
5         this->n = n;
6         x.resize(n + 3);
7         y.resize(n + 3);
8         fac.resize(n + 3);
9         invfac.resize(n + 3);
10        init(n);
11    }
12    void init(int n) {
13        iota(x.begin(), x.end(), 0);
14        for (int i = 1; i <= n + 2; i++) {
15            Z t;
16            y[i] = y[i - 1] + t.power(i, n);
17        }
18        fac[0] = 1;
19        for (int i = 1; i <= n + 2; i++) {
20            fac[i] = fac[i - 1] * i;
21        }
22        invfac[n + 2] = fac[n + 2].inv();
23        for (int i = n + 1; i >= 0; i--) {
24            invfac[i] = invfac[i + 1] * (i + 1);
25        }
26    }
27    Z solve(LL k) {
28        if (k <= n + 2) {
29            return y[k];
30        }
31        vector<Z> sub(n + 3);
32        for (int i = 1; i <= n + 2; i++) {
33            sub[i] = k - x[i];
34        }
35        vector<Z> mul(n + 3);
36        mul[0] = 1;
37        for (int i = 1; i <= n + 2; i++) {
38            mul[i] = mul[i - 1] * sub[i];
39        }
40        Z ans = 0;
41        for (int i = 1; i <= n + 2; i++) {
42            ans = ans + y[i] * mul[n + 2] * sub[i].inv() * pow(-1, n + 2 - i) * invfac
                [i - 1] *
43                invfac[n + 2 - i];
44        }
45        return ans;
46    }
47 };

```

2.3.10 离散傅里叶变换 dft 与其逆变换 idft

```

1 vector<int> rev;
2 template<int P> vector<MInt<P>> roots{0, 1};
3
4 template<int P> constexpr MInt<P> findPrimitiveRoot() {
5     MInt<P> i = 2;
6     int k = __builtin_ctz(P - 1);
7     while (true) {
8         if (power(i, (P - 1) / 2) != 1) {
9             break;
10        }
11        i += 1;
12    }
13    return power(i, (P - 1) >> k);
14 }
15
16 template<int P> constexpr MInt<P> primitiveRoot = findPrimitiveRoot<P>();
17 template<> constexpr MInt<998244353> primitiveRoot<998244353>{31};
18
19 template<int P> constexpr void dft(vector<MInt<P>> &a) { // 离散傅里叶变换
20     int n = a.size();
21
22     if (int(rev.size()) != n) {
23         int k = __builtin_ctz(n) - 1;
24         rev.resize(n);
25         for (int i = 0; i < n; i++) {
26             rev[i] = rev[i >> 1] >> 1 | (i & 1) << k;
27         }
28     }
29
30     for (int i = 0; i < n; i++) {
31         if (rev[i] < i) {
32             swap(a[i], a[rev[i]]);
33         }
34     }
35     if (roots<P>.size() < n) {
36         int k = __builtin_ctz(roots<P>.size());
37         roots<P>.resize(n);
38         while ((1 << k) < n) {
39             auto e = power(primitiveRoot<P>, 1 << (__builtin_ctz(P - 1) - k - 1));
40             for (int i = 1 << (k - 1); i < (1 << k); i++) {
41                 roots<P>[2 * i] = roots<P>[i];
42                 roots<P>[2 * i + 1] = roots<P>[i] * e;
43             }
44             k++;
45         }
46     }
47     for (int k = 1; k < n; k *= 2) {
48         for (int i = 0; i < n; i += 2 * k) {
49             for (int j = 0; j < k; j++) {
50                 MInt<P> u = a[i + j];
51                 MInt<P> v = a[i + j + k] * roots<P>[k + j];

```

```

52         a[i + j] = u + v;
53         a[i + j + k] = u - v;
54     }
55 }
56 }
57 }
58 template<int P> constexpr void idft(vector<MInt<P>> &a) { // 逆变换
59     int n = a.size();
60     reverse(a.begin() + 1, a.end());
61     dft(a);
62     MInt<P> inv = (1 - P) / n;
63     for (int i = 0; i < n; i++) {
64         a[i] *= inv;
65     }
66 }

```

2.4 矩阵

2.4.1 矩阵模板

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  class Martix {
4  public:
5  #define gx int
6      int row, col;
7      vector<vector<gx>> > data;
8      Martix(int row, int col) {
9          this->row = row, this->col = col, data.resize(row);
10         for (int i = 0; i < row; i++)
11             data[i].resize(col);
12     }
13     Martix mul(Martix rhs, int mod = 1e9 + 7) { // 矩阵乘法
14         Martix res(row, rhs.col);
15         for (int i = 0; i < row; i++)
16             for (int j = 0; j < rhs.col; j++)
17                 for (int k = 0; k < col; k++)
18                     res.data[i][j] = (res.data[i][j] + data[i][k] * rhs.data[k][j]) %
19                                     mod;
20         return res;
21     }
22     void unit() { // 单位矩阵
23         for (int i = 0; i < row; i++)
24             for (int j = 0; j < col; j++)
25                 if (i == j) data[i][j] = 1;
26     }
27     Martix qpow(int mi, int mod = 1e9 + 7) const { // 矩阵快速幂
28         Martix res(row, col), tmp = *this;
29         res.unit();
30         while (mi) {
31             if (mi & 1)

```

```

32         res = res.mul(tmp, mod);
33         tmp = tmp.mul(tmp, mod);
34         mi >>= 1;
35     }
36     return res;
37 }
38
39 void print() const {
40     for (int i = 0; i < row; i++)
41         for (int j = 0; j < col; j++)
42             cout << data[i][j] << " \n"[j == col - 1];
43 }
44
45 void fill(gx x) {
46     for (int i = 0; i < row; i++)
47         for (int j = 0; j < col; j++)
48             data[i][j] = x;
49 }
50 };

```

2.4.2 矩阵快速幂 (推荐使用数论-矩阵)

```

1  // -----
2  // 矩阵快速幂  $O(n^3 \log k)$ 
3  // -----
4  #include <bits/stdc++.h>
5  using namespace std;
6  using ll = long long;
7  const int N = 505;
8  const ll MOD = 1000000007;
9
10 template<class T>
11 void matmul(T A[][N], T B[][N], T C[][N], int n) { //C=A*B, 对MOD取模
12     static T tmp[N][N];
13     for(int i=0;i<n;i++) for(int j=0;j<n;j++) tmp[i][j]=0;
14     for(int i=0;i<n;i++){
15         for(int k=0;k<n;k++) if(A[i][k]){
16             ll v = A[i][k];
17             for(int j=0;j<n;j++){
18                 tmp[i][j] = (tmp[i][j] + v * B[k][j]) % MOD;
19             }
20         }
21     }
22     for(int i=0;i<n;i++) for(int j=0;j<n;j++) C[i][j] = tmp[i][j];
23 }
24
25 template<class T>
26 void matpow(T a[][N], int n, int k){ //原地矩阵快速幂: 把a改写为  $a^k$ 
27     static T ans[N][N];
28     for(int i=0;i<n;i++){
29         for(int j=0;j<n;j++){
30             ans[i][j] = (i==j);

```

```

31     }
32 }
33 for(; k; k >>= 1){
34     if(k & 1) matmul(ans, a, ans, n);
35     matmul(a, a, a, n);
36 }
37 for(int i=0;i<n;i++){
38     for(int j=0;j<n;j++){
39         a[i][j] = ans[i][j];
40     }
41 }
42 }
43
44 int main(){
45     ios::sync_with_stdio(false);
46     cin.tie(nullptr);
47
48     int n, k; cin >> n >> k;
49     static ll a[N][N];
50
51     for(int i = 0; i < n; i++){ //读入
52         for(int j = 0; j < n; j++){cin >> a[i][j]; a[i][j] %= MOD;}
53
54     matpow(a, n, k); //a 就地变成 a^k
55
56     return 0;
57 }

```

2.4.3 位运算——线性基

```

1  class XorBasis {
2      vector<int> b;
3
4  public:
5      // n 为值域最大值 U 的二进制长度，例如 U=1e9 时 n=30
6      XorBasis(int n) : b(n) {}
7
8      void insert(int x) {
9          // 从高到低遍历，保证计算 max_xor 的时候，参与 XOR 的基的最高位（或者说二进制长度）
          // 是互不相同的
10         for (int i = b.size() - 1; i >= 0; i--) {
11             if (x >> i) { // 由于大于 i 的位都被我们异或成了 0，所以 x >> i 的结果只能是 0
                // 或 1
12                 if (b[i] == 0) { // x 和之前的基是线性无关的
13                     b[i] = x; // 新增一个基，最高位为 i
14                     return;
15                 }
16                 x ^= b[i]; // 保证每个基的二进制长度互不相同
17             }
18         }
19         // 正常循环结束，此时 x=0，说明一开始的 x 可以被已有基表出，不是一个线性无关基
20     }

```

```

21
22     int max_xor() {
23         int res = 0;
24         // 从高到低贪心：越高的位，越必须是 1
25         // 由于每个位的基至多一个，所以每个位只需考虑异或一个基，若能变大，则异或之
26         for (int i = b.size() - 1; i >= 0; i--) {
27             res = max(res, res ^ b[i]);
28         }
29         return res;
30     }
31 };
32
33 作者：灵茶山艾府
34 链接：https://leetcode.cn/discuss/post/dHn9Vk/
35 来源：力扣（LeetCode）
36 著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

```

2.4.4 矩阵加速

```

1  const int mod = 1e9 + 7;
2  LL T, n, t[5][5], a[5][5], b[5][5];
3  void matrixQp(LL y){
4      while (y){
5          if (y & 1){
6              memset(t, 0, sizeof t);
7              for (int i = 1; i <= 3; i ++ )
8                  for (int j = 1; j <= 1; j ++ )
9                      for (int k = 1; k <= 3; k ++ )
10                         t[i][j] = ( t[i][j] + (a[i][k] * b[k][j]) % mod ) % mod;
11              memcpy(b, t, sizeof t);
12          }
13          y >>= 1;
14          memset(t, 0, sizeof t);
15          for (int i = 1; i <= 3; i ++ )
16              for (int j = 1; j <= 3; j ++ )
17                  for (int k = 1; k <= 3; k ++ )
18                     t[i][j] = ( t[i][j] + (a[i][k] * a[k][j]) % mod ) % mod;
19          memcpy(a, t, sizeof t);
20      }
21 }
22 void init(){
23     b[1][1] = b[2][1] = b[3][1] = 1;
24     memset(a, 0, sizeof a);
25     a[1][1] = a[2][1] = a[1][3] = a[3][2] = 1;
26 }
27 void solve(){
28     cin >> n;
29     if (n <= 3) cout << "1\n";
30     else{
31         init();
32         matrixQp(n - 3);
33         cout << b[1][1] << "\n";

```

```

34     }
35 }
36 int main(){
37     cin >> T;
38     while ( T -- )
39         solve();
40     return 0;
41 }

```

2.4.5 矩阵四则运算

封装来自。矩阵乘法复杂度 $\mathcal{O}(N^3)$ 。

```

1  const int SIZE = 2;
2  struct Matrix {
3      ll M[SIZE + 5][SIZE + 5];
4      void clear() { memset(M, 0, sizeof(M)); }
5      void reset() { //初始化
6          clear();
7          for (int i = 1; i <= SIZE; ++i) M[i][i] = 1;
8      }
9      Matrix friend operator*(const Matrix &A, const Matrix &B) {
10         Matrix Ans;
11         Ans.clear();
12         for (int i = 1; i <= SIZE; ++i)
13             for (int j = 1; j <= SIZE; ++j)
14                 for (int k = 1; k <= SIZE; ++k)
15                     Ans.M[i][j] = (Ans.M[i][j] + A.M[i][k] * B.M[k][j]) % mod;
16         return Ans;
17     }
18     Matrix friend operator+(const Matrix &A, const Matrix &B) {
19         Matrix Ans;
20         Ans.clear();
21         for (int i = 1; i <= SIZE; ++i)
22             for (int j = 1; j <= SIZE; ++j)
23                 Ans.M[i][j] = (A.M[i][j] + B.M[i][j]) % mod;
24         return Ans;
25     }
26 };
27
28 inline int mypow(LL n, LL k, int p = MOD) {
29     LL r = 1;
30     for (; k >>= 1, n = n * n % p) {
31         if (k & 1) r = r * n % p;
32     }
33     return r;
34 }
35 bool ok = 1;
36 Matrix getinv(Matrix a) { //矩阵求逆
37     int n = SIZE, m = SIZE * 2;
38     for (int i = 1; i <= n; i++) a.M[i][i + n] = 1;
39     for (int i = 1; i <= n; i++) {
40         int pos = i;

```

```

41     for (int j = i + 1; j <= n; j++)
42         if (abs(a.M[j][i]) > abs(a.M[pos][i])) pos = j;
43     if (i != pos) swap(a.M[i], a.M[pos]);
44     if (!a.M[i][i]) {
45         puts("No Solution");
46         ok = 0;
47     }
48     ll inv = q_pow(a.M[i][i], mod - 2);
49     for (int j = 1; j <= n; j++)
50         if (j != i) {
51             ll mul = a.M[j][i] * inv % mod;
52             for (int k = i; k <= m; k++)
53                 a.M[j][k] = ((a.M[j][k] - a.M[i][k] * mul) % mod + mod) % mod;
54         }
55     for (int j = 1; j <= m; j++) a.M[i][j] = a.M[i][j] * inv % mod;
56 }
57 Matrix res;
58 res.clear();
59 for (int i = 1; i <= n; i++)
60     for (int j = 1; j <= m; j++)
61         res.M[i][j] = a.M[i][n + j];
62 return res;
63 }

```

2.4.6 矩阵快速幂

以 $\mathcal{O}(N^3 \log M)$ 的复杂度计算。

```

1  const int N = 110, mod = 1e9 + 7;
2  LL n, k, a[N][N], b[N][N], t[N][N];
3  void matrixQp(LL y){
4      while (y){
5          if (y & 1){
6              memset(t, 0, sizeof t);
7              for (int i = 1; i <= n; i++)
8                  for (int j = 1; j <= n; j++)
9                      for (int k = 1; k <= n; k++)
10                         t[i][j] = ( t[i][j] + (a[i][k] * b[k][j]) % mod ) % mod;
11              memcpy(b, t, sizeof t);
12          }
13          y >>= 1;
14          memset(t, 0, sizeof t);
15          for (int i = 1; i <= n; i++)
16              for (int j = 1; j <= n; j++)
17                  for (int k = 1; k <= n; k++)
18                     t[i][j] = ( t[i][j] + (a[i][k] * a[k][j]) % mod ) % mod;
19          memcpy(a, t, sizeof t);
20      }
21 }
22 int main(){
23     cin >> n >> k;
24     for (int i = 1; i <= n; i++)
25         for (int j = 1; j <= n; j++) {

```



```

26         cin >> b[i][j];
27         a[i][j] = b[i][j];
28     }
29     matrixQp(k - 1);
30     for (int i = 1; i <= n; i++)
31         for (int j = 1; j <= n; j++)
32             cout << b[i][j] << " \n"[j == n];
33     return 0;
34 }

```

2.5 博弈论

2.5.1 Anti-Nim 游戏 (反 Nim 游戏)

```

1  ### Anti-Nim 游戏 (反 Nim 游戏)
2
3  > 有  $N$  堆石子，给出每一堆的石子数量，两名玩家轮流行动，按以下规则取石子：
4  >
5  > 规定：每人每次任选一堆，取走正整数颗石子，拿到最后一颗石子的一方**出局**；
6  >
7  > 双方均采用最优策略，询问谁会获胜。
8
9  - 所有堆的石头数量均不超过  $1$ ，且  $\sum N$  为偶数（也可看作“且有偶数堆”）；
10 - 至少有一堆的石头数量大于  $1$ ，且  $\sum N \neq 0$ 。

```

2.5.2 Anti-SG 游戏 (反 SG 游戏)

```

1  ### Anti-SG 游戏 (反 SG 游戏)
2
3  SG 游戏中最先不能行动的一方获胜。
4
5  **以下局面先手必胜：**
6
7  - **单局游戏的SG值均不超过  $1$ ，且总SG值为  $0$ ；
8  - **至少有一局单局游戏的SG值大于  $1$ ，且总SG值不为  $0$ 。
9
10 在本质上，这与 Anti-Nim 游戏的结论一致。

```

2.5.3 Every-SG 游戏

```

1  ### Every-SG 游戏
2
3  > 给出一个有向无环图，其中  $K$  个顶点上放置了石子，两名玩家轮流行动，按以下规则操作石子：
4  >
5  > 移动图上所有还能够移动的石子；
6  >
7  > 无法移动石子的一方出局。双方均采用最优策略，询问谁会获胜。
8
9 定义  $step$  为某一局游戏至多需要经过的回合数。
10
11 **以下局面先手必胜： $step$  为奇数** 。

```

2.5.4 Lasker' s-Nim 游戏 (Multi-SG 游戏)

```

1  ### Lasker' s-Nim 游戏 ( Multi-SG 游戏)
2
3  > 有 $N$ 堆石子, 给出每一堆的石子数量, 两名玩家轮流行动, 每人每次任选以下规定的一种操作石
   子:
4  >
5  > - 任选一堆, 取走正整数颗石子;
6  > - 任选数量大于 $2$ 的一堆, 分成两堆非空石子。
7  >
8  > 拿到最后一颗石子的一方获胜。双方均采用最优策略, 询问谁会获胜。
9
10 **本题使用SG函数求解, SG值定义为: **
11
12 $$\pmb{ SG(x) =
13 \begin{cases}
14 x-1 & \text{, } x \bmod 4 = 0 \\
15 x & \text{, } x \bmod 4 = 1 \\
16 x & \text{, } x \bmod 4 = 2 \\
17 x+1 & \text{, } x \bmod 4 = 3
18 \end{cases} }$$

```

2.5.5 Moore' s Nim 游戏 (Nim-K 游戏)

```

1  ### Moore' s Nim 游戏 (Nim-K 游戏)
2
3  > 有 $N$ 堆石子, 给出每一堆的石子数量, 两名玩家轮流行动, 按以下规则取石子:
4  >
5  > 规定: 每人每次任选不超过 $K$ 堆, 对每堆都取走不同的正整数颗石子, 拿到最后一颗石子的一方获
   胜。
6  >
7  > 双方均采用最优策略, 询问谁会获胜。
8
9  把每一堆石子的石子数用二进制表示, 定义 $one_i$ 为二进制第 $i$ 位上 $1$ 的个数。
10
11 **以下局面先手必胜: **
12
13 **对于每一位, ** $\pmb{one_1, one_2, \dots, one_N}$ **均不为** $\pmb{K+1}$ **的倍数。 **

```

2.5.6 Nim 博弈

```

1  ### Nim 博弈
2
3  > 有 $N$ 堆石子, 给出每一堆的石子数量, 两名玩家轮流行动, 按以下规则取石子:
4  >
5  > 规定: 每人每次任选一堆, 取走正整数颗石子, 拿到最后一颗石子的一方获胜 (注: 几个特点是**不能
   跨堆**、**不能不拿**)。
6  >
7  > 双方均采用最优策略, 询问谁会获胜。
8
9  记初始时各堆石子的数量 $(a_1, a_2, \dots, a_n)$, 定义尼姆和 $Sum_N = a_1 \oplus a_2 \oplus \dots \oplus a_n$。

```

```

10
11 **当** $\pmb{Sum\_N = 0}$ **时先手必败，反之先手必胜。**

```

2.5.7 Nim 游戏具体取法

```

1 ### Nim 游戏具体取法
2
3 先计算出尼姆和，再对每一堆石子计算  $A_i \oplus Sum\_N$ ，记为  $X_i$ 。
4
5 若得到的值  $X_i < A_i$ ， $X_i$  即为一个可行解，即**剩下**  $X_i$  **颗石头，取走**  $A_i - X_i$  **颗石头**（这里取小于号是因为至少要取走 1 颗石子）。

```

2.5.8 SG 游戏（有向图游戏）

我们使用以下几条规则来定义暴力求解的过程：

- 使用数字来表示输赢情况，0 代表局面必败，非 0 代表**存在必胜可能**，我们称这个数字为这个局面的 SG 值；
- 找到最终态，根据题意人为定义最终态的输赢情况；
- 对于非最终态的某个节点，其 SG 值为所有子节点的 SG 值取 mex；
- 单个游戏的输赢态即对应根节点的 SG 值是否为 0，为 0 代表先手必败，非 0 代表先手必胜；
- 多个游戏的总 SG 值为单个游戏 SG 值的异或和。

使用哈希表，以 $\mathcal{O}(N + M)$ 的复杂度计算。

```

1 int n, m, a[N], num[N];
2 int sg(int x) {
3     if (num[x] != -1) return num[x];
4
5     unordered_set<int> S;
6     for (int i = 1; i <= m; ++ i)
7         if (x >= a[i])
8             S.insert(sg(x - a[i]));
9
10    for (int i = 0; ; ++ i)
11        if (S.count(i) == 0)
12            return num[x] = i;
13 }
14 void Solve() {
15     cin >> m;
16     for (int i = 1; i <= m; ++ i) cin >> a[i];
17     cin >> n;
18
19     int ans = 0; memset(num, -1, sizeof num);
20     for (int i = 1; i <= n; ++ i) {
21         int x; cin >> x;

```

```

22     ans ^= sg(x);
23 }
24
25 if (ans == 0) no;
26 else yes;
27 }

```

2.5.9 威佐夫博弈

有两堆石子，给出每一堆的石子数量，两名玩家轮流行动，每人每次任选以下规定的一种操作石子：- 任选一堆，取走正整数颗石子；- 从两队中同时取走正整数颗石子。拿到最后一颗石子的一方获胜。双方均采用最优策略，询问谁会获胜。

以下局面先手必败：

$(1, 2), (3, 5), (4, 7), (6, 10), \dots$ 具体而言，每一对的第一个数为此前没出现过的最小整数，第二个数为第一个数加上 $1, 2, 3, 4, \dots$ 。

更一般地，对于第 k 对数，第一个数为 $First_k = \left\lfloor \frac{k*(1+\sqrt{5})}{2} \right\rfloor$ ，第二个数为 $Second_k = First_k + k$ 。

其中，在两堆石子的数量均大于 10^9 次时，由于需要使用高精度计算，我们需要人为定义 $\frac{1+\sqrt{5}}{2}$ 的取值为 $lorry = 1.618033988749894848204586834$ 。

```

1 const double lorry = (sqrt(5.0) + 1.0) / 2.0;
2 //const double lorry = 1.618033988749894848204586834;
3 void Solve() {
4     int n, m; cin >> n >> m;
5     if (n < m) swap(n, m);
6     double x = n - m;
7     if ((int)(lorry * x) == m) cout << "lose\n";
8     else cout << "win\n";
9 }

```

2.5.10 巴什博弈

```

1 ### 巴什博弈
2
3 > 有 $N$ 个石子，两名玩家轮流行动，按以下规则取石子：
4 >
5 > 规定：每人每次可以取走 $X(1 \leq X \leq M)$ 个石子，拿到最后一颗石子的一方获胜。
6 >
7 > 双方均采用最优策略，询问谁会获胜。
8
9 > 两名玩家轮流报数。
10 >
11 > 规定：第一个报数的人可以报 $X(1 \leq X \leq M)$，后报数的人需要比前者所报数大 $Y(1 \leq Y \leq M)$，率先报到 $N$ 的人获胜。
12 >
13 > 双方均采用最优策略，询问谁会获胜。
14

```

- 15 - $N=K \cdot (M+1)$ (其中 $K \in \mathbb{N}^+$) , 后手必胜 (后手可以控制每一回合结束时双方恰好取走 $M+1$ 个, 重复 K 轮后即胜利);
- 16 - $N=K \cdot (M+1)+R$ (其中 $K \in \mathbb{N}^+, 0 < R < M+1$) , 先手必胜 (先手先取走 R 个, 之后控制每一回合结束时双方恰好取走 $M+1$ 个, 重复 K 轮后即胜利)。

2.5.11 扩展巴什博弈

- ```
1 ### 扩展巴什博弈
2
3 > 有 N 颗石子, 两名玩家轮流行动, 按以下规则取石子:。
4 >
5 > 规定: 每人每次可以取走 $X(a \leq X \leq b)$ 个石子, 如果最后剩余物品的数量小于 a 个, 则不能再取, 拿到最后一颗石子的一方获胜。
6 >
7 > 双方均采用最优策略, 询问谁会获胜。
8
9 - $N = K \cdot (a+b)$ 时, 后手必胜;
10 - $N = K \cdot (a+b)+R_1$ (其中 $K \in \mathbb{N}^+, 0 < R_1 < a$) 时, 后手必胜 (这些数量不够再取一次, 先手无法逆转局面);
11 - $N = K \cdot (a+b)+R_2$ (其中 $K \in \mathbb{N}^+, a \leq R_2 \leq b$) 时, 先手必胜;
12 - $N = K \cdot (a+b)+R_3$ (其中 $K \in \mathbb{N}^+, b < R_3 < a+b$) 时, 先手必胜 (这些数量不够再取一次, 后手无法逆转局面);
```

### 2.5.12 斐波那契博弈

有一堆石子, 数量为  $N$ , 两名玩家轮流行动, 按以下规则取石子: 先手第 1 次可以取任意多颗, 但不能全部取完, 此后每人取的石子数不能超过上个人的两倍, 拿到最后一颗石子的一方获胜。双方均采用最优策略, 询问谁会获胜。

当且仅当  $N$  为斐波那契数时先手必败。

```
1 int fib[100] = {1, 2};
2 map<int, bool> mp;
3 void Force() {
4 for (int i = 2; i <= 86; ++ i) fib[i] = fib[i - 1] + fib[i - 2];
5 for (int i = 0; i <= 86; ++ i) mp[fib[i]] = 1;
6 }
7 void Solve() {
8 int n; cin >> n;
9 if (mp[n] == 1) cout << "lose\n";
10 else cout << "win\n";
11 }
```

### 2.5.13 无向图删边游戏 (Fusion Principle 定理)

- ```
1  ### 无向图删边游戏 (Fusion Principle 定理)
2
3  > 给出一张  $N$  个节点的无向联通图, 有一个点作为图的根, 两名玩家轮流行动, 按以下规则操作:
4  >
5  > 选择任意一条边删除, 不与根相连的部分会同步被删去;
```

```

6 >
7 > 删掉最后一条边的一方获胜。双方均采用最优策略，询问谁会获胜。
8
9 - **对于奇环，我们将其缩成一个新点+一条新边；**
10 - **对于偶环，我们将其缩成一个新点；**
11 - **所有连接到原来环上的边全部与新点相连。**
12
13 此时，本模型转化为“树上删边游戏”。
14
15 <div style="page-break-after:always">/END/</div>

```

2.5.14 树上删边游戏

给出一棵 N 个节点的有根树，两名玩家轮流行动，按以下规则操作：选择任意一棵子树并删除（即删去任意一条边，不与根相连的部分会同步被删去）；删掉最后一棵子树的一方获胜（换句话说，删去根节点的一方失败）。双方均采用最优策略，询问谁会获胜。

结论：当根节点 SG 值非 1 时先手必胜。

相较于传统 SG 值的定义，本题的 SG 函数值定义为：

- 叶子节点的 SG 值为 0。
- 非叶子节点的 SG 值为其所有孩子节点 SG 值 +1 的异或和。

```

1 auto dfs = [&](auto self, int x, int fa) -> int {
2     int x = 0;
3     for (auto y : ver[x]) {
4         if (y == fa) continue;
5         x ^= self(self, y, x);
6     }
7     return x + 1;
8 };
9 cout << (dfs(dfs, 1, 0) == 1 ? "Bob\n" : "Alice\n");

```

2.5.15 阶梯 - Nim 博弈

```

1 ### 阶梯 - Nim 博弈
2
3 > 有  $N$  级台阶，每一级台阶上均有一定数量的石子，给出每一级石子的数量，两名玩家轮流行动，按
   以下规则操作石子：
4 >
5 > 规定：每人每次任选一级台阶，拿走正整数颗石子放到下一级台阶中，已经拿到地面上的石子不能再
   拿，拿到最后一颗石子的一方获胜。
6 >
7 > 双方均采用最优策略，询问谁会获胜。
8
9 **对奇数台阶做传统  $\text{Nim}$  博弈，当  $\sum_{N=0}$  时先手必败，反之先
   手必胜。**

```

2.6 动态规划

2.6.1 打家劫舍

```

1  选择任意一个 nums[i] , 删除它并获得 nums[i] 的点数。之后, 你必须删除 所有 等于 nums[i] -
   1 和 nums[i] + 1 的元素。开始你拥有 0 个点数。返回你能通过这些操作获得的最大点数。
2  class Solution {
3  public:
4      int deleteAndEarn(vector<int>& nums) {
5          vector<long long>a(10005,0);
6          for(auto x:nums) a[x]+=x;
7          int n = 10001;
8          vector<int>dp(n+5,0);
9          for(int i=1;i<n;i++){
10             if(i<=1) dp[i] = a[i];
11             else if(i==2) dp[i] = dp[0] + a[i];
12             else dp[i] = max(dp[i-2],dp[i-3])+a[i];
13         }
14         if(n==1) return dp[0];
15         return max(dp[n-1],dp[n-2]);
16     }
17 };

```

2.6.2 最长上升子序列 + 最长公共递增子序列 (LCIS) +LCS

```

1  // -----
2  // LIS
3  // -----
4  //1. 贪心
5  #include <bits/stdc++.h>
6  // #define int long long //(可选: 如果需要更大的整数范围)
7  #define PII pair<int,int>
8  #define endl '\n'
9  #define LL __int128
10
11 using namespace std;
12 const int N=1e5+10,M=1e3+10,mod=998244353,INF=0x3f3f3f3f;
13
14 int a[N],b[N],c[N],pre[N],f[N];
15 string ans[N];
16 string res;
17
18 signed main(){
19     std::ios::sync_with_stdio(false);
20     std::cin.tie(nullptr);
21     int n;cin>>n;
22     for(int i=1;i<=n;i++)
23         cin>>a[i];
24
25     int len=0; // 当前最长递增子序列的长度
26     // ans[k] 存储以第 k 个数为结尾的最长递增子序列的构成元素 (可选, 根据需要决定是否启用)
27     for(int i=1;i<=n;i++)
28     {

```

```

29     auto k= lower_bound(f+1,f+len+1,a[i])-f; // 二分查找, 找到 a[i] 应该插入的位置
30     f[k]=a[i]; // 更新 f 数组
31     len=max(len,(int)k); // 更新最长递增子序列的长度
32     if(len==k){
33         res+=" "+to_string(a[i]); // 如果当前数扩展了最长递增子序列, 则将其加入结果字符串
34     }
35     // cout<<f[k]<<endl;
36     // cout<<a[i]<<' '<<k<<' '<<ans[k]<<endl;
37     // ans[k]=ans[k-1]+" "+to_string(a[i]); // 更新以第 k 个数为结尾的最长递增子序列的构成元素 (
38
39 }
40 // cout<<ans[len]<<endl; // 输出最长递增子序列的构成元素
41 cout<<res<<endl; // 输出结果字符串
42 cout<<len; // 输出最长递增子序列的长度
43 return 0;
44 }
45
46 //2.dp
47 #include<iostream>
48
49 using namespace std;
50 const int N=1010;
51 int a[N],f[N];
52
53 int main()
54 {
55     int n;
56     cin>>n;
57     for(int i=1;i<=n;i++)
58     {
59         f[i]=1; //这属于初始化, 因为对于每一个数字, 自己单独一定是一个上升子序列
60         cin>>a[i];
61     }
62     for(int i=1;i<=n;i++) //i表示枚举到以a[i]结尾的最长上升子序列
63         for(int j=1;j<i;j++) //j表示从数组中下标小于i的数枚举, 看能不能加到a[i]加到以a[j]
64             if(a[i]>a[j]) //如果满足要求, 那就表明a[i]可以加到以j结尾的最长上升子序列
65                 f[i]=max(f[i],f[j]+1); //对于每一种可能取最大值
66     int res=0;
67     for(int i=1;i<=n;i++) //求最大值
68         res=max(res,f[i]);
69     cout<<res<<endl;
70     return 0;
71 }
72
73 // -----
74 // 最长公共递增子序列 (LCIS) : //不是最长公共哦
75 // -----
76 #include <iostream>
77 #include <vector>
78 #include <algorithm>

```



```
79 using namespace std;
80
81 // 求解最长公共上升子序列及其路径
82 pair<int, vector<int>> LCIS(const vector<int>& A, const vector<int>& B) {
83     int n = A.size(), m = B.size();
84     vector<int> dp(m, 0); // dp[j]: 以B[j]结尾的LCIS长度
85     vector<int> pre(m, -1); // pre[j]: dp[j]对应的前驱位置 (在B中的位置)
86     vector<int> from(m, -1); // 记录当前 dp[j] 是由哪个 j 转移过来的
87
88     for (int i = 0; i < n; i++) {
89         int current = 0;
90         int last = -1;
91         for (int j = 0; j < m; j++) {
92             if (A[i] == B[j]) {
93                 if (current + 1 > dp[j]) {
94                     dp[j] = current + 1;
95                     pre[j] = last;
96                 }
97             } else if (B[j] < A[i]) {
98                 if (dp[j] > current) {
99                     current = dp[j];
100                     last = j;
101                 }
102             }
103         }
104     }
105
106     // 找出最大长度的位置
107     int max_len = 0, pos = -1;
108     for (int j = 0; j < m; j++) {
109         if (dp[j] > max_len) {
110             max_len = dp[j];
111             pos = j;
112         }
113     }
114
115     // 回溯路径
116     vector<int> path;
117     while (pos != -1) {
118         path.push_back(B[pos]);
119         pos = pre[pos];
120     }
121     reverse(path.begin(), path.end());
122
123     return {max_len, path};
124 }
125
126 int main() {
127     int n, m;
128     cin >> n >> m;
129     vector<int> A(n), B(m);
130     for (int i = 0; i < n; i++) cin >> A[i];
131     for (int j = 0; j < m; j++) cin >> B[j];
```

```

132
133     auto [len, path] = LCIS(A, B);
134     cout << "LCIS长度: " << len << endl;
135     cout << "LCIS路径: ";
136     for (int x : path) cout << x << " ";
137     cout << endl;
138     return 0;
139 }
140
141 // -----
142 // LCS-最长公共子序列
143 // -----
144 #include <bits/stdc++.h>
145 using namespace std;
146
147 int main(){
148     ios::sync_with_stdio(false);
149     cin.tie(NULL);
150
151     string s, t;
152     cin >> s >> t;
153     int n = s.size(), m = t.size();
154     // dp[i][j]: LCS length of s[0..i-1] and t[0..j-1]
155     vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
156
157     for(int i = 1; i <= n; i++){
158         for(int j = 1; j <= m; j++){
159             if(s[i-1] == t[j-1])
160                 dp[i][j] = dp[i-1][j-1] + 1;
161             else
162                 dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
163         }
164     }
165
166     // 回溯构造一个最长公共子序列
167     string lcs;
168     int i = n, j = m;
169     while(i > 0 && j > 0){
170         if(s[i-1] == t[j-1]){
171             lcs.push_back(s[i-1]);
172             i--; j--;
173         } else if(dp[i-1][j] >= dp[i][j-1]){
174             i--;
175         } else {
176             j--;
177         }
178     }
179     reverse(lcs.begin(), lcs.end());
180
181     // 输出长度和一个 LCS
182     cout << dp[n][m] << "\n"
183         << lcs << "\n";
184     return 0;

```

```

185 }
186
187 // -----
188 // 编辑距离 (Edit Distance) 问题
189 // -----
190 #include <iostream>
191 #include <string>
192 #include <vector>
193 #include <algorithm>
194 using namespace std;
195
196 int main() {
197     string A, B;
198     cin >> A >> B;
199     int n = A.size(), m = B.size();
200     vector<vector<int>> f(n + 1, vector<int>(m + 1, 0));
201
202     // 初始化
203     for (int i = 0; i <= n; ++i) f[i][0] = i;
204     for (int j = 0; j <= m; ++j) f[0][j] = j;
205
206     for (int i = 1; i <= n; ++i)
207         for (int j = 1; j <= m; ++j) {
208             f[i][j] = min({
209                 f[i - 1][j] + 1, // 删除
210                 f[i][j - 1] + 1, // 插入
211                 f[i - 1][j - 1] + (A[i - 1] != B[j - 1]) // 替换或不变
212             });
213         }
214
215     cout << f[n][m] << endl;
216     return 0;
217 }

```

2.6.3 划分型 dp

```

1  fill(dp, dp + N, -INT_MAX); //
2  dp[0] = 0;
3
4  for (int i = 1; i <= n; i++) {
5      dp[i] = dp[i - 1];
6      int sum = 0; // 当前组的和
7      for (int j = i; j >= 1; j--) {
8          sum += a[j]; // 从i到j的和
9          if (sum >= 0) {
10             dp[i] = max(dp[i], dp[j - 1] + 1);
11         }
12     }
13 }
14
15 if (dp[n] <= 0) {
16     cout << "Impossible" << endl;

```

```

17 } else {
18     cout << dp[n] << endl;
19 }
20
21 // -----
22 // 在这个数组中选择*最多* m 个"不相交"的区间
23 // -----
24 // dp[i][j] 表示在前 i 个元素里,*恰好*选用 j 个不相交区间所能得到的最大「增益」之和;
25 //问: 为什么dp定义是*恰好*, 但最后只能得到*最多*?
26 // -> 初始化: ll dp[N][N]; // 全局, 静态区, 所有元素都被初始化为 0
27 //问: 怎么初始化才能得到*恰好*?
28 // -> fill(dp[0],dp[0]+N,-INT_MAX); dp[0][0]=0;
29 for(int i=1; i<=n; i++) {
30     for(int j=1; j<=m; j++) dp[i][j]=max(dp[i][j],dp[i-1][j]); //不选
31
32     for(int j=1; j<=m; j++) {
33         ll sum=0;
34         for(int k=i; k>=1; k--) {
35             sum+=d[k];
36             dp[i][j]=max(dp[i][j],dp[k-1][j-1]+sum); //选
37         }
38     }
39 }
40 cout<<ans+dp[n][m]<<endl;
41
42 // -----
43 // 在这个数组中选择*最多* k 个"相邻"的区间, 所有元素都要用
44 // -----
45
46 //---! ! ! 超时做法! ! ! ---O(n^2 * k )----和上面"不相交"的代码区分
47 void solve() {
48     int n,k;
49     cin>>n>>k;
50     vector<int> a(n+1);
51     for(int i=1;i<=n;i++){
52         cin>>a[i];
53     }
54     vector<int> w(k+1);
55     for(int i=1;i<=k;i++){
56         cin>>w[i];
57     }
58     vector< vector<int> > dp( n+10, vector<int>(k+10,-2e18));
59     dp[0][0] = 0;
60     for(int i=1;i<=n;i++){
61         int j;
62         for( j=1;j<=k;j++){ //不划分, 但也要累加权值, 视作当前区间的扩展
63             dp[i][j] = max(dp[i][j],dp[i-1][j])+a[i]*w[j];
64         }
65
66         for( j=1;j<=k;j++){ //划分
67             int sum = 0;
68             for(int x = i;x>=j;x--){
69                 sum+= a[x];

```

```

70         dp[i][j] = max(dp[i][j], dp[x-1][j-1] + sum*w[j]);
71     }
72 }
73 }
74 cout<<dp[n][k];
75 }
76
77 //---AC做法-----//还可滚
78 vector< vector<int> > dp( n+10, vector<int>(k+10, -2e18));
79 dp[0][0] = 0;
80 for(int i=1; i<=n; i++){
81     int j;
82     for( j=1; j<=k; j++){
83         //将a[i]新开一个区间, 还是并入旧区间, 每次两种选择
84         dp[i][j] = max(dp[i-1][j], dp[i-1][j-1]) + a[i]*w[j];
85     }
86 }
87 cout<<dp[n][k];

```

2.6.4 背包

```

1  背包有多种模型：
2  1.0/1背包，在这种情况下，每个物品只有一个。
3  2. 完全背包，在这种情况下，每个物品有无穷个。
4  3. 多重背包，在这种情况下，每个物品有ci个，ci为第i个物品的数量。
5  4. 分组背包，在这种情况下，存在 n个组别，每个组别有若干个物品，但是每组至多只能选择一个。
6  5. 依赖背包，在这种情况下，某些物品之间存在依赖关系。
7
8  // -----
9  // 01背包
10 // -----
11 // -----二维数组版本
12 for(int i = 1; i <= n; ++i){
13     for(int j = 0; j <= m; ++j){
14         // 不装第 i 件
15         dp[i][j] = dp[i-1][j];
16         // 装第 i 件 (若能装下)
17         if(j >= v[i]){
18             dp[i][j] = max(dp[i][j], dp[i-1][j - v[i]] + p[i]);
19         }
20     }
21 }
22 // -----一维数组版本
23 for (int i = 1; i <= n; ++i) {
24     for (int j = m; j - v[i] >= 0; --j) {
25         dp[j] = max(dp[j], dp[j - v[i]] + p[i]);
26     }
27 }
28 // dp[i][j] 代表处理到第 i 个物品，容量为 j 时的最大值。
29
30 // -----bitset优化版本-----
31 const int N = 2e5 + 100;

```

```

32 int n,w;
33
34 bitset<N> f;
35
36 void solve()
37 {
38
39     cin >> n >> w;
40     f[0] = 1;
41     for(int i = 0;i < n;i ++){
42     {
43         int x; cin >> x; f |= (f << x);
44     }
45
46     for(int i = w;i >= 0;i --) if(f[i])
47     {
48         cout << i << endl; return;
49     }
50 }
51
52
53 // -----
54 // 完全背包
55 // -----
56 #include<bits/stdc++.h>
57 using namespace std;
58 #define int long long
59 const int maxn = 1e4+5; const int maxV = 1e7+5;
60 int n, v[maxn] ,w[maxn];
61 int dp[maxV];
62 signed main()
63 {
64     int V;
65     cin>>V>>n;
66     // cout
67     for(int i=1;i<=n;i++){
68         cin>>v[i];cin>>w[i];
69     }
70     for(int i=1;i<=n;i++){
71         for(int j=v[i];j<=V;j++){
72             dp[j] = max(dp[j],dp[j-v[i]]+w[i]);
73         }
74     }
75     cout<<dp[V]<<endl;
76     return 0;
77 }
78 // dp[i][j] 代表处理到第 i 个物品，容量为 j 时的最大值。
79
80 // -----
81 // 多重背包
82 // -----
83
84 //-----二进制拆分-----

```

```

85
86 原理：因为任何正整数  $s$  都可以用“若干个不同的 2 的幂”再加上一个小于该最大幂的“剩余”来唯
    一表示
87
88  int n, m;
89  cin >> n >> m;
90  vector<int> weight(n), value(n), cnt(n);
91  for (int i = 0; i < n; i++) {
92      // 读取格式：先读重量，再读价值，最后读数量
93      cin >> weight[i] >> value[i] >> cnt[i];
94  }
95
96  // dp[j] 表示容量为 j 时的最大价值
97  vector<ll> dp(m + 1, 0);
98
99  // 对每个物品做二进制拆分
100 for (int i = 0; i < n; i++) {
101     int v = weight[i], w = value[i], s = cnt[i];
102     int k = 1;
103     // 把 s 拆成若干个 2 的幂次和最后一个剩余
104     while (k <= s) {
105         int tk = k; // 当前拆分出“tk 件”原物品
106         s -= tk;
107         int ww = tk * v;
108         int vv = tk * w;
109         // 0-1 背包逆序枚举
110         for (int j = m; j >= ww; j--) {
111             dp[j] = max(dp[j], dp[j - ww] + vv);
112         }
113         k <<= 1;
114     }
115     // 剩下的那一部分（不足 k，却还剩余 s 件）
116     if (s > 0) {
117         int ww = s * v;
118         int vv = s * w;
119         for (int j = m; j >= ww; j--) {
120             dp[j] = max(dp[j], dp[j - ww] + vv);
121         }
122     }
123 }
124
125 cout << dp[m] << "\n";
126 return 0;
127
128 //-----单调队列-----
129 #include <bits/stdc++.h>
130 using namespace std;
131
132 const int N = 1010, M = 20010;
133 int n, m;
134 int v[N], w[N], s[N];
135
136 int main() {

```

```

137     cin >> n >> m;
138     for (int i = 1; i <= n; ++i) cin >> w[i] >> v[i] >> s[i];
139
140     vector<int> dp(m+1);
141     for (int i = 1; i <= n; ++i) { //1:枚物品
142         vector<int> ndp(m+1);
143         for (int r = 0; r < v[i]; ++r) { //2:枚余数
144             deque<int> q;
145             for (int j = r; j <= m; j += v[i]) { //3:枚背包, v[i]为步长 2、3联动->方便区分个数
146
147                 while (!q.empty() && dp[q.back()] + ((j - q.back()) / v[i]) * w[i] <=
148                     dp[j])
149                     q.pop_back();
150
151                 q.push_back(j);
152
153                 while (!q.empty() && (j - q.front()) / v[i] > s[i]) q.pop_front();
154
155                 ndp[j] = max(ndp[j], dp[q.front()] + ((j - q.front()) / v[i]) * w[i]);
156             }
157         }
158         dp.swap(ndp);
159     }
160     cout << dp[m] << endl;
161     return 0;
162 }

```

2.6.5 数位 dp

```

1  // -----
2  // 数位和sum
3  // -----
4  #include <bits/stdc++.h>
5  using namespace std;
6
7  typedef long long ll;
8  constexpr int maxn = 20, M = 1e9 + 7;
9  int a[maxn];
10 ll f[maxn][9 * 18 + 5]; // 总共最多 19 * (9 * 18)个状态
11 // f[pos][sum]: 在[pos + 1, len]中的数位和为sum,
12 // [1, pos]的数位任意填。满足这样约束的数的总和
13
14 ll dfs(int pos, bool limit, int sum){
15     if(!pos) return sum;
16     if(!limit && ~f[pos][sum]) //没限制并且dp值已搜索过
17         return f[pos][sum];
18     int up = limit?a[pos]:9;
19     ll res = 0;
20     for(int i=0;i<=up;i++){
21         res = (res + dfs(pos-1, limit && i == up, sum+i) )%M;

```



```

22     }
23     if(!limit) //记搜, 可复用: limit=false的才记忆化
24         f[pos][sum] = res;
25     return res;
26 }
27 ll solve(ll x){
28     int len = 0;
29     while(x){
30         a[++len] = x%10;
31         x/=10;
32     }
33     return dfs(len,true,0); //初始状态可以理解为len之前全部卡前导0的上
34 }
35 int T;
36 int main(){
37     cin.tie(0)->sync_with_stdio(false);
38     memset(f, -1, sizeof f); //可复用的, 多组样例都可使用!!!
39     cin >> T;
40     while(T --)
41     {
42         ll l, r;
43         cin >> l >> r;
44         ll ans = (solve(r) - solve(l - 1) + M) % M;
45         if(ans < 0) ans += M;
46         cout << ans << '\n';
47     }
48 }
49
50
51 // -----
52 // 前导0 digit个数cnt
53 // -----
54 #include <bits/stdc++.h>
55 using namespace std;
56
57 typedef long long LL;
58 constexpr int N = 14;
59 int a[N];
60 int digit;//当前需要统计的数字
61 // f[pos][cnt][0/1]: 当最高位已经填了(没有前导0时)在[pos + 1, len]中
62 // digit填了cnt个, [1, pos]任意填, digit出现的次数
63 // 第三维0表示digit=0,第三维为1表示digit!=0
64 // 当limit=false时, 容易知道填1-9的数量是相同的
65 LL f[N][N][2];
66
67 LL dfs(int pos, bool limit, bool lead0, int cnt){
68     if (!pos) // 递归边界
69         return cnt;
70     auto &now = f[pos][cnt][digit != 0];
71     if (!limit && !lead0 && ~now) // 没限制并且dp值已搜索过
72         return now;
73     int up = limit ? a[pos] : 9;
74     LL res = 0;

```

```

75     for(int i = 0; i <= up; i ++){
76     {
77         //填0的时候需注意是否为前导
78         //前导零不算入0的个数
79         int tmp = cnt + (i == digit);
80         if(lead0 && digit == 0 && i == 0)
81             tmp = 0;
82         res += dfs(pos - 1, limit && i == up, lead0 && i == 0, tmp);
83     }
84     if (!limit && !lead0) //无限制并且没有前导0
85         now = res;
86     return res;
87 }
88
89 LL ans[10];
90 void solve(LL x, int f){
91     int len = 0;
92     while (x > 0)
93     {
94         a[++len] = x % 10;
95         x /= 10;
96     }
97     for(int i = 0; i <= 9; i ++){
98     {
99         digit = i;
100         ans[i] += f * dfs(len, true, true, 0);
101     }
102 }
103
104 int main(){
105     memset(f, -1, sizeof f);
106     LL l, r;
107     cin >> l >> r;
108     solve(r, 1); //加上[1, r]
109     solve(l - 1, -1); //扣除掉[1, l-1]
110     for(int i = 0; i <= 9; i ++){
111         cout << ans[i] << ' ';
112     }
113
114     // -----
115     // 前导0 数数 和上一位有关: 以last为一维状态 (windy数
116     // -----
117     #include <bits/stdc++.h>
118     using namespace std;
119
120     typedef long long LL;
121     constexpr int N = 11, INF = 1e9;
122     int a[N];
123     // f[pos][last]: 长度为pos+1的以last开头的windy数的数量
124     // 相当于[1, pos]没有填, 第pos+1位填了last
125     int f[N][10];
126
127     int dfs(int pos, bool limit, bool lead0, int last)

```

```

128 {
129     if (!pos) // 递归边界
130         return 1;
131     if (!limit && last != INF && ~f[pos][last]) // 没限制并且dp值已搜索过, 并且last填了
132         return f[pos][last];
133     int up = limit ? a[pos] : 9;
134     int res = 0;
135     for(int i = 0; i <= up; i++)
136     {
137         //填0的时候需注意是否为前导
138         if(lead0) // 如果是前导0表示还没有开始填数, 则需要让last依旧不约束下一个数
139             res = res + dfs(pos - 1, limit && i == up, lead0 && i == 0, i == 0 ? last
140                 : i);
141         else
142             if (abs(last - i) >= 2) // 与上一个数差值至少为2
143                 res = res + dfs(pos - 1, limit && i == up, false, i);
144     }
145     if (!limit && last != INF) // 记搜, 并且last填了
146         f[pos][last] = res;
147     return res;
148 }
149 int solve(LL x)
150 {
151     int len = 0;
152     while (x > 0)
153     {
154         a[++len] = x % 10;
155         x /= 10;
156     }
157     return dfs(len, true, true, INF); // last设置为一个没有约束下一个数位的数
158 }
159
160 int main()
161 {
162     memset(f, -1, sizeof f);
163     int l, r;
164     cin >> l >> r;
165     LL ans = solve(r) - solve(l - 1);
166     cout << ans << '\n';
167 }
168
169 // -----
170 // 两个dp数组: 方案数+答案
171 // 给定正整数 N 和 K。求所有不大于 N 且满足以下条件的正整数 x 的和,
172 // x 的popcount正好是 K。
173 // -----
174 #include <bits/stdc++.h>
175 using namespace std;
176 using ll = long long;
177 constexpr int maxn = 64;
178 constexpr ll M = 998244353;
179

```

```

180 int a[maxn]; // 存二进制
181 ll k;
182 ll dp_cnt[maxn][61], // dp_cnt[pos][cnt]:不受限时,从pos开始、已用cnt个1的方案数
183     dp_sum[maxn][61]; // dp_sum[pos][cnt]:不受限时,这些方案对应的数之和
184 bool vis[maxn][61]; // vis[pos][cnt] 表示 dp_cnt/sum 已缓存
185
186 pair<ll,ll> dfs(int pos, bool limit, int cnt) {
187     if (cnt > k)
188         return {0, 0};
189     if (pos == 0) {
190         // 全部位都决定完了: 若恰好用满 k 个 1, 则计 1 个方案; 加和值为 0 (没有更低的位了)
191         if (cnt == k) return {1, 0};
192         else return {0, 0};
193     }
194     if (!limit && vis[pos][cnt]) {
195         return {dp_cnt[pos][cnt], dp_sum[pos][cnt]};
196     }
197     ll total_cnt = 0, total_sum = 0;
198     int up = limit ? a[pos] : 1;
199     for (int d = 0; d <= up; ++d) {
200         auto [cnt_sub, sum_sub] = dfs(pos - 1, limit && (d == up), cnt + d);
201         // 把子状态的 lower-bits 贡献先加上
202         total_sum = (total_sum + sum_sub) % M;
203         // 如果在 pos 这一位放了 1, 就要加上 2^(pos-1) * cnt_sub
204         if (d == 1) {
205             ll bit_val = ( (1LL << (pos - 1)) % M );
206             total_sum = (total_sum + bit_val * cnt_sub) % M;
207         }
208         total_cnt = (total_cnt + cnt_sub) % M;
209     }
210     if (!limit) {
211         vis[pos][cnt] = true;
212         dp_cnt[pos][cnt] = total_cnt;
213         dp_sum[pos][cnt] = total_sum;
214     }
215     return {total_cnt, total_sum};
216 }
217
218 ll solve(ll x) {
219     // 拆二进制到 a[1..len]
220     int len = 0;
221     while (x) {
222         a[++len] = x & 1;
223         x >>= 1;
224     }
225     // 清缓存
226     memset(vis, 0, sizeof vis);
227     // 从最高位 len 开始, cnt=0, tight = true
228     auto [cnt, sum] = dfs(len, true, 0);
229     return sum;
230 }
231
232 int T;

```

```

233 signed main(){
234     ios::sync_with_stdio(false);
235     cin.tie(nullptr);
236
237     cin >> T;
238     while (T--){
239         ll N;
240         cin >> N >> k;
241         cout << solve(N) << "\n";
242     }
243     return 0;
244 }
245
246 作者: 灵茶山艾府
247 链接: https://leetcode.cn/discuss/post/tXLS3i/
248 来源: 力扣 (LeetCode)
249 著作权归作者所有。商业转载请联系作者获得授权, 非商业转载请注明出处。
250 // 代码示例: 返回 [low, high] 中的恰好包含 target 个 0 的数字个数
251 // 比如 digitDP(0, 10, 1) == 2
252 // 要点: 我们统计的是 0 的个数, 需要区分【前导零】和【数字中的零】, 前导零不能计入, 而数字
      中的零需要计入
253 int digitDP(int low, int high, int target) {
254     string low_s = to_string(low);
255     string high_s = to_string(high);
256     int n = high_s.size();
257     int diff_lh = n - low_s.size();
258     vector memo(n, vector<int>(target + 1, -1));
259
260     auto dfs = [&](this auto&& dfs, int i, int cnt0, bool limit_low, bool limit_high
      ) -> int {
261         if (cnt0 > target) {
262             return 0; // 不合法
263         }
264         if (i == n) {
265             return cnt0 == target;
266         }
267
268         if (!limit_low && !limit_high && memo[i][cnt0] >= 0) {
269             return memo[i][cnt0];
270         }
271
272         int lo = limit_low && i >= diff_lh ? low_s[i - diff_lh] - '0' : 0;
273         int hi = limit_high ? high_s[i] - '0' : 9;
274
275         int res = 0;
276         int d = lo;
277         // 通过 limit_low 和 i 可以判断能否不填数字, 无需 is_num 参数
278         // 如果前导零不影响答案, 去掉这个 if block
279         if (limit_low && i < diff_lh) {
280             // 不填数字, 上界不受约束
281             res = dfs(i + 1, cnt0, true, false);
282             d = 1;
283         }

```

```

284     for (; d <= hi; d++) {
285         // 统计 0 的个数
286         res += dfs(i + 1, cnt0 + (d == 0), limit_low && d == lo, limit_high && d
                == hi);
287         // res %= MOD;
288     }
289
290     if (!limit_low && !limit_high) {
291         memo[i][cnt0] = res;
292     }
293     return res;
294 };
295
296 return dfs(0, 0, true, true);
297 }

```

2.6.6 01 背包

有 n 件物品和一个容量为 W 的背包，第 i 件物品的体积为 $w[i]$ ，价值为 $v[i]$ ，求解将哪些物品装入背包中使总价值最大。

思路：

当放入一个体积为 $w[i]$ 的物品后，价值增加了 $v[i]$ ，于是我们可以构建一个二维的 $dp[i][j]$ 数组，装入第 i 件物品时，背包容量为 j 能实现的 **最大价值**，可以得到 **转移方程** $dp[i][j] = \max(dp[i-1][j], dp[i-1][j-w[i]] + v[i])$ 。

我们可以发现，第 i 个物品的状态是由第 $i-1$ 个物品转移过来的，每次的 j 转移过来后，第 $i-1$ 个方程的 j 已经没用了，于是我们想到可以把二维方程压缩成 **一维的**，用以 **优化空间复杂度**。

```

1  for (int i = 1; i <= n; i++)
2      for (int j = 0; j <= W; j++){
3          dp[i][j] = dp[i-1][j];
4          if (j >= w[i])
5              dp[i][j] = max(dp[i][j], dp[i-1][j-w[i]] + v[i]);
6      }
7
8  for (int i = 1; i <= n; i++) //当前装第 i 件物品
9      for (int j = W; j >= w[i]; j--) //背包容量为 j
10         dp[j] = max(dp[j], dp[j-w[i]] + v[i]); //判断背包容量为 j 的情况下能是实现总价
            值最大是多少

```

2.6.7 二维费用的背包

有 n 件物品和一个容量为 W 的背包，背包能承受的最大重量为 M ，每件物品只能用一次，第 i 件物品的体积是 $w[i]$ ，重量为 $m[i]$ ，价值为 $v[i]$ ，求解将哪些物品放入背包中使总体积不超过背包容量，总重量不超过背包最大容量，且总价值最大。

思路：

背包的限制条件由一个变成两个，那么我们的循环再多一维即可。

```

1 for (int i = 1; i <= n; i++)
2     for (int j = W; j >= w; j--) //容量限制
3         for (int k = M; k >= m; k--) //重量限制
4             dp[j][k] = max(dp[j][k], dp[j - w][k - m] + v);

```

2.6.8 分组背包

有 n 组物品，一个容量为 W 的背包，每组物品有若干，同一组的物品最多选一个，第 i 组第 j 件物品的体积为 $w[i][j]$ ，价值为 $v[i][j]$ ，求解将哪些物品装入背包，可使物品总体积不超过背包容量，且使总价值最大。

思路：

考虑每组中的某件物品选不选，可以选的话，去下一组选下一个，否则在这组继续寻找可以选的物品，当这组遍历完后，去下一组寻找。

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int N = 110;
4 int n, W, s[N], w[N][N], v[N][N], dp[N];
5 int main(){
6     cin >> n >> W;
7     for (int i = 1; i <= n; i++){
8         scanf("%d", &s[i]);
9         for (int j = 1; j <= s[i]; j++)
10             scanf("%d %d", &w[i][j], &v[i][j]);
11     }
12     for (int i = 1; i <= n; i++){
13         for (int j = W; j >= 0; j--){
14             for (int k = 1; k <= s[i]; k++){
15                 if (j - w[i][k] >= 0)
16                     dp[j] = max(dp[j], dp[j - w[i][k]] + v[i][k]);
17             }
18             cout << dp[W] << "\n";
19         }
20     }
21 }

```

2.6.9 多重背包

有 n 种物品和一个容量为 W 的背包，第 i 种物品的体积为 $w[i]$ ，价值为 $v[i]$ ，数量为 $s[i]$ ，求解将哪些物品装入背包中使总价值最大。

思路：

对于每一种物品，都有 $s[i]$ 种取法，我们可以将其转化为 **01 背包** 问题

上述方法的时间复杂度为 $O(n * m * s)$ 。

尽管采用了 **二进制优化**，时间复杂度还是太高，采用 **单调队列优化**，将时间复杂度优化至 $O(n * m)$

```

1 for (int i = 1; i <= n; i++){
2     for (int j = W; j >= 0; j--){
3         for (int k = 0; k <= s[i]; k++){

```

```

4         if (j - k * w[i] < 0) break;
5         dp[j] = max(dp[j], dp[j - k * w[i]] + k * v[i]);
6     }
7
8     for (int i = 1; i <= n; i++){
9         scanf("%lld%lld%lld", &x, &y, &s); //x 为体积, y 为价值, s 为数量
10        t = 1;
11        while (s >= t){
12            w[++num] = x * t;
13            v[num] = y * t;
14            s -= t;
15            t *= 2;
16        }
17        w[++num] = x * s;
18        v[num] = y * s;
19    }
20    for (int i = 1; i <= num; i++)
21        for (int j = W; j >= w[i]; j--)
22            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
23
24    #include <bits/stdc++.h>
25    using namespace std;
26    const int N = 2e5 + 10;
27    int n, W, w, v, s, f[N], g[N], q[N];
28    int main(){
29        ios::sync_with_stdio(false); cin.tie(0);
30        cin >> n >> W;
31        for (int i = 0; i < n; i++){
32            memcpy ( g, f, sizeof f);
33            cin >> w >> v >> s;
34            for (int j = 0; j < w; j++){
35                int head = 0, tail = -1;
36                for (int k = j; k <= W; k += w){
37                    if ( head <= tail && k - s * w > q[head] ) head ++ ; //保证队列长度 <= s
38                    while ( head <= tail && g[q[tail]] - (q[tail] - j) / w * v <= g[k] - (k
                        - j) / w * v ) tail -- ; //保证队列单调递减
39                    q[ ++ tail] = k;
40                    f[k] = g[q[head]] + (k - q[head]) / w * v;
41                }
42            }
43        }
44        cout << f[W] << "\n";
45        return 0;
46    }

```

2.6.10 完全背包

有 n 件物品和一个容量为 W 的背包，第 i 件物品的体积为 $w[i]$ ，价值为 $v[i]$ ，每件物品有无限个，求解将哪些物品装入背包中使总价值最大。

思路：

思路和 01 背包差不多，但是每一件物品有无限个，其实就是从每种物品中取 $0, 1, 2, \dots$ 件

物品加入背包中

实际上，我们可以发现，取 k 件物品可以从取 $k - 1$ 件转移过来，那么我们就可以将 k 的循环优化掉

和 01 背包类似地压缩成一维

```

1  for (int i = 1; i <= n; i++)
2      for (int j = 0; j <= W; j++)
3          for (int k = 0; k * w[i] <= j; k++) //选取几个物品
4              dp[i][j] = max(dp[i][j], dp[i - 1][j - k * w[i]] + k * v[i]);
5
6  for (int i = 1; i <= n; i++)
7      for (int j = 0; j <= W; j++){
8          dp[i][j] = dp[i - 1][j];
9          if (j >= w[i])
10             dp[i][j] = max(dp[i][j], dp[i][j - w[i]] + v[i]);
11     }
12
13  for (int i = 1; i <= n; i++)
14      for (int j = w[i]; j <= W; j++)
15          dp[j] = max(dp[j], dp[j - w[i]] + v[i]);

```

2.6.11 常用例题

题意：在一篇文章（包含大小写英文字母、数字、和空白字符（制表/空格/回车））中寻找 helloworld（任意一个字母的大小写都行）的子序列出现了多少次，输出结果对 $10^9 + 7$ 的余数。

字符串 DP，构建一个二维 DP 数组， $dp[i][j]$ 的 i 表示文章中的第几个字符， j 表示寻找的字符串的第几个字符，当字符串中的字符和文章中的字符相同时，即找到符合条件的字符， $dp[i][j] = dp[i - 1][j] + dp[i - 1][j - 1]$ ，因为字符串中的每个字符不会对后面的结果产生影响，所以 DP 方程可以优化成一维的，由于字符串中有重复的字符，所以比较时应该从后往前。

题意：（最长括号匹配）给一个只包含 ‘(’, ‘)’, ‘[’, ‘]’ 的非空字符串，“()” 和 “[]” 是匹配的，寻找字符串中最长的括号匹配的子串，若有两串长度相同，输出靠前的一串。

设给定的字符串为 s ，可以定义数组 $dp[i]$ 表示以 $s[i]$ 结尾的字符串里最长的括号匹配的字符。显然，从 $i - dp[i] + 1$ 到 i 的字符串是括号匹配的，当找到一个字符是 ‘)’ 或 ‘]’ 时，再去判断第 $i - 1 - dp[i - 1]$ 的字符和第 i 位的字符是否匹配，如果是，那么 $dp[i] = dp[i - 1] + 2 + dp[i - 2 - dp[i - 1]]$ 。

题意：去掉区间内包含 “4” 和 “62” 的数字，输出剩余的数字个数

串

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define LL long long

```

```

4  const int mod = 1e9 + 7;
5  char c, s[20] = "!helloworld";
6  LL dp[20];
7  int main(){
8      dp[0] = 1;
9      while ((c = getchar()) != EOF)
10         for (int i = 10; i >= 1; i--){
11             if (c == s[i] || c == s[i] - 32)
12                 dp[i] = (dp[i] + dp[i - 1]) % mod;
13             cout << dp[10] << "\n";
14             return 0;
15         }
16
17 #include <bits/stdc++.h>
18 using namespace std;
19 const int maxn = 1e6 + 10;
20 string s;
21 int len, dp[maxn], ans, id;
22 int main(){
23     cin >> s;
24     len = s.length();
25     for (int i = 1; i < len; i++){
26         if ((s[i] == '(' && s[i - 1 - dp[i - 1]] == '(') || (s[i] == '[' && s[i - 1 - dp[i - 1]] == '[')){
27             dp[i] = dp[i - 1] + 2 + dp[i - 2 - dp[i - 1]];
28             if (dp[i] > ans) {
29                 ans = dp[i]; //记录长度
30                 id = i; //记录位置
31             }
32         }
33     }
34     for (int i = id - ans + 1; i <= id; i++){
35         cout << s[i];
36     }
37     cout << "\n";
38     return 0;
39 }
40 int T,n,m,len,a[20]; //a数组用于判断每一位能取到的最大值
41 ll l,r,dp[20][15];
42 ll dfs(int pos,int pre,int limit){ //记搜
43     //pos搜到的位置, pre前一位数
44     //limit判断是否有最高位限制
45     if(pos>len) return 1; //剪枝
46     if(dp[pos][pre]!=-1 && !limit) return dp[pos][pre]; //记录当前值
47     ll ret=0; //暂时记录当前方案数
48     int res=limit?a[len-pos+1]:9; //res当前位能取到的最大值
49     for(int i=0;i<=res;i++){
50         if(!(i==4 || (pre==6 && i==2)))
51             ret+=dfs(pos+1,i,i==res&&limit);
52     }
53     if(!limit) dp[pos][pre]=ret; //当前状态方案数记录
54     return ret;
55 }
56 ll part(ll x){ //把数按位拆分

```

```

56     len=0;
57     while(x) a[++len]=x%10,x/=10;
58     memset(dp,-1,sizeof dp);//初始化-1 (因为有可能某些情况下的方案数是0)
59     return dfs(1,0,1);//进入记搜
60 }
61 int main(){
62     cin>>n;
63     while(n--){
64         cin>>l>>r;
65         if(l==0 && r==0)break;
66         if(l) printf("%lld\n",part(r)-part(l-1));//[1,r](1!=0)
67         else printf("%lld\n",part(r)-part(l));//从0开始要特判
68     }
69 }

```

2.6.12 数位 DP

```

1  /* pos 表示当前枚举到第几位
2  sum 表示 d 出现的次数
3  limit 为 1 表示枚举的数字有限制
4  zero 为 1 表示有前导 0
5  d 表示要计算出出现次数的数 */
6  const int N = 15;
7  LL dp[N][N];
8  int num[N];
9  LL dfs(int pos, LL sum, int limit, int zero, int d) {
10     if (pos == 0) return sum;
11     if (!limit && !zero && dp[pos][sum] != -1) return dp[pos][sum];
12     LL ans = 0;
13     int up = (limit ? num[pos] : 9);
14     for (int i = 0; i <= up; i++) {
15         ans += dfs(pos - 1, sum + ((!zero || i) && (i == d)), limit && (i == num[pos]
16             ),
17             zero && (i == 0), d);
18     }
19     if (!limit && !zero) dp[pos][sum] = ans;
20     return ans;
21 }
22 LL solve(LL x, int d) {
23     memset(dp, -1, sizeof dp);
24     int len = 0;
25     while (x) {
26         num[++len] = x % 10;
27         x /= 10;
28     }
29     return dfs(len, 0, 1, 1, d);
30 }

```

2.6.13 有依赖的背包

有 n 个物品和一个容量为 W 的背包，物品之间有依赖关系，且之间的依赖关系组成一颗 **树** 的形状，如果选择一个物品，则必须选择它的 **父节点**，第 i 件物品的体积是 $w[i]$ ，价值为 $v[i]$ ，依赖的父节点的编号为 $p[i]$ ，若 $p[i]$ 等于 -1，则为 **根节点**。求将哪些物品装入背包中，使总体积不超过总容量，且总价值最大。

思路：

定义 $f[i][j]$ 为以第 i 个节点为根，容量为 j 的背包的最大价值。那么结果就是 $f[root][W]$ ，为了知道根节点的最大价值，得通过其子节点来更新。所以采用递归的方式。对于每一个点，先将这个节点装入背包，然后找到剩余容量可以实现的最大价值，最后更新父节点的最大价值即可。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  const int N = 110;
4  int n, W, w[N], v[N], p, f[N][N], root;
5  vector<int> g[N];
6  void dfs(int u){
7      for (int i = w[u]; i <= W; i ++ )
8          f[u][i] = v[u];
9      for (auto v : g[u]){
10         dfs(v);
11         for (int j = W; j >= w[u]; j -- )
12             for (int k = 0; k <= j - w[u]; k ++ )
13                 f[u][j] = max(f[u][j], f[u][j - k] + f[v][k]);
14     }
15 }
16 int main(){
17     cin >> n >> W;
18     for (int i = 1; i <= n; i ++ ){
19         cin >> w[i] >> v[i] >> p;
20         if (p == -1) root = i;
21         else g[p].push_back(i);
22     }
23     dfs(root);
24     cout << f[root][W] << "\n";
25     return 0;
26 }

```

2.6.14 混合背包

放入背包的物品可能只有 1 件 (01 背包)，也可能有无限件 (完全背包)，也可能只有可数的几件 (多重背包)。

思路：

分类讨论即可，哪一类就用哪种方法去 dp 。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int n, W, w, v, s;

```

```

4 int main(){
5     cin >> n >> W;
6     vector <int> f(W + 1);
7     for (int i = 0; i < n; i ++ ){
8         cin >> w >> v >> s;
9         if (s == -1){
10             for (int j = W; j >= w; j -- )
11                 f[j] = max(f[j], f[j - w] + v);
12         }
13         else if (s == 0){
14             for (int j = w; j <= W; j ++ )
15                 f[j] = max(f[j], f[j - w] + v);
16         }
17         else {
18             int t = 1, cnt = 0;
19             vector <int> x(s + 1), y(s + 1);
20             while (s >= t){
21                 x[++cnt] = w * t;
22                 y[cnt] = v * t;
23                 s -= t;
24                 t *= 2;
25             }
26             x[++cnt] = w * s;
27             y[cnt] = v * s;
28             for (int i = 1; i <= cnt; i ++ )
29                 for (int j = W; j >= x[i]; j -- )
30                     f[j] = max(f[j], f[j - x[i]] + y[i]);
31         }
32     }
33     cout << f[W] << "\n";
34     return 0;
35 }

```

2.6.15 状压 DP

题意: 在 $n \times n$ 的棋盘里面放 k 个国王, 使他们互不攻击, 共有多少种摆放方案。国王能攻击到它上下左右, 以及左上右下右上左下八个方向上附近的各一个格子, 共 8 个格子。

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define LL long long
4 const int N = 15, M = 150, K = 1500;
5 LL n, k;
6 LL cnt[K]; //每个状态的二进制中 1 的数量
7 LL tot; //合法状态的数量
8 LL st[K]; //合法的状态
9 LL dp[N][M][K]; //第 i 行, 放置了 j 个国王, 状态为 k 的方案数
10 int main(){
11     ios::sync_with_stdio(false); cin.tie(0);
12     cin >> n >> k;
13     for (int s = 0; s < (1 << n); s ++ ){ //找出合法状态
14         LL sum = 0, t = s;

```

```

15     while(t){ //计算 1 的数量
16         sum += (t & 1);
17         t >>= 1;
18     }
19     cnt[s] = sum;
20     if ( ((s << 1) | (s >> 1)) & s) == 0 ){ //判断合法性
21         st[ ++ tot] = s;
22     }
23 }
24 dp[0][0][0] = 1;
25 for (int i = 1; i <= n + 1; i ++ ){
26     for (int j1 = 1; j1 <= tot; j1 ++ ){ //当前的状态
27         LL s1 = st[j1];
28         for (int j2 = 1; j2 <= tot; j2 ++ ){ //上一行的状态
29             LL s2 = st[j2];
30             if ( ((s2 | (s2 << 1) | (s2 >> 1)) & s1) == 0 ){
31                 for (int j = 0; j <= k; j ++ ){
32                     if (j - cnt[s1] >= 0)
33                         dp[i][j][s1] += dp[i - 1][j - cnt[s1]][s2];
34                 }
35             }
36         }
37     }
38 }
39 cout << dp[n + 1][k][0] << "\n";
40 return 0;
41 }

```

2.6.16 背包问题求具体方案

```

1 signed main() {
2     int Task = 1;
3     for (cin >> Task; Task; Task--) {
4         int n, m;
5         cin >> n >> m;
6
7         vector<int> w(n + 1), v(n + 1);
8         vector<vector<int>> dp(n + 2, vector<int>(m + 2));
9         for (int i = 1; i <= n; i++) {
10             cin >> w[i] >> v[i];
11         }
12
13         for (int i = n; i >= 1; i--) {
14             for (int j = 0; j <= m; j++) {
15                 dp[i][j] = dp[i + 1][j];
16                 if (j >= w[i]) {
17                     dp[i][j] = max(dp[i][j], dp[i + 1][j - w[i]] + v[i]);
18                 }
19             }
20         }
21
22         vector<int> ans;

```

```

23     for (int i = 1; i <= n; i++) {
24         if (m - w[i] >= 0 && dp[i][m] == dp[i + 1][m - w[i]] + v[i]) {
25             ans.push_back(i);
26             // cout << i << " ";
27             m -= w[i];
28         }
29     }
30     cout << ans.size() << "\n";
31     for (auto i : ans) {
32         cout << i << " ";
33     }
34     cout << "\n";
35 }
36 }

```

2.6.17 背包问题求方案数

有 n 件物品和一个容量为 W 的背包，每件物品只能用一次，第 i 件物品的重量为 $w[i]$ ，价值为 $v[i]$ ，求解将哪些物品放入背包使总重量不超过背包容量，且总价值最大，输出 **最优选法的方案数**，答案可能很大，输出答案模 $10^9 + 7$ 的结果。

思路：

开一个储存方案数的数组 cnt ， $cnt[i]$ 表示容量为 i 时的 **方案数**，先将 cnt 的每一个值都初始化为 1，因为 **不装任何东西就是一种方案**，如果装入这件物品使总的价值 **更大**，那么装入后的方案数 **等于装之前的方案数**，如果装入后总价值 **相等**，那么方案数就是 **二者之和**

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define LL long long
4  const int mod = 1e9 + 7, N = 1010;
5  LL n, W, cnt[N], f[N], w, v;
6  int main(){
7      cin >> n >> W;
8      for (int i = 0; i <= W; i++)
9          cnt[i] = 1;
10     for (int i = 0; i < n; i++){
11         cin >> w >> v;
12         for (int j = W; j >= w; j--)
13             if (f[j] < f[j - w] + v){
14                 f[j] = f[j - w] + v;
15                 cnt[j] = cnt[j - w];
16             }
17             else if (f[j] == f[j - w] + v){
18                 cnt[j] = (cnt[j] + cnt[j - w]) % mod;
19             }
20     }
21     cout << cnt[W] << "\n";
22     return 0;
23 }

```

3 数据结构科技

3.1 基础

3.1.1 并查集

```
1 int fa[maxn];
2
3 void init(){
4     for(int i=1 ; i<=maxn; i++) fa[i] = i;
5 }
6
7 int root(int x){
8     return fa[x] = fa[x] == x ? x :root(fa[x]);
9 }
10
11 void merge(int x,int y){
12     x = root(x);
13     y = root(y);
14     if(x != y) fa[x] = fa[y];
15 }
16
17
18 struct DSU {
19     vector<int> p;
20     DSU(int n): p(n, -1) {}
21     int fa(int x) { return p[x] < 0 ? x : p[x] = fa(p[x]); }
22     void merge(int a, int b) {
23         a = fa(a); b = fa(b);
24         if (a == b) return;
25         if (p[a] > p[b]) swap(a, b);
26         p[a] += p[b];
27         p[b] = a;
28     }
29 };
```

3.1.2 单调栈，单调队列

```
1 // -----
2 // 单调栈:
3 // -----
4 pair<vector<int>, vector<int>> nearestGreater(vector<int>& nums) {
5     int n = nums.size();
6     // left[i] 是 nums[i] 左侧最近的严格大于 nums[i] 的数的下标, 若不存在则为 -1
7     vector<int> left(n);
8     vector<int> st{-1}; // 哨兵
9     for (int i = 0; i < n; i++) {
10         int x = nums[i];
11         while (st.size() > 1 && nums[st.back()] <= x) { // 如果求严格小于, 改成 >=
12             st.pop_back();
13         }
14         left[i] = st.back();
15         st.push_back(i);
16     }
```



```

16     }
17
18     // right[i] 是 nums[i] 右侧最近的严格大于 nums[i] 的数的下标, 若不存在则为 n
19     vector<int> right(n);
20     st = {n}; // 哨兵
21     for (int i = n - 1; i >= 0; i--) {
22         int x = nums[i];
23         while (st.size() > 1 && nums[st.back()] <= x) {
24             st.pop_back();
25         }
26         right[i] = st.back();
27         st.push_back(i);
28     }
29
30     return {left, right};
31 }
32
33 作者: 灵茶山艾府
34 链接: https://leetcode.cn/discuss/post/9oZFK9/
35 来源: 力扣 (LeetCode)
36 著作权归作者所有。商业转载请联系作者获得授权, 非商业转载请注明出处。
37
38 #include <vector>
39 #include <stack>
40 using namespace std;
41 /*
42     问题类型 | 单调栈类型 | 比较条件
43     下一个更大元素 (右侧) | 单调递减栈 | nums[i] > stk.top()
44     下一个更小元素 (右侧) | 单调递增栈 | nums[i] < stk.top()
45     上一个更大元素 (左侧) | 单调递减栈 | 从右向左遍历
46     上一个更小元素 (左侧) | 单调递增栈 | 从右向左遍历
47 */
48 // 单调栈模板 (单调递减栈: 栈底到栈顶递减)
49 vector<int> nextGreaterElement(vector<int>& nums) {
50     int n = nums.size();
51     vector<int> res(n, -1); // 默认-1表示无更大元素
52     stack<int> stk; // 存储下标 (若需值可直接存nums[i])
53
54     for (int i = 0; i < n; i++) {
55         // 当前元素 > 栈顶元素时, 说明找到栈顶元素的下一个更大值
56         while (!stk.empty() && nums[i] > nums[stk.top()]) {
57             res[stk.top()] = nums[i]; // 记录结果
58             stk.pop(); // 弹出已处理的元素
59         }
60         stk.push(i); // 当前元素入栈 (存下标便于处理)
61     }
62     return res;
63 }
64
65 // -----
66 // 单调栈分段:
67 // -----
68 // Compute L and R boundaries for each index via monotonic stack

```

```

69 // L[i]: index of previous smaller element, R[i]: next smaller (strict) element
70 void monotonic_bounds(const vector<ll>& b, vector<int>& L, vector<int>& R) {
71     int n = b.size() - 1;
72     stack<int> stk;
73     for (int i = 1; i <= n; i++) {
74         while (!stk.empty() && b[stk.top()] >= b[i]) stk.pop();
75         L[i] = stk.empty() ? 0 : stk.top();
76         stk.push(i);
77     }
78     while (!stk.empty()) stk.pop();
79     for (int i = n; i >= 1; i--) {
80         while (!stk.empty() && b[stk.top()] > b[i]) stk.pop();
81         R[i] = stk.empty() ? n+1 : stk.top();
82         stk.push(i);
83     }
84 }
85 // 调用
86 vector<int> L(n+1), R(n+1);
87 monotonic_bounds(b, L, R);
88
89 // Find root index
90 int root = 1;
91 for (int i = 1; i <= n; i++) if (L[i] == 0 && R[i] == n+1) { root = i; break; }
92
93 // -----
94 // 单调队列:
95 // -----
96 //deque:
97 void solve() {
98     int n, k;
99     cin >> n >> k;
100     vector<int> a(n);
101     for(auto &x:a) cin >> x;
102     deque<int> q; // 存的是下标
103     vector<int> Max;
104     vector<int> Min;
105     for (int i = 0; i < n; ++i) {
106         while (!q.empty() && a[q.back()] >= a[i]) q.pop_back();
107         q.push_back(i);
108         while (!q.empty() && q.front() <= i - k) q.pop_front();
109         if(i>=k-1) Min.push_back(a[q.front()]);
110     }
111     q.clear();
112     for (int i = 0; i < n; ++i) {
113         while (!q.empty() && a[q.back()] <= a[i]) q.pop_back();
114         q.push_back(i);
115         while (!q.empty() && q.front() <= i - k) q.pop_front();
116         if(i>=k-1) Max.push_back(a[q.front()]);
117     }
118     for(auto i:Min) cout<<i<<" "; cout<<endl;
119     for(auto i:Max) cout<<i<<" "; cout<<endl;
120 }

```

3.1.3 树状数组 + 求中位数

```

1  int sz = ;
2  vector<int> tr(sz+1,0);
3  auto upd=[&](int i,int v){ for(; i<=sz;i+=i&-i) tr[i]+=v; };
4  auto get=[&](int i){ int s=0; for(;i;i-=i&-i) s+=tr[i]; return s; };
5
6  auto upd = [&](vector<ll>& tr, int i, ll v){for(; i <= sz; i += i & -i){tr[i] = (tr[
    i] + v) % M;}};
7  auto get = [&](vector<ll>& tr, int i){ll s = 0;for(; i > 0; i -= i & -i){s = (s + tr
    [i]) % M;}return s;};
8
9  // -----
10 // 区间加, 区间查
11 // -----
12 int sz =
13 vector<int> tr(sz+10);
14 vector<int> tr2(sz+10);
15 auto upd = [&](int i ,int v){
16     int p = i;
17     for(;i<=sz;i+=i&-i){tr[i]+=v;tr2[i]+=(p-1)*v;}
18 };
19 auto qry = [&](int i){
20     int p = i;
21     int s=0;for(;i;i-=i&-i){s+=p*tr[i]+tr2[i];} return s;
22 };
23 // -----
24 // 树状数组 max 单点修改 (不支持改小), 区间查询
25 // -----
26 int sz = ;
27 vector<int> tr(sz+1, -inf); //这里要初始化为最小值
28
29 // 更新位置 i 的值为 v (覆盖, 而且v>=i 才有效, 小于由于不会更新失效)
30 auto upd = [&](int i, int v) {
31     for (; i <= sz; i += i & -i ). tr[i] = max(tr[i], v);
32 };
33 // 查询前缀 [1..i] 的最大值
34 auto get = [&](int i) {
35     int res = -inf; //负无穷
36     for (; i ; i -= i & -i ) res = max(res, tr[i]);
37     return res;
38 };
39
40 //点增版本upd:
41 auto upd = [&](int i, int delta) {
42     a[i] += delta;
43     for (; i <= sz; i += i & -i) {
44         tr[i] = max(tr[i], a[i]);
45     }
46 };
47 // -----
48 // 找第kth的位置: 树状数组维护桶,
49 // -----

```

```

50 int sz = ;
51 vector<int> tr(sz+1,0);
52 auto upd=[&](int i,int v){ for(; i<=sz;i+=i&-i) tr[i]+=v; };
53 auto get=[&](int i){ int s=0; for(;i;i-=i&-i) s+=tr[i]; return s; };
54
55 auto kth = [&](int k){
56     int idx = 0;
57     int LOG = __lg(sz);
58     int pw = 1 << LOG;
59     ll acc = 0;
60     while(pw){
61         int next = idx + pw;
62         if( next <= sz && acc + tr[idx + pw] < k){
63             acc += tr[next];
64             idx = next;
65         }
66         pw >>= 1;
67     }
68     return idx + 1; // +1 !!!
69 };

```

3.1.4 树状数组: 逆序对

```

1 P1908
2 #include <iostream>
3 #include <vector>
4 #include <algorithm>
5 using namespace std;
6 typedef long long ll;
7
8 int main() {
9     ios::sync_with_stdio(false);cin.tie(nullptr);
10
11     int n;
12     cin >> n;
13     vector<int> a(n), d(n);
14     for (int i = 0; i < n; ++i) {
15         cin >> a[i];
16         d[i] = a[i];
17     }
18
19     // Coordinate compression
20     sort(d.begin(), d.end());
21     d.erase(unique(d.begin(), d.end()), d.end());
22
23     int sz = d.size();
24     vector<int> tr(sz + 1, 0);
25
26     auto upd=[&](int i,int v){ for(; i<=sz;i+=i&-i) tr[i]+=v; };
27     auto qry=[&](int i){ int s=0; for(;i;i-=i&-i) s+=tr[i]; return s; };
28
29     ll inv = 0;

```

```

30     for (int i = n - 1; i >= 0; --i) {
31         int pos = lower_bound(d.begin(), d.end(), a[i]) - d.begin() + 1;
32         inv += qry(pos - 1);
33         upd(pos, 1);
34     }
35
36     cout << inv << '\n';
37
38     return 0;
39 }

```

3.1.5 二维数点离线 (树状数组)

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  using ll = long long;
4
5  struct Event { int x, y, id, sgn; };
6
7  int main(){
8      ios::sync_with_stdio(false);
9      cin.tie(nullptr);
10
11     int n, m;
12     cin >> n >> m;
13     vector<pair<int,int> > pts(n);
14     vector<int> ys;
15     ys.reserve(n);
16     for(int i = 0; i < n; i++){
17         cin >> pts[i].first >> pts[i].second;
18         ys.push_back(pts[i].second);
19     }
20     // 离散化 y
21     sort(ys.begin(), ys.end());
22     ys.erase(unique(ys.begin(), ys.end()), ys.end());
23
24     auto getY = [&](int y){
25         int idx = int(upper_bound(ys.begin(), ys.end(), y) - ys.begin()) - 1;
26         return idx;
27     };
28
29     // 构造查询事件
30     vector<Event> ev;
31     ev.reserve(4LL * m);
32     for(int i = 0; i < m; i++){
33         int a, b, c, d;
34         cin >> a >> b >> c >> d;
35         ev.push_back({ c, getY(d), i, +1 });
36         ev.push_back({ a-1, getY(d), i, -1 });
37         ev.push_back({ c, getY(b-1), i, -1 });
38         ev.push_back({ a-1, getY(b-1), i, +1 });
39     }

```

```

40
41 // 按 x 排序
42 sort(pts.begin(), pts.end(), [](auto &A, auto &B){ return A.first < B.first; });
43 sort(ev.begin(), ev.end(), [](auto &A, auto &B){ return A.x < B.x; });
44
45 // 树状数组模板
46 int sz = ys.size();
47 vector<int> pos(sz+1, 0);
48 auto upd = [&](int i, int v){ for(; i <= sz; i += i & -i) pos[i] += v; };
49 auto qry = [&](int i){ int s = 0; for(; i > 0; i -= i & -i) s += pos[i]; return
    s; };
50
51 vector<ll> ans(m, 0);
52 int p = 0;
53 for(auto &e : ev){
54     // 将所有 x <= e.x 的点加入 BIT
55     while(p < n && pts[p].first <= e.x){
56         int yi = int(lower_bound(ys.begin(), ys.end(), pts[p].second) - ys.begin()
57             ) + 1;
58         upd(yi, 1);
59         p++;
60     }
61     if(e.y >= 0){
62         ans[e.id] += ll(e.sgn) * qry(e.y + 1);
63     }
64
65     for(int i = 0; i < m; i++) cout << ans[i] << '\n';
66     return 0;
67 }

```

3.1.6 ST 表

```

1
2 // -----
3 // 0-base: a[0..n-1] : 0-base和1-base 别! 混! 用!
4 // -----
5 vector<int> a(n);
6 for (int i = 0; i < n; i++) cin >> a[i];
7
8
9 vector<int> lg2(n + 10); //预处理floor(log2(i))
10 //查询没有到1e6次可用__lg代替
11 lg2[1] = 0;
12 for (int i = 2; i <= n; i++) lg2[i] = lg2[i / 2] + 1;
13
14 int LOG = lg2[n] + 1; // 最大级数
15 vector<vector<int>> st(LOG, vector<int>(n));
16 st[0] = a; // k = 0 层直接拷贝原数组
17
18 for (int k = 1; k < LOG; k++) {
19     int len = 1 << k;

```

```

20     for (int i = 0; i + len <= n; i++) {
21         st[k][i] = max(st[k - 1][i], st[k - 1][i + (len >> 1)]);
22     }
23 }
24
25 auto query = [&](int l, int r) { //0-base
26     int len = r - l + 1;
27     int k = lg2[len];
28     return max(st[k][l], st[k][r - (1 << k) + 1]);
29 };
30 // -----
31 // 位与/位或 ST表
32 // -----
33
34 转移: st[j][i] = st[j-1][i] & st[j-1][i + (1<<(j-1))];
35
36 查询: return st[k][l] | st[k][r - (1 << k) + 1]);
37
38 // -----
39 // 1-base: a[1..n]
40 // -----
41 vector<int> a(n + 1); // 下标从 1 开始
42 for (int i = 1; i <= n; i++) cin >> a[i];
43
44
45 int LOG = 21;
46 vector<vector<int>> st(LOG, vector<int>(n + 10));
47 st[0] = a;
48
49 for (int k = 1; k <= 20; k++) {
50     int len = 1<<k;
51     for (int i = 1; i + len - 1 <= n; i++) {
52         st[k][i] = max(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
53     }
54 }
55
56 auto query = [&](int l, int r) {
57     int k = __lg(r - l + 1);
58     return max(st[k][l], st[k][r - (1 << k) + 1]);
59 };
60 // -----
61 // 1-base: a[1..n]
62 //在不用要求幂等 (x opt x = x) 的前提下, 用倍增 (即二进制分解) 的方式来做 ST 表
63 // 要求: 只要操作是结合的, 有一个单位元 (做初始值用)
64 // -----
65 #include <bits/stdc++.h>
66 using namespace std;
67
68 int main() {
69     ios::sync_with_stdio(false);
70     cin.tie(nullptr);
71
72     int n;

```

```

73     cin >> n;
74     vector<int> a(n+1);
75     for (int i = 1; i <= n; i++) {
76         cin >> a[i];
77     }
78
79     // 预处理
80     int LOG = 0;
81     while ((1 << LOG) <= n) LOG++;
82     vector<vector<int>>> st(LOG, vector<int>(n+2));
83     // k = 0 层就是原数组
84     for (int i = 1; i <= n; i++) {
85         st[0][i] = a[i];
86     }
87     // 递推: st[k][i] 存的是区间 [i, i+2^k-1] 上的答案
88     for (int k = 1; k < LOG; k++) {
89         int len = 1 << (k-1);
90         for (int i = 1; i + (1<<k) - 1 <= n; i++) {
91             st[k][i] = max(st[k-1][i], st[k-1][i + len]);
92         }
93     }
94
95     // 倍增跳查询
96     auto query = [&](int l, int r) {
97         int len = r - l + 1;
98         // 对 max, 单位元设为最小值
99         int res = numeric_limits<int>::min();
100        for (int k = 0; k < LOG; k++) {
101            if (len & (1 << k)) {
102                res = max(res, st[k][l]);
103                l += (1 << k);
104            }
105        }
106        return res;
107    };
108
109    // 示例: 处理 m 次查询
110    int m;
111    cin >> m;
112    while (m--) {
113        int l, r;
114        cin >> l >> r;
115        cout << query(l, r) << "\n";
116    }
117    return 0;
118 }

```

3.1.7 对顶堆

```

1 对顶堆
2 #include<bits/stdc++.h>
3 using namespace std;

```



```
4 #define int long long
5 #define N 100001
6 string s;
7 int i,j,t,n,a[1000005],m,k,q,temp1,temp2;
8 priority_queue <int> Q;
9 priority_queue <int,vector<int>,greater<int>> Q1;
10 void insert(int x){
11     if(x<Q1.top()){
12         Q.push(x);
13     }
14     else Q1.push(x);
15
16     if(Q1.size()>Q.size()){
17         int tt=Q1.top();
18         Q1.pop();
19         Q.push(tt);
20     }
21     else if(Q1.size()<Q.size()){
22         int tt=Q.top();
23         Q.pop();
24         Q1.push(tt);
25     }
26 }
27 signed main() {
28     // ios::sync_with_stdio(false);
29     // cin.tie(0), cout.tie(0);
30     cin>>t>>n;
31     cin>>temp1;
32     Q1.push(temp1);
33
34     for(i=1;i<n;i++){
35         cin>>temp1;
36         insert(temp1);
37     }
38
39     cout<<(Q1.size()>Q.size()?Q1.top():Q.top())<<"\n";
40
41     while(t--){
42         cin>>n;
43         while(n--){
44             cin>>temp1;
45             insert(temp1);
46         }
47         cout<<(Q1.size()>Q.size()?Q1.top():Q.top())<<"\n";
48     }
49     return 0;
50 }
```

3.1.8 归并排序逆序对

```
1 #include <iostream>
2 #include <vector>
```

```

3
4 // Merge sort and count inversions.
5 long long merge_sort(std::vector<int>& nums, int b, int e) {
6     // Return 0 when length <= 1.
7     if (e - b <= 1) return 0;
8     long long res = 0;
9     int m = (b + e) / 2;
10    // Merge_sort two halves.
11    res += merge_sort(nums, b, m);
12    res += merge_sort(nums, m, e);
13    // Temporary vector to store sorted array.
14    std::vector<int> tmp(e - b);
15    int i = b, j = m, k = 0;
16    while (i < m && j < e) {
17        if (nums[j] < nums[i]) {
18            tmp[k] = nums[j++];
19            // In this case, all elements in [i,m) are larger than element j.
20            res += m - i;
21        } else {
22            tmp[k] = nums[i++];
23        }
24        ++k;
25    }
26    // Deal with remaining elements.
27    for (; i < m; ++i, ++k) {
28        tmp[k] = nums[i];
29    }
30    for (; j < e; ++j, ++k) {
31        tmp[k] = nums[j];
32    }
33    // Copy back to original vector.
34    for (i = b, k = 0; i < e; ++i, ++k) {
35        nums[i] = tmp[k];
36    }
37    return res;
38 }
39
40 int main() {
41     int n;
42     std::cin >> n;
43     std::vector<int> nums(n);
44     for (int& num : nums) {
45         std::cin >> num;
46     }
47     std::cout << merge_sort(nums, 0, n);
48     return 0;
49 }

```

3.1.9 ST 表

用于解决区间可重复贡献问题，需要满足 x 运算符 $x = x$ （如区间最大值： $\max(x, x) = x$ 、区间 gcd： $\gcd(x, x) = x$ 等），但是不支持修改操作。 $\mathcal{O}(N \log N)$ 预处理， $\mathcal{O}(1)$ 查询。

```

1 struct ST {
2     const int n, k;
3     vector<int> in1, in2;
4     vector<vector<int>> Max, Min;
5     ST(int n) : n(n), in1(n + 1), in2(n + 1), k(__lg(n)) {
6         Max.resize(k + 1, vector<int>(n + 1));
7         Min.resize(k + 1, vector<int>(n + 1));
8     }
9     void init() {
10         for (int i = 1; i <= n; i++) {
11             Max[0][i] = in1[i];
12             Min[0][i] = in2[i];
13         }
14         for (int i = 0, t = 1; i < k; i++, t <= 1) {
15             const int T = n - (t < 1) + 1;
16             for (int j = 1; j <= T; j++) {
17                 Max[i + 1][j] = max(Max[i][j], Max[i][j + t]);
18                 Min[i + 1][j] = min(Min[i][j], Min[i][j + t]);
19             }
20         }
21     }
22     int getMax(int l, int r) {
23         if (l > r) {
24             swap(l, r);
25         }
26         int k = __lg(r - l + 1);
27         return max(Max[k][l], Max[k][r - (1 < k) + 1]);
28     }
29     int getMin(int l, int r) {
30         if (l > r) {
31             swap(l, r);
32         }
33         int k = __lg(r - l + 1);
34         return min(Min[k][l], Min[k][r - (1 < k) + 1]);
35     }
36 };

```

3.1.10 二维树状数组

半动态矩阵和（单点修改）

封装一：该版本不能同时进行区间修改 + 区间求和。空间占用为 $\mathcal{O}(NM)$ 、建树复杂度为 $\mathcal{O}(NM)$ 、单次查询复杂度为 $\mathcal{O}(\log N \cdot \log M)$ 。

动态矩阵和（区间修改）

封装二：仅支持区间修改 + 区间求和。但是时空复杂度均比上一个版本多 4 倍。

```

1 template<class T> struct BIT_2D {
2     int n, m;
3     vector<vector<T>> w;
4
5     BIT_2D(int n, int m) : n(n), m(m) {

```

```

6         w.resize(n + 1, vector<T>(m + 1));
7     }
8     void pointAdd(int x, int y, T k) {
9         for (int i = x; i <= n; i += i & -i) {
10             for (int j = y; j <= m; j += j & -j) {
11                 w[i][j] += k;
12             }
13         }
14     }
15     void matrixAdd(int x, int y, int X, int Y, T k) { // 区块修改: 二维差分
16         X++, Y++;
17         pointAdd(x, y, k), pointAdd(X, y, -k);
18         pointAdd(X, Y, k), pointAdd(x, Y, -k);
19     }
20     T pointSum(int x, int y) {
21         T ans = T();
22         for (int i = x; i; i -= i & -i) {
23             for (int j = y; j; j -= j & -j) {
24                 ans += w[i][j];
25             }
26         }
27         return ans;
28     }
29     T matrixSum(int x, int y, int X, int Y) { // 区块查询: 二维前缀和
30         x--, y--;
31         return pointSum(X, Y) - pointSum(x, Y) - pointSum(X, y) + pointSum(x, y);
32     }
33 };
34
35 template<class T> struct BIT_2D {
36     int n, m;
37     vector<vector<T>> b1, b2, b3, b4;
38
39     BIT_2D(int n, int m) : n(n), m(m) {
40         b1.resize(n + 1, vector<T>(m + 1));
41         b2.resize(n + 1, vector<T>(m + 1));
42         b3.resize(n + 1, vector<T>(m + 1));
43         b4.resize(n + 1, vector<T>(m + 1));
44     }
45     void matrixAdd(int x, int y, int X, int Y, T k) { // 区块修改: 二维差分
46         X++, Y++;
47         auto add = [&](int x, int y, T k) {
48             for (int i = x; i <= n; i += i & -i) {
49                 for (int j = y; j <= m; j += j & -j) {
50                     b1[i][j] += k;
51                     b2[i][j] += k * (x - 1);
52                     b3[i][j] += k * (y - 1);
53                     b4[i][j] += k * (x - 1) * (y - 1);
54                 }
55             }
56         };
57         add(x, y, k), add(X, y, -k);
58         add(X, Y, k), add(x, Y, -k);

```

```

59     }
60     T matrixSum(int x, int y, int X, int Y) { // 区块查询: 二维前缀和
61         x--, y--;
62         auto ask = [&](int x, int y) {
63             T ans = T();
64             for (int i = x; i; i -= i & -i) {
65                 for (int j = y; j; j -= j & -j) {
66                     ans += x * y * b1[i][j];
67                     ans -= y * b2[i][j];
68                     ans -= x * b3[i][j];
69                     ans += b4[i][j];
70                 }
71             }
72             return ans;
73         };
74         return ask(X, Y) - ask(x, Y) - ask(X, y) + ask(x, y);
75     }
76 };

```

3.1.11 坐标压缩与离散化

简单版本

封装

```

1  sort(alls.begin(), alls.end());
2  alls.erase(unique(alls.begin(), alls.end()), alls.end());
3  auto get = [&](int x) {
4      return lower_bound(alls.begin(), alls.end(), x) - alls.begin();
5  };
6
7  template <typename T> struct Compress_ {
8      int n, shift = 0; // shift 用于标记下标偏移量
9      vector<T> alls;
10
11      Compress_() {}
12      Compress_(auto in) : alls(in) {
13          init();
14      }
15      void add(T x) {
16          alls.emplace_back(x);
17      }
18      template <typename... Args> void add(T x, Args... args) {
19          add(x), add(args...);
20      }
21      void init() {
22          alls.emplace_back(numeric_limits<T>::max());
23          sort(alls.begin(), alls.end());
24          alls.erase(unique(alls.begin(), alls.end()), alls.end());
25          this->n = alls.size();
26      }
27      int size() {
28          return n;

```

```

29     }
30     int operator[](T x) { // 返回 x 元素的新下标
31         return upper_bound(alls.begin(), alls.end(), x) - alls.begin() + shift;
32     }
33     T Get(int x) { // 根据新下标返回原来元素
34         assert(x - shift < n);
35         return x - shift < n ? alls[x - shift] : -1;
36     }
37     bool count(T x) { // 查找元素 x 是否存在
38         return binary_search(alls.begin(), alls.end(), x);
39     }
40     friend auto &operator<< (ostream &o, const auto &j) {
41         cout << "{";
42         for (auto it : j.alls) {
43             o << it << " ";
44         }
45         return o << "}";
46     }
47 };
48 using Compress = Compress_<int>;

```

3.1.12 基于状压的线性 RMQ 算法

严格 $\mathcal{O}(N)$ 预处理, $\mathcal{O}(1)$ 查询。

```

1  template<class T, class Cmp = less<T>> struct RMQ {
2      const Cmp cmp = Cmp();
3      static constexpr unsigned B = 64;
4      using u64 = unsigned long long;
5      int n;
6      vector<vector<T>> a;
7      vector<T> pre, suf, ini;
8      vector<u64> stk;
9      RMQ() {}
10     RMQ(const vector<T> &v) {
11         init(v);
12     }
13     void init(const vector<T> &v) {
14         n = v.size();
15         pre = suf = ini = v;
16         stk.resize(n);
17         if (!n) {
18             return;
19         }
20         const int M = (n - 1) / B + 1;
21         const int lg = __lg(M);
22         a.assign(lg + 1, vector<T>(M));
23         for (int i = 0; i < M; i++) {
24             a[0][i] = v[i * B];
25             for (int j = 1; j < B && i * B + j < n; j++) {
26                 a[0][i] = min(a[0][i], v[i * B + j], cmp);
27             }
28         }

```

```

29     for (int i = 1; i < n; i++) {
30         if (i % B) {
31             pre[i] = min(pre[i], pre[i - 1], cmp);
32         }
33     }
34     for (int i = n - 2; i >= 0; i--) {
35         if (i % B != B - 1) {
36             suf[i] = min(suf[i], suf[i + 1], cmp);
37         }
38     }
39     for (int j = 0; j < lg; j++) {
40         for (int i = 0; i + (2 << j) <= M; i++) {
41             a[j + 1][i] = min(a[j][i], a[j][i + (1 << j)], cmp);
42         }
43     }
44     for (int i = 0; i < M; i++) {
45         const int l = i * B;
46         const int r = min(1U * n, l + B);
47         u64 s = 0;
48         for (int j = 1; j < r; j++) {
49             while (s && cmp(v[j], v[__lg(s) + 1])) {
50                 s ^= 1ULL << __lg(s);
51             }
52             s |= 1ULL << (j - 1);
53             stk[j] = s;
54         }
55     }
56 }
57 T operator()(int l, int r) {
58     if (l / B != (r - 1) / B) {
59         T ans = min(suf[l], pre[r - 1], cmp);
60         l = l / B + 1;
61         r = r / B;
62         if (l < r) {
63             int k = __lg(r - l);
64             ans = min({ans, a[k][l], a[k][r - (1 << k)]}, cmp);
65         }
66         return ans;
67     } else {
68         int x = B * (l / B);
69         return ini[__builtin_ctzll(stk[r - 1] >> (l - x)) + 1];
70     }
71 }
72 };

```

3.1.13 并查集 (全功能)

```

1 struct DSU {
2     vector<int> fa, p, e, f;
3
4     DSU(int n) {
5         fa.resize(n + 1);

```

```

6     iota(fa.begin(), fa.end(), 0);
7     p.resize(n + 1, 1);
8     e.resize(n + 1);
9     f.resize(n + 1);
10  }
11  int get(int x) {
12      while (x != fa[x]) {
13          x = fa[x] = fa[fa[x]];
14      }
15      return x;
16  }
17  bool merge(int x, int y) { // 设x是y的祖先
18      if (x == y) f[get(x)] = 1;
19      x = get(x), y = get(y);
20      e[x]++;
21      if (x == y) return false;
22      if (x < y) swap(x, y); // 将编号小的合并到大的上
23      fa[y] = x;
24      f[x] |= f[y], p[x] += p[y], e[x] += e[y];
25      return true;
26  }
27  bool same(int x, int y) {
28      return get(x) == get(y);
29  }
30  bool F(int x) { // 判断连通块内是否存在自环
31      return f[get(x)];
32  }
33  int size(int x) { // 输出连通块中点的数量
34      return p[get(x)];
35  }
36  int E(int x) { // 输出连通块中边的数量
37      return e[get(x)];
38  }
39  };

```

3.1.14 树状数组

半动态区间和（单点修改）

动态区间和（区间修改）

静态区间最值（单点修改）

逆序对扩展

性质：交换序列的任意两元素，序列的逆序数的奇偶性必定发生改变。

前驱后继扩展（常规 + 第 k 小值查询 + 元素排名查询 + 元素前驱后继查询）

注意，被查询的值都应该小于等于 N ，否则会越界；如果离散化不可使用，则需要使用平衡树替代。

```

1  struct BIT {
2      vector<i64> w;
3      int n;

```



```

4   BIT(int n) : n(n), w(n + 1) {}
5   void add(int x, i64 v) {
6       for (; x <= n; x += x & -x) {
7           w[x] += v;
8       }
9   }
10  i64 rangeAsk(int l, int r) { // 差分实现区间和查询
11      auto ask = [&](int x) {
12          i64 ans = 0;
13          for (; x; x -= x & -x) {
14              ans += w[x];
15          }
16          return ans;
17      };
18      return ask(r) - ask(l - 1);
19  }
20 };
21
22 struct BIT {
23     vector<i64> a, b;
24     int n;
25
26     BIT(int n) : n(n), a(n + 1), b(n + 1) {}
27     void rangeAdd(int l, int r, i64 val) { // 区间修改
28         auto add = [&](int pos, i64 val) {
29             for (int i = pos; i <= n; i += i & -i) {
30                 a[i] += val;
31                 b[i] += pos * val;
32             }
33         };
34         add(l, val), add(r + 1, -val);
35     }
36     i64 rangeSum(int l, int r) { // 区间和查询
37         auto sum = [&](int x) {
38             i64 ans = 0;
39             for (int i = x; i; i -= i & -i) {
40                 ans += (x + 1) * a[i] - b[i];
41             }
42             return ans;
43         };
44         return sum(r) - sum(l - 1);
45     }
46 };
47
48 struct BIT {
49     vector<i64> w;
50     int n;
51     BIT(int n) : n(n), w(2 * n + 1) {}
52
53     void modify(int pos, i64 val) {
54         for (w[pos += n] = val; pos > 1; pos /= 2) {
55             w[pos / 2] = max(w[pos], w[pos ^ 1]);
56         }

```

```

57     }
58
59     i64 rangeMax(int l, int r) {
60         r++; // 使用开区间实现
61         i64 res = -1e18;
62         for (l += n, r += n; l < r; l /= 2, r /= 2) {
63             if (l % 2) res = max(res, w[l++]);
64             if (r % 2) res = max(res, w[--r]);
65         }
66         return res;
67     }
68 };
69
70 struct BIT {
71     int n;
72     vector<int> w, chk; // chk 为传入的待处理数组
73     BIT(auto &in) : n(in.size() - 1), w(in.size()), chk(in) {}
74     void add(int x, i64 v) {...}
75     i64 rangeAsk(int l, int r) {...}
76     i64 get() {
77         vector<array<int, 2>> alls;
78         for (int i = 1; i <= n; i++) {
79             alls.push_back({chk[i], i});
80         }
81         sort(alls.begin(), alls.end());
82         i64 ans = 0;
83         for (auto [val, idx] : alls) {
84             ans += ask(idx + 1, n);
85             add(idx, 1);
86         }
87         return ans;
88     }
89 };
90
91 struct BIT {
92     int n;
93     vector<int> w;
94     BIT(int n) : n(n), w(n + 1) {}
95     void add(int x, int v) {
96         for (; x <= n; x += x & -x) {
97             w[x] += v;
98         }
99     }
100     int kth(int x) { // 查找第 k 小的值
101         int ans = 0;
102         for (int i = __lg(n); i >= 0; i--) {
103             int val = ans + (1 << i);
104             if (val < n && w[val] < x) {
105                 x -= w[val];
106                 ans = val;
107             }
108         }
109         return ans + 1;

```

```

110     }
111     int get(int x) { // 查找 x 的排名
112         int ans = 1;
113         for (x--; x; x -= x & -x) {
114             ans += w[x];
115         }
116         return ans;
117     }
118     int pre(int x) { return kth(get(x) - 1); } // 查找 x 的前驱
119     int suf(int x) { return kth(get(x + 1)); } // 查找 x 的后继
120 };
121 const int N = 10000000; // 可以用于在线处理平衡二叉树的全部要求
122 signed main() {
123     BIT bit(N + 1); // 在线处理不能够离散化，一定要开到比最大值更大
124     int n;
125     cin >> n;
126     for (int i = 1; i <= n; i++) {
127         int op, x;
128         cin >> op >> x;
129         if (op == 1) bit.add(x, 1); // 插入 x
130         else if (op == 2) bit.add(x, -1); // 删除任意一个 x
131         else if (op == 3) cout << bit.get(x) << "\n"; // 查询 x 的排名
132         else if (op == 4) cout << bit.kth(x) << "\n"; // 查询排名为 x 的数
133         else if (op == 5) cout << bit.pre(x) << "\n"; // 求小于 x 的最大值 (前驱)
134         else if (op == 6) cout << bit.suf(x) << "\n"; // 求大于 x 的最小值 (后继)
135     }
136 }

```

3.2 线段树

3.2.1 线段树板子 (jiangly)

```

1  A - 线段树 (SegmentTree 基础区间加乘)
2  -----
3
4  [2023-10-18](https://cf.dianhsu.com/gym/104417/submission/223800089)
5
6  struct SegmentTree {
7      int n;
8      std::vector<int> tag, sum;
9      SegmentTree(int n_) : n(n_), tag(4 * n, 1), sum(4 * n) {}
10
11     void pull(int p) {
12         sum[p] = (sum[2 * p] + sum[2 * p + 1]) % P;
13     }
14
15     void mul(int p, int v) {
16         tag[p] = 1LL * tag[p] * v % P;
17         sum[p] = 1LL * sum[p] * v % P;
18     }
19
20     void push(int p) {

```

```
21     mul(2 * p, tag[p]);
22     mul(2 * p + 1, tag[p]);
23     tag[p] = 1;
24 }
25
26 int query(int p, int l, int r, int x, int y) {
27     if (l >= y || r <= x) {
28         return 0;
29     }
30     if (l >= x && r <= y) {
31         return sum[p];
32     }
33     int m = (l + r) / 2;
34     push(p);
35     return (query(2 * p, l, m, x, y) + query(2 * p + 1, m, r, x, y)) % P;
36 }
37
38 int query(int x, int y) {
39     return query(1, 0, n, x, y);
40 }
41
42 void rangeMul(int p, int l, int r, int x, int y, int v) {
43     if (l >= y || r <= x) {
44         return;
45     }
46     if (l >= x && r <= y) {
47         return mul(p, v);
48     }
49     int m = (l + r) / 2;
50     push(p);
51     rangeMul(2 * p, l, m, x, y, v);
52     rangeMul(2 * p + 1, m, r, x, y, v);
53     pull(p);
54 }
55
56 void rangeMul(int x, int y, int v) {
57     rangeMul(1, 0, n, x, y, v);
58 }
59
60 void add(int p, int l, int r, int x, int v) {
61     if (r - l == 1) {
62         sum[p] = (sum[p] + v) % P;
63         return;
64     }
65     int m = (l + r) / 2;
66     push(p);
67     if (x < m) {
68         add(2 * p, l, m, x, v);
69     } else {
70         add(2 * p + 1, m, r, x, v);
71     }
72     pull(p);
73 }
```

```

74
75     void add(int x, int v) {
76         add(1, 0, n, x, v);
77     }
78 };
79
80
81 B - 线段树 (SegmentTree+Info 查找前驱后继)
82 -----
83
84 [2023-08-11](https://ac.nowcoder.com/acm/contest/view-submission?submissionId
    =63382128)
85
86 template<class Info>
87 struct SegmentTree {
88     int n;
89     std::vector<Info> info;
90     SegmentTree() : n(0) {}
91     SegmentTree(int n_, Info v_ = Info()) {
92         init(n_, v_);
93     }
94     template<class T>
95     SegmentTree(std::vector<T> init_) {
96         init(init_);
97     }
98     void init(int n_, Info v_ = Info()) {
99         init(std::vector(n_, v_));
100    }
101    template<class T>
102    void init(std::vector<T> init_) {
103        n = init_.size();
104        info.assign(4 << std::lg(n), Info());
105        std::function<void(int, int, int)> build = [&](int p, int l, int r) {
106            if (r - l == 1) {
107                info[p] = init_[l];
108                return;
109            }
110            int m = (l + r) / 2;
111            build(2 * p, l, m);
112            build(2 * p + 1, m, r);
113            pull(p);
114        };
115        build(1, 0, n);
116    }
117    void pull(int p) {
118        info[p] = info[2 * p] + info[2 * p + 1];
119    }
120    void modify(int p, int l, int r, int x, const Info &v) {
121        if (r - l == 1) {
122            info[p] = v;
123            return;
124        }
125        int m = (l + r) / 2;

```

```

126     if (x < m) {
127         modify(2 * p, l, m, x, v);
128     } else {
129         modify(2 * p + 1, m, r, x, v);
130     }
131     pull(p);
132 }
133 void modify(int p, const Info &v) {
134     modify(1, 0, n, p, v);
135 }
136 Info rangeQuery(int p, int l, int r, int x, int y) {
137     if (l >= y || r <= x) {
138         return Info();
139     }
140     if (l >= x && r <= y) {
141         return info[p];
142     }
143     int m = (l + r) / 2;
144     return rangeQuery(2 * p, l, m, x, y) + rangeQuery(2 * p + 1, m, r, x, y);
145 }
146 Info rangeQuery(int l, int r) {
147     return rangeQuery(1, 0, n, l, r);
148 }
149 template<class F>
150 int findFirst(int p, int l, int r, int x, int y, F pred) {
151     if (l >= y || r <= x || !pred(info[p])) {
152         return -1;
153     }
154     if (r - l == 1) {
155         return l;
156     }
157     int m = (l + r) / 2;
158     int res = findFirst(2 * p, l, m, x, y, pred);
159     if (res == -1) {
160         res = findFirst(2 * p + 1, m, r, x, y, pred);
161     }
162     return res;
163 }
164 template<class F>
165 int findFirst(int l, int r, F pred) {
166     return findFirst(1, 0, n, l, r, pred);
167 }
168 template<class F>
169 int findLast(int p, int l, int r, int x, int y, F pred) {
170     if (l >= y || r <= x || !pred(info[p])) {
171         return -1;
172     }
173     if (r - l == 1) {
174         return l;
175     }
176     int m = (l + r) / 2;
177     int res = findLast(2 * p + 1, m, r, x, y, pred);
178     if (res == -1) {

```

```

179         res = findLast(2 * p, l, m, x, y, pred);
180     }
181     return res;
182 }
183 template<class F>
184 int findLast(int l, int r, F pred) {
185     return findLast(1, 0, n, l, r, pred);
186 }
187 };
188 struct Info {
189     int cnt = 0;
190     i64 sum = 0;
191     i64 ans = 0;
192 };
193 Info operator+(Info a, Info b) {
194     Info c;
195     c.cnt = a.cnt + b.cnt;
196     c.sum = a.sum + b.sum;
197     c.ans = a.ans + b.ans + a.cnt * b.sum - a.sum * b.cnt;
198     return c;
199 }
200 ...
201
202 C - 线段树 (SegmentTree+Info+Merge 区间合并)
203 -----
204 [2022-04-23](https://codeforces.com/contest/1672/submission/154766851)
205
206 template<class Info>
207 struct SegmentTree {
208     int n;
209     std::vector<Info> info;
210     SegmentTree() : n(0) {}
211     SegmentTree(int n_, Info v_ = Info()) {
212         init(n_, v_);
213     }
214     template<class T>
215     SegmentTree(std::vector<T> init_) {
216         init(init_);
217     }
218     void init(int n_, Info v_ = Info()) {
219         init(std::vector(n_, v_));
220     }
221     template<class T>
222     void init(std::vector<T> init_) {
223         n = init_.size();
224         info.assign(4 << std::lg(n), Info());
225         std::function<void(int, int, int)> build = [&](int p, int l, int r) {
226             if (r - l == 1) {
227                 info[p] = init_[l];
228                 return;
229             }
230             int m = (l + r) / 2;
231             build(2 * p, l, m);

```

```

232         build(2 * p + 1, m, r);
233         pull(p);
234     };
235     build(1, 0, n);
236 }
237 void pull(int p) {
238     info[p] = info[2 * p] + info[2 * p + 1];
239 }
240 void modify(int p, int l, int r, int x, const Info &v) {
241     if (r - l == 1) {
242         info[p] = v;
243         return;
244     }
245     int m = (l + r) / 2;
246     if (x < m) {
247         modify(2 * p, l, m, x, v);
248     } else {
249         modify(2 * p + 1, m, r, x, v);
250     }
251     pull(p);
252 }
253 void modify(int p, const Info &v) {
254     modify(1, 0, n, p, v);
255 }
256 Info rangeQuery(int p, int l, int r, int x, int y) {
257     if (l >= y || r <= x) {
258         return Info();
259     }
260     if (l >= x && r <= y) {
261         return info[p];
262     }
263     int m = (l + r) / 2;
264     return rangeQuery(2 * p, l, m, x, y) + rangeQuery(2 * p + 1, m, r, x, y);
265 }
266 Info rangeQuery(int l, int r) {
267     return rangeQuery(1, 0, n, l, r);
268 }
269 template<class F>
270 int findFirst(int p, int l, int r, int x, int y, F pred) {
271     if (l >= y || r <= x || !pred(info[p])) {
272         return -1;
273     }
274     if (r - l == 1) {
275         return l;
276     }
277     int m = (l + r) / 2;
278     int res = findFirst(2 * p, l, m, x, y, pred);
279     if (res == -1) {
280         res = findFirst(2 * p + 1, m, r, x, y, pred);
281     }
282     return res;
283 }
284 template<class F>

```



```

285     int findFirst(int l, int r, F pred) {
286         return findFirst(1, 0, n, l, r, pred);
287     }
288     template<class F>
289     int findLast(int p, int l, int r, int x, int y, F pred) {
290         if (l >= y || r <= x || !pred(info[p])) {
291             return -1;
292         }
293         if (r - l == 1) {
294             return l;
295         }
296         int m = (l + r) / 2;
297         int res = findLast(2 * p + 1, m, r, x, y, pred);
298         if (res == -1) {
299             res = findLast(2 * p, l, m, x, y, pred);
300         }
301         return res;
302     }
303     template<class F>
304     int findLast(int l, int r, F pred) {
305         return findLast(1, 0, n, l, r, pred);
306     }
307 };
308
309 struct Info {
310     int x = 0;
311     int cnt = 0;
312 };
313
314 Info operator+(Info a, Info b) {
315     if (a.x == b.x) {
316         return {a.x, a.cnt + b.cnt};
317     } else if (a.cnt > b.cnt) {
318         return {a.x, a.cnt - b.cnt};
319     } else {
320         return {b.x, b.cnt - a.cnt};
321     }
322 }
323
324 2A - 懒标记线段树 (LazySegmentTree 基础区间修改)
325 -----
326
327 [2023-07-17](https://ac.nowcoder.com/acm/contest/view-submission?submissionId
328     =62804432)
329
330 template<class Info, class Tag>
331 struct LazySegmentTree {
332     const int n;
333     std::vector<Info> info;
334     std::vector<Tag> tag;
335     LazySegmentTree(int n) : n(n), info(4 << std::__lg(n)), tag(4 << std::__lg(n))
336         {}
337     LazySegmentTree(std::vector<Info> init) : LazySegmentTree(init.size()) {

```

```

336     std::function<void(int, int, int)> build = [&](int p, int l, int r) {
337         if (r - l == 1) {
338             info[p] = init[l];
339             return;
340         }
341         int m = (l + r) / 2;
342         build(2 * p, l, m);
343         build(2 * p + 1, m, r);
344         pull(p);
345     };
346     build(1, 0, n);
347 }
348 void pull(int p) {
349     info[p] = info[2 * p] + info[2 * p + 1];
350 }
351 void apply(int p, const Tag &v) {
352     info[p].apply(v);
353     tag[p].apply(v);
354 }
355 void push(int p) {
356     apply(2 * p, tag[p]);
357     apply(2 * p + 1, tag[p]);
358     tag[p] = Tag();
359 }
360 void modify(int p, int l, int r, int x, const Info &v) {
361     if (r - l == 1) {
362         info[p] = v;
363         return;
364     }
365     int m = (l + r) / 2;
366     push(p);
367     if (x < m) {
368         modify(2 * p, l, m, x, v);
369     } else {
370         modify(2 * p + 1, m, r, x, v);
371     }
372     pull(p);
373 }
374 void modify(int p, const Info &v) {
375     modify(1, 0, n, p, v);
376 }
377 Info rangeQuery(int p, int l, int r, int x, int y) {
378     if (l >= y || r <= x) {
379         return Info();
380     }
381     if (l >= x && r <= y) {
382         return info[p];
383     }
384     int m = (l + r) / 2;
385     push(p);
386     return rangeQuery(2 * p, l, m, x, y) + rangeQuery(2 * p + 1, m, r, x, y);
387 }
388 Info rangeQuery(int l, int r) {

```

```

389     return rangeQuery(1, 0, n, l, r);
390 }
391 void rangeApply(int p, int l, int r, int x, int y, const Tag &v) {
392     if (l >= y || r <= x) {
393         return;
394     }
395     if (l >= x && r <= y) {
396         apply(p, v);
397         return;
398     }
399     int m = (l + r) / 2;
400     push(p);
401     rangeApply(2 * p, l, m, x, y, v);
402     rangeApply(2 * p + 1, m, r, x, y, v);
403     pull(p);
404 }
405 void rangeApply(int l, int r, const Tag &v) {
406     return rangeApply(1, 0, n, l, r, v);
407 }
408 void half(int p, int l, int r) {
409     if (info[p].act == 0) {
410         return;
411     }
412     if ((info[p].min + 1) / 2 == (info[p].max + 1) / 2) {
413         apply(p, {-(info[p].min + 1) / 2});
414         return;
415     }
416     int m = (l + r) / 2;
417     push(p);
418     half(2 * p, l, m);
419     half(2 * p + 1, m, r);
420     pull(p);
421 }
422 void half() {
423     half(1, 0, n);
424 }
425 };
426
427 constexpr i64 inf = 1E18;
428
429 struct Tag {
430     i64 add = 0;
431
432     void apply(Tag t) {
433         add += t.add;
434     }
435 };
436
437 struct Info {
438     i64 min = inf;
439     i64 max = -inf;
440     i64 sum = 0;
441     i64 act = 0;

```

```

442
443     void apply(Tag t) {
444         min += t.add;
445         max += t.add;
446         sum += act * t.add;
447     }
448 };
449
450 Info operator+(Info a, Info b) {
451     Info c;
452     c.min = std::min(a.min, b.min);
453     c.max = std::max(a.max, b.max);
454     c.sum = a.sum + b.sum;
455     c.act = a.act + b.act;
456     return c;
457 }
458
459 2B - 懒标记线段树 (LazySegmentTree 查找前驱后继)
460 -----
461 [2023-07-17](https://ac.nowcoder.com/acm/contest/view-submission?submissionId
    =62804432)
462
463 template<class Info, class Tag>
464 struct LazySegmentTree {
465     int n;
466     std::vector<Info> info;
467     std::vector<Tag> tag;
468     LazySegmentTree() : n(0) {}
469     LazySegmentTree(int n_, Info v_ = Info()) {
470         init(n_, v_);
471     }
472     template<class T>
473     LazySegmentTree(std::vector<T> init_) {
474         init(init_);
475     }
476     void init(int n_, Info v_ = Info()) {
477         init(std::vector(n_, v_));
478     }
479     template<class T>
480     void init(std::vector<T> init_) {
481         n = init_.size();
482         info.assign(4 << std::__lg(n), Info());
483         tag.assign(4 << std::__lg(n), Tag());
484         std::function<void(int, int, int)> build = [&](int p, int l, int r) {
485             if (r - l == 1) {
486                 info[p] = init_[l];
487                 return;
488             }
489             int m = (l + r) / 2;
490             build(2 * p, l, m);
491             build(2 * p + 1, m, r);
492             pull(p);
493         };

```

```

494     build(1, 0, n);
495 }
496 void pull(int p) {
497     info[p] = info[2 * p] + info[2 * p + 1];
498 }
499 void apply(int p, const Tag &v) {
500     info[p].apply(v);
501     tag[p].apply(v);
502 }
503 void push(int p) {
504     apply(2 * p, tag[p]);
505     apply(2 * p + 1, tag[p]);
506     tag[p] = Tag();
507 }
508 void modify(int p, int l, int r, int x, const Info &v) {
509     if (r - l == 1) {
510         info[p] = v;
511         return;
512     }
513     int m = (l + r) / 2;
514     push(p);
515     if (x < m) {
516         modify(2 * p, l, m, x, v);
517     } else {
518         modify(2 * p + 1, m, r, x, v);
519     }
520     pull(p);
521 }
522 void modify(int p, const Info &v) {
523     modify(1, 0, n, p, v);
524 }
525 Info rangeQuery(int p, int l, int r, int x, int y) {
526     if (l >= y || r <= x) {
527         return Info();
528     }
529     if (l >= x && r <= y) {
530         return info[p];
531     }
532     int m = (l + r) / 2;
533     push(p);
534     return rangeQuery(2 * p, l, m, x, y) + rangeQuery(2 * p + 1, m, r, x, y);
535 }
536 Info rangeQuery(int l, int r) {
537     return rangeQuery(1, 0, n, l, r);
538 }
539 void rangeApply(int p, int l, int r, int x, int y, const Tag &v) {
540     if (l >= y || r <= x) {
541         return;
542     }
543     if (l >= x && r <= y) {
544         apply(p, v);
545         return;
546     }

```

```

547     int m = (l + r) / 2;
548     push(p);
549     rangeApply(2 * p, l, m, x, y, v);
550     rangeApply(2 * p + 1, m, r, x, y, v);
551     pull(p);
552 }
553 void rangeApply(int l, int r, const Tag &v) {
554     return rangeApply(1, 0, n, l, r, v);
555 }
556 template<class F>
557 int findFirst(int p, int l, int r, int x, int y, F pred) {
558     if (l >= y || r <= x || !pred(info[p])) {
559         return -1;
560     }
561     if (r - l == 1) {
562         return l;
563     }
564     int m = (l + r) / 2;
565     push(p);
566     int res = findFirst(2 * p, l, m, x, y, pred);
567     if (res == -1) {
568         res = findFirst(2 * p + 1, m, r, x, y, pred);
569     }
570     return res;
571 }
572 template<class F>
573 int findFirst(int l, int r, F pred) {
574     return findFirst(1, 0, n, l, r, pred);
575 }
576 template<class F>
577 int findLast(int p, int l, int r, int x, int y, F pred) {
578     if (l >= y || r <= x || !pred(info[p])) {
579         return -1;
580     }
581     if (r - l == 1) {
582         return l;
583     }
584     int m = (l + r) / 2;
585     push(p);
586     int res = findLast(2 * p + 1, m, r, x, y, pred);
587     if (res == -1) {
588         res = findLast(2 * p, l, m, x, y, pred);
589     }
590     return res;
591 }
592 template<class F>
593 int findLast(int l, int r, F pred) {
594     return findLast(1, 0, n, l, r, pred);
595 }
596 };
597
598 struct Tag {
599     i64 a = 0, b = 0;

```

```

600     void apply(Tag t) {
601         a = std::min(a, b + t.a);
602         b += t.b;
603     }
604 };
605
606 int k;
607
608 struct Info {
609     i64 x = 0;
610     void apply(Tag t) {
611         x += t.a;
612         if (x < 0) {
613             x = (x % k + k) % k;
614         }
615         x += t.b - t.a;
616     }
617 };
618 Info operator+(Info a, Info b) {
619     return {a.x + b.x};
620 }
621
622 2C - 懒标记线段树 (LazySegmentTree 二分修改)
623 -----
624 [2023-03-03](https://atcoder.jp/contests/joi2023yo2/submissions/39363123)
625
626 constexpr int inf = 1E9 + 1;
627 template<class Info, class Tag>
628 struct LazySegmentTree {
629     const int n;
630     std::vector<Info> info;
631     std::vector<Tag> tag;
632     LazySegmentTree(int n) : n(n), info(4 << std::__lg(n)), tag(4 << std::__lg(n))
633     {}
634     LazySegmentTree(std::vector<Info> init) : LazySegmentTree(init.size()) {
635         std::function<void(int, int, int)> build = [&](int p, int l, int r) {
636             if (r - l == 1) {
637                 info[p] = init[l];
638                 return;
639             }
640             int m = (l + r) / 2;
641             build(2 * p, l, m);
642             build(2 * p + 1, m, r);
643             pull(p);
644         };
645         build(1, 0, n);
646     }
647     void pull(int p) {
648         info[p] = info[2 * p] + info[2 * p + 1];
649     }
650     void apply(int p, const Tag &v) {
651         info[p].apply(v);
652         tag[p].apply(v);

```

```

652     }
653     void push(int p) {
654         apply(2 * p, tag[p]);
655         apply(2 * p + 1, tag[p]);
656         tag[p] = Tag();
657     }
658     void modify(int p, int l, int r, int x, const Info &v) {
659         if (r - l == 1) {
660             info[p] = v;
661             return;
662         }
663         int m = (l + r) / 2;
664         push(p);
665         if (x < m) {
666             modify(2 * p, l, m, x, v);
667         } else {
668             modify(2 * p + 1, m, r, x, v);
669         }
670         pull(p);
671     }
672     void modify(int p, const Info &v) {
673         modify(1, 0, n, p, v);
674     }
675     Info rangeQuery(int p, int l, int r, int x, int y) {
676         if (l >= y || r <= x) {
677             return Info();
678         }
679         if (l >= x && r <= y) {
680             return info[p];
681         }
682         int m = (l + r) / 2;
683         push(p);
684         return rangeQuery(2 * p, l, m, x, y) + rangeQuery(2 * p + 1, m, r, x, y);
685     }
686     Info rangeQuery(int l, int r) {
687         return rangeQuery(1, 0, n, l, r);
688     }
689     void rangeApply(int p, int l, int r, int x, int y, const Tag &v) {
690         if (l >= y || r <= x) {
691             return;
692         }
693         if (l >= x && r <= y) {
694             apply(p, v);
695             return;
696         }
697         int m = (l + r) / 2;
698         push(p);
699         rangeApply(2 * p, l, m, x, y, v);
700         rangeApply(2 * p + 1, m, r, x, y, v);
701         pull(p);
702     }
703     void rangeApply(int l, int r, const Tag &v) {
704         return rangeApply(1, 0, n, l, r, v);

```



```

705     }
706     void maintainL(int p, int l, int r, int pre) {
707         if (info[p].difl > 0 && info[p].maxlowl < pre) {
708             return;
709         }
710         if (r - l == 1) {
711             info[p].max = info[p].maxlowl;
712             info[p].maxl = info[p].maxr = l;
713             info[p].maxlowl = info[p].maxlowr = -inf;
714             return;
715         }
716         int m = (l + r) / 2;
717         push(p);
718         maintainL(2 * p, l, m, pre);
719         pre = std::max(pre, info[2 * p].max);
720         maintainL(2 * p + 1, m, r, pre);
721         pull(p);
722     }
723     void maintainL() {
724         maintainL(1, 0, n, -1);
725     }
726     void maintainR(int p, int l, int r, int suf) {
727         if (info[p].difr > 0 && info[p].maxlowr < suf) {
728             return;
729         }
730         if (r - l == 1) {
731             info[p].max = info[p].maxlowl;
732             info[p].maxl = info[p].maxr = l;
733             info[p].maxlowl = info[p].maxlowr = -inf;
734             return;
735         }
736         int m = (l + r) / 2;
737         push(p);
738         maintainR(2 * p + 1, m, r, suf);
739         suf = std::max(suf, info[2 * p + 1].max);
740         maintainR(2 * p, l, m, suf);
741         pull(p);
742     }
743     void maintainR() {
744         maintainR(1, 0, n, -1);
745     }
746 };
747
748 struct Tag {
749     int add = 0;
750
751     void apply(Tag t) & {
752         add += t.add;
753     }
754 };
755
756 struct Info {
757     int max = -1;

```

```

758     int maxl = -1;
759     int maxr = -1;
760     int difl = inf;
761     int difr = inf;
762     int maxlowl = -inf;
763     int maxlowr = -inf;
764
765     void apply(Tag t) & {
766         if (max != -1) {
767             max += t.add;
768         }
769         difl += t.add;
770         difr += t.add;
771     }
772 };
773
774 Info operator+(Info a, Info b) {
775     Info c;
776     if (a.max > b.max) {
777         c.max = a.max;
778         c.maxl = a.maxl;
779         c.maxr = a.maxr;
780     } else if (a.max < b.max) {
781         c.max = b.max;
782         c.maxl = b.maxl;
783         c.maxr = b.maxr;
784     } else {
785         c.max = a.max;
786         c.maxl = a.maxl;
787         c.maxr = b.maxr;
788     }
789
790     c.difl = std::min(a.difl, b.difl);
791     c.difr = std::min(a.difr, b.difr);
792     if (a.max != -1) {
793         c.difl = std::min(c.difl, a.max - b.maxlowl);
794     }
795     if (b.max != -1) {
796         c.difr = std::min(c.difr, b.max - a.maxlowr);
797     }
798
799     if (a.max == -1) {
800         c.maxlowl = std::max(a.maxlowl, b.maxlowl);
801     } else {
802         c.maxlowl = a.maxlowl;
803     }
804     if (b.max == -1) {
805         c.maxlowr = std::max(a.maxlowr, b.maxlowr);
806     } else {
807         c.maxlowr = b.maxlowr;
808     }
809     return c;
810 }

```

3.2.2 线段树: 维护最大子段和

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  using ll = long long;
4  struct Node {
5      ll sum; // 区间总和
6      ll pref; // 前缀最大和: 以区间左端点开始的最大子段和
7      ll suff; // 后缀最大和: 以区间右端点结束的最大子段和
8      ll best; // 区间最大子数组和
9      Node(ll v = 0) : sum(v), pref(max(0LL, v)), suff(max(0LL, v)), best(max(0LL, v))
10 {}
11 };
12 // 合并左右子节点信息, 得到父节点信息
13 Node merge(const Node &L, const Node &R) {
14     Node res;
15     // 区间总和= 左总和+ 右总和
16     res.sum = L.sum + R.sum;
17     // 前缀最大和: 要么完全在左侧, 要么左侧全选再加右侧前缀
18     res.pref = max(L.pref, L.sum + R.pref);
19     // 后缀最大和: 要么完全在右侧, 要么右侧全选再加左侧后缀
20     res.suff = max(R.suff, R.sum + L.suff);
21     // 区间最大子数组和: 要么在左子区间、要么在右子区间、要么跨越左右
22     res.best = max({L.best, R.best, L.suff + R.pref});
23     return res;
24 }
25 struct SegTree {
26     int n;
27     vector<Node> t;
28     SegTree(int _n) : n(_n), t(4*n) {}
29     void build(int p, int l, int r, const vector<ll> &a) {
30
31         if (l == r) {
32             t[p] = Node(a[l]);
33             return;
34         }
35         int m = (l + r) >> 1;
36         build(p<<1, l, m, a);
37         build(p<<1|1, m+1, r, a);
38         t[p] = merge(t[p<<1], t[p<<1|1]);
39     }
40     void update(int p, int l, int r, int idx, ll v) {
41         if (l == r) {
42             t[p] = Node(v);
43             return;
44         }
45         int m = (l + r) >> 1;
46         if (idx <= m) update(p<<1, l, m, idx, v);
47         else update(p<<1|1, m+1, r, idx, v);
48         t[p] = merge(t[p<<1], t[p<<1|1]);
49     }
50     Node query(int p, int l, int r, int ql, int qr) {
51         if (ql <= l && r <= qr) return t[p];

```

```

52 int m = (l + r) >> 1;
53 if (qr <= m) return query(p<<1, l, m, ql, qr);
54 if (ql > m) return query(p<<1|1, m+1, r, ql, qr);
55 Node L = query(p<<1, l, m, ql, qr);
56 Node R = query(p<<1|1, m+1, r, ql, qr);
57 return merge(L, R);
58 }
59 };
60 int main() {
61 ios::sync_with_stdio(false);
62 cin.tie(nullptr);
63 int n, q;
64 cin >> n >> q;
65 vector<ll> a(n+1);
66 for (int i = 1; i <= n; i++) {
67 cin >> a[i];
68 }
69 SegTree st(n);
70 st.build(1, 1, n, a);
71 while (q--) {
72 char op;
73 cin >> op;
74 if (op == 'U') {
75 int idx; ll v;
76 cin >> idx >> v;
77 st.update(1, 1, n, idx, v);
78 }
79 } else if (op == 'Q') {
80 int l, r;
81 cin >> l >> r;
82 auto res = st.query(1, 1, n, l, r);
83 cout << res.best << "\n";
84 }
85 }
86 return 0;
87 }
88 3.9

```

3.2.3 线段树

区间加法修改、区间最小值查询

同时需要处理区间加法与乘法修改

区间赋值/推平

如果存在推平为 0 的操作，那么需要将 lazy 初始赋值为 -1。

区间取模

原题需要进行“单点赋值 + 区间取模 + 区间求和” See。该操作不需要懒标记。

需要额外维护一个区间最大值，当模数大于区间最大值时剪枝，否则进行单点取模。由于单点 $\text{MOD} < x$ 时 $x \bmod \text{MOD} < \frac{x}{2}$ ，故单点取模至 0 最劣只需要 $\log x$ 次。

区间异或修改

原题需要维护” 区间异或修改 + 区间求和 “See 。

拆位运算

原题同上。使用若干棵线段树维护每一位的值，区间异或转变为区间翻转。

```

1  template<class T> struct Segt {
2      struct node {
3          int l, r;
4          T w, rmq, lazy;
5      };
6      vector<T> w;
7      vector<node> t;
8
9      Segt() {}
10     Segt(int n) { init(n); }
11     Segt(vector<int> in) {
12         int n = in.size() - 1;
13         w.resize(n + 1);
14         for (int i = 1; i <= n; i++) {
15             w[i] = in[i];
16         }
17         init(in.size() - 1);
18     }
19
20     #define GL (k << 1)
21     #define GR (k << 1 | 1)
22
23     void init(int n) {
24         w.resize(n + 1);
25         t.resize(n * 4 + 1);
26         auto build = [&](auto self, int l, int r, int k = 1) {
27             if (l == r) {
28                 t[k] = {l, r, w[l], w[l], -1}; // 如果有赋值为 0 的操作，则懒标记必须要 -1
29                 return;
30             }
31             t[k] = {l, r, 0, 0, -1};
32             int mid = (l + r) / 2;
33             self(self, l, mid, GL);
34             self(self, mid + 1, r, GR);
35             pushup(k);
36         };
37         build(build, 1, n);
38     }
39     void pushdown(node &p, T lazy) { /* 【在此更新下递函数】 */
40         p.w += (p.r - p.l + 1) * lazy;
41         p.rm += lazy;
42         p.lazy += lazy;
43     }
44     void pushdown(int k) {
45         if (t[k].lazy == -1) return;
46         pushdown(t[GL], t[k].lazy);
47         pushdown(t[GR], t[k].lazy);
48         t[k].lazy = -1;

```

```

49     }
50     void pushup(int k) {
51         auto pushup = [&](node &p, node &l, node &r) { /* 【在此更新上传函数】 */
52             p.w = l.w + r.w;
53             p.rmq = min(l.rmq, r.rmq); // RMQ -> min/max
54         };
55         pushup(t[k], t[GL], t[GR]);
56     }
57     void modify(int l, int r, T val, int k = 1) { // 区间修改
58         if (l <= t[k].l && t[k].r <= r) {
59             pushdown(t[k], val);
60             return;
61         }
62         pushdown(k);
63         int mid = (t[k].l + t[k].r) / 2;
64         if (l <= mid) modify(l, r, val, GL);
65         if (mid < r) modify(l, r, val, GR);
66         pushup(k);
67     }
68     T rmq(int l, int r, int k = 1) { // 区间询问最小值
69         if (l <= t[k].l && t[k].r <= r) {
70             return t[k].rmq;
71         }
72         pushdown(k);
73         int mid = (t[k].l + t[k].r) / 2;
74         T ans = numeric_limits<T>::max(); // RMQ -> 为 max 时需要修改为 ::lowest()
75         if (l <= mid) ans = min(ans, rmq(l, r, GL)); // RMQ -> min/max
76         if (mid < r) ans = min(ans, rmq(l, r, GR)); // RMQ -> min/max
77         return ans;
78     }
79     T ask(int l, int r, int k = 1) { // 区间询问
80         if (l <= t[k].l && t[k].r <= r) {
81             return t[k].w;
82         }
83         pushdown(k);
84         int mid = (t[k].l + t[k].r) / 2;
85         T ans = 0;
86         if (l <= mid) ans += ask(l, r, GL);
87         if (mid < r) ans += ask(l, r, GR);
88         return ans;
89     }
90     void debug(int k = 1) {
91         cout << "[" << t[k].l << ", " << t[k].r << "]: ";
92         cout << "w = " << t[k].w << ", ";
93         cout << "Min = " << t[k].rmq << ", ";
94         cout << "lazy = " << t[k].lazy << ", ";
95         cout << endl;
96         if (t[k].l == t[k].r) return;
97         debug(GL), debug(GR);
98     }
99 };
100
101 template <class T> struct Segt_ {

```

```

102 struct node {
103     int l, r;
104     T w, add, mul = 1; // 注意初始赋值
105 };
106 vector<T> w;
107 vector<node> t;
108
109 Segt_(int n) {
110     w.resize(n + 1);
111     t.resize((n << 2) + 1);
112     build(1, n);
113 }
114 Segt_(vector<int> in) {
115     int n = in.size() - 1;
116     w.resize(n + 1);
117     for (int i = 1; i <= n; i++) {
118         w[i] = in[i];
119     }
120     t.resize((n << 2) + 1);
121     build(1, n);
122 }
123 void pushdown(node &p, T add, T mul) { // 在此更新下递函数
124     p.w = p.w * mul + (p.r - p.l + 1) * add;
125     p.add = p.add * mul + add;
126     p.mul *= mul;
127 }
128 void pushup(node &p, node &l, node &r) { // 在此更新上传函数
129     p.w = l.w + r.w;
130 }
131 #define GL (k << 1)
132 #define GR (k << 1 | 1)
133 void pushdown(int k) { // 不需要动
134     pushdown(t[GL], t[k].add, t[k].mul);
135     pushdown(t[GR], t[k].add, t[k].mul);
136     t[k].add = 0, t[k].mul = 1;
137 }
138 void pushup(int k) { // 不需要动
139     pushup(t[k], t[GL], t[GR]);
140 }
141 void build(int l, int r, int k = 1) {
142     if (l == r) {
143         t[k] = {l, r, w[l]};
144         return;
145     }
146     t[k] = {l, r};
147     int mid = (l + r) / 2;
148     build(l, mid, GL);
149     build(mid + 1, r, GR);
150     pushup(k);
151 }
152 void modify(int l, int r, T val, int k = 1) { // 区间修改
153     if (l <= t[k].l && t[k].r <= r) {
154         t[k].w += (t[k].r - t[k].l + 1) * val;

```

```

155         t[k].add += val;
156         return;
157     }
158     pushdown(k);
159     int mid = (t[k].l + t[k].r) / 2;
160     if (l <= mid) modify(l, r, val, GL);
161     if (mid < r) modify(l, r, val, GR);
162     pushup(k);
163 }
164 void modify2(int l, int r, T val, int k = 1) { // 区间修改
165     if (l <= t[k].l && t[k].r <= r) {
166         t[k].w *= val;
167         t[k].add *= val;
168         t[k].mul *= val;
169         return;
170     }
171     pushdown(k);
172     int mid = (t[k].l + t[k].r) / 2;
173     if (l <= mid) modify2(l, r, val, GL);
174     if (mid < r) modify2(l, r, val, GR);
175     pushup(k);
176 }
177 T ask(int l, int r, int k = 1) { // 区间询问, 不合并
178     if (l <= t[k].l && t[k].r <= r) {
179         return t[k].w;
180     }
181     pushdown(k);
182     int mid = (t[k].l + t[k].r) / 2;
183     T ans = 0;
184     if (l <= mid) ans += ask(l, r, GL);
185     if (mid < r) ans += ask(l, r, GR);
186     return ans;
187 }
188 void debug(int k = 1) {
189     cout << "[" << t[k].l << ", " << t[k].r << "]: ";
190     cout << "w = " << t[k].w << ", ";
191     cout << "add = " << t[k].add << ", ";
192     cout << "mul = " << t[k].mul << ", ";
193     cout << endl;
194     if (t[k].l == t[k].r) return;
195     debug(GL), debug(GR);
196 }
197 };
198
199 void pushdown(node &p, T lazy) { /* 【在此更新下递函数】 */
200     p.w = (p.r - p.l + 1) * lazy;
201     p.lazy = lazy;
202 }
203 void modify(int l, int r, T val, int k = 1) {
204     if (l <= t[k].l && t[k].r <= r) {
205         t[k].w = val;
206         t[k].lazy = val;
207         return;

```



```

208     }
209     // 剩余部分不变
210 }
211
212 void modifyMod(int l, int r, T val, int k = 1) {
213     if (l <= t[k].l && t[k].r <= r) {
214         if (t[k].rmq < val) return; // 重要剪枝
215     }
216     if (t[k].l == t[k].r) {
217         t[k].w %= val;
218         t[k].rmq %= val;
219         return;
220     }
221     int mid = (t[k].l + t[k].r) / 2;
222     if (l <= mid) modifyMod(l, r, val, GL);
223     if (mid < r) modifyMod(l, r, val, GR);
224     pushup(k);
225 }
226
227 struct Segt { // #define GL (k << 1) // #define GR (k << 1 | 1)
228     struct node {
229         int l, r;
230         int w[N], lazy; // 注意这里为了方便计算, w 只需要存位
231     };
232     vector<int> base;
233     vector<node> t;
234
235     Segt(vector<int> in) : base(in) {
236         int n = in.size() - 1;
237         t.resize(n * 4 + 1);
238         auto build = [&](auto self, int l, int r, int k = 1) {
239             t[k] = {l, r}; // 前置赋值
240             if (l == r) {
241                 for (int i = 0; i < N; i++) {
242                     t[k].w[i] = base[l] >> i & 1;
243                 }
244                 return;
245             }
246             int mid = (l + r) / 2;
247             self(self, l, mid, GL);
248             self(self, mid + 1, r, GR);
249             pushup(k);
250         };
251         build(build, 1, n);
252     }
253     void pushdown(node &p, int lazy) { /* 【在此更新下递函数】 */
254         int len = p.r - p.l + 1;
255         for (int i = 0; i < N; i++) {
256             if (lazy >> i & 1) { // 即 p.w = (p.r - p.l + 1) - p.w;
257                 p.w[i] = len - p.w[i];
258             }
259         }
260         p.lazy ^= lazy;

```

```

261     }
262     void pushdown(int k) { // 【不需要动】
263         if (t[k].lazy == 0) return;
264         pushdown(t[GL], t[k].lazy);
265         pushdown(t[GR], t[k].lazy);
266         t[k].lazy = 0;
267     }
268     void pushup(int k) {
269         auto pushup = [&](node &p, node &l, node &r) { /* 【在此更新上传函数】 */
270             for (int i = 0; i < N; i++) {
271                 p.w[i] = l.w[i] + r.w[i]; // 即 p.w = l.w + r.w;
272             }
273         };
274         pushup(t[k], t[GL], t[GR]);
275     }
276     void modify(int l, int r, int val, int k = 1) { // 区间修改
277         if (l <= t[k].l && t[k].r <= r) {
278             pushdown(t[k], val);
279             return;
280         }
281         pushdown(k);
282         int mid = (t[k].l + t[k].r) / 2;
283         if (l <= mid) modify(l, r, val, GL);
284         if (mid < r) modify(l, r, val, GR);
285         pushup(k);
286     }
287     i64 ask(int l, int r, int k = 1) { // 区间求和
288         if (l <= t[k].l && t[k].r <= r) {
289             i64 ans = 0;
290             for (int i = 0; i < N; i++) {
291                 ans += t[k].w[i] * (1LL << i);
292             }
293             return ans;
294         }
295         pushdown(k);
296         int mid = (t[k].l + t[k].r) / 2;
297         i64 ans = 0;
298         if (l <= mid) ans += ask(l, r, GL);
299         if (mid < r) ans += ask(l, r, GR);
300         return ans;
301     }
302 };
303
304 template<class T> struct Segt_ { // GL 为 (k << 1), GR 为 (k << 1 | 1)
305     struct node {
306         int l, r;
307         T w;
308         bool lazy; // 注意懒标记用布尔型足以
309     };
310     vector<T> w;
311     vector<node> t;
312
313     Segt_() {}

```

```

314 void init(vector<int> in) {
315     int n = in.size() - 1;
316     w.resize(n * 4 + 1);
317     for (int i = 0; i <= n; i++) { w[i] = in[i]; }
318     t.resize(n * 4 + 1);
319     build(1, n);
320 }
321 void pushdown(node &p, bool lazy = 1) { // 【在此更新下递函数】
322     p.w = (p.r - p.l + 1) - p.w;
323     p.lazy ^= lazy;
324 }
325 void pushup(node &p, node &l, node &r) { // 【在此更新上传函数】
326     p.w = l.w + r.w;
327 }
328 void pushdown(int k) { // 【不需要动】
329     if (t[k].lazy == 0) return;
330     pushdown(t[GL]), pushdown(t[GR]); // 注意这里不再需要传入第二个参数
331     t[k].lazy = 0;
332 }
333 void pushup(int k) { pushup(t[k], t[GL], t[GR]); } // 【不需要动】
334 void build(int l, int r, int k = 1) {
335     if (l == r) {
336         t[k] = {l, r, w[l], 0}; // 注意懒标记初始为 0
337         return;
338     }
339     t[k] = {l, r};
340     int mid = (l + r) / 2;
341     build(l, mid, GL);
342     build(mid + 1, r, GR);
343     pushup(k);
344 }
345 void reverse(int l, int r, int k = 1) { // 区间翻转
346     if (l <= t[k].l && t[k].r <= r) {
347         pushdown(t[k], 1);
348         return;
349     }
350     pushdown(k);
351     int mid = (t[k].l + t[k].r) / 2;
352     if (l <= mid) reverse(l, r, GL);
353     if (mid < r) reverse(l, r, GR);
354     pushup(k);
355 }
356 T ask(int l, int r, int k = 1) { // 区间求和
357     if (l <= t[k].l && t[k].r <= r) {
358         return t[k].w;
359     }
360     pushdown(k);
361     int mid = (t[k].l + t[k].r) / 2;
362     T ans = 0;
363     if (l <= mid) ans += ask(l, r, GL);
364     if (mid < r) ans += ask(l, r, GR);
365     return ans;
366 }

```

```

367 };
368 signed main() {
369     int n; cin >> n;
370     vector in(20, vector<int>(n + 1));
371     Segt_<i64> segt[20]; // 拆位建线段树
372     for (int i = 1, x; i <= n; i++) { cin >> x;
373         for (int bit = 0; bit < 20; bit++) {
374             in[bit][i] = x >> bit & 1;
375         }
376     }
377     for (int i = 0; i < 20; i++) {
378         segt[i].init(in[i]);
379     }
380
381     int m, op;
382     for (cin >> m; m; m--) { cin >> op;
383         if (op == 1) {
384             int l, r; i64 ans = 0; cin >> l >> r;
385             for (int i = 0; i < 20; i++) {
386                 ans += segt[i].ask(l, r) * (1LL << i);
387             }
388             cout << ans << "\n";
389         } else {
390             int l, r, val; cin >> l >> r >> val;
391             for (int i = 0; i < 20; i++) {
392                 if (val >> i & 1) { segt[i].reverse(l, r); }
393             }
394         }
395     }
396 }

```

3.2.4 轻重链剖分/树链剖分

将线段树处理的部分分离，方便修改。支持链上查询/修改、子树查询/修改，建树时间复杂度 $\mathcal{O}(N \log N)$ ，单次查询时间复杂度 $\mathcal{O}(\log^2 N)$ 。

```

1  struct Segt {
2      struct node {
3          int l, r, w, lazy;
4      };
5      vector<int> w;
6      vector<node> t;
7
8      Segt() {}
9      #define GL (k << 1)
10     #define GR (k << 1 | 1)
11
12     void init(vector<int> in) {
13         int n = in.size() - 1;
14         w.resize(n + 1);
15         for (int i = 1; i <= n; i++) {
16             w[i] = in[i];

```

```

17     }
18     t.resize(n * 4 + 1);
19     auto build = [&](auto self, int l, int r, int k = 1) {
20         if (l == r) {
21             t[k] = {l, r, w[l], 0}; // 如果有赋值为 0 的操作, 则懒标记必须要 -1
22             return;
23         }
24         t[k] = {l, r};
25         int mid = (l + r) / 2;
26         self(self, l, mid, GL);
27         self(self, mid + 1, r, GR);
28         pushup(k);
29     };
30     build(build, 1, n);
31 }
32 void pushdown(node &p, int lazy) { /* 【在此更新下递函数】 */
33     p.w += (p.r - p.l + 1) * lazy;
34     p.lazy += lazy;
35 }
36 void pushdown(int k) { // 不需要动
37     if (t[k].lazy == 0) return;
38     pushdown(t[GL], t[k].lazy);
39     pushdown(t[GR], t[k].lazy);
40     t[k].lazy = 0;
41 }
42 void pushup(int k) { // 不需要动
43     auto pushup = [&](node &p, node &l, node &r) { /* 【在此更新上传函数】 */
44         p.w = l.w + r.w;
45     };
46     pushup(t[k], t[GL], t[GR]);
47 }
48 void modify(int l, int r, int val, int k = 1) {
49     if (l <= t[k].l && t[k].r <= r) {
50         pushdown(t[k], val);
51         return;
52     }
53     pushdown(k);
54     int mid = (t[k].l + t[k].r) / 2;
55     if (l <= mid) modify(l, r, val, GL);
56     if (mid < r) modify(l, r, val, GR);
57     pushup(k);
58 }
59 int ask(int l, int r, int k = 1) {
60     if (l <= t[k].l && t[k].r <= r) {
61         return t[k].w;
62     }
63     pushdown(k);
64     int mid = (t[k].l + t[k].r) / 2;
65     int ans = 0;
66     if (l <= mid) ans += ask(l, r, GL);
67     if (mid < r) ans += ask(l, r, GR);
68     return ans;
69 }

```

```
70 };
71
72 struct HLD {
73     int n, idx;
74     vector<vector<int>> ver;
75     vector<int> siz, dep;
76     vector<int> top, son, parent;
77     vector<int> in, id, val;
78     Segt segt;
79
80     HLD(int n) {
81         this->n = n;
82         ver.resize(n + 1);
83         siz.resize(n + 1);
84         dep.resize(n + 1);
85
86         top.resize(n + 1);
87         son.resize(n + 1);
88         parent.resize(n + 1);
89
90         idx = 0;
91         in.resize(n + 1);
92         id.resize(n + 1);
93         val.resize(n + 1);
94     }
95     void add(int x, int y) { // 建立双向边
96         ver[x].push_back(y);
97         ver[y].push_back(x);
98     }
99     void dfs1(int x) {
100         siz[x] = 1;
101         dep[x] = dep[parent[x]] + 1;
102         for (auto y : ver[x]) {
103             if (y == parent[x]) continue;
104             parent[y] = x;
105             dfs1(y);
106             siz[x] += siz[y];
107             if (siz[y] > siz[son[x]]) {
108                 son[x] = y;
109             }
110         }
111     }
112     void dfs2(int x, int up) {
113         id[x] = ++idx;
114         val[idx] = in[x]; // 建立编号
115         top[x] = up;
116         if (son[x]) dfs2(son[x], up);
117         for (auto y : ver[x]) {
118             if (y == parent[x] || y == son[x]) continue;
119             dfs2(y, y);
120         }
121     }
122     void modify(int l, int r, int val) { // 链上修改
```

```

123     while (top[l] != top[r]) {
124         if (dep[top[l]] < dep[top[r]]) {
125             swap(l, r);
126         }
127         segt.modify(id[top[l]], id[l], val);
128         l = parent[top[l]];
129     }
130     if (dep[l] > dep[r]) {
131         swap(l, r);
132     }
133     segt.modify(id[l], id[r], val);
134 }
135 void modify(int root, int val) { // 子树修改
136     segt.modify(id[root], id[root] + siz[root] - 1, val);
137 }
138 int ask(int l, int r) { // 链上查询
139     int ans = 0;
140     while (top[l] != top[r]) {
141         if (dep[top[l]] < dep[top[r]]) {
142             swap(l, r);
143         }
144         ans += segt.ask(id[top[l]], id[l]);
145         l = parent[top[l]];
146     }
147     if (dep[l] > dep[r]) {
148         swap(l, r);
149     }
150     return ans + segt.ask(id[l], id[r]);
151 }
152 int ask(int root) { // 子树查询
153     return segt.ask(id[root], id[root] + siz[root] - 1);
154 }
155 void work(auto in, int root = 1) { // 在此初始化
156     assert(in.size() == n + 1);
157     this->in = in;
158     dfs1(root);
159     dfs2(root, root);
160     segt.init(val); // 建立线段树
161 }
162 void work(int root = 1) { // 在此初始化
163     dfs1(root);
164     dfs2(root, root);
165     segt.init(val); // 建立线段树
166 }
167 };

```

3.3 平衡树

3.3.1 Treap: 数堆

```

1 //https://www.luogu.com.cn/problem/P3369
2

```

```
3 #include <stdio>
4 #include <stdlib>
5 using namespace std;
6
7 struct N {
8     int k, v, l, r, s;
9 } t[100005];
10 int cnt = 0, rt = 0;
11
12 int newN(int k) {
13     t[++cnt] = {k, rand(), 0, 0, 1};
14     return cnt;
15 }
16
17 void up(int x) {
18     if (x) t[x].s = t[t[x].l].s + t[t[x].r].s + 1;
19 }
20
21 void split(int x, int k, int &a, int &b) {
22     if (!x) { a = b = 0; return; }
23     if (t[x].k <= k) {
24         a = x;
25         split(t[x].r, k, t[x].r, b);
26     } else {
27         b = x;
28         split(t[x].l, k, a, t[x].l);
29     }
30     up(x);
31 }
32
33 int merge(int a, int b) {
34     if (!a || !b) return a | b;
35     if (t[a].v > t[b].v) {
36         t[a].r = merge(t[a].r, b);
37         up(a);
38         return a;
39     } else {
40         t[b].l = merge(a, t[b].l);
41         up(b);
42         return b;
43     }
44 }
45
46 void ins(int k) {
47     int x, y;
48     split(rt, k-1, x, y);
49     rt = merge(merge(x, newN(k)), y);
50 }
51
52 void del(int k) {
53     int x, y, z;
54     split(rt, k, x, z);
55     split(x, k-1, x, y);
```



```

56     if (y) y = merge(t[y].l, t[y].r);
57     rt = merge(merge(x, y), z);
58 }
59
60 int rk(int k) {
61     int x, y, r;
62     split(rt, k-1, x, y);
63     r = t[x].s + 1;
64     rt = merge(x, y);
65     return r;
66 }
67
68 int kth(int r) {
69     int x = rt;
70     while (1) {
71         if (t[t[x].l].s + 1 == r) break;
72         if (t[t[x].l].s + 1 > r) x = t[x].l;
73         else r -= t[t[x].l].s + 1, x = t[x].r;
74     }
75     return t[x].k;
76 }
77
78 int pre(int k) {
79     int x, y, p;
80     split(rt, k-1, x, y);
81     p = x;
82     while (t[p].r) p = t[p].r;
83     rt = merge(x, y);
84     return t[p].k;
85 }
86
87 int suc(int k) {
88     int x, y, s;
89     split(rt, k, x, y);
90     s = y;
91     while (t[s].l) s = t[s].l;
92     rt = merge(x, y);
93     return t[s].k;
94 }
95
96 int main() {
97     int n, op, x;
98     scanf("%d", &n);
99     while (n--) {
100         scanf("%d%d", &op, &x);
101         switch (op) {
102             case 1: ins(x); break; //插入
103             case 2: del(x); break; //删除
104             case 3: printf("%d\n", rk(x)); break; //x排名 (比x小的数的个数+1)
105             case 4: printf("%d\n", kth(x)); break; //排名x的数
106             case 5: printf("%d\n", pre(x)); break; //查询x的前驱
107             case 6: printf("%d\n", suc(x)); break; //查询x的后继
108         }

```

```

109     }
110     return 0;
111 }

```

3.3.2 pbds 扩展库实现平衡二叉树

记得加上相应的头文件，同时需要注意定义时的参数，一般只需要修改第三个参数：即定义的是大根堆还是小根堆。

附常见成员函数: `c++ empty()` / `size()` `insert(x)` // 插入元素 `x` `erase(x)` // 删除元素/迭代器 `x` `order_of_key(x)` // 返回元素 `x` 的排名 `find_by_order(x)` // 返回排名为 `x` 的元素迭代器 `lower_bound(x)` / `upper_bound(x)` // 返回迭代器 `join(Tree)` // 将 `Tree` 树的全部元素并入当前的树 `split(x, Tree)` // 将大于 `x` 的元素放入 `Tree` 树

```

1  #include <ext/pb_ds/assoc_container.hpp>
2  using namespace __gnu_pbds;
3  using V = pair<int, int>;
4  tree<V, null_type, less<V>, rb_tree_tag, tree_order_statistics_node_update> ver;
5  map<int, int> dic;
6
7  int n; cin >> n;
8  for (int i = 1, op, x; i <= n; i++) {
9      cin >> op >> x;
10     if (op == 1) { // 插入一个元素x, 允许重复
11         ver.insert({x, ++dic[x]});
12     } else if (op == 2) { // 删除元素x, 若有重复, 则任意删除一个
13         ver.erase({x, dic[x]--});
14     } else if (op == 3) { // 查询元素x的排名 (排名定义为比当前数小的数的个数+1)
15         cout << ver.order_of_key({x, 1}) + 1 << endl;
16     } else if (op == 4) { // 查询排名为x的元素
17         cout << ver.find_by_order(--x)->first << endl;
18     } else if (op == 5) { // 查询元素x的前驱
19         int idx = ver.order_of_key({x, 1}) - 1; // 无论x存不存在, idx都代表x的位置, 需要-1
20         cout << ver.find_by_order(idx)->first << endl;
21     } else if (op == 6) { // 查询元素x的后继
22         int idx = ver.order_of_key({x, dic[x]}); // 如果x不存在, 那么idx就是x的后继
23         if (ver.find({x, 1}) != ver.end()) idx++; // 如果x存在, 那么idx是x的位置, 需要+1
24         cout << ver.find_by_order(idx)->first << endl;
25     }
26 }

```

3.3.3 vector 模拟实现平衡二叉树

```

1  #define ALL(x) x.begin(), x.end()
2  #define pre lower_bound
3  #define suf upper_bound
4  int n; cin >> n;

```

```

5 vector<int> ver;
6 for (int i = 1, op, x; i <= n; i++) {
7     cin >> op >> x;
8     if (op == 1) ver.insert(pre(ALL(ver), x), x);
9     if (op == 2) ver.erase(pre(ALL(ver), x));
10    if (op == 3) cout << pre(ALL(ver), x) - ver.begin() + 1 << endl;
11    if (op == 4) cout << ver[x - 1] << endl;
12    if (op == 5) cout << ver[pre(ALL(ver), x) - ver.begin() - 1] << endl;
13    if (op == 6) cout << ver[suf(ALL(ver), x) - ver.begin()] << endl;
14 }

```

3.3.4 珂朵莉树 (OD Tree)

区间赋值的数据结构都可以骗分，在数据随机的情况下，复杂度可以保证，时间复杂度： $\mathcal{O}(N \log \log N)$ 。

```

1 struct ODT {
2     struct node {
3         int l, r;
4         mutable LL v;
5         node(int l, int r = -1, LL v = 0) : l(l), r(r), v(v) {}
6         bool operator<(const node &o) const {
7             return l < o.l;
8         }
9     };
10    set<node> s;
11    ODT() {
12        s.clear();
13    }
14    auto split(int pos) {
15        auto it = s.lower_bound(node(pos));
16        if (it != s.end() && it->l == pos) return it;
17        it--;
18        int l = it->l, r = it->r;
19        LL v = it->v;
20        s.erase(it);
21        s.insert(node(l, pos - 1, v));
22        return s.insert(node(pos, r, v)).first;
23    }
24    void assign(int l, int r, LL x) {
25        auto itr = split(r + 1), itl = split(l);
26        s.erase(itl, itr);
27        s.insert(node(l, r, x));
28    }
29    void add(int l, int r, LL x) {
30        auto itr = split(r + 1), itl = split(l);
31        for (auto it = itl; it != itr; it++) {
32            it->v += x;
33        }
34    }
35    LL kth(int l, int r, int k) {
36        vector<pair<LL, int>> a;

```

```

37     auto itr = split(r + 1), itl = split(1);
38     for (auto it = itl; it != itr; it++) {
39         a.push_back(pair<LL, int>(it->v, it->r - it->l + 1));
40     }
41     sort(a.begin(), a.end());
42     for (auto [val, len] : a) {
43         k -= len;
44         if (k <= 0) return val;
45     }
46 }
47 LL power(LL a, int b, int mod) {
48     a %= mod;
49     LL res = 1;
50     for (; b; b /= 2, a = a * a % mod) {
51         if (b % 2) {
52             res = res * a % mod;
53         }
54     }
55     return res;
56 }
57 LL powersum(int l, int r, int x, int mod) {
58     auto itr = split(r + 1), itl = split(1);
59     LL ans = 0;
60     for (auto it = itl; it != itr; it++) {
61         ans = (ans + power(it->v, x, mod) * (it->r - it->l + 1) % mod) % mod;
62     }
63     return ans;
64 }
65 };

```

3.4 莫队

3.4.1 Mo 算法

```

1  // -----
2  // 区间不同数个数
3  // -----
4  #include <bits/stdc++.h>
5  using namespace std;
6  using ll = long long;
7
8  struct Query {
9      int l, r, idx, block;
10 };
11
12 int B; // 块大小
13 vector<int> a; // 原数组
14 vector<ll> ans; // 存放每个查询答案
15 vector<int> cnt; // 计数数组
16 ll cur_ans = 0; // 当前答案
17
18 int main(){

```

```

19 ios::sync_with_stdio(false);
20 cin.tie(nullptr);
21
22 int n, q;
23 cin >> n ;
24 a.resize(n);
25 for(int i = 0; i < n; i++) {
26     cin >> a[i];
27     // 如果元素范围大, 可做离散化
28 }
29 cin>>q;
30 // 离散化示例 (如需)
31 // {
32 // vector<int> ys = a;
33 // sort(ys.begin(), ys.end());
34 // ys.erase(unique(ys.begin(), ys.end()), ys.end());
35 // for(int &x: a)
36 // x = lower_bound(ys.begin(), ys.end(), x) - ys.begin();
37 // cnt.assign(ys.size(), 0);
38 // }
39 //不离散化
40 {
41     cnt.assign(1e6+10,0);
42 }
43
44 B = max(1, int(sqrt(n)));
45 vector<Query> qs(q);
46 ans.assign(q, 0);
47
48 for(int i = 0; i < q; i++){
49     int l, r;
50     cin >> l >> r;
51     --l; --r;
52     qs[i] = {l, r, i, l / B};
53 }
54
55 sort(qs.begin(), qs.end(), [](auto &A, auto &B){
56     if (A.block != B.block) return A.block < B.block;
57     // 奇偶块交错
58     return (A.block & 1) ? (A.r > B.r) : (A.r < B.r);
59 });
60
61 // 示例: 统计区间内不同数字个数
62 auto add = [&](int x){
63     if (cnt[x]++ == 0) cur_ans++;
64 };
65 auto remove_ = [&](int x){
66     if (--cnt[x] == 0) cur_ans--;
67 };
68
69 int curL = 0, curR = -1;
70 for(auto &qv: qs){
71     while(curL > qv.l) add(a[--curL]);

```

```

72     while(curR < qv.r) add(a[++curR]);
73     while(curL < qv.l) remove_(a[curL++]);
74     while(curR > qv.r) remove_(a[curR--]);
75     ans[qv.idx] = cur_ans;
76 }
77
78 for(ll v : ans) cout << v << "\n";
79 return 0;
80 }

```

3.4.2 带修改的莫队（带时间维度的莫队）

以 $\mathcal{O}(N^{\frac{5}{3}})$ 的复杂度完成 Q 次询问的离线查询，其中每个分块的大小取 $N^{\frac{2}{3}} = \sqrt[3]{100000^2} = 2154$ （直接取会略快），也可以使用 `pow(n, 0.6666)` 划分。

```

1 signed main() {
2     int n, q;
3     cin >> n >> q;
4     vector<int> w(n + 1);
5     for (int i = 1; i <= n; i++) {
6         cin >> w[i];
7     }
8
9     vector<array<int, 4>> query = {{}}; // {左区间, 右区间, 累计修改次数, 下标}
10    vector<array<int, 2>> modify = {{}}; // {修改的值, 修改的元素下标}
11    for (int i = 1; i <= q; i++) {
12        char op;
13        cin >> op;
14        if (op == 'Q') {
15            int l, r;
16            cin >> l >> r;
17            query.push_back({l, r, (int)modify.size() - 1, (int)query.size()});
18        } else {
19            int idx, w;
20            cin >> idx >> w;
21            modify.push_back({w, idx});
22        }
23    }
24
25    int Knum = 2154; // 计算块长
26    vector<int> K(n + 1);
27    for (int i = 1; i <= n; i++) { // 固定块长
28        K[i] = (i - 1) / Knum + 1;
29    }
30    sort(query.begin() + 1, query.end(), [&](auto x, auto y) {
31        if (K[x[0]] != K[y[0]]) return x[0] < y[0];
32        if (K[x[1]] != K[y[1]]) return x[1] < y[1];
33        return x[3] < y[3];
34    });
35
36    int l = 1, r = 0, val = 0;
37    int t = 0; // 累计修改次数

```

```

38     vector<int> ans(query.size());
39     for (int i = 1; i < query.size(); i++) {
40         auto [ql, qr, qt, id] = query[i];
41         auto add = [&](int x) -> void {};
42         auto del = [&](int x) -> void {};
43         auto time = [&](int x, int l, int r) -> void {};
44         while (l > ql) add(w[--l]);
45         while (r < qr) add(w[++r]);
46         while (l < ql) del(w[l++]);
47         while (r > qr) del(w[r--]);
48         while (t < qt) time(++t, ql, qr);
49         while (t > qt) time(t--, ql, qr);
50         ans[id] = val;
51     }
52     for (int i = 1; i < ans.size(); i++) {
53         cout << ans[i] << endl;
54     }
55 }

```

3.4.3 普通莫队

以 $O(N\sqrt{N})$ 的复杂度完成 Q 次询问的离线查询，其中每个分块的大小取 $\sqrt{N} = \sqrt{10^5} = 317$ ，也可以使用 $n / \min<\text{int}>(n, \text{sqrt}(q))$ 、 $\text{ceil}((\text{double})n / (\text{int})\text{sqrt}(n))$ 或者 $\text{sqrt}(n)$ 划分。

需要注意的是，在普通莫队中， K 数组的作用是根据左边界的值进行排序，当询问次数很少时 ($q \ll n$)，可以直接合并到 query 数组中。

```

1  signed main() {
2      int n;
3      cin >> n;
4      vector<int> w(n + 1);
5      for (int i = 1; i <= n; i++) {
6          cin >> w[i];
7      }
8
9      int q;
10     cin >> q;
11     vector<array<int, 3>> query(q + 1);
12     for (int i = 1; i <= q; i++) {
13         int l, r;
14         cin >> l >> r;
15         query[i] = {l, r, i};
16     }
17
18     int Knum = n / min<int>(n, sqrt(q)); // 计算块长
19     vector<int> K(n + 1);
20     for (int i = 1; i <= n; i++) { // 固定块长
21         K[i] = (i - 1) / Knum + 1;
22     }
23     sort(query.begin() + 1, query.end(), [&](auto x, auto y) {
24         if (K[x[0]] != K[y[0]]) return x[0] < y[0];

```

```

25     if (K[x[0]] & 1) return x[1] < y[1];
26     return x[1] > y[1];
27 });
28
29 int l = 1, r = 0, val = 0;
30 vector<int> ans(q + 1);
31 for (int i = 1; i <= q; i++) {
32     auto [ql, qr, id] = query[i];
33     auto add = [&](int x) -> void {};
34     auto del = [&](int x) -> void {};
35     while (l > ql) add(w[--l]);
36     while (r < qr) add(w[++r]);
37     while (l < ql) del(w[l++]);
38     while (r > qr) del(w[r--]);
39     ans[id] = val;
40 }
41 for (int i = 1; i <= q; i++) {
42     cout << ans[i] << endl;
43 }
44 }
45
46 vector<array<int, 4>> query(q);
47 for (int i = 1; i <= q; i++) {
48     int l, r;
49     cin >> l >> r;
50     query[i] = {l, r, i, (l - 1) / Knum + 1}; // 合并
51 }
52 sort(query.begin() + 1, query.end(), [&](auto x, auto y) {
53     if (x[3] != y[3]) return x[3] < y[3];
54     if (x[3] & 1) return x[1] < y[1];
55     return x[1] > y[1];
56 });

```

3.5 高阶

3.5.1 高精度: 重载运算符

```

1  #include<cstdio>
2  #include<cstring>
3  #include<iostream>
4  using namespace std;
5  const int maxn = 200;
6  struct bign{
7  int len, s[maxn];
8  bign() {
9  memset(s, 0, sizeof(s));
10 len = 1;
11 }
12 bign(int num) {
13 *this = num;
14 }
15 //定义为const参数, 作用是不能对const参数的值做修改

```



```

16 bign(const char* num) {
17     *this = num;
18 }
19 /*以上是构造方法，初始化时对执行相应的方法*/
20 bign operator = (int num) {
21     char s[maxn];
22     sprintf(s, "%d", num);
23     *this = s;
24     return *this;
25 }
26 //函数定义后的const关键字，它表明 “x.str()不会改变x”
27 string str() const {
28     string res = "";
29     for(int i = 0; i < len; i++) res = (char)(s[i] + '0') + res;
30     if(res == "") res = "0";
31     return res;
32 }
33 void clean() {
34
35     while(len > 1 && !s[len-1]) len--;
36 }
37 /* 以下是重载操作符*/
38 bign operator = (const char* num) {
39     //逆序存储，方便计算
40     len = strlen(num);
41     for(int i = 0; i < len; i++) s[i] = num[len-i-1] - '0';
42     return *this;
43 }
44 bign operator + (const bign& b) const{
45     bign c;
46     c.len = 0;
47     for(int i = 0, g = 0; g || i < max(len, b.len); i++) {
48         int x = g;
49         if(i < len) x += s[i];
50         if(i < b.len) x += b.s[i];
51         c.s[c.len++] = x % 10;
52         g = x / 10;
53     }
54     return c;
55 }
56 bign operator * (const bign& b) {
57     bign c; c.len = len + b.len;
58     for(int i = 0; i < len; i++)
59         for(int j = 0; j < b.len; j++)
60             c.s[i+j] += s[i] * b.s[j];
61     for(int i = 0; i < c.len-1; i++){
62         c.s[i+1] += c.s[i] / 10;
63         c.s[i] %= 10;
64     }
65     c.clean();
66     return c;
67 }
68 bign operator - (const bign& b) {

```

```

69 bign c; c.len = 0;
70 for(int i = 0, g = 0; i < len; i++) {
71     int x = s[i] - g;
72     if(i < b.len) x -= b.s[i];
73     if(x >= 0) g = 0;
74     else {
75         g = 1;
76         x += 10;
77     }
78     c.s[c.len++] = x;
79 }
80 c.clean();
81 return c;
82 }
83
84 // bign operator /(bign a, bign b){
85 //
86 bign c;
87 //
88 c.len=a.len-b.len+1;
89 //
90 for(int i=c.len;i>=1;i--){
91     //
92     bign t=cpy(b,i);
93     //
94     while(a>t||a==t){
95         //
96         c.num[i]++;
97         //
98         a=a-t;
99         //
100 while(a.len>0&&!a.num[a.len]) a.len--;
101 //
102 }
103 //
104 }
105 //
106 while(c.len>0&&!c.num[c.len]) c.len--;
107 //
108 return c;
109 // }
110 bool operator < (const bign& b) const{
111     if(len != b.len) return len < b.len;
112     for(int i = len-1; i >= 0; i--)
113         if(s[i] != b.s[i]) return s[i] < b.s[i];
114     return false;
115 }
116 bool operator > (const bign& b) const{
117     return b < *this;
118 }
119 bool operator <= (const bign& b) {
120     return !(b > *this);
121 }

```

```

122 bool operator == (const bign& b) {
123     return !(b < *this) && !(*this < b);
124 }
125 bign operator += (const bign& b) {
126     *this = *this + b;
127     return *this;
128 }
129 };
130 istream& operator >> (istream &in, bign& x) {
131     string s;
132     in >> s;
133     x = s.c_str();
134     return in;
135 }
136 ostream& operator << (ostream &out, const bign& x) {
137     out << x.str();
138     return out;
139 }
140
141 signed main() {
142     bign A,B;
143     cin>>A>>B;
144     return 0;
145 }
146 3.10

```

3.5.2 KD Tree

在第 k 维上的单次查询复杂度最坏为 $\mathcal{O}(n^{1-k^{-1}})$ 。

```

1 struct KDT {
2     constexpr static int N = 1e5 + 10, K = 2;
3     double alpha = 0.725;
4     struct node {
5         int info[K];
6         int mn[K], mx[K];
7     } tr[N];
8     int ls[N], rs[N], siz[N], id[N], d[N];
9     int idx, rt, cur;
10    int ans;
11    KDT() {
12        rt = 0;
13        cur = 0;
14        memset(ls, 0, sizeof ls);
15        memset(rs, 0, sizeof rs);
16        memset(d, 0, sizeof d);
17    }
18    void apply(int p, int son) {
19        if (son) {
20            for (int i = 0; i < K; i++) {
21                tr[p].mn[i] = min(tr[p].mn[i], tr[son].mn[i]);
22                tr[p].mx[i] = max(tr[p].mx[i], tr[son].mx[i]);
23            }

```

```

24         siz[p] += siz[son];
25     }
26 }
27 void maintain(int p) {
28     for (int i = 0; i < K; i++) {
29         tr[p].mn[i] = tr[p].info[i];
30         tr[p].mx[i] = tr[p].info[i];
31     }
32     siz[p] = 1;
33     apply(p, ls[p]);
34     apply(p, rs[p]);
35 }
36 int build(int l, int r) {
37     if (l > r) return 0;
38     vector<double> avg(K);
39     for (int i = 0; i < K; i++) {
40         for (int j = l; j <= r; j++) {
41             avg[i] += tr[id[j]].info[i];
42         }
43         avg[i] /= (r - l + 1);
44     }
45     vector<double> var(K);
46     for (int i = 0; i < K; i++) {
47         for (int j = l; j <= r; j++) {
48             var[i] += (tr[id[j]].info[i] - avg[i]) * (tr[id[j]].info[i] - avg[i]);
49         }
50     }
51     int mid = (l + r) / 2;
52     int x = max_element(var.begin(), var.end()) - var.begin();
53     nth_element(id + l, id + mid, id + r + 1, [&](int a, int b) {
54         return tr[a].info[x] < tr[b].info[x];
55     });
56     d[id[mid]] = x;
57     ls[id[mid]] = build(l, mid - 1);
58     rs[id[mid]] = build(mid + 1, r);
59     maintain(id[mid]);
60     return id[mid];
61 }
62 void print(int p) {
63     if (!p) return;
64     print(ls[p]);
65     id[++idx] = p;
66     print(rs[p]);
67 }
68 void rebuild(int &p) {
69     idx = 0;
70     print(p);
71     p = build(1, idx);
72 }
73 bool bad(int p) {
74     return alpha * siz[p] <= max(siz[ls[p]], siz[rs[p]]);
75 }
76 void insert(int &p, int cur) {

```

```

77     if (!p) {
78         p = cur;
79         maintain(p);
80         return;
81     }
82     if (tr[p].info[d[p]] > tr[cur].info[d[p]]) insert(ls[p], cur);
83     else insert(rs[p], cur);
84     maintain(p);
85     if (bad(p)) rebuild(p);
86 }
87 void insert(vector<int> &a) {
88     cur++;
89     for (int i = 0; i < K; i++) {
90         tr[cur].info[i] = a[i];
91     }
92     insert(rt, cur);
93 }
94 bool out(int p, vector<int> &a) {
95     for (int i = 0; i < K; i++) {
96         if (a[i] < tr[p].mn[i]) {
97             return true;
98         }
99     }
100     return false;
101 }
102 bool in(int p, vector<int> &a) {
103     for (int i = 0; i < K; i++) {
104         if (a[i] < tr[p].info[i]) {
105             return false;
106         }
107     }
108     return true;
109 }
110 bool all(int p, vector<int> &a) {
111     for (int i = 0; i < K; i++) {
112         if (a[i] < tr[p].mx[i]) {
113             return false;
114         }
115     }
116     return true;
117 }
118 void query(int p, vector<int> &a) {
119     if (!p) return;
120     if (out(p, a)) return;
121     if (all(p, a)) {
122         ans += siz[p];
123         return;
124     }
125     if (in(p, a)) ans++;
126     query(ls[p], a);
127     query(rs[p], a);
128 }
129 int query(vector<int> &a) {

```

```

130     ans = 0;
131     query(rt, a);
132     return ans;
133 }
134 };

```

3.5.3 主席树 (可持久化线段树)

以 $\mathcal{O}(N \log N)$ 的时间复杂度建树、查询、修改。

```

1 struct PresidentTree {
2     static constexpr int N = 2e5 + 10;
3     int cntNodes, root[N];
4     struct node {
5         int l, r;
6         int cnt;
7     } tr[4 * N + 17 * N];
8     void modify(int &u, int v, int l, int r, int x) {
9         u = ++cntNodes;
10        tr[u] = tr[v];
11        tr[u].cnt++;
12        if (l == r) return;
13        int mid = (l + r) / 2;
14        if (x <= mid)
15            modify(tr[u].l, tr[v].l, l, mid, x);
16        else
17            modify(tr[u].r, tr[v].r, mid + 1, r, x);
18    }
19    int kth(int u, int v, int l, int r, int k) {
20        if (l == r) return l;
21        int res = tr[tr[v].l].cnt - tr[tr[u].l].cnt;
22        int mid = (l + r) / 2;
23        if (k <= res)
24            return kth(tr[u].l, tr[v].l, l, mid, k);
25        else
26            return kth(tr[u].r, tr[v].r, mid + 1, r, k - res);
27    }
28 };

```

3.5.4 分数运算类

定义了分数的四则运算，如果需要处理浮点数，那么需要将函数中的 `gcd` 运算替换为 `fgcd`。

```

1 template<class T> struct Frac {
2     T x, y;
3     Frac() : Frac(0, 1) {}
4     Frac(T x_) : Frac(x_, 1) {}
5     Frac(T x_, T y_) : x(x_), y(y_) {
6         if (y < 0) {
7             y = -y;
8             x = -x;
9         }

```

```

10     }
11
12     constexpr double val() const {
13         return 1. * x / y;
14     }
15     constexpr Frac norm() const { // 调整符号、转化为最简形式
16         T p = gcd(x, y);
17         return {x / p, y / p};
18     }
19     friend constexpr auto &operator<<(ostream &o, const Frac &j) {
20         T p = gcd(j.x, j.y);
21         if (j.y == p) {
22             return o << j.x / p;
23         } else {
24             return o << j.x / p << "/" << j.y / p;
25         }
26     }
27     constexpr Frac &operator/=(const Frac &i) {
28         x *= i.y;
29         y *= i.x;
30         if (y < 0) {
31             x = -x;
32             y = -y;
33         }
34         return *this;
35     }
36     constexpr Frac &operator+=(const Frac &i) { return x = x * i.y + y * i.x, y *= i
        .y, *this; }
37     constexpr Frac &operator-=(const Frac &i) { return x = x * i.y - y * i.x, y *= i
        .y, *this; }
38     constexpr Frac &operator*=(const Frac &i) { return x *= i.x, y *= i.y, *this; }
39     friend constexpr Frac operator+(const Frac i, const Frac j) { return i += j; }
40     friend constexpr Frac operator-(const Frac i, const Frac j) { return i -= j; }
41     friend constexpr Frac operator*(const Frac i, const Frac j) { return i *= j; }
42     friend constexpr Frac operator/(const Frac i, const Frac j) { return i /= j; }
43     friend constexpr Frac operator-(const Frac i) { return Frac(-i.x, i.y); }
44     friend constexpr bool operator<(const Frac i, const Frac j) { return i.x * j.y <
        i.y * j.x; }
45     friend constexpr bool operator>(const Frac i, const Frac j) { return i.x * j.y >
        i.y * j.x; }
46     friend constexpr bool operator==(const Frac i, const Frac j) { return i.x * j.y
        == i.y * j.x; }
47     friend constexpr bool operator!=(const Frac i, const Frac j) { return i.x * j.y
        != i.y * j.x; }
48 };

```

3.5.5 取模运算类

集成了常见的取模四则运算，运算速度与手动取模相差无几，效率极高。

```

1 using i64 = long long;
2
3 template<class T> constexpr T mypow(T n, i64 k) {

```

```

4   T r = 1;
5   for (; k; k /= 2, n *= n) {
6       if (k % 2) {
7           r *= n;
8       }
9   }
10  return r;
11 }
12
13 template<class T> constexpr T power(int n) {
14     return mypow(T(2), n);
15 }
16
17 template<const int &MOD> struct Zmod {
18     int x;
19     Zmod(signed x = 0) : x(norm(x % MOD)) {}
20     Zmod(i64 x) : x(norm(x % MOD)) {}
21
22     constexpr int norm(int x) const noexcept {
23         if (x < 0) [[unlikely]] {
24             x += MOD;
25         }
26         if (x >= MOD) [[unlikely]] {
27             x -= MOD;
28         }
29         return x;
30     }
31     explicit operator int() const {
32         return x;
33     }
34     constexpr int val() const {
35         return x;
36     }
37     constexpr Zmod operator-() const {
38         Zmod val = norm(MOD - x);
39         return val;
40     }
41     constexpr Zmod inv() const {
42         assert(x != 0);
43         return mypow(*this, MOD - 2);
44     }
45     friend constexpr auto &operator>>(istream &in, Zmod &j) {
46         int v;
47         in >> v;
48         j = Zmod(v);
49         return in;
50     }
51     friend constexpr auto &operator<<(ostream &o, const Zmod &j) {
52         return o << j.val();
53     }
54     constexpr Zmod &operator++() {
55         x = norm(x + 1);
56         return *this;

```



```

57     }
58     constexpr Zmod &operator--() {
59         x = norm(x - 1);
60         return *this;
61     }
62     constexpr Zmod operator++(signed) {
63         Zmod res = *this;
64         ++*this;
65         return res;
66     }
67     constexpr Zmod operator--(signed) {
68         Zmod res = *this;
69         --*this;
70         return res;
71     }
72     constexpr Zmod &operator+=(const Zmod &i) {
73         x = norm(x + i.x);
74         return *this;
75     }
76     constexpr Zmod &operator-=(const Zmod &i) {
77         x = norm(x - i.x);
78         return *this;
79     }
80     constexpr Zmod &operator*=(const Zmod &i) {
81         x = i64(x) * i.x % MOD;
82         return *this;
83     }
84     constexpr Zmod &operator/=(const Zmod &i) {
85         return *this *= i.inv();
86     }
87     constexpr Zmod &operator%=(const int &i) {
88         return x %= i, *this;
89     }
90     friend constexpr Zmod operator+(const Zmod i, const Zmod j) {
91         return Zmod(i) += j;
92     }
93     friend constexpr Zmod operator-(const Zmod i, const Zmod j) {
94         return Zmod(i) -= j;
95     }
96     friend constexpr Zmod operator*(const Zmod i, const Zmod j) {
97         return Zmod(i) *= j;
98     }
99     friend constexpr Zmod operator/(const Zmod i, const Zmod j) {
100         return Zmod(i) /= j;
101     }
102     friend constexpr Zmod operator%(const Zmod i, const int j) {
103         return Zmod(i) %= j;
104     }
105     friend constexpr bool operator==(const Zmod i, const Zmod j) {
106         return i.val() == j.val();
107     }
108     friend constexpr bool operator!=(const Zmod i, const Zmod j) {
109         return i.val() != j.val();

```

```

110     }
111     friend constexpr bool operator<(const Zmod i, const Zmod j) {
112         return i.val() < j.val();
113     }
114     friend constexpr bool operator>(const Zmod i, const Zmod j) {
115         return i.val() > j.val();
116     }
117 };
118
119 int MOD[] = {998244353, 1000000007};
120 using Z = Zmod<MOD[1]>;

```

3.5.6 大整数类（高精度计算）

```

1  const int base = 1000000000;
2  const int base_digits = 9; // 分解为九个数位一个数字
3  struct bigint {
4      vector<int> a;
5      int sign;
6
7      bigint() : sign(1) {}
8      bigint operator-() const {
9          bigint res = *this;
10         res.sign = -sign;
11         return res;
12     }
13     bigint(long long v) {
14         *this = v;
15     }
16     bigint(const string &s) {
17         read(s);
18     }
19     void operator=(const bigint &v) {
20         sign = v.sign;
21         a = v.a;
22     }
23     void operator=(long long v) {
24         a.clear();
25         sign = 1;
26         if (v < 0) sign = -1, v = -v;
27         for (; v > 0; v = v / base) {
28             a.push_back(v % base);
29         }
30     }
31
32     // 基础加减乘除
33     bigint operator+(const bigint &v) const {
34         if (sign == v.sign) {
35             bigint res = v;
36             for (int i = 0, carry = 0; i < (int)max(a.size(), v.a.size()) || carry; ++i) {
37                 if (i == (int)res.a.size()) {

```

```

38         res.a.push_back(0);
39     }
40     res.a[i] += carry + (i < (int)a.size() ? a[i] : 0);
41     carry = res.a[i] >= base;
42     if (carry) {
43         res.a[i] -= base;
44     }
45 }
46 return res;
47 }
48 return *this - (-v);
49 }
50 bigint operator-(const bigint &v) const {
51     if (sign == v.sign) {
52         if (abs() >= v.abs()) {
53             bigint res = *this;
54             for (int i = 0, carry = 0; i < (int)v.a.size() || carry; ++i) {
55                 res.a[i] -= carry + (i < (int)v.a.size() ? v.a[i] : 0);
56                 carry = res.a[i] < 0;
57                 if (carry) {
58                     res.a[i] += base;
59                 }
60             }
61             res.trim();
62             return res;
63         }
64         return -(v - *this);
65     }
66     return *this + (-v);
67 }
68 void operator*=(int v) {
69     check(v);
70     for (int i = 0, carry = 0; i < (int)a.size() || carry; ++i) {
71         if (i == (int)a.size()) {
72             a.push_back(0);
73         }
74         long long cur = a[i] * (long long)v + carry;
75         carry = (int)(cur / base);
76         a[i] = (int)(cur % base);
77     }
78     trim();
79 }
80 void operator/=(int v) {
81     check(v);
82     for (int i = (int)a.size() - 1, rem = 0; i >= 0; --i) {
83         long long cur = a[i] + rem * (long long)base;
84         a[i] = (int)(cur / v);
85         rem = (int)(cur % v);
86     }
87     trim();
88 }
89 int operator%(int v) const {
90     if (v < 0) {

```

```

91         v = -v;
92     }
93     int m = 0;
94     for (int i = a.size() - 1; i >= 0; --i) {
95         m = (a[i] + m * (long long)base) % v;
96     }
97     return m * sign;
98 }
99
100 void operator+=(const bigint &v) {
101     *this = *this + v;
102 }
103 void operator-=(const bigint &v) {
104     *this = *this - v;
105 }
106 bigint operator*(int v) const {
107     bigint res = *this;
108     res *= v;
109     return res;
110 }
111 bigint operator/(int v) const {
112     bigint res = *this;
113     res /= v;
114     return res;
115 }
116 void operator%=(const int &v) {
117     *this = *this % v;
118 }
119
120 bool operator<(const bigint &v) const {
121     if (sign != v.sign) return sign < v.sign;
122     if (a.size() != v.a.size()) return a.size() * sign < v.a.size() * v.sign;
123     for (int i = a.size() - 1; i >= 0; i--)
124         if (a[i] != v.a[i]) return a[i] * sign < v.a[i] * sign;
125     return false;
126 }
127 bool operator>(const bigint &v) const {
128     return v < *this;
129 }
130 bool operator<=(const bigint &v) const {
131     return !(v < *this);
132 }
133 bool operator>=(const bigint &v) const {
134     return !(*this < v);
135 }
136 bool operator==(const bigint &v) const {
137     return !(*this < v) && !(v < *this);
138 }
139 bool operator!=(const bigint &v) const {
140     return *this < v || v < *this;
141 }
142
143 bigint abs() const {

```

```

144     bigint res = *this;
145     res.sign *= res.sign;
146     return res;
147 }
148 void check(int v) { // 检查输入的是否为负数
149     if (v < 0) {
150         sign = -sign;
151         v = -v;
152     }
153 }
154 void trim() { // 去除前导零
155     while (!a.empty() && !a.back()) a.pop_back();
156     if (a.empty()) sign = 1;
157 }
158 bool isZero() const { // 判断是否等于零
159     return a.empty() || (a.size() == 1 && !a[0]);
160 }
161 friend bigint gcd(const bigint &a, const bigint &b) {
162     return b.isZero() ? a : gcd(b, a % b);
163 }
164 friend bigint lcm(const bigint &a, const bigint &b) {
165     return a / gcd(a, b) * b;
166 }
167 void read(const string &s) {
168     sign = 1;
169     a.clear();
170     int pos = 0;
171     while (pos < (int)s.size() && (s[pos] == '-' || s[pos] == '+')) {
172         if (s[pos] == '-') sign = -sign;
173         ++pos;
174     }
175     for (int i = s.size() - 1; i >= pos; i -= base_digits) {
176         int x = 0;
177         for (int j = max(pos, i - base_digits + 1); j <= i; j++) x = x * 10 + s[j]
            - '0';
178         a.push_back(x);
179     }
180     trim();
181 }
182 friend istream &operator>>(istream &stream, bigint &v) {
183     string s;
184     stream >> s;
185     v.read(s);
186     return stream;
187 }
188 friend ostream &operator<<(ostream &stream, const bigint &v) {
189     if (v.sign == -1) stream << '-';
190     stream << (v.a.empty() ? 0 : v.a.back());
191     for (int i = (int)v.a.size() - 2; i >= 0; --i)
192         stream << setw(base_digits) << setfill('0') << v.a[i];
193     return stream;
194 }
195

```

```

196  /* 大整数乘除大整数部分 */
197  typedef vector<long long> vll;
198  bigint operator*(const bigint &v) const { // 大整数乘大整数
199      vector<int> a6 = convert_base(this->a, base_digits, 6);
200      vector<int> b6 = convert_base(v.a, base_digits, 6);
201      vll a(a6.begin(), a6.end());
202      vll b(b6.begin(), b6.end());
203      while (a.size() < b.size()) a.push_back(0);
204      while (b.size() < a.size()) b.push_back(0);
205      while (a.size() & (a.size() - 1)) a.push_back(0), b.push_back(0);
206      vll c = karatsubaMultiply(a, b);
207      bigint res;
208      res.sign = sign * v.sign;
209      for (int i = 0, carry = 0; i < (int)c.size(); i++) {
210          long long cur = c[i] + carry;
211          res.a.push_back((int)(cur % 1000000));
212          carry = (int)(cur / 1000000);
213      }
214      res.a = convert_base(res.a, 6, base_digits);
215      res.trim();
216      return res;
217  }
218  friend pair<bigint, bigint> divmod(const bigint &a1,
219                                   const bigint &b1) { // 大整数除大整数, 同时返回答案与余
220                                           数
221      int norm = base / (b1.a.back() + 1);
222      bigint a = a1.abs() * norm;
223      bigint b = b1.abs() * norm;
224      bigint q, r;
225      q.a.resize(a.a.size());
226      for (int i = a.a.size() - 1; i >= 0; i--) {
227          r *= base;
228          r += a.a[i];
229          int s1 = r.a.size() <= b.a.size() ? 0 : r.a[b.a.size()];
230          int s2 = r.a.size() <= b.a.size() - 1 ? 0 : r.a[b.a.size() - 1];
231          int d = ((long long)base * s1 + s2) / b.a.back();
232          r -= b * d;
233          while (r < 0) r += b, --d;
234          q.a[i] = d;
235      }
236      q.sign = a1.sign * b1.sign;
237      r.sign = a1.sign;
238      q.trim();
239      r.trim();
240      return make_pair(q, r / norm);
241  }
242  static vector<int> convert_base(const vector<int> &a, int old_digits, int
243      new_digits) {
244      vector<long long> p(max(old_digits, new_digits) + 1);
245      p[0] = 1;
246      for (int i = 1; i < (int)p.size(); i++) p[i] = p[i - 1] * 10;
247      vector<int> res;
248      long long cur = 0;

```

```

247     int cur_digits = 0;
248     for (int i = 0; i < (int)a.size(); i++) {
249         cur += a[i] * p[cur_digits];
250         cur_digits += old_digits;
251         while (cur_digits >= new_digits) {
252             res.push_back((int)(cur % p[new_digits]));
253             cur /= p[new_digits];
254             cur_digits -= new_digits;
255         }
256     }
257     res.push_back((int)cur);
258     while (!res.empty() && !res.back()) res.pop_back();
259     return res;
260 }
261 static vll karatsubaMultiply(const vll &a, const vll &b) {
262     int n = a.size();
263     vll res(n + n);
264     if (n <= 32) {
265         for (int i = 0; i < n; i++) {
266             for (int j = 0; j < n; j++) {
267                 res[i + j] += a[i] * b[j];
268             }
269         }
270         return res;
271     }
272
273     int k = n >> 1;
274     vll a1(a.begin(), a.begin() + k);
275     vll a2(a.begin() + k, a.end());
276     vll b1(b.begin(), b.begin() + k);
277     vll b2(b.begin() + k, b.end());
278
279     vll a1b1 = karatsubaMultiply(a1, b1);
280     vll a2b2 = karatsubaMultiply(a2, b2);
281
282     for (int i = 0; i < k; i++) a2[i] += a1[i];
283     for (int i = 0; i < k; i++) b2[i] += b1[i];
284
285     vll r = karatsubaMultiply(a2, b2);
286     for (int i = 0; i < (int)a1b1.size(); i++) r[i] -= a1b1[i];
287     for (int i = 0; i < (int)a2b2.size(); i++) r[i] -= a2b2[i];
288
289     for (int i = 0; i < (int)r.size(); i++) res[i + k] += r[i];
290     for (int i = 0; i < (int)a1b1.size(); i++) res[i] += a1b1[i];
291     for (int i = 0; i < (int)a2b2.size(); i++) res[i + n] += a2b2[i];
292     return res;
293 }
294
295 void operator*=(const bigint &v) {
296     *this = *this * v;
297 }
298 bigint operator/(const bigint &v) const {
299     return divmod(*this, v).first;

```

```

300     }
301     void operator/=(const bigint &v) {
302         *this = *this / v;
303     }
304     bigint operator%(const bigint &v) const {
305         return divmod(*this, v).second;
306     }
307     void operator%=(const bigint &v) {
308         *this = *this % v;
309     }
310 };

```

3.5.7 小波矩阵树：高效静态区间第 K 大查询

手写 `bitset` 压位，以 $\mathcal{O}(N \log N)$ 的时间复杂度和 $\mathcal{O}(N + \frac{N \log N}{64})$ 的空间建树后，实现单次 $\mathcal{O}(\log N)$ 复杂度的区间第 k 大值询问。已经过偏移，请使用 `1-idx`。

```

1  #define __count(x) __builtin_popcountll(x)
2  struct Wavelet {
3      vector<int> val, sum;
4      vector<u64> bit;
5      int t, n;
6
7      int getSum(int i) {
8          return sum[i >> 6] + __count(bit[i >> 6] & ((1ULL << (i & 63)) - 1));
9      }
10
11     Wavelet(vector<int> v) : val(v), n(v.size()) {
12         sort(val.begin(), val.end());
13         val.erase(unique(val.begin(), val.end()), val.end());
14
15         int n_ = val.size();
16         t = __lg(2 * n_ - 1);
17         bit.resize((t * n + 64) >> 6);
18         sum.resize(bit.size());
19         vector<int> cnt(n_ + 1);
20
21         for (int &x : v) {
22             x = lower_bound(val.begin(), val.end(), x) - val.begin();
23             cnt[x + 1]++;
24         }
25         for (int i = 1; i < n_; ++i) {
26             cnt[i] += cnt[i - 1];
27         }
28         for (int j = 0; j < t; ++j) {
29             for (int i : v) {
30                 int tmp = i >> (t - 1 - j);
31                 int pos = (tmp >> 1) << (t - j);
32                 auto setBit = [&](int i, u64 v) {
33                     bit[i >> 6] |= (v << (i & 63));
34                 };
35                 setBit(j * n + cnt[pos], tmp & 1);

```



```

36         cnt[pos]++;
37     }
38     for (int i : v) {
39         cnt[(i >> (t - j)) << (t - j)]--;
40     }
41 }
42 for (int i = 1; i < sum.size(); ++i) {
43     sum[i] = sum[i - 1] + __count(bit[i - 1]);
44 }
45 }
46
47 int small(int l, int r, int k) {
48     r++, k--;
49     for (int j = 0, x = 0, y = n, res = 0;; ++j) {
50         if (j == t) return val[res];
51         int A = getSum(n * j + x), B = getSum(n * j + 1);
52         int C = getSum(n * j + r), D = getSum(n * j + y);
53         int ab_zeros = r - 1 - C + B;
54         if (ab_zeros > k) {
55             res = res << 1;
56             y -= D - A;
57             l -= B - A;
58             r -= C - A;
59         } else {
60             res = (res << 1) | 1;
61             k -= ab_zeros;
62             x += y - x - D + A;
63             l += y - l - D + B;
64             r += y - r - D + C;
65         }
66     }
67 }
68 int large(int l, int r, int k) {
69     return small(l, r, r - l - k);
70 }
71 };

```

3.5.8 常见例题

题意：（带修莫队 - 维护队列）要求能够处理以下操作：

- 'Q' l r：询问区间 $[l, r]$ 有几个颜色；
- 'R' idx w：将下标 idx 的颜色修改为 w。

输入格式为：第一行 n 和 q ($1 \leq n, q \leq 133333$) 分别代表区间长度和操作数量；第二行 n 个整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^6$) 代表初始颜色；随后 q 行为具体操作。

动态规划

```

1  const int N = 1e6 + 7;
2  signed main() {
3      int n, q;

```

```

4   cin >> n >> q;
5   vector<int> w(n + 1);
6   for (int i = 1; i <= n; i++) {
7       cin >> w[i];
8   }
9
10  vector<array<int, 4>> query = {{}}; // {左区间, 右区间, 累计修改次数, 下标}
11  vector<array<int, 2>> modify = {{}}; // {修改的值, 修改的元素下标}
12  for (int i = 1; i <= q; i++) {
13      char op;
14      cin >> op;
15      if (op == 'Q') {
16          int l, r;
17          cin >> l >> r;
18          query.push_back({l, r, (int)modify.size() - 1, (int)query.size()});
19      } else {
20          int idx, w;
21          cin >> idx >> w;
22          modify.push_back({w, idx});
23      }
24  }
25
26  int Knum = 2154; // 计算块长
27  vector<int> K(n + 1);
28  for (int i = 1; i <= n; i++) { // 固定块长
29      K[i] = (i - 1) / Knum + 1;
30  }
31  sort(query.begin() + 1, query.end(), [&](auto x, auto y) {
32      if (K[x[0]] != K[y[0]]) return x[0] < y[0];
33      if (K[x[1]] != K[y[1]]) return x[1] < y[1];
34      return x[3] < y[3];
35  });
36
37  int l = 1, r = 0, val = 0;
38  int t = 0; // 累计修改次数
39  vector<int> ans(query.size()), cnt(N);
40  for (int i = 1; i < query.size(); i++) {
41      auto [ql, qr, qt, id] = query[i];
42      auto add = [&](int x) -> void {
43          if (cnt[x] == 0) ++ val;
44          ++ cnt[x];
45      };
46      auto del = [&](int x) -> void {
47          -- cnt[x];
48          if (cnt[x] == 0) -- val;
49      };
50      auto time = [&](int x, int l, int r) -> void {
51          if (l <= modify[x][1] && modify[x][1] <= r) { //当修改的位置在询问期间内部时
52              //才会改变num的值
53              del(w[modify[x][1]]);
54              add(modify[x][0]);
55          }
56          swap(w[modify[x][1]], modify[x][0]); //直接交换修改数组的值与原始值, 减少额外

```

```

    的数组开销, 且方便复原
56     };
57     while (l > ql) add(w[--l]);
58     while (r < qr) add(w[++r]);
59     while (l < ql) del(w[l++]);
60     while (r > qr) del(w[r--]);
61     while (t < qt) time(++t, ql, qr);
62     while (t > qt) time(t--, ql, qr);
63     ans[id] = val;
64 }
65 for (int i = 1; i < ans.size(); i++) {
66     cout << ans[i] << endl;
67 }
68 }

```

3.5.9 常见结论

题意:(区间移位问题)要求将整个序列左移/右移若干个位置,例如,原序列为 $A = (a_1, a_2, \dots, a_n)$, 右移 x 位后变为 $A = (a_{x+1}, a_{x+2}, \dots, a_n, a_1, a_2, \dots, a_x)$ 。

区间的端点只是一个数字,即使被改变了,通过一定的转换也能够还原,所以我们可以 $\mathcal{O}(1)$ 解决这一问题。为了方便计算,我们规定下标从 0 开始,即整个线段的区间为 $[0, n)$, 随后,使用一个偏移量 `shift` 记录。使用 `shift = (shift + x) % n`; 更新偏移量; 此后的区间查询/修改前, 再将坐标偏移回去即可, 下方代码使用区间修改作为示例。

```

1  cin >> l >> r >> x;
2  l--; // 坐标修改为 0 开始
3  r--;
4  l = (l + shift) % n; // 偏移
5  r = (r + shift) % n;
6  if (l > r) { // 区间分离则分别操作
7      segt.modify(l, n - 1, x);
8      segt.modify(0, r, x);
9  } else {
10     segt.modify(l, r, x);
11 }

```

3.5.10 线性基 (高斯消元法)

设向量长度为 N (一般取 63), 总数为 M , 时间复杂度为 $\mathcal{O}(NM)$ 。

```

1  struct LB { // Linear Basis
2      using i64 = long long;
3      const int BASE = 63;
4      vector<i64> d, p;
5      int cnt, flag;
6
7      LB() {
8          d.resize(BASE + 1);
9          p.resize(BASE + 1);
10         cnt = flag = 0;

```

```

11     }
12     bool insert(i64 val) {
13         for (int i = BASE - 1; i >= 0; i--) {
14             if (val & (1ll << i)) {
15                 if (!d[i]) {
16                     d[i] = val;
17                     return true;
18                 }
19                 val ^= d[i];
20             }
21         }
22         flag = 1; //可以异或出0
23         return false;
24     }
25     bool check(i64 val) { // 判断 val 是否可能被异或得到
26         for (int i = BASE - 1; i >= 0; i--) {
27             if (val & (1ll << i)) {
28                 if (!d[i]) {
29                     return false;
30                 }
31                 val ^= d[i];
32             }
33         }
34         return true;
35     }
36     i64 ask_max() {
37         i64 res = 0;
38         for (int i = BASE - 1; i >= 0; i--) {
39             if ((res ^ d[i]) > res) res ^= d[i];
40         }
41         return res;
42     }
43     i64 ask_min() {
44         if (flag) return 0; // 特判 0
45         for (int i = 0; i <= BASE - 1; i++) {
46             if (d[i]) return d[i];
47         }
48     }
49     void rebuild() { // 第k小值独立预处理
50         for (int i = BASE - 1; i >= 0; i--) {
51             for (int j = i - 1; j >= 0; j--) {
52                 if (d[i] & (1ll << j)) d[i] ^= d[j];
53             }
54         }
55         for (int i = 0; i <= BASE - 1; i++) {
56             if (d[i]) p[cnt++] = d[i];
57         }
58     }
59     i64 kthquery(i64 k) { // 查询能被异或得到的第 k 小值, 如不存在则返回 -1
60         if (flag) k--; // 特判 0, 如果不需要 0, 直接删去
61         if (!k) return 0;
62         i64 res = 0;
63         if (k >= (1ll << cnt)) return -1;

```

```

64     for (int i = BASE - 1; i >= 0; i--) {
65         if (k & (1LL << i)) res ^= p[i];
66     }
67     return res;
68 }
69 void Merge(const LB &b) { // 合并两个线性基
70     for (int i = BASE - 1; i >= 0; i--) {
71         if (b.d[i]) {
72             insert(b.d[i]);
73         }
74     }
75 }
76 };

```

4 杂项

4.1 基础算法

4.1.1 OJ 测试

对于一个未知属性的 OJ，应当在正式赛前进行以下全部测试：

GNU C++ 版本测试

编译器位数测试

评测器环境测试

Windows 系统输出 -1；反之则为一个随机数。

运算速度测试

本地-20(64)	CodeForces-20(64)	AtCoder-20(64)	牛客-17(64)	学院 OJ	CodeForces-17(32)	马蹄集
4E3 量级-硬跑	2454	2886	874	4121	4807	2854
4E3 量级-手动加速	556	686	873	1716	1982	2246

```

1  for (int i : {1, 2}) {} // GNU C++11 支持范围表达式
2
3  auto cc = [&](int x) { x++; }; // GNU C++11 支持 auto 与 lambda 表达式
4  cc(2);
5
6  tuple<string, int, int> V; // GNU C++11 引入
7  array<int, 3> C; // GNU C++11 引入
8
9  auto dfs = [&](auto self, int x) -> void { // GNU C++14 支持 auto 自递归
10     if (x > 10) return;
11     self(self, x + 1);
12 };
13 dfs(dfs, 1);
14
15 vector in(1, vector<int>(1)); // GNU C++17 支持 vector 模板类型缺失
16
17 map<int, int> dic;

```

```

18 for (auto [u, v] : dic) {} // GNU C++17 支持 auto 解绑
19 dic.contains(12); // GNU C++20 支持 contains 函数
20
21 constexpr double Pi = numbers::pi; // C++20 支持
22
23 using i64 = __int128; // 64 位 GNU C++11 支持
24
25 #define int long long
26 map<int, int> dic;
27 int x = dic.size() - 1;
28 cout << x << endl;
29
30 // #pragma GCC optimize("Ofast", "unroll-loops")
31 #include <bits/stdc++.h>
32 using namespace std;
33
34 signed main() {
35     int n = 4E3, cnt = 0;
36     bitset<30> ans;
37     for (int i = 1; i <= n; i++) {
38         for (int j = 1; j <= n; j += 2) {
39             for (int k = 1; k <= n; k += 4) {
40                 ans |= i | j | k;
41                 cnt++;
42             }
43         }
44     }
45     cout << cnt << "\n";
46 }
47
48 // #pragma GCC optimize("Ofast", "unroll-loops")
49
50 #include <bits/stdc++.h>
51 using namespace std;
52 mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
53
54 signed main() {
55     size_t n = 34000000, seed = 0;
56     for (int i = 1; i <= n; i++) {
57         seed ^= rnd();
58     }
59
60     return 0;
61 }

```

4.1.2 Python 常用语法

读入与定义

- 读入多个变量并转换类型: `X, Y = map(int, input().split())`
- 读入列表: `X = eval(input())`

- 多维数组定义: `X = [[0 for j in range(0, 100)] for i in range(0, 200)]`

格式化输出

- 保留小数输出: `print("{:.12f}".format(X))` 指保留 12 位小数
- 对齐与宽度: `print("{:<12f}".format(X))` 指左对齐, 保留 12 个宽度

排序

- 倒序排序: 使用 `reverse` 实现倒序 `X.sort(reverse=True)`
- 自定义排序: 下方代码实现了先按第一关键字降序、再按第二关键字升序排序。

```
python X.sort(key=lambda x: x[1]) X.sort(key=lambda x: x[0], reverse=True)
```

文件 IO

- 打开要读取的文件: `r = open('X.txt', 'r', encoding='utf-8')`
- 打开要写入的文件: `w = open('Y.txt', 'w', encoding='utf-8')`
- 按行写入: `w.write(XX)`

增加输出流长度、递归深度

自定义结构体

自定义结构体并且自定义排序

数据结构

- 模拟于 C_{++}^{map} , 定义: `dic = dict()`
- 模拟栈与队列: 使用常见的 `list` 即可完成, `list.insert(0, X)` 实现头部插入、`list.pop()` 实现尾部弹出、`list.pop(0)` 实现头部弹出

其他

- 获取 ASCII 码: `ord()` 函数
- 转换为 ASCII 字符: `chr()` 函数

```
1 import sys
2 sys.set_int_max_str_digits(200000)
3 sys.setrecursionlimit(100000)
4
5 class node:
6     def __init__(self, A, B, C):
7         self.A = A
8         self.B = B
9         self.C = C
10
```

```

11 w = []
12 for i in range(1, 5):
13     a, b, c = input().split()
14     w.append(node(a, b, c))
15 w.sort(key=lambda x: x.C, reverse=True)
16 for i in w:
17     print(i.A, i.B, i.C)

```

4.1.3 cout 输出流控制

设置字段宽度: `setw(x)`, 该函数可以使得补全 x 位输出, 默认用空格补全。

设置填充字符: `setfill(x)`, 该函数可以设定补全类型, 注意这里的 x 只能为 `char` 类型。

```

1 bool Solve() {
2     cout << 12 << endl;
3     cout << setw(12) << 12 << endl;
4     return 0;
5 }
6
7 bool Solve() {
8     cout << 12 << endl;
9     cout << setw(12) << setfill('*') << 12 << endl;
10    return 0;
11 }

```

4.1.4 int128 输入输出流控制

`int128` 只在基于 `Lumix` 系统的环境下可用, 需要 `C++20`。38 位精度, 除输入输出外与普通数据类型无差别。该封装支持负数读入, 需要注意 `write` 函数结尾不输出多余空格与换行。

```

1 namespace my128 { // 读入优化封装, 支持__int128
2     using i64 = __int128_t;
3     i64 abs(const i64 &x) {
4         return x > 0 ? x : -x;
5     }
6     auto &operator>>(istream &it, i64 &j) {
7         string val;
8         it >> val;
9         reverse(val.begin(), val.end());
10        i64 ans = 0;
11        bool f = 0;
12        char c = val.back();
13        val.pop_back();
14        for (; c < '0' || c > '9'; c = val.back(), val.pop_back()) {
15            if (c == '-') {
16                f = 1;
17            }
18        }
19        for (; c >= '0' && c <= '9'; c = val.back(), val.pop_back()) {
20            ans = ans * 10 + c - '0';
21        }

```



```

22     j = f ? -ans : ans;
23     return it;
24 }
25 auto &operator<<(ostream &os, const i64 &j) {
26     string ans;
27     function<void(i64)> write = [&](i64 x) {
28         if (x < 0) ans += '-', x = -x;
29         if (x > 9) write(x / 10);
30         ans += x % 10 + '0';
31     };
32     write(j);
33     return os << ans;
34 }
35 } // namespace my128

```

4.1.5 头文件

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  template<class... Args> void _(Args... args) {
5      auto _ = [&](auto x) { cout << x << " "; };
6      cout << "---->";
7      int arr[] = {_(args), 0}...;
8      cout << "\n";
9  }
10
11 signed main() {
12     int Task = 1;
13     for (cin >> Task; Task; Task--) {}
14 }
15 int __OI_INIT__ = []() {
16     ios::sync_with_stdio(0), cin.tie(0);
17     cout.tie(0);
18     cout << fixed << setprecision(12);
19     return 0;
20 }();

```

4.1.6 实数域三分

限制次数实现。

树上问题

```

1  ld l = -1E9, r = 1E9;
2  for (int t = 1; t <= 100; t++) {
3      ld mid1 = (l * 2 + r) / 3;
4      ld mid2 = (l + r * 2) / 3;
5      if (judge(mid1) < judge(mid2)) {
6          r = mid2;
7      } else {
8          l = mid1;

```

```

9     }
10 }
11 cout << l << endl;

```

4.1.7 实数域二分

目前主流的写法是限制二分次数。

```

1 for (int t = 1; t <= 100; t++) {
2     ld mid = (l + r) / 2;
3     if (judge(mid)) r = mid;
4     else l = mid;
5 }
6 cout << l << endl;

```

4.1.8 对拍版子

- 文件控制
- C++ 版 bat

```

1 // BAD.cpp, 存放待寻找错误的代码
2 freopen("A.txt", "r", stdin);
3 freopen("BAD.out", "w", stdout);
4
5 // 1.cpp, 存放暴力或正确的代码
6 freopen("A.txt", "r", stdin);
7 freopen("1.out", "w", stdout);
8
9 // Ask.cpp
10 freopen("A.txt", "w", stdout);
11
12 int main() {
13     int T = 1E5;
14     while(T--) {
15         system("BAD.exe");
16         system("1.exe");
17         system("A.exe");
18         if (system("fc BAD.out 1.out")) {
19             puts("WA");
20             return 0;
21         }
22     }
23 }

```

4.1.9 常用函数

```

1 i64 mysqrt(i64 n) { // 针对 sqrt 无法精确计算 ll 型
2     i64 ans = sqrt(n);
3     while ((ans + 1) * (ans + 1) <= n) ans++;

```

```

4   while (ans * ans > n) ans--;
5   return ans;
6 }
7 int mylcm(int x, int y) {
8     return x / gcd(x, y) * y;
9 }
10
11 template<class T> int log2floor(T n) { // 针对 log2 无法精确计算 ll 型; 向下取整
12     assert(n > 0);
13     for (T i = 0, chk = 1;; i++, chk *= 2) {
14         if (chk <= n && n < chk * 2) {
15             return i;
16         }
17     }
18 }
19 template<class T> int log2ceil(T n) { // 向上取整
20     assert(n > 0);
21     for (T i = 0, chk = 1;; i++, chk *= 2) {
22         if (n <= chk) {
23             return i;
24         }
25     }
26 }
27 int log2floor(int x) {
28     return 31 - __builtin_clz(x);
29 }
30 int log2ceil(int x) { // 向上取整
31     return log2floor(x) + (__builtin_popcount(x) != 1);
32 }
33
34 template <class T> T sign(const T &a) {
35     return a == 0 ? 0 : (a < 0 ? -1 : 1);
36 }
37 template <class T> T floor(const T &a, const T &b) { // 注意大数据计算时会丢失精度
38     T A = abs(a), B = abs(b);
39     assert(B != 0);
40     return sign(a) * sign(b) > 0 ? A / B : -(A + B - 1) / B;
41 }
42 template <class T> T ceil(const T &a, const T &b) { // 注意大数据计算时会丢失精度
43     T A = abs(a), B = abs(b);
44     assert(b != 0);
45     return sign(a) * sign(b) > 0 ? (A + B - 1) / B : -A / B;
46 }

```

4.1.10 快读

```

1 // #define DEBUG 1 // 调试开关
2 struct IO {
3     #define MAXSIZE (1 << 20)
4     #define isdigit(x) (x >= '0' and x <= '9')
5     char buf[MAXSIZE], *p1, *p2;
6     char pbuf[MAXSIZE], *pp;

```

```

7  #if DEBUG
8  #else
9      IO() : p1(buf), p2(buf), pp(pbuf) {}
10
11      ~IO() { fwrite(pbuf, 1, pp - pbuf, stdout); }
12  #endif
13      char gc() {
14  #if DEBUG // 调试, 可显示字符
15          return getchar();
16  #endif
17          if(p1 == p2) p2 = (p1 = buf) + fread(buf, 1, MAXSIZE, stdin);
18          return p1 == p2 ? ' ' : *p1++;
19      }
20
21      void read(int &x) {
22          bool neg = false;
23          x = 0;
24          char ch = gc();
25          for(; !isdigit(ch); ch = gc())
26              if(ch == '-') neg = true;
27          if(neg)
28              for(; isdigit(ch); ch = gc()) x = x * 10 + ('0' - ch);
29          else
30              for(; isdigit(ch); ch = gc()) x = x * 10 + (ch - '0');
31      }
32
33      void read(char *s) {
34          char ch = gc();
35          for(; isspace(ch); ch = gc());
36          for(; !isspace(ch); ch = gc()) *s++ = ch;
37          *s = 0;
38      }
39
40      void read(char &c) { for(c = gc(); isspace(c); c = gc()); }
41
42      void push(const char &c) {
43  #if DEBUG // 调试, 可显示字符
44          putchar(c);
45  #else
46          if(pp - pbuf == MAXSIZE) fwrite(pbuf, 1, MAXSIZE, stdout), pp = pbuf;
47          *pp++ = c;
48  #endif
49      }
50
51      void write(int x) {
52          bool neg = false;
53          if(x < 0) {
54              neg = true;
55              push('-');
56          }
57          static int sta[40];
58          int top = 0;
59          do {

```

```

60     sta[top++] = x % 10;
61     x /= 10;
62 } while (x);
63 if(neg)
64     while (top) push('0' - sta[--top]);
65 else
66     while (top) push('0' + sta[--top]);
67 }
68
69 void write(int x, char lastChar) { write(x), push(lastChar); }
70 } io;

```

4.1.11 手工哈希

```

1 struct myhash {
2     static uint64_t hash(uint64_t x) {
3         x += 0x9e3779b97f4a7c15;
4         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
5         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
6         return x ^ (x >> 31);
7     }
8     size_t operator()(uint64_t x) const {
9         static const uint64_t SEED = chrono::steady_clock::now().time_since_epoch().
            count();
10        return hash(x + SEED);
11    }
12    size_t operator()(pair<uint64_t, uint64_t> x) const {
13        static const uint64_t SEED = chrono::steady_clock::now().time_since_epoch().
            count();
14        return hash(x.first + SEED) ^ (hash(x.second + SEED) >> 1);
15    }
16 };
17 // unordered_map<int, int, myhash>

```

4.1.12 整数域三分

```

1 while (l < r) {
2     int mid = (l + r) / 2;
3     if (check(mid) <= check(mid + 1)) r = mid;
4     else l = mid + 1;
5 }
6 cout << check(l) << endl;

```

4.1.13 整数域二分

旧版（无法处理负数情况）

- 在递增序列 a 中查找 $\geq x$ 数中最小的一个（即 x 或 x 的后继）
- 在递增序列 a 中查找 $\leq x$ 数中最大的一个（即 x 或 x 的前驱）

新版

- x 或 x 的后继
- x 或 x 的前驱

```
1 while (l < r) {
2     int mid = (l + r) / 2;
3     if (a[mid] >= x) {
4         r = mid;
5     } else {
6         l = mid + 1;
7     }
8 }
9 return a[l];
10
11 while (l < r) {
12     int mid = (l + r + 1) / 2;
13     if (a[mid] <= x) {
14         l = mid;
15     } else {
16         r = mid - 1;
17     }
18 }
19 return a[l];
20
21 int l = 0, r = 1E8, ans = r;
22 while (l <= r) {
23     int mid = (l + r) / 2;
24     if (judge(mid)) {
25         r = mid - 1;
26         ans = mid;
27     } else {
28         l = mid + 1;
29     }
30 }
31 return ans;
32
33 int l = 0, r = 1E8, ans = 1;
34 while (l <= r) {
35     int mid = (l + r) / 2;
36     if (judge(mid)) {
37         l = mid + 1;
38         ans = mid;
39     } else {
40         r = mid - 1;
41     }
42 }
43 return ans;
```

4.1.14 编译器设置

```
1 g++ -O2 -std=c++20 -pipe
2 -Wall -Wextra -Wconversion /* 这部分是警告相关, 可能用不到 */
3 -fstack-protector
4 -Wl,--stack=268435456
```

4.1.15 读取一行数字, 个数未知

```
1 string s;
2 getline(cin, s);
3 stringstream ss;
4 ss << s;
5 while (ss >> s) {
6     auto res = stoi(s);
7     cout << res * 100 << endl;
8 }
```

4.1.16 随机数生成与样例构造

```
1
2 mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
3 int r(int a, int b) {
4     return rnd() % (b - a + 1) + a;
5 }
6
7 void graph(int n, int root = -1, int m = -1) {
8     vector<pair<int, int>> t;
9     for (int i = 1; i < n; i++) { // 先建立一棵以0为根节点的树
10         t.emplace_back(i, r(0, i - 1));
11     }
12
13     vector<pair<int, int>> edge;
14     set<pair<int, int>> uni;
15     if (root == -1) root = r(0, n - 1); // 确定根节点
16     for (auto [x, y] : t) { // 偏移建树
17         x = (x + root) % n + 1;
18         y = (y + root) % n + 1;
19         edge.emplace_back(x, y);
20         uni.emplace(x, y);
21     }
22
23     if (m != -1) { // 如果是图, 则在树的基础上继续加边
24         for (int i = n; i <= m; i++) {
25             while (true) {
26                 int x = r(1, n), y = r(1, n);
27                 if (x == y) continue; // 拒绝自环
28                 if (uni.count({x, y})) continue; // 拒绝重边
29                 edge.emplace_back(x, y);
30                 uni.emplace(x, y);
31             }
32         }
33     }
```

```

34
35     random_shuffle(edge.begin(), edge.end()); // 打乱节点
36     for (auto [x, y] : edge) {
37         cout << x << " " << y << endl;
38     }
39 }

```

4.2 STL

4.2.1 容器与成员函数

元组 tuple

数组 array

变长数组 vector

栈 stack

栈顶入，栈顶出。先进后出。

队列 queue

队尾进，队头出。先进先出。

双向队列 deque

优先队列 priority_queue

默认升序（大根堆），自定义排序需要重载 $<$ 。

字符串 string

有序、多重有序集合 set、multiset

默认升序（大根堆），set 去重，multiset 不去重， $\mathcal{O}(\log N)$ 。

特殊函数 next 和 prev 详解：

erase(x)；有两种删除方式：

- 当 x 为某一元素时，删除所有这个数，复杂度为 $\mathcal{O}(num_x + \log N)$ ；
- 当 x 为迭代器时，删除这个迭代器。

map、multimap

默认升序（大根堆），map 去重，multimap 不去重， $\mathcal{O}(\log S)$ ，其中 S 为元素数量。

erase(x)；有两种删除方式：

- 当 x 为某一元素时，删除所有以这个元素为下标的二元组，复杂度为 $\mathcal{O}(num_x + \log N)$ ；
- 当 x 为迭代器时，删除这个迭代器。

慎用随机访问！——当不确定某次查询是否存在于容器中时，不要直接使用下标查询，而是先使用 count() 或者 find() 方法检查 key 值，防止不必要的零值二元组被构造。

慎用自带的 pair、tuple 作为 key 值类型！使用自定义结构体！

bitset

将数据转换为二进制，从高位到低位排序，以 0 为最低位。当位数相同时支持全部的位运算。

哈希系列 unordered

通常指代 `unordered_map`、`unordered_set`、`unordered_multimap`、`unordered_multiset`，与原版相比不进行排序。

如果将不支持哈希的类型作为 `key` 值代入，编译器就无法正常运行，这时需要我们为其手写哈希函数。而我们写的这个哈希函数的正确性其实并不是特别重要（但是不可以没有），当发生冲突时编译器会调用 `key` 的 `operator ==` 函数进行进一步判断。参考

对 `pair`、`tuple` 定义哈希

对结构体定义哈希

需要两个条件，一个是在结构体中重载等于号（区别于非哈希容器需要重载小于号，如上所述，当冲突时编译器需要根据重载的等于号判断），第二是写一个哈希函数。注意 `hash<>()` 的尖括号中的类型匹配。

对 `vector` 定义哈希

以下两个方法均可。注意 `hash<>()` 的尖括号中的类型匹配。

```

1 //获取obj对象中的第index个元素——get<index>(obj)
2 //需要注意的是这里的index只能手动输入，使用for循环这样的自动输入是不可以的
3 tuple<string, int, int> Student = {"Wida", 23, 45000};
4 cout << get<0>(Student) << endl; //获取Student对象中的第一个元素，这里的输出结果应为 “
   Wida”
5
6 array<int, 3> x; // 建立一个包含三个元素的数组x
7
8 [] // 跟正常数组一样，可以使用随机访问
9 cout << x[0]; // 获取数组中的第一个元素
10
11 resize(n) // 重设容器大小，但是不改变已有元素的值
12 assign(n, 0) // 重设容器大小为n，且替换容器内的内容为0
13
14 // 尽量不要使用[]的形式声明多维变长数组，而是使用嵌套的方式替代
15 vector<int> ver[n + 1]; // 不好的声明方式
16 vector<vector<int>> ver(n + 1);
17
18 // 嵌套时只需要在最后一个注明变量类型
19 vector dis(n + 1, vector<int>(m + 1));
20 vector dis(m + 1, vector(n + 1, vector<int>(n + 1)));
21
22 //没有clear函数
23 size() / empty()
24 push(x) //向栈顶插入x
25 top() //获取栈顶元素
26 pop() //弹出栈顶元素
27
28 //没有clear函数
29 size() / empty()
30 push(x) //向队尾插入x
31 front() / back() //获取队头、队尾元素
32 pop() //弹出队头元素
33
34 //没有clear函数，但是可以用重新构造替代
35 queue<int> q;

```

```

36 q = queue<int>();
37
38 size() / empty() / clear()
39 push_front(x) / push_back(x)
40 pop_front(x) / pop_back(x)
41 front() / back()
42 begin() / end()
43 []
44
45 //没有clear函数
46 priority_queue<int, vector<int>, greater<int> > p; //重定义为降序（小根堆）
47 push(x); //向栈顶插入x
48 top(); //获取栈顶元素
49 pop(); //弹出栈顶元素
50
51 //重载运算符【注意，符号相反!!!】
52 struct Node {
53     int x; string s;
54     friend bool operator < (const Node &a, const Node &b) {
55         if (a.x != b.x) return a.x > b.x;
56         return a.s > b.s;
57     }
58 };
59
60 size() / empty() / clear()
61
62 //从字符串S的S[start]开始，取出长度为len的子串——S.substr(start, len)
63 //len省略时默认取到结尾，超过字符串长度时也默认取到结尾
64 cout << S.substr(1, 12);
65
66 find(x) / rfind(x); //顺序、逆序查找x，返回下标，没找到时返回一个极大值【! 建议与 size()
    比较，而不要和 -1 比较，后者可能出错】
67 //注意，没有count函数
68
69 set<int, greater<> > s; //重定义为降序（小根堆）
70 size() / empty() / clear()
71 begin() / end()
72 ++ / -- //返回前驱、后继
73
74 insert(x); //插入x
75 find(x) / rfind(x); //顺序、逆序查找x，返回迭代器【迭代器!!!】，没找到时返回end()
76 count(x); //返回x的个数
77 lower_bound(x); //返回第一个>=x的迭代器【迭代器!!!】
78 upper_bound(x); //返回第一个>x的迭代器【迭代器!!!】
79
80 auto it = s.find(x); // 建立一个迭代器
81 prev(it) / next(it); // 默认返回迭代器it的前/后一个迭代器
82 prev(it, 2) / next(it, 2); // 可选参数可以控制返回前/后任意个迭代器
83
84 /* 以下是一些应用 */
85 auto pre = prev(s.lower_bound(x)); // 返回第一个<x的迭代器
86 int ed = *prev(S.end(), 1); // 返回最后一个元素
87

```

```

88 //连续头部删除
89 set<int> S = {0, 9, 98, 1087, 894, 34, 756};
90 auto it = S.begin();
91 int len = S.size();
92 for (int i = 0; i < len; ++ i) {
93     if (*it >= 500) continue;
94     it = S.erase(it); //删除所有小于500的元素
95 }
96 //错误用法如下【千万不能这样用!!!】
97 //for (auto it : S) {
98 // if (it >= 500) continue;
99 // S.erase(it); //删除所有小于500的元素
100 //}
101
102 map<int, int, greater<> > mp; //重定义为降序（小根堆）
103 size() / empty() / clear()
104 begin() / end()
105 ++ / -- //返回前驱、后继
106
107 insert({x, y}); //插入二元组
108 [] //随机访问, multimap不支持
109 count(x); //返回x为下标的个数
110 lower_cound(x); //返回第一个下标>=x的迭代器
111 upper_cound(x); //返回第一个下标>x的迭代器
112
113 int q = 0;
114 if (mp.count(i)) q = mp[i];
115
116 struct fff {
117     LL x, y;
118     friend bool operator < (const fff &a, const fff &b) {
119         if (a.x != b.x) return a.x < b.x;
120         return a.y < b.y;
121     }
122 };
123 map<fff, int> mp;
124
125 // 如果输入的是01字符串, 可以直接使用">>"读入
126 bitset<10> s;
127 cin >> s;
128
129 //使用只含01的字符串构造——bitset<容器长度>B (字符串)
130 string S; cin >> S;
131 bitset<32> B (S);
132
133 //使用整数构造 (两种方式)
134 int x; cin >> x;
135 bitset<32> B1 (x);
136 bitset<32> B2 = x;
137
138 // 构造时, 尖括号里的数字不能是变量
139 int x; cin >> x;
140 bitset<x> ans; // 错误构造

```

```

141
142 [] //随机访问
143 set(x) //将第x位置1, x省略时默认全部位置1
144 reset(x) //将第x位置0, x省略时默认全部位置0
145 flip(x) //将第x位取反, x省略时默认全部位取反
146 to_ullong() //重转换为ULL类型
147 to_string() //重转换为ULL类型
148 count() //返回1的个数
149 any() //判断是否至少有一个1
150 none() //判断是否全为0
151
152 _Find_fisrt() // 找到从低位到高位第一个1的位置
153 _Find_next(x) // 找到当前位置x的下一个1的位置, 复杂度 O(n/w + count)
154
155 bitset<23> B1("11101001"), B2("11101000");
156 cout << (B1 ^ B2) << "\n"; //按位异或
157 cout << (B1 | B2) << "\n"; //按位或
158 cout << (B1 & B2) << "\n"; //按位与
159 cout << (B1 == B2) << "\n"; //比较是否相等
160 cout << B1 << " " << B2 << "\n"; //你可以直接使用cout输出
161
162 struct hash_pair {
163     template <class T1, class T2>
164     size_t operator()(const pair<T1, T2> &p) const {
165         return hash<T1>()(p.fi) ^ hash<T2>()(p.se);
166     }
167 };
168 unordered_set<pair<int, int>, int, hash_pair> S;
169 unordered_map<tuple<int, int, int>, int, hash_pair> M;
170
171 struct fff {
172     string x, y;
173     int z;
174     friend bool operator == (const fff &a, const fff &b) {
175         return a.x == b.x || a.y == b.y || a.z == b.z;
176     }
177 };
178 struct hash_fff {
179     size_t operator()(const fff &p) const {
180         return hash<string>()(p.x) ^ hash<string>()(p.y) ^ hash<int>()(p.z);
181     }
182 };
183 unordered_map<fff, int, hash_fff> mp;
184
185 struct hash_vector {
186     size_t operator()(const vector<int> &p) const {
187         size_t seed = 0;
188         for (auto it : p) {
189             seed ^= hash<int>()(it);
190         }
191         return seed;
192     }
193 };

```

```

194 unordered_map<vector<int>, int, hash_vector> mp;
195
196 namespace std {
197     template<> struct hash<vector<int>> {
198         size_t operator()(const vector<int> &p) const {
199             size_t seed = 0;
200             for (int i : p) {
201                 seed ^= hash<int>()(i) + 0x9e3779b9 + (seed << 6) + (seed >> 2);
202             }
203             return seed;
204         }
205     };
206 }
207 unordered_set<vector<int> > S;

```

4.2.2 库函数

pb_ds 库

其中 `gp_hash_table` 使用的最多，其等价于 `unordered_map`，内部是无序的。

查找后继 `lower_bound`、`upper_bound`

`lower` 表示 $\geq$ ，`upper` 表示 $>$ 。使用前记得先进行排序。

数组打乱 `shuffle`

二分搜索 `binary_search`

用于查找某一元素是否在容器中，相当于 `find` 函数。在使用前需要先进行排序。

批量递增赋值函数 `iota`

对容器递增初始化。

数组去重函数 `unique`

在使用前需要先进行排序。

其作用是，对于区间 [开始位置，结束位置)，不停的把后面不重复的元素移到前面来，也可以说是用不重复的元素占领重复元素的位置。并且返回去重后容器中不重复序列的最后一个元素的下一个元素。所以在进行操作后，数组、容器的大小并没有发生改变。

bit 库与位运算函数 `__builtin__`

注：以上函数的 `long long` 版本只需要在函数后面加上 `ll` 即可（例如 `__builtin_popcountll(x)`），`unsigned long long` 加上 `ull`。

数字转字符串函数

`itoa` 虽然能将整数转换成任意进制的字符串，但是其不是标准的 C 函数，且为 Windows 独有，且不支持 `long long`，建议手写。

字符串转数字

全排列算法 `next_permutation`、`prev_permutation`

在提及这个函数时，我们先需要补充几点字典序相关的知识。

对于三个字符所组成的序列 {a,b,c}，其按照字典序的 6 种排列分别为：

{abc}, {acb}, {bac}, {bca}, {cab}, {cba} 其排序原理是：先固定 a (序列内最小元素)，再对之后的元素排列。而 $b < c$ ，所以 $abc < acb$ 。同理，先固定 b (序列内次小元素)，再对之后的元素排列。即可得出以上序列。

next_permutation 算法，即是按照字典序顺序输出的全排列；相对应的，prev_permutation 则是按照逆字典序顺序输出的全排列。可以是数字，亦可以是其他类型元素。其直接在序列上进行更新，故直接输出序列即可。

字符串转换为数值函数 stoi

可以快捷的将一串字符串转换为指定进制的数字。

使用方法

- stoi(字符串, 0, x 进制)：将一串 x 进制的字符串转换为 int 型数字。
- stoll(字符串, 0, x 进制)：将一串 x 进制的字符串转换为 long long 型数字。
- stoull stod stold 同理。

数值转换为字符串函数 to_string

允许将各种数值类型转换为字符串类型。

判断非递减 is_sorted

累加 accumulate

迭代器 iterator

其他函数

exp2(x)：返回 2^x

log2(x)：返回 $\log_2(x)$

gcd(x, y) / lcm(x, y)：以 log 的复杂度返回 $\gcd(|x|, |y|)$ 与 $\text{lcm}(|x|, |y|)$ ，且返回值符号也为正数。

```

1  #include <bits/extc++.h>
2  #include <ext/pb_ds/assoc_container.hpp>
3  template<class S, class T> using omap = __gnu_pbds::gp_hash_table<S, T, myhash>;
4
5  //返回a数组[start,end)区间中第一个>=x的地址【地址!!!】
6  cout << lower_bound(a + start, a + end, x);
7
8  cout << lower_bound(a, a + n, x) - a; //在a数组中查找第一个>=x的元素下标
9  upper_bound(a, a + n, k) - lower_bound(a, a + n, k) //查找k在a中出现了几次
10
11 mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
12 shuffle(ver.begin(), ver.end(), rnd);
13
14 //在a数组[start,end)区间中查找x是否存在，返回bool型
15 cout << binary_search(a + start, a + end, x);
16
17 //将a数组[start,end)区间复制成“x, x+1, x+2, ...”
18 iota(a + start, a + end, x);
19
20 //将a数组[start,end)区间去重，返回迭代器

```

```

21 unique(a + start, a + end);
22
23 //与erase函数结合, 达到去重+删除的目的
24 a.erase(unique(ALL(a)), a.end());
25
26 __builtin_popcount(x) // 返回x二进制下含1的数量, 例如x=15=(1111)时答案为4
27
28 __builtin_ffs(x) // 返回x右数第一个1的位置(1-idx), 1(1) 返回 1, 8(1000) 返回 4, 26
    (11010) 返回 2
29
30 __builtin_ctz(x) // 返回x二进制下后导0的个数, 1(1) 返回 0, 8(1000) 返回 3
31
32 bit_width(x) // 返回x二进制下的位数, 9(1001) 返回 4, 26(11010) 返回 5
33
34 // to_string函数会直接将你的各种类型的数字转换为字符串。
35 // string to_string(T val);
36 double val = 12.12;
37 cout << to_string(val);
38
39 // 【不建议使用】itoa允许你将整数转换成任意进制的字符串, 参数为待转换整数、目标字符数组、进
    制。
40 // char* itoa(int value, char* string, int radix);
41 char ans[10] = {};
42 itoa(12, ans, 2);
43 cout << ans << endl; /*1100*/
44
45 // 长整型函数名ltoa, 最高支持到int型上限2^31。ultoa同理。
46
47 // stoi直接使用
48 cout << stoi("12") << endl;
49
50 // 【不建议使用】stoi转换进制, 参数为待转换字符串、起始位置、进制。
51 // int stoi(string value, int st, int radix);
52 cout << stoi("1010", 0, 2) << endl; /*10*/
53 cout << stoi("c", 0, 16) << endl; /*12*/
54 cout << stoi("0x3f3f3f3f", 0, 0) << endl; /*1061109567*/
55
56 // 长整型函数名stoll, 最高支持到long long型上限2^63。stoull、stod、stold同理。
57
58 // atoi直接使用, 空字符返回0, 允许正负符号, 数字字符前有其他字符返回0, 数字字符前有空白字符
    自动去除
59 cout << atoi("12") << endl;
60 cout << atoi(" 12") << endl; /*12*/
61 cout << atoi("-12abc") << endl; /*-12*/
62 cout << atoi("abc12") << endl; /*0*/
63
64 // 长整型函数名atoll, 最高支持到long long型上限2^63。
65
66 int n;
67 cin >> n;
68 vector<int> a(n);
69 // iota(a.begin(), a.end(), 1);
70 for (auto &it : a) cin >> it;

```

```

71 sort(a.begin(), a.end());
72
73 do {
74     for (auto it : a) cout << it << " ";
75     cout << endl;
76 } while (next_permutation(a.begin(), a.end()));
77
78 //将数值num转换为字符串s
79 string s = to_string(num);
80
81 //a数组[start,end)区间是否是递增的, 返回bool型
82 cout << is_sorted(a + start, a + end);
83
84 //将a数组[start,end)区间的元素进行累加, 并输出累加和+x的值
85 cout << accumulate(a + start, a + end, x);
86
87 //构建一个UUU容器的正向迭代器, 名字叫it
88 UUU::iterator it;
89
90 vector<int>::iterator it; //创建一个正向迭代器, ++ 操作时指向下一个
91 vector<int>::reverse_iterator it; //创建一个反向迭代器, ++ 操作时指向上一个

```

4.2.3 程序标准化

使用 Lambda 函数

- function 统一写法

需要注意的是, 虽然 `function` 定义时已经声明了返回值类型了, 但是有的时候会出错 (例如, 声明返回 `long long` 但是返回 `int`, 原因没去了解), 所以推荐在后面使用 `->` 再行声明一遍。

- auto 非递归写法

不需要使用递归函数时, 直接用 `auto` 替换 `function` 即可。

- auto 递归写法

相较于 `function` 写法, 需要额外引用一遍自身。

使用构造函数

可以将一些必要的声明和预处理放在构造函数, 在编译时, 无论放置在程序的哪个位置, 都会先于主函数进行。下方是我将输入流控制声明的过程。

卡常

```

1 function<void(int, int)> clac = [&](int x, int y) -> void {
2 };
3 clac(1, 2);
4
5 function<bool(int)> dfs = [&](int x) -> bool {
6     return dfs(x + 1);

```



```

7  };
8  dfs(1);
9
10 auto clac = [&](int x, int y) -> void {
11 };
12
13 auto dfs = [&](auto self, int x) -> bool {
14     return self(self, x + 1);
15 };
16 dfs(dfs, 1);
17
18 int __FAST_IO__ = []() { // 函数名称可以随意修改
19     ios::sync_with_stdio(0), cin.tie(0);
20     cout.tie(0);
21     cout << fixed << setprecision(12);
22     freopen("out.txt", "r", stdin);
23     freopen("in.txt", "w", stdout);
24     return 0;
25 }();

```

4.3 杂类

4.3.1 蒙特卡洛模拟概率问题

```

1  #include <cstdio>
2  #include <algorithm>
3  #include <random>
4
5  const int N = 16, M = 300000;
6
7  int main() {
8      double p[N][N];
9      for (int i = 0; i < N; i++)
10         for (int j = 0; j < N; j++)
11             scanf("%lf", &p[i][j]);
12
13     int w[N] = {0}, t[N], nt[N];
14     std::random_device rd;
15     std::mt19937 gen(rd());
16     std::uniform_real_distribution<double> dis(0.0, 1.0);
17
18     for (int s = 0; s < M; s++) {
19         for (int i = 0; i < N; i++) t[i] = i;
20         int c = N;
21         for (int r = 0; r < 4; r++) {
22             int nc = 0;
23             for (int i = 0; i < c; i += 2) {
24                 int a = t[i], b = t[i + 1];
25                 double rnd = dis(gen);
26                 if (rnd <= p[a][b]) nt[nc++] = a;
27                 else nt[nc++] = b;
28             }

```

```

29         std::sort(nt, nt + nc);
30         for (int i = 0; i < nc; i++) t[i] = nt[i];
31         c /= 2;
32     }
33     w[t[0]]++;
34 }
35
36 int champ = 0, maxc = w[0];
37 for (int i = 1; i < N; i++)
38     if (w[i] > maxc) {
39         maxc = w[i];
40         champ = i;
41     }
42
43 printf("%d\n", champ);
44 return 0;
45 }

```

4.3.2 区间哈希 1

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef unsigned long long ui64;
4  mt19937_64 rnd(time(0));
5  using p32 = pair<int, int>;
6  /*
7   给你n个区间
8   判断区间是否两两相交
9   此题的相交指的是 包含(不等于) 与 相离之外的所有情况
10  所以包括了区间等于
11  */
12 void solve() {
13     int n;
14     cin >> n;
15     vector<p32> nodes; // 存储所有事件点 (左端点和右端点)
16     vector<ui64> hsh(n + 1); // 区间赋值 随机哈希值
17     vector<ui64> rec1(n + 1); // 记录第i个区间左右端点处的哈希累积结果
18     for (int i = 1; i <= n; i++) hsh[i] = rnd();
19
20     // 2. 收集事件点: 每个区间拆分为"左端点事件"和"右端点事件"
21     for (int i = 1; i <= n; i++) {
22         int l, r;
23         cin >> l >> r;
24         nodes.push_back({l, i});
25         // (用+n区分左右事件)
26         nodes.push_back({r, i + n});
27     }
28
29     // 3. 按坐标x升序排序事件点: 保证从左到右处理区间的开始和结束
30     sort(nodes.begin(), nodes.end());
31
32     // 4. 初始化哈希累积变量

```

```

33     ui64 H = 0; // 当前被覆盖的区间的哈希异或和 (动态更新)
34     ui64 sum = 0; // 所有区间哈希值的异或和 (代表"所有区间都覆盖"的状态)
35     for (int i = 1; i <= n; i++) {
36         sum ^= hsh[i]; // 计算  $G = hsh[1] \oplus hsh[2] \oplus \dots \oplus hsh[n]$ 
37     }
38
39     // 5. 处理所有事件点, 更新H和rec1
40     for (auto [x, y] : nodes) { // [x, y]: x是坐标, y是事件类型
41         if (y <= n) { // 左端点事件: 区间y开始覆盖
42             // 记录区间y的左端点处, 当前已覆盖的区间哈希和 (此时还未加入y自身)
43             rec1[y] = H;
44             // 将区间y的哈希加入H (表示y开始被覆盖)
45             H ^= hsh[y];
46         } else { // 右端点事件: 区间(y - n)结束覆盖 (y = 原区间编号 + n)
47             int idx = y - n; // 还原为原区间编号
48             // 记录区间idx的右端点处, 剩余覆盖的区间哈希和 (此时已移除idx自身)
49             rec1[idx] ^= H;
50             // 将区间idx的哈希从H中移除 (表示idx结束覆盖)
51             H ^= hsh[idx];
52         }
53     }
54
55     // 6. 验证是否存在公共交集: 所有区间的rec1[i]必须等于G
56     // 原理: 若存在公共点x, 所有区间都覆盖x, 则每个区间的[左, 右]必包含x
57     // 此时rec1[i]会累积所有区间的哈希异或和G (即所有区间同时覆盖x)
58     for (int i = 1; i <= n; i++) {
59         if (rec1[i] != sum) { // 存在区间不满足, 说明无公共交集
60             cout << "No\n";
61             return;
62         }
63     }
64     // 所有区间都满足, 说明存在公共交集 (两两相交)
65     cout << "Yes\n";
66 }
67 signed main() {
68     int times = 1; cin >> times;
69     while (times--) solve();
70     return 0;
71 }

```

4.3.3 区间哈希 2

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef unsigned long long ui64;
4  mt19937_64 rnd(time(NULL));
5  #define int long long
6  using p32 = pair<int, int>;
7  /*
8   在一个无限长的数轴上, 有n个集合, 每个集合给定一个范围
9   [l,r], 其中 l 和 r 为整数, 且满足  $l \leq r$ 
10  我们称两个整数 x 和 y 性质不同, 当且仅当存在

```

```

11 至少一个集合 S, 使得 x 属于 S 但 y 不属于 S
12 或者反过来
13 你的任务是计算在这个数轴上最多可以选出多少个数,
14 使得这些数的性质两两不同。
15 */
16 int n;
17 void solve() {
18     cin >> n;
19     vector<p32> pos; // 存储所有事件点 (记录集合开始/结束的位置)
20     map<int, int> mp; // 用于记录所有不同的"特征标签" (去重)
21
22     for (int i = 1; i <= n; i++) {
23         int l, r, flag;
24         cin >> l >> r; // 输入第i个集合的范围[l, r]
25         flag = rnd(); // 给当前集合分配一个随机哈希值作为唯一标识
26         // 事件点1: x=l时, 数开始属于该集合 (特征标签需加入flag)
27         pos.push_back({l, flag});
28         // 事件点2: x=r+1时, 数不再属于该集合 (特征标签需移除flag)
29         pos.push_back({r + 1, flag});
30     }
31     // 特殊情况: 若没有集合, 所有数性质相同, 最多选1个
32     if (n == 0) {
33         cout << "1\n";
34         return;
35     }
36     // 按x坐标升序排序事件点, 确保从左到右遍历数轴
37     sort(pos.begin(), pos.end());
38     int tmp = 0; // 存储当前位置的"特征标签": 由所有包含当前数的集合的哈希值异或而成
39     // 遍历所有事件点, 处理特征标签的变化
40     for (int i = 0; i < 2 * n; i++) {
41         auto [x, y] = pos[i]; // 当前事件点的坐标x和集合标识y
42         // 处理所有x坐标相同的事件点 (同一位置可能有多个集合开始/结束)
43         while (x == pos[i].first) {
44             // 异或操作: 加入或移除集合标识 (异或的逆操作是自身, 适合动态更新)
45             tmp = tmp ^ pos[i].second;
46             i++;
47             if (i == 2 * n) break; // 防止越界
48         }
49         // 记录当前特征标签 (同一标签只存一次, map自动去重)
50         mp[tmp] = 1;
51     }
52     // map的大小即为不同特征标签的数量, 也就是最多可选择的性质不同的数的数量
53     cout << mp.size() << '\n';
54 }
55 signed main() {
56     ios::sync_with_stdio(false);
57     cin.tie(0);
58     int times = 1; cin >> times;
59     while (times--) solve();
60     return 0;
61 }

```

4.3.4 N*M 数独字典序最小方案

```

1  ### N*M 数独字典序最小方案
2
3  规则：每个宫大小为  $2^N \times 2^M$ ，大图一共由  $M \times N$  个宫组成（总大小即  $2^{N^2} \times 2^{M^2}$ ），要求每行、每列、每宫都要出现  $1$  到  $2^{N \times 2^M}$  的全部数字。输出字典序最小方案。
4
5  下例为  $2, 1$  和  $1, 2$  时数独字典序最小的示意。
6
7  
8
9  公式： $(i, j)$  格所填的内容为  $\big(i \bmod 2^N \oplus \left\lfloor \frac{j}{2^M} \right\rfloor \cdot 2^M + \left\lfloor \frac{i}{2^N} \right\rfloor \oplus j \bmod 2^M \big) + 1$ ，注意  $i, j$  从  $0$  开始。

```

4.3.5 单调队列

查询区间 k 的最大最小值。

```

1  deque<int> D;
2  int n, k, x, a[MAX];
3  int main(){
4      IOS();
5      cin >> n >> k;
6      for(int i=1; i<=n; i++) cin >> a[i];
7      for(int i=1; i<=n; i++){
8          while(!D.empty() && a[D.back()]<=a[i]) D.pop_back();
9          D.emplace_back(i);
10         if(!D.empty()) if(i-D.front()>=k) D.pop_front();
11         if(i>=k) cout<<a[D.front()]<<endl;
12     }
13     return 0;
14 }

```

4.3.6 幻方

构造题用，其有一些性质（保证 N 为奇数）：1 到 N^2 每个数字恰好使用一次，且每行、每列及两条对角线上的数字之和都相同，且为奇数 See。

构造方式：将 1 写在第一行的中间，随后不断向右上角位置填下一个数字，直到填满。

```

1  int n;
2  cin >> n;
3  int x = 1, y = (n + 1) / 2;
4  vector ans(n + 1, vector<int>(n + 1));
5  for (int i = 1; i <= n * n; i++) {
6      ans[x][y] = i;
7      if (!ans[(x - 2 + n) % n + 1][y % n + 1]){
8          x = (x - 2 + n) % n + 1;
9          y = y % n + 1;
10     } else {

```

```

11     x = x % n + 1;
12 }
13 }

```

4.3.7 德州扑克

读入牌型，并且支持两副牌之间的大小比较。代码参考

```

1 struct card {
2     int suit, rank;
3     friend bool operator < (const card &a, const card &b) {
4         return a.rank < b.rank;
5     }
6     friend bool operator == (const card &a, const card &b) {
7         return a.rank == b.rank;
8     }
9     friend bool operator != (const card &a, const card &b) {
10        return a.rank != b.rank;
11    }
12    friend auto &operator>> (istream &it, card &C) {
13        string S, T; it >> S;
14        T = "__23456789TJQKA"; //点数
15        FOR (i, 0, T.sz - 1) {
16            if (T[i] == S[0]) C.rank = i;
17        }
18        T = "_SHCD"; //花色
19        FOR (i, 0, T.sz - 1) {
20            if (T[i] == S[1]) C.suit = i;
21        }
22        return it;
23    }
24 };
25 struct game {
26     int level;
27     vector<card> peo;
28     int a, b, c, d, e;
29     int u, v, w, x, y;
30     bool Rk10() { //Rk10: Royal Flush, 五张牌同花色, 且点数为AKQJT (14,13,12,11,10)
31         sort(ALL(peo));
32         reverse(ALL(peo));
33         a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e = peo[4].rank;
34         u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y = peo[4].suit;
35
36         if (u != v || v != w || w != x || x != y) return 0;
37         if (a == 14 && b == 13 && c == 12 && d == 11 && e == 10) return 1;
38         return 0;
39     }
40     bool Dif(vector<card> &peo) { //专门用于检查A2345这种顺子的情况 (这是最小的顺子)
41         a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e = peo[4].rank;

```

```

42     u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y = peo
      [4].suit;
43
44     if (a != 14 || b != 5 || c != 4 || d != 3 || e != 2) return 0;
45     vector<card> peo2 = {peo[1], peo[2], peo[3], peo[4], peo[0]}; //重新排序
46     peo = peo2;
47     return 1;
48 }
49 bool Rk9() { //Rk9: Straight Flush, 五张牌同花色, 且顺连 【r1 > r2 > r3 > r4 > r5】
50     sort(ALL(peo));
51     reverse(ALL(peo));
52     a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e = peo
      [4].rank;
53     u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y = peo
      [4].suit;
54
55     if (u != v || v != w || w != x || x != y) return 0;
56     if (Dif(peo)) return 1; //特判: A2345
57     if (a == b + 1 && b == c + 1 && c == d + 1 && d == e + 1) return 1;
58     return 0;
59 }
60 bool Rk8() { //Rk8: Four of a Kind, 四张牌点数一样 【r1 = r2 = r3 = r4】
61     sort(ALL(peo));
62     a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e = peo
      [4].rank;
63     u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y = peo
      [4].suit;
64
65     if (a == b && b == c && c == d) return 1;
66     if (b == c && c == d && d == e) {
67         reverse(ALL(peo));
68         return 1;
69     }
70     return 0;
71 }
72 bool Rk7() { //Rk7: Fullhouse, 三张牌点数一样, 另外两张点数也一样 【r1 = r2 = r3, r4
      = r5】
73     sort(ALL(peo));
74     a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e = peo
      [4].rank;
75     u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y = peo
      [4].suit;
76
77     if (a == b && b == c && d == e) return 1;
78     if (a == b && c == d && d == e) {
79         reverse(ALL(peo));
80         return 1;
81     }
82     return 0;
83 }
84 bool Rk6() { //Rk6: Flush, 五张牌同花色 【r1 > r2 > r3 > r4 > r5】
85     sort(ALL(peo));
86     reverse(ALL(peo));

```

```

87     a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e = peo
      [4].rank;
88     u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y = peo
      [4].suit;
89
90     if (u != v || v != w || w != x || x != y) return 0;
91     return 1;
92 }
93 bool Rk5() { //Rk5: Straight, 五张牌顺连 【r1 > r2 > r3 > r4 > r5】
94     sort(ALL(peo));
95     reverse(ALL(peo));
96     a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e = peo
      [4].rank;
97     u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y = peo
      [4].suit;
98
99     if (Dif(peo)) return 1; //特判: A2345
100    if (a == b + 1 && b == c + 1 && c == d + 1 && d == e + 1) return 1;
101    return 0;
102 }
103 bool Rk4() { //Rk4: Three of a kind, 三张牌点数一样 【r1 = r2 = r3, r4 > r5】
104     sort(ALL(peo));
105     reverse(ALL(peo));
106     a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e = peo
      [4].rank;
107     u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y = peo
      [4].suit;
108
109     if (a == b && b == c) return 1;
110     if (b == c && c == d) {
111         swap(peo[3], peo[0]);
112         return 1;
113     }
114     if (c == d && d == e) {
115         swap(peo[3], peo[0]);
116         swap(peo[4], peo[1]);
117         return 1;
118     }
119     return 0;
120 }
121 bool Rk3() { //Rk3: Two Pairs, 两张牌点数一样, 另外有两张点数也一样 (两个对子) 【r1 =
      r2 > r3 = r4】
122     sort(ALL(peo));
123     reverse(ALL(peo));
124     a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e = peo
      [4].rank;
125     u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y = peo
      [4].suit;
126
127     if (a == b && c == d) return 1;
128     if (a == b && d == e) {
129         swap(peo[2], peo[4]);
130         return 1;

```



```

131     }
132     if (b == c && d == e) {
133         swap(peo[0], peo[2]);
134         swap(peo[2], peo[4]);
135         return 1;
136     }
137     return 0;
138 }
139 bool Rk2() { //Rk2: One Pairs, 两张牌点数一样 (一个对子) 【r1 = r2, r3 > r4 > r5】
140     sort(ALL(peo));
141     reverse(ALL(peo));
142     a = peo[0].rank, b = peo[1].rank, c = peo[2].rank, d = peo[3].rank, e = peo
143         [4].rank;
144     u = peo[0].suit, v = peo[1].suit, w = peo[2].suit, x = peo[3].suit, y = peo
145         [4].suit;
146     vector<card> peo2;
147     if (a == b) return 1;
148     if (b == c) {
149         peo2 = {peo[1], peo[2], peo[0], peo[3], peo[4]};
150         peo = peo2;
151         return 1;
152     }
153     if (c == d) {
154         peo2 = {peo[2], peo[3], peo[0], peo[1], peo[4]};
155         peo = peo2;
156         return 1;
157     }
158     if (d == e) {
159         peo2 = {peo[3], peo[4], peo[0], peo[1], peo[2]};
160         peo = peo2;
161         return 1;
162     }
163     return 0;
164 }
165 bool Rk1() { //Rk1: high card
166     sort(ALL(peo));
167     reverse(ALL(peo));
168     return 1;
169 }
170 game (vector<card> New_peo) {
171     peo = New_peo;
172     if (Rk10()) { level = 10; return; }
173     if (Rk9()) { level = 9; return; }
174     if (Rk8()) { level = 8; return; }
175     if (Rk7()) { level = 7; return; }
176     if (Rk6()) { level = 6; return; }
177     if (Rk5()) { level = 5; return; }
178     if (Rk4()) { level = 4; return; }
179     if (Rk3()) { level = 3; return; }
180     if (Rk2()) { level = 2; return; }
181     if (Rk1()) { level = 1; return; }
182 }

```

```

182     friend bool operator < (const game &a, const game &b) {
183         if (a.level != b.level) return a.level < b.level;
184         FOR (i, 0, 4) if (a.peo[i] != b.peo[i]) return a.peo[i] < b.peo[i];
185         return 0;
186     }
187     friend bool operator == (const game &a, const game &b) {
188         if (a.level != b.level) return 0;
189         FOR (i, 0, 4) if (a.peo[i] != b.peo[i]) return 0;
190         return 1;
191     }
192 };
193 void debug(vector<card> peo) {
194     for (auto it : peo) cout << it.rank << " " << it.suit << " ";
195     cout << "\n\n";
196 }
197 int clac(vector<card> Ali, vector<card> Bob) {
198     game atype(Ali), btype(Bob);
199     if (atype < btype) return -1;
200     else if (atype == btype) return 0;
201     return 1;
202 }

```

4.3.8 日期换算（基姆拉尔森公式）

已知年月日，求星期数。

```

1 int week(int y,int m,int d){
2     if(m<=2)m+=12,y--;
3     return (d+2*m+3*(m+1)/5+y+y/4-y/100+y/400)%7+1;
4 }

```

4.3.9 最长严格/非严格递增子序列 (LIS)

一维

注意子序列是不连续的。使用二分搜索，以 $\mathcal{O}(N \log N)$ 复杂度通过，另也有 $\mathcal{O}(N^2)$ 的 dp 解法。dis dis

二维 + 输出方案

```

1 vector<int> val; // 堆数
2 for (int i = 1, x; i <= n; i++) {
3     cin >> x;
4     int it = upper_bound(val.begin(), val.end(), x) - val.begin(); // low/upp: 严格/
        非严格递增
5     if (it >= val.size()) { // 新增一堆
6         val.push_back(x);
7     } else { // 更新对应位置元素
8         val[it] = x;
9     }
10 }
11 cout << val.size() << endl;
12

```

```

13 vector<array<int, 3>> in(n + 1);
14 for (int i = 1; i <= n; i++) {
15     cin >> in[i][0] >> in[i][1];
16     in[i][2] = i;
17 }
18 sort(in.begin() + 1, in.end(), [&](auto x, auto y) {
19     if (x[0] != y[0]) return x[0] < y[0];
20     return x[1] > y[1];
21 });
22
23 vector<int> val{0}, idx{0}, pre(n + 1);
24 for (int i = 1; i <= n; i++) {
25     auto [x, y, z] = in[i];
26     int it = lower_bound(val.begin(), val.end(), y) - val.begin(); // low/upp: 严格/
        非严格递增
27     if (it >= val.size()) { // 新增一堆
28         pre[z] = idx.back();
29         val.push_back(y);
30         idx.push_back(z);
31     } else { // 更新对应位置元素
32         pre[z] = idx[it - 1];
33         val[it] = y;
34         idx[it] = z;
35     }
36 }
37
38 vector<int> ans;
39 for (int i = idx.back(); i != 0; i = pre[i]) {
40     ans.push_back(i);
41 }
42 reverse(ans.begin(), ans.end());
43 cout << ans.size() << "\n";
44 for (auto it : ans) {
45     cout << it << " ";
46 }

```

4.3.10 浮点数比较

比较下列浮点数的大小: $x^{yz}, x^{zy}, (x^y)^z, (x^z)^y, y^{xz}, y^{zx}, (y^x)^z, (y^z)^x, z^{xy}, z^{yx}, (z^x)^y$ 和 $(z^y)^x$ 。

```

1 vector<pair<ld, int>> val = {
2     {log(x) * pow(y, z), 0}, {log(x) * pow(z, y), 1}, {log(x) * y * z, 2},
3     {log(x) * z * y, 3}, {log(y) * pow(x, z), 4}, {log(y) * pow(z, x), 5},
4     {log(y) * x * z, 6}, {log(y) * z * x, 7}, {log(z) * pow(x, y), 8},
5     {log(z) * pow(y, x), 9}, {log(z) * x * y, 10}, {log(z) * y * x, 11}};
6
7 sort(val.begin(), val.end(), [&](auto x, auto y) {
8     if (equal(x.first, y.first)) return x.second < y.second; // queal比较两个浮点数是
        否相等
9     return x.first > y.first;
10 });
11 cout << ans[val.front().second] << endl;

```

4.3.11 物品装箱

有 N 个物品，第 i 个物品为 $a[i]$ ，有无限个容量为 C 的空箱子。两种装箱方式，输出需要多少个箱子才能装完所有物品。

从前往后装（线段树解）

选择最前面的能放下物品的箱子放入物品。

选择最优的箱子装（multiset 解）

选择能放下物品且剩余容量最小的箱子放物品

```

1  const int N = 1e6 + 10;
2  int T, n, a[N], c, tr[N << 2];
3  void pushup(int u){
4      tr[u] = max(tr[u << 1], tr[u << 1 | 1]);
5  }
6  void build(int u, int l, int r){
7      if (l == r) tr[u] = c;
8      else {
9          int mid = l + r >> 1;
10         build(u << 1, l, mid);
11         build(u << 1 | 1, mid + 1, r);
12         pushup(u);
13     }
14 }
15 void update(int u, int l, int r, int p, int k){
16     if (l > p || r < p) return;
17     if (l == r) tr[u] -= k;
18     else {
19         int mid = l + r >> 1;
20         update(u << 1, l, mid, p, k);
21         update(u << 1 | 1, mid + 1, r, p, k);
22         pushup(u);
23     }
24 }
25 int query(int u, int l, int r, int k){
26     if (l == r){
27         if (tr[u] >= k) return l;
28         return n + 1;
29     }
30     int mid = l + r >> 1;
31     if (tr[u << 1] >= k) return query(u << 1, l, mid, k);
32     else return query(u << 1 | 1, mid + 1, r, k);
33 }
34 int main() {
35     cin >> n >> c;
36     for (int i = 1; i <= n; i++) cin >> a[i];
37     build(1, 1, n);
38     for (int i = 1; i <= n; i++)
39         update(1, 1, n, query(1, 1, n, a[i]), a[i]);
40     cout << query(1, 1, n, c) - 1 << " ";
41 }
42
43 void solve(){

```

```

44     cin >> n >> c;
45     for (int i = 1; i <= n; i++) cin >> a[i];
46     multiset<int> s;
47     for (int i = 1; i <= n; i++){
48         auto it = s.lower_bound(a[i]);
49         if (it == s.end()) s.insert(c - a[i]);
50         else {
51             int x = *it;
52             // multiset 可以存放重复数据，如果是删除某个值的话，会去掉多个箱子
53             // 导致答案错误，所以直接删除对应位置的元素
54             s.erase(it);
55             s.insert(x - a[i]);
56         }
57     }
58     cout << s.size() << "\n";
59 }

```

4.3.12 约瑟夫问题

n 个人编号 $0, 1, 2, \dots, n-1$ ，每次数到 k 出局，求最后剩下的人的编号。

$\mathcal{O}(N)$ 。

$\mathcal{O}(K \log N)$ ，适用于 K 较小的情况。

```

1  int jos(int n,int k){
2      int res=0;
3      repeat(i,1,n+1)res=(res+k)%i;
4      return res; // res+1, 如果编号从1开始
5  }
6
7  int jos(int n,int k){
8      if(n==1 || k==1)return n-1;
9      if(k>n)return (jos(n-1,k)+k)%n; // 线性算法
10     int res=jos(n-n/k,k)-n%k;
11     if(res<0)res+=n; // mod n
12     else res+=res/(k-1); // 还原位置
13     return res; // res+1, 如果编号从1开始
14 }

```

4.3.13 统计区间不同数字的数量（离线查询）

核心在于使用 pre 数组滚动维护每一个数字出现的最后位置，配以树状数组统计数量。由于滚动维护具有后效性，所以需要离线操作，从前往后更新。时间复杂度 $\mathcal{O}(N \log N)$ ，常数瓶颈在于 map，用手造哈希或者离散化可以优化到理想区间；同时也有莫队做法，复杂度稍劣。例题链接。

```

1  signed main() {
2      int n;
3      cin >> n;
4      vector<int> in(n + 1);
5      for (int i = 1; i <= n; i++) {

```

```

6     cin >> in[i];
7 }
8
9 int q;
10 cin >> q;
11 vector<array<int, 3>> query;
12 for (int i = 0; i < q; i++) {
13     int l, r;
14     cin >> l >> r;
15     query.push_back({r, l, i});
16 }
17 sort(query.begin(), query.end());
18
19 vector<pair<int, int>> ans;
20 map<int, int> pre;
21 int st = 1;
22 BIT bit(n);
23 for (auto [r, l, id] : query) {
24     for (int i = st; i <= r; i++, st++) {
25         if (pre.count(in[i])) { // 消除此前操作的影响
26             bit.add(pre[in[i]], -1);
27         }
28         bit.add(i, 1);
29         pre[in[i]] = i; // 更新操作
30     }
31     ans.push_back({id, bit.ask(r) - bit.ask(l - 1)});
32 }
33
34 sort(ans.begin(), ans.end());
35 for (auto [id, w] : ans) {
36     cout << w << endl;
37 }
38 }

```

4.3.14 网格路径计数

```

1  ### 网格路径计数
2
3  从  $(0, 0)$  走到  $(a, b)$ , 规定每次只能从  $(x, y)$  走到左下或者右下, 方案数记为  $f(a, b)$ 
4
5  -  $f(a, b) = \sum_{k=0}^a \binom{a+b}{2k}$  ;
6  - 若路径和直线  $y=k, k \notin [0, b]$  不能有交点, 则方案数为  $f(a, b) - f(a, 2k-b)$  ;
7  - 若路径和两条直线  $y=k_1, y=k_2$  ( $k_1 < 0 \leq b < k_2$ ) 不能有交点, 方案数记为  $g(a, b, k_1, k_2)$ , 可以使用  $\mathcal{O}(N)$  递归求解;
8  - 若路径必须碰到  $y=k_1$  但是不能碰到  $y=k_2$ , 方案数记为  $h(a, b, k_1, k_2)$ , 可以使用  $\mathcal{O}(N)$  递归求解 (递归过程中两条直线距离会越来越大)。
9
10 从  $(0, 0)$  走到  $(a, 0)$ , 规定每次只能走到左下或者右下, 且必须有**恰好一次**传送 (向下  $b$  单位), 且不能走到  $x$  轴下方, 方案数为  $\sum_{k=0}^a \binom{a+1}{\frac{a-b}{2}+k+1}$ 。

```

4.3.15 选数 (DFS 解)

从 N 个整数中任选 K 个整数相加。使用 DFS 求解。

```

1  int n, k; cin >> n >> k;
2  vector<int> in(n), now(n);
3  for (auto &it : in) { cin >> it; }
4  auto dfs = [&](auto self, int k, int bit, int idx) -> void {
5      for (int i = idx; i < n; i++) {
6          now[bit] = in[i];
7          if (bit < k - 1) { self(self, k, bit + 1, i + 1); }
8          if (bit == k - 1) {
9              int add = 0;
10             for (int j = 0; j < k; j++) {
11                 add += now[j];
12             }
13             cout << add << endl;
14         }
15     }
16 };
17 dfs(dfs, k, 0, 0);

```

4.3.16 选数 (位运算状压)

```

1  int n, k; cin >> n >> k;
2  vector<int> in(n);
3  for (auto &it : in) { cin >> it; }
4  int comb = (1 << k) - 1, U = 1 << n;
5  while (comb < U) {
6      int add = 0;
7      for (int i = 0; i < n; i++) {
8          if (1 << i & comb) {
9              add += in[i];
10         }
11     }
12     cout << add << "\n";
13
14     int x = comb & -comb;
15     int y = comb + x;
16     int z = comb & ~y;
17     comb = (z / x >> 1) | y;
18 }

```

4.3.17 阿达马矩阵 (Hadamard matrix)

构造题用，其有一些性质：将 0 看作 -1 ；1 看作 $+1$ ，整个矩阵可以构成一个 2^k 维向量组，任意两个行、列向量的点积均为 0 See。例如，在 $k = 2$ 时行向量 $\vec{2}$ 和行向量 $\vec{3}$ 的点积为 $1 \cdot 1 + (-1) \cdot 1 + 1 \cdot (-1) + (-1) \cdot (-1) = 0$ 。

构造方式：
$$\begin{bmatrix} T & T \\ T & !T \end{bmatrix}$$

```

1  int n;
2  cin >> n;
3  int N = pow(2, n);
4  vector ans(N, vector<int>(N));
5  ans[0][0] = 1;
6  for (int t = 0; t < n; t++) {
7      int m = pow(2, t);
8      for (int i = 0; i < m; i++) {
9          for (int j = m; j < 2 * m; j++) {
10             ans[i][j] = ans[i][j - m];
11         }
12     }
13     for (int i = m; i < 2 * m; i++) {
14         for (int j = 0; j < m; j++) {
15             ans[i][j] = ans[i - m][j];
16         }
17     }
18     for (int i = m; i < 2 * m; i++) {
19         for (int j = m; j < 2 * m; j++) {
20             ans[i][j] = 1 - ans[i - m][j - m];
21         }
22     }
23 }

```

4.3.18 高精度快速幂

求解 $n^k \bmod p$, 其中 $0 \leq n, k \leq 10^{1000000}$, $1 \leq p \leq 10^9$ 。容易发现 n 可以直接取模, 瓶颈在于 k See。

魔改十进制快速幂 (暴力计算)

该算法复杂度 $\mathcal{O}(\text{len}(k))$ 。

扩展欧拉定理 (欧拉降幂公式)

$$n^k \equiv \begin{cases} n^{k \bmod \varphi(p)} & \gcd(n, p) = 1 \\ n^{k \bmod \varphi(p) + \varphi(p)} & \gcd(n, p) \neq 1 \wedge k \geq \varphi(p) \\ n^k & \gcd(n, p) \neq 1 \wedge k < \varphi(p) \end{cases}$$

最终我们可以将幂降到 $\varphi(p)$ 的级别, 使得能够直接使用快速幂解题, 复杂度瓶颈在求解欧拉函数 $\mathcal{O}(\sqrt{p})$ 。

```

1  int mypow10(int n, vector<int> k, int p) {
2      int r = 1;
3      for (int i = k.size() - 1; i >= 0; i--) {
4          for (int j = 1; j <= k[i]; j++) {
5              r = r * n % p;
6          }
7          int v = 1;
8          for (int j = 0; j <= 9; j++) {
9              v = v * n % p;

```



```
10     }
11     n = v;
12 }
13 return r;
14 }
15 signed main() {
16     string n_, k_;
17     int p;
18     cin >> n_ >> k_ >> p;
19
20     int n = 0; // 转化并计算 n % p
21     for (auto it : n_) {
22         n = n * 10 + it - '0';
23         n %= p;
24     }
25     vector<int> k; // 转化 k
26     for (auto it : k_) {
27         k.push_back(it - '0');
28     }
29     cout << mypow10(n, k, p) << endl; // 暴力快速幂
30 }
31
32 int phi(int n) { //求解 phi(n)
33     int ans = n;
34     for (int i = 2; i <= n / i; i++) {
35         if (n % i == 0) {
36             ans = ans / i * (i - 1);
37             while (n % i == 0) {
38                 n /= i;
39             }
40         }
41     }
42     if (n > 1) { //特判 n 为质数的情况
43         ans = ans / n * (n - 1);
44     }
45     return ans;
46 }
47 signed main() {
48     string n_, k_;
49     int p;
50     cin >> n_ >> k_ >> p;
51
52     int n = 0; // 转化并计算 n % p
53     for (auto it : n_) {
54         n = n * 10 + it - '0';
55         n %= p;
56     }
57     int mul = phi(p), type = 0, k = 0; // 转化 k
58     for (auto it : k_) {
59         k = k * 10 + it - '0';
60         type |= (k >= mul);
61         k %= mul;
62     }
```

```

63     if (type) {
64         k += mul;
65     }
66     cout << mypow(n, k, p) << endl;
67 }

```

4.3.19 高精度进制转换

2 – 62 进制相互转换。输入格式：“转换前进制转换后进制要转换的数据”。注释：进制排序为 0-9, A-Z, a-z。

```

1  struct numpy {
2      vector<int> mp; // 将字符转化为数字
3      vector<char> mp2; // 将数字转化为字符
4      numpy() : mp(123), mp2(62) { // 0-9A-Za-z
5          for (int i = 0; i < 10; i++) mp[i + 48] = i, mp2[i] = i + 48;
6          for (int i = 10; i < 36; i++) mp[i + 55] = i, mp2[i] = i + 55;
7          for (int i = 36; i < 62; i++) mp[i + 61] = i, mp2[i] = i + 61;
8      }
9
10     // 转换前进制 转换后进制 要转换的数据
11     string solve(int a, int b, const string &s) {
12         vector<int> nums, ans;
13         for (auto c : s) {
14             nums.push_back(mp[c]);
15         }
16         reverse(nums.begin(), nums.end());
17         while (nums.size()) { // 短除法, 将整个大数一直除 b, 取余数
18             int remainder = 0;
19             for (int i = nums.size() - 1; ~i; i--) {
20                 nums[i] += remainder * a;
21                 remainder = nums[i] % b;
22                 nums[i] /= b;
23             }
24             ans.push_back(remainder); // 得到余数
25             while (nums.size() && nums.back() == 0) {
26                 nums.pop_back(); // 去掉前导 0
27             }
28         }
29         reverse(ans.begin(), ans.end());
30
31         string sh;
32         for (int i : ans) sh += mp2[i];
33         return sh;
34     }
35 };

```

4.4 字符串

4.4.1 Manacher

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  void solve() {
4      string s;
5      cin >> s;
6      int n = s.size();
7      /*以位置i 为中心的
8      最长回文串的半径长度 (半径长度 d1[i] d2[i]
9      均为从位置i 到回文串最右端位置包含的字符个数) 。*/
10     vector<int> d1(n);
11     for (int i = 0, l = 0, r = -1; i < n; i++) {
12         int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
13         while (0 <= i - k && i + k < n && s[i - k] == s[i + k]) {
14             k++;
15         }
16         d1[i] = k--;
17         if (i + k > r) {
18             l = i - k;
19             r = i + k;
20         }
21     }
22     vector<int> d2(n);
23     for (int i = 0, l = 0, r = -1; i < n; i++) {
24         int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
25         while (0 <= i - k - 1 && i + k < n && s[i - k - 1] == s[i + k]) {
26             k++;
27         }
28         d2[i] = k--;
29         if (i + k > r) {
30             l = i - k - 1;
31             r = i + k;
32         }
33     }
34 }
35 int main() {
36     solve();
37     return 0;
38 }

```

4.4.2 kmp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  vector<int> mxMatch(string s){
5      int len = s.length(), p = 0;
6      vector<int> pi(len);
7      //abababzabababa
8      //0012340123456?(5)
9      for(int i=1;i<len;i++){
10         while(p and s[i]!=s[p]) p = pi[p-1];
11         //既然s[i]!=s[p] 就让s[i]比较s[pi[p-1]] 以此往复.....

```

```

12     //因为pi[p-1]是s[p]的最大匹配长度 也就是次大匹配
13     if(s[i] == s[p]) p++;
14     pi[i] = p;
15 }
16 return pi;
17 }
18 vector<int> kmp(string text,string s){
19     int len = text.length(),ls=s.length(),p=0;
20     vector<int>pos,next;
21     next = mxMatch(s);
22     for(int i=0;i<len;i++){
23         while(p and text[i]!=s[p]) p = next[p-1];
24         if(text[i]==s[p]) p++;
25         if(p==ls){
26             pos.push_back(i-ls+1);
27             p = next[p-1];//此处找到, 那么后面继续找次大匹配
28         }
29         //这样理解更easy
30         //pi[i] = p,ok[i] = (p[i]==ls)
31     }
32     return pos;
33 }
34 void solve() {
35     string s1,s2;
36     cin >> s1 >> s2;
37     kmp(s1,s2);
38     return;
39 }

```

4.4.3 拓展 kmp/z 函数

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  /*
4  给定一个字符串,
5  求后缀s1-sn si为从最后开始截取长度为i的子串
6  公共前缀值之和
7  输入: s = "babab"
8  输出: 9
9  解释:
10 s1 == "b", 最长公共前缀是 "b", 得分为 1 。
11 s2 == "ab", 没有公共前缀, 得分为 0 。
12 s3 == "bab", 最长公共前缀为 "bab", 得分为 3 。
13 s4 == "abab", 没有公共前缀, 得分为 0 。
14 s5 == "babab", 最长公共前缀为 "babab", 得分为 5 。
15 得分和为 1 + 0 + 3 + 0 + 5 = 9 , 所以我们返回 9 。
16 */
17 int main(){
18     string s;
19     cin >> s;
20     int n = s.length();
21     long ans = n;

```

```

22     vector<int> z(n);
23     for (int i = 1, l = 0, r = 0; i < n; ++i) {
24         z[i] = max(min(z[i - 1], r - i + 1), 0);
25         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
26             l = i;
27             r = i + z[i];
28             ++z[i];
29         }
30         ans += z[i];
31     }
32     for(int i=0;i<n;i++) cout<<z[i]<<" \n"[i==n-1];
33     cout << ans;
34 }

```

4.4.4 ac 自动机

```

1  #include <iostream>
2  #include <queue>
3  #include <cstring>
4
5  using namespace std;
6
7  const int MAXN = 100005; //最大节点数
8  const int MAXM = 26; //最大字符集大小
9
10 struct Trie_Node //Trie树节点
11 {
12     int id; //节点编号
13     int nxt[MAXM]; //子节点
14     int fail; //失败指针
15     int val; //节点的输出值
16 }trie[MAXN];
17
18 int idx; //Trie树节点计数器
19
20 void trie_clear() //清空Trie树
21 {
22     memset(trie, 0, sizeof(trie));
23     idx = 0;
24 }
25
26 void trie_insert(char *s, int val) //往Trie树中插入一个字符串
27 {
28     int len = strlen(s);
29     int u = 0;
30     for(int i = 0; i < len; i++)
31     {
32         int c = s[i] - 'a';
33         if(!trie[u].nxt[c]) //新建节点
34             trie[u].nxt[c] = ++idx;
35         u = trie[u].nxt[c];
36     }

```

```

37     trie[u].val = val; //该节点的输出值为该字符串的权值
38 }
39
40 void ac_build() //构建AC自动机
41 {
42     queue<int> q;
43     for(int c = 0; c < MAXM; c++)
44     {
45         if(trie[0].nxt[c]) //所有的根节点的子节点的失败指针指向根节点
46         {
47             trie[trie[0].nxt[c]].fail = 0;
48             q.push(trie[0].nxt[c]);
49         }
50     }
51     while(!q.empty()) //BFS求出AC自动机的失败指针
52     {
53         int u = q.front();
54         q.pop();
55         for(int c = 0; c < MAXM; c++)
56         {
57             int v = trie[u].nxt[c];
58             if(v)
59             {
60                 trie[v].fail = trie[trie[u].fail].nxt[c]; //与父节点的失败指针相同分两种
                    情况
61                 if(trie[trie[v].fail].val) //如果该节点的失败指针是可接受节点，那么当前节点
                    就是可接受节点的后缀
62                     trie[v].val = trie[trie[v].fail].val; //更新该节点的输出值
63                 q.push(v);
64             }
65             else
66             {
67                 trie[u].nxt[c] = trie[trie[u].fail].nxt[c];
68             }
69         }
70     }
71 }
72
73 char s[MAXN]; //输入的文本串
74 int pos[MAXN]; //所有模式串在文本串中的位置
75 bool vis[MAXN]; //标记每个出现过的节点
76 int ans; //输出值的和
77
78 void ac_query() //在Trie树上匹配文本串
79 {
80     int u = 0;
81     int len = strlen(s);
82     for(int i = 0; i < len; i++) //从根节点开始匹配
83     {
84         int c = s[i] - 'a';
85         u = trie[u].nxt[c]; //转移到下一个节点
86         int v = u;
87         while(v) //利用失败指针更新所有可接受的节点

```

```

88     {
89         if(trie[v].val && !vis[v])
90         {
91             vis[v] = true;
92             ans += trie[v].val;
93         }
94         v = trie[v].fail;
95     }
96 }
97 }
98
99 int main()
100 {
101     int n;
102     while(cin >> n)
103     {
104         trie_clear();
105         for(int i = 1; i <= n; i++)
106         {
107             int val;
108             cin >> s >> val;
109             trie_insert(s, val);
110         }
111         ac_build();
112         cin >> s;
113         memset(vis, false, sizeof(vis));
114         ans = 0;
115         ac_query();
116         cout << ans << endl;
117     }
118     return 0;
119 }

```

4.4.5 最小表示法

```

1
2 // 返回 s 的字典序最小的循环同构串
3 // 时间复杂度 O(n), 证明见末尾
4 func smallestRepresentation(s string) string {
5     n := len(s)
6     s += s
7     // 注: 如果要返回一个和原串不同的字符串, 初始化 i=1, j=2
8     i := 0
9     for j := 1; j < n; {
10         // 暴力比较: 是 i 开头的字典序小, 还是 j 开头的字典序小?
11         // 相同就继续往后比, 至多循环 n 次 (如果循环 n 次, 说明所有字母都相同, 不用再比了)
12         k := 0
13         for k < n && s[i+k] == s[j+k] {
14             k++
15         }
16         if k >= n {
17             break

```

```

18     }
19
20     if s[i+k] < s[j+k] { // 注：如果求字典序最大，改成 >
21         // 从 i 开始比从 j 开始更小（排除 j）
22         // 此外：
23         // 从 i+1 开始比从 j+1 开始更小，所以从 j+1 开始不可能是答案，排除
24         // 从 i+2 开始比从 j+2 开始更小，所以从 j+2 开始不可能是答案，排除
25         // ...
26         // 从 i+k 开始比从 j+k 开始更小，所以从 j+k 开始不可能是答案，排除
27         // 所以下一个「可能是答案」的开始位置是 j+k+1
28         j += k + 1
29     } else {
30         // 从 j 开始比从 i 开始更小，更新 i=j（也意味着我们排除了 i）
31         // 此外：
32         // 从 j+1 开始比从 i+1 开始更小，所以从 i+1 开始不可能是答案，排除
33         // 从 j+2 开始比从 i+2 开始更小，所以从 i+2 开始不可能是答案，排除
34         // ...
35         // 从 j+k 开始比从 i+k 开始更小，所以从 i+k 开始不可能是答案，排除
36         // 所以把 j 跳到 i+k+1，不过这可能比 j+1 小，所以与 j+1 取 max
37         // 综上所述，下一个「可能是答案」的开始位置是 max(j+1, i+k+1)
38         i, j = j, max(j, i+k)+1
39     }
40     // 每次要么排除 k+1 个与 i 相关的位置（这样的位置至多 n 个），要么排除 k+1 个与 j
41     // 相关的位置（这样的位置至多 n 个）
42     // 所以上面关于 k 的循环， $\sum k \leq 2n$ ，所以二重循环的总循环次数是  $O(n)$  的
43 }
44 return s[i : i+n]
45 }

```

4.4.6 trie

```

1 // -----
2 // tire （封装）：字符前缀统计
3 // -----
4 #include <iostream>
5 #include <string>
6 #include <vector>
7
8 using namespace std;
9
10 struct Node {
11     Node* child[256] = {nullptr};
12     int cnt = 0;
13 };
14
15 class Trie {
16 public:
17     Trie() : root(new Node()) {}
18
19     void insert(const string& word) {
20         Node* node = root;
21         for (char c : word) {

```



```
22         if (node->child[c] == nullptr) {
23             node->child[c] = new Node();
24         }
25         node = node->child[c];
26         node->cnt++;
27     }
28 }
29
30 int search(const string& prefix) {
31     Node* node = root;
32     for (char c : prefix) {
33         if (node->child[c] == nullptr) {
34             return 0;
35         }
36         node = node->child[c];
37     }
38     return node->cnt;
39 }
40
41 private:
42     Node* root;
43 };
44
45 int main() {
46     int n, q;
47     cin >> n >> q;
48     Trie trie;
49
50     for (int i = 0; i < n; i++) {
51         string S;
52         cin >> S;
53         trie.insert(S);
54     }
55
56     for (int i = 0; i < q; i++) {
57         string T;
58         cin >> T;
59         int cnt = trie.search(T);
60         cout << cnt << endl;
61     }
62
63     return 0;
64 }
65
66 // -----
67 // tire (Lambda) 不同的字符串个数统计
68 // -----
69 #include <bits/stdc++.h>
70 using namespace std;
71
72 struct Node {
73     int ch[26];
74     bool e;
```

```

75     //Node(){ memset(ch, 0, sizeof ch); e = false; }
76 };
77 int main(){
78     ios::sync_with_stdio(false);
79     cin.tie(nullptr);
80
81     int n;
82     cin >> n;
83
84     vector<Node> tr;
85     //tr.reserve(4000005);
86     tr.emplace_back(); // 根节点
87     int root = 0, idx = 1; // root=根, idx=下一个节点索引
88
89     auto insertStr = [&](const string &s){
90         int p = root;
91         for(char c: s){
92             int d = c - 'a';
93             if(!tr[p].ch[d]){
94                 tr[p].ch[d] = idx++;
95                 tr.emplace_back();
96             }
97             p = tr[p].ch[d];
98         }
99         tr[p].e = true;
100     };
101
102     // 查重: true 表示新串
103     auto qry = [&](const string &s)->bool{
104         int p = root;
105         for(char c: s){
106             int d = c - 'a';
107             if(!tr[p].ch[d]) return true;
108             p = tr[p].ch[d];
109         }
110         return !tr[p].e;
111     };
112
113     long long ans = 0;
114     string s;
115     if(n > 0){
116         cin >> s;
117         insertStr(s);
118         ans = 1;
119         n--;
120     }
121     while(n--){
122         cin >> s;
123         if(qry(s)) ans++;
124         insertStr(s);
125     }
126
127     cout << ans << "\n";

```

```

128     return 0;
129 }
130
131
132 // -----
133 // 01tire2 : 最大 (两两) 异或值
134 // -----
135 struct Node {
136     int ch[2], cnt;
137     Node() { ch[0] = ch[1] = cnt = 0; }
138 };
139
140 void sol() {
141     int n;
142     cin >> n;
143
144     vector<int> a(n+1);
145     for(int i=1; i<=n; i++){cin>>a[i];}
146
147     vector<Node> tr(n * 20 + 5);
148     int root = 1, tn = 2; //tn: 「下一个可用节点编号」
149
150     auto insertVal = [&](int x){
151         int p = root;
152         tr[p].cnt++;
153         for(int i = 27; i >= 0; --i){
154             int b = (x >> i) & 1;
155             if(!tr[p].ch[b]) tr[p].ch[b] = tn++;
156             p = tr[p].ch[b];
157             tr[p].cnt++;
158         }
159     };
160
161     auto sol_max = [&](int x){
162         ll s = 0;
163         int p = root;
164         for(int i = 27; p && i >= 0; --i){
165             int jm = (x >> i) & 1;
166             int c = tr[p].ch[jm^1];
167             if(c) {
168                 s |= (1<<i);
169                 p = tr[p].ch[jm^1 ];
170             }
171             else {
172                 p = tr[p].ch[jm];
173             }
174         }
175         return s;
176     };
177
178     insertVal(a[1]);
179     ll ans = 0;
180     for(int i = 2; i <= n; ++i){

```

```

181     ans = max( ans,sol_max(a[i]) );
182     insertVal(a[i]);
183 }
184
185 cout << ans << "\n";
186 return;
187 }
188
189 // -----
190 // 01tire3 : 1.树上 (异或) 前缀 2.根据limit算异或pair<i,j>的贡献 (个数)
191 // -----
192 using namespace MATH;
193 using namespace Prime;
194 using i128 = __int128_t;
195 using vt=vector<int>;
196 using ld=long double;
197 int T;
198 const int maxn = 2e5+10;
199 //int a[maxn];
200
201 struct Node {
202     int ch[2], cnt;
203     Node() { ch[0] = ch[1] = cnt = 0; }
204 };
205
206 void sol() {
207     int n, K;
208     cin >> n >> K;
209     vector<vector<pair<int,int>>> g(n+1);
210     for(int i = 1, u, v, w; i < n; i++){
211         cin >> u >> v >> w;
212         g[u].emplace_back(v, w);
213         g[v].emplace_back(u, w);
214     }
215
216     //前缀
217     vector<int> a(n+1);
218     vector<char> vis(n+1, 0);
219     queue<int> q;
220     q.push(1);
221     vis[1] = 1;
222     while(!q.empty()){
223         int u = q.front(); q.pop();
224         for(auto e : g[u]){
225             int v = e.first, w = e.second;
226             if(!vis[v]){
227                 vis[v] = 1;
228                 a[v] = a[u] ^ w;
229                 q.push(v);
230             }
231         }
232     }
233 }

```

```

234     vector<Node> tr((ll)n * 31 + 5);
235     int root = 1, tn = 2;
236
237     auto insertVal = [&](int x){
238         int p = root;
239         tr[p].cnt++;
240         for(int i = 30; i >= 0; --i){
241             int b = (x >> i) & 1;
242             if(!tr[p].ch[b]) tr[p].ch[b] = tn++;
243             p = tr[p].ch[b];
244             tr[p].cnt++;
245         }
246     };
247
248     auto sol_less = [&](int x){
249         ll s = 0;
250         int p = root;
251         for(int i = 30; p && i >= 0; --i){
252             int xb = (x >> i) & 1;
253             if((K >> i) & 1){
254                 // 如果 K 在第 i 位为 1, 则可以把与 x 在该位相同的分支全算上
255                 int c = tr[p].ch[xb];
256                 if(c) s += tr[c].cnt;
257                 // 并转到相反分支继续深搜
258                 p = tr[p].ch[xb ^ 1];
259             } else {
260                 // 如果 K 在该位为 0, 则只能继续走相同分支
261                 p = tr[p].ch[xb];
262             }
263         }
264         return s;
265     };
266
267     insertVal(a[1]);
268     ll res = 0;
269     for(int i = 2; i <= n; ++i){
270         res += sol_less(a[i]);
271         insertVal(a[i]);
272     }
273
274     ll tot = 1LL * n * (n - 1) / 2;
275     cout << (tot - res) << "\n";
276     return;
277 }

```

4.4.7 字符串哈希

```

1     const ull B = 131; // 基数
2     string s;
3     cin >> s;
4     int n = s.size();
5

```

```

6 // 1-base 1-base 1-base ! ! !
7
8 vector<ull> h(n+1, 0), p(n+1, 1);
9 for (int i = 1; i <= n; i++) {
10     h[i] = h[i-1] * B + (unsigned char)s[i-1]; //hash值
11     p[i] = p[i-1] * B; //位权
12 }
13
14 auto getHash = [&](int l, int r) { // l, r 都是 1~n 范围
15     return h[r] - h[l-1] * p[r-l+1];
16 };
17
18
19 int l1, r1, l2, r2;
20 cin >> l1 >> r1 >> l2 >> r2;
21 cout << (getHash(l1, r1) == getHash(l2, r2) ? "Yes" : "No");

```

4.4.8 AC 自动机

定义 $|s_i|$ 是模板串的长度, $|S|$ 是文本串的长度, $|\Sigma|$ 是字符集的大小 (常数, 一般为 26), 时间复杂度为 $\mathcal{O}(\sum |s_i| + |S|)$ 。

```

1 // Trie+Kmp, 多模式串匹配
2 struct ACAutomaton {
3     static constexpr int N = 1e6 + 10;
4     int ch[N][26], fail[N], cntNodes;
5     int cnt[N];
6     ACAutomaton() {
7         cntNodes = 1;
8     }
9     void insert(string s) {
10         int u = 1;
11         for (auto c : s) {
12             int &v = ch[u][c - 'a'];
13             if (!v) v = ++cntNodes;
14             u = v;
15         }
16         cnt[u]++;
17     }
18     void build() {
19         fill(ch[0], ch[0] + 26, 1);
20         queue<int> q;
21         q.push(1);
22         while (!q.empty()) {
23             int u = q.front();
24             q.pop();
25             for (int i = 0; i < 26; i++) {
26                 int &v = ch[u][i];
27                 if (!v)
28                     v = ch[fail[u]][i];
29                 else {
30                     fail[v] = ch[fail[u]][i];

```

```

31         q.push(v);
32     }
33 }
34 }
35 }
36 LL query(string t) {
37     LL ans = 0;
38     int u = 1;
39     for (auto c : t) {
40         u = ch[u][c - 'a'];
41         for (int v = u; v && ~cnt[v]; v = fail[v]) {
42             ans += cnt[v];
43             cnt[v] = -1;
44         }
45     }
46     return ans;
47 }
48 };

```

4.4.9 Z 函数 (扩展 KMP)

获取字符串 s 和 $s[i, n-1]$ (即以 $s[i]$ 开头的后缀) 的最长公共前缀 (LCP) 的长度, 总复杂度 $\mathcal{O}(N)$ 。

```

1 vector<int> zFunction(string s) {
2     int n = s.size();
3     vector<int> z(n);
4     z[0] = n;
5     for (int i = 1, j = 1; i < n; i++) {
6         z[i] = max(0, min(j + z[j] - i, z[i - j]));
7         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
8             z[i]++;
9         }
10        if (i + z[i] > j + z[j]) {
11            j = i;
12        }
13    }
14    return z;
15 }

```

4.4.10 后缀数组 SA

以 $\mathcal{O}(N)$ 的复杂度求解。

```

1 struct SuffixArray {
2     int n;
3     vector<int> sa, rk, lc;
4     SuffixArray(const string &s) {
5         n = s.length();
6         sa.resize(n);
7         lc.resize(n - 1);
8         rk.resize(n);

```

```

9      iota(sa.begin(), sa.end(), 0);
10     sort(sa.begin(), sa.end(), [&](int a, int b) { return s[a] < s[b]; });
11     rk[sa[0]] = 0;
12     for (int i = 1; i < n; ++i) {
13         rk[sa[i]] = rk[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
14     }
15     int k = 1;
16     vector<int> tmp, cnt(n);
17     tmp.reserve(n);
18     while (rk[sa[n - 1]] < n - 1) {
19         tmp.clear();
20         for (int i = 0; i < k; ++i) {
21             tmp.push_back(n - k + i);
22         }
23         for (auto i : sa) {
24             if (i >= k) {
25                 tmp.push_back(i - k);
26             }
27         }
28         fill(cnt.begin(), cnt.end(), 0);
29         for (int i = 0; i < n; ++i) {
30             ++cnt[rk[i]];
31         }
32         for (int i = 1; i < n; ++i) {
33             cnt[i] += cnt[i - 1];
34         }
35         for (int i = n - 1; i >= 0; --i) {
36             sa[--cnt[rk[tmp[i]]]] = tmp[i];
37         }
38         swap(rk, tmp);
39         rk[sa[0]] = 0;
40         for (int i = 1; i < n; ++i) {
41             rk[sa[i]] = rk[sa[i - 1]] + (tmp[sa[i - 1]] < tmp[sa[i]] || sa[i - 1] +
42                                     k == n ||
43                                     tmp[sa[i - 1] + k] < tmp[sa[i] + k]);
44         }
45         k *= 2;
46     }
47     for (int i = 0, j = 0; i < n; ++i) {
48         if (rk[i] == 0) {
49             j = 0;
50             continue;
51         }
52         for (j -= j > 0;
53              i + j < n && sa[rk[i] - 1] + j < n && s[i + j] == s[sa[rk[i] - 1] + j];) {
54             ++j;
55         }
56         lc[rk[i] - 1] = j;
57     }
58 };

```


4.4.11 后缀自动机 SAM

定义 $|\Sigma|$ 是字符集的大小, 复杂度为 $\mathcal{O}(N \log |\Sigma|)$ 。

```

1 // 有向无环图
2 struct SuffixAutomaton {
3     static constexpr int N = 1e6;
4     struct node {
5         int len, link, nxt[26];
6         int siz;
7     } t[N << 1];
8     int cntNodes;
9     SuffixAutomaton() {
10         cntNodes = 1;
11         fill(t[0].nxt, t[0].nxt + 26, 1);
12         t[0].len = -1;
13     }
14     int extend(int p, int c) {
15         if (t[p].nxt[c]) {
16             int q = t[p].nxt[c];
17             if (t[q].len == t[p].len + 1) {
18                 return q;
19             }
20             int r = ++cntNodes;
21             t[r].siz = 0;
22             t[r].len = t[p].len + 1;
23             t[r].link = t[q].link;
24             copy(t[q].nxt, t[q].nxt + 26, t[r].nxt);
25             t[q].link = r;
26             while (t[p].nxt[c] == q) {
27                 t[p].nxt[c] = r;
28                 p = t[p].link;
29             }
30             return r;
31         }
32         int cur = ++cntNodes;
33         t[cur].len = t[p].len + 1;
34         t[cur].siz = 1;
35         while (!t[p].nxt[c]) {
36             t[p].nxt[c] = cur;
37             p = t[p].link;
38         }
39         t[cur].link = extend(p, c);
40         return cur;
41     }
42 };

```

4.4.12 回文自动机 PAM (回文树)

应用: 1. 本质不同的回文串个数: $idx - 2$; 2. 回文子串出现次数。

对于一个字符串 s , 它的本质不同回文子串个数最多只有 $|s|$ 个, 那么, 构造 s 的回文树的时间复杂度是 $\mathcal{O}(|s|)$ 。

```

1 struct PalindromeAutomaton {
2     constexpr static int N = 5e5 + 10;
3     int tr[N][26], fail[N], len[N];
4     int cntNodes, last;
5     int cnt[N];
6     string s;
7     PalindromeAutomaton(string s) {
8         memset(tr, 0, sizeof tr);
9         memset(fail, 0, sizeof fail);
10        len[0] = 0, fail[0] = 1;
11        len[1] = -1, fail[1] = 0;
12        cntNodes = 1;
13        last = 0;
14        this->s = s;
15    }
16    void insert(char c, int i) {
17        int u = get_fail(last, i);
18        if (!tr[u][c - 'a']) {
19            int v = ++cntNodes;
20            fail[v] = tr[get_fail(fail[u], i)][c - 'a'];
21            tr[u][c - 'a'] = v;
22            len[v] = len[u] + 2;
23            cnt[v] = cnt[fail[v]] + 1;
24        }
25        last = tr[u][c - 'a'];
26    }
27    int get_fail(int u, int i) {
28        while (i - len[u] - 1 <= -1 || s[i - len[u] - 1] != s[i]) {
29            u = fail[u];
30        }
31        return u;
32    }
33 };

```

4.4.13 子串与子序列

```

1 ### 子串与子序列
2
3 |中文名称|常见英文名称|解释|
4 |:--:|:--:|:--:|
5 |子串|$\\tt substring$|连续的选择一段字符（可以全选、可以不选）组成的新字符串|
6 |子序列|$\\tt subsequence$|从左到右取出若干个字符（可以不取、可以全取、可以不连续）组成的新字符串|

```

4.4.14 子序列自动机

对于给定的长度为 n 的主串 s ，以 $\mathcal{O}(n)$ 的时间复杂度预处理、 $\mathcal{O}(m + \log \text{size}s)$ 的复杂度判定长度为 m 的询问串是否是主串的子序列。

自动离散化、自动类型匹配封装

朴素封装

原时间复杂度中的 $\text{size}:s$ 需要手动设置。类型需要手动设置。

博弈论

```

1  template<class T> struct SequenceAutomaton {
2      vector<T> alls;
3      vector<vector<int>>> ver;
4
5      SequenceAutomaton(auto in) {
6          for (auto &i : in) {
7              alls.push_back(i);
8          }
9          sort(alls.begin(), alls.end());
10         alls.erase(unique(alls.begin(), alls.end()), alls.end());
11
12         ver.resize(alls.size() + 1);
13         for (int i = 0; i < in.size(); i++) {
14             ver[get(in[i])].push_back(i + 1);
15         }
16     }
17     bool count(T x) {
18         return binary_search(alls.begin(), alls.end(), x);
19     }
20     int get(T x) {
21         return lower_bound(alls.begin(), alls.end(), x) - alls.begin();
22     }
23     bool contains(auto in) {
24         int at = 0;
25         for (auto &i : in) {
26             if (!count(i)) {
27                 return false;
28             }
29
30             auto j = get(i);
31             auto it = lower_bound(ver[j].begin(), ver[j].end(), at + 1);
32             if (it == ver[j].end()) {
33                 return false;
34             }
35             at = *it;
36         }
37         return true;
38     }
39 };
40
41 struct SequenceAutomaton {
42     vector<vector<int>>> ver;
43
44     SequenceAutomaton(vector<int> &in, int size) : ver(size + 1) {
45         for (int i = 0; i < in.size(); i++) {
46             ver[in[i]].push_back(i + 1);
47         }
48     }
49     bool contains(vector<int> &in) {
50         int at = 0;

```

```

51     for (auto &i : in) {
52         auto it = lower_bound(ver[i].begin(), ver[i].end(), at + 1);
53         if (it == ver[i].end()) {
54             return false;
55         }
56         at = *it;
57     }
58     return true;
59 }
60 };

```

4.4.15 字典树 trie

基础封装

01 字典树

```

1  struct Trie {
2      int ch[N][63], cnt[N], idx = 0;
3      map<char, int> mp;
4      void init() {
5          LL id = 0;
6          for (char c = 'a'; c <= 'z'; c++) mp[c] = ++id;
7          for (char c = 'A'; c <= 'Z'; c++) mp[c] = ++id;
8          for (char c = '0'; c <= '9'; c++) mp[c] = ++id;
9      }
10     void insert(string s) {
11         int u = 0;
12         for (int i = 0; i < s.size(); i++) {
13             int v = mp[s[i]];
14             if (!ch[u][v]) ch[u][v] = ++idx;
15             u = ch[u][v];
16             cnt[u]++;
17         }
18     }
19     LL query(string s) {
20         int u = 0;
21         for (int i = 0; i < s.size(); i++) {
22             int v = mp[s[i]];
23             if (!ch[u][v]) return 0;
24             u = ch[u][v];
25         }
26         return cnt[u];
27     }
28     void Clear() {
29         for (int i = 0; i <= idx; i++) {
30             cnt[i] = 0;
31             for (int j = 0; j <= 62; j++) {
32                 ch[i][j] = 0;
33             }
34         }
35         idx = 0;
36     }

```

```

37 } trie;
38
39 struct Trie {
40     int n, idx;
41     vector<vector<int>> ch;
42     Trie(int n) {
43         this->n = n;
44         idx = 0;
45         ch.resize(30 * (n + 1), vector<int>(2));
46     }
47     void insert(int x) {
48         int u = 0;
49         for (int i = 30; ~i; i--) {
50             int &v = ch[u][x >> i & 1];
51             if (!v) v = ++idx;
52             u = v;
53         }
54     }
55     int query(int x) {
56         int u = 0, res = 0;
57         for (int i = 30; ~i; i--) {
58             int v = x >> i & 1;
59             if (ch[u][!v]) {
60                 res += (1 << i);
61                 u = ch[u][!v];
62             } else {
63                 u = ch[u][v];
64             }
65         }
66         return res;
67     }
68 };

```

4.4.16 字符串哈希

双哈希封装

随机质数列表：1111111121、1211111123、1311111119。

前后缀去重

sample please ease 去重后得到 samplease。

```

1  const int N = 1 << 21;
2  static const int mod1 = 1E9 + 7, base1 = 127;
3  static const int mod2 = 1E9 + 9, base2 = 131;
4  using U = Zmod<mod1>;
5  using V = Zmod<mod2>;
6  vector<U> val1;
7  vector<V> val2;
8  void init(int n = N) {
9      val1.resize(n + 1), val2.resize(n + 2);
10     val1[0] = 1, val2[0] = 1;
11     for (int i = 1; i <= n; i++) {

```

```

12     val1[i] = val1[i - 1] * base1;
13     val2[i] = val2[i - 1] * base2;
14 }
15 }
16 struct String {
17     vector<U> hash1;
18     vector<V> hash2;
19     string s;
20
21     String(string s_) : s(s_), hash1{1}, hash2{1} {
22         for (auto it : s) {
23             hash1.push_back(hash1.back() * base1 + it);
24             hash2.push_back(hash2.back() * base2 + it);
25         }
26     }
27     pair<U, V> get() { // 输出整串的哈希值
28         return {hash1.back(), hash2.back()};
29     }
30     pair<U, V> substring(int l, int r) { // 输出子串的哈希值
31         if (l > r) swap(l, r);
32         U ans1 = hash1[r + 1] - hash1[l] * val1[r - l + 1];
33         V ans2 = hash2[r + 1] - hash2[l] * val2[r - l + 1];
34         return {ans1, ans2};
35     }
36     pair<U, V> modify(int idx, char x) { // 修改 idx 位为 x
37         int n = s.size() - 1;
38         U ans1 = hash1.back() + val1[n - idx] * (x - s[idx]);
39         V ans2 = hash2.back() + val2[n - idx] * (x - s[idx]);
40         return {ans1, ans2};
41     }
42 };
43
44 string compress(vector<string> in) { // 前后缀压缩
45     vector<U> hash1{1};
46     vector<V> hash2{1};
47     string ans = "#";
48     for (auto s : in) {
49         s = "#" + s;
50         int st = 0;
51         U chk1 = 0;
52         V chk2 = 0;
53         for (int j = 1; j < s.size() && j < ans.size(); j++) {
54             chk1 = chk1 * base1 + s[j];
55             chk2 = chk2 * base2 + s[j];
56             if ((hash1.back() == hash1[ans.size() - 1 - j] * val1[j] + chk1) &&
57                 (hash2.back() == hash2[ans.size() - 1 - j] * val2[j] + chk2)) {
58                 st = j;
59             }
60         }
61         for (int j = st + 1; j < s.size(); j++) {
62             ans += s[j];
63             hash1.push_back(hash1.back() * base1 + s[j]);
64             hash2.push_back(hash2.back() * base2 + s[j]);

```

```

65     }
66 }
67 return ans.substr(1);
68 }

```

4.4.17 字符串模式匹配算法 KMP

应用：1. 在字符串中查找子串；2. 最小周期：字符串长度-整个字符串的 **border** ；
3. 最小循环节：区别于周期，当字符串长度 $n \bmod (n - \text{next}[n]) = 0$ 时，等于最小周期，否则为 n 。

以最坏 $\mathcal{O}(N + M)$ 的时间计算 t 在 s 中出现的全部位置。

```

1 auto kmp = [&](string s, string t) {
2     int n = s.size(), m = t.size();
3     vector<int> kmp(m + 1), ans;
4     s = "@" + s;
5     t = "@" + t;
6     for (int i = 2, j = 0; i <= m; i++) {
7         while (j && t[i] != t[j + 1]) {
8             j = kmp[j];
9         }
10        j += t[i] == t[j + 1];
11        kmp[i] = j;
12    }
13    for (int i = 1, j = 0; i <= n; i++) {
14        while (j && s[i] != t[j + 1]) {
15            j = kmp[j];
16        }
17        if (s[i] == t[j + 1] && ++j == m) {
18            ans.push_back(i - m + 1); // t 在 s 中出现的位置
19        }
20    }
21    return ans;
22 };

```

4.4.18 最长公共子序列 LCS

求解两个串的最长公共子序列的长度。

小数据解

针对 10^3 以内的数据。

大数据解

针对 10^5 以内的数据。

```

1 const int N = 1e3 + 10;
2 char a[N], b[N];
3 int n, m, f[N][N];
4 void solve(){
5     cin >> n >> m >> a + 1 >> b + 1;
6     for (int i = 1; i <= n; i++)

```

```

7     for (int j = 1; j <= m; j++){
8         f[i][j] = max(f[i - 1][j], f[i][j - 1]);
9         if (a[i] == b[j]) f[i][j] = max(f[i][j], f[i - 1][j - 1] + 1);
10    }
11    cout << f[n][m] << "\n";
12 }
13 int main(){
14     solve();
15     return 0;
16 }
17
18 const int INF = 0x7fffffff;
19 int n, a[maxn], b[maxn], f[maxn], p[maxn];
20 int main(){
21     cin >> n;
22     for (int i = 1; i <= n; i++){
23         scanf("%d", &a[i]);
24         p[a[i]] = i; //将第二个序列中的元素映射到第一个中
25     }
26     for (int i = 1; i <= n; i++){
27         scanf("%d", &b[i]);
28         f[i] = INF;
29     }
30     int len = 0;
31     f[0] = 0;
32     for (int i = 1; i <= n; i++){
33         if (p[b[i]] > f[len]) f[++len] = p[b[i]];
34         else {
35             int l = 0, r = len;
36             while (l < r){
37                 int mid = (l + r) >> 1;
38                 if (f[mid] > p[b[i]]) r = mid;
39                 else l = mid + 1;
40             }
41             f[l] = min(f[l], p[b[i]]);
42         }
43     }
44     cout << len << "\n";
45     return 0;
46 }

```

4.4.19 马拉车

$\mathcal{O}(N)$ 时间求出字符串的最长回文子串。

```

1 string s;
2 cin >> s;
3 int n = s.length();
4 string t = "-#";
5 for (int i = 0; i < n; i++) {
6     t += s[i];
7     t += '#';
8 }

```



```

9  int m = t.length();
10 t += '+';
11 int mid = 0, r = 0;
12 vector<int> p(m);
13 for (int i = 1; i < m; i++) {
14     p[i] = i < r ? min(p[2 * mid - i], r - i) : 1;
15     while (t[i - p[i]] == t[i + p[i]]) p[i]++;
16     if (i + p[i] > r) {
17         r = i + p[i];
18         mid = i;
19     }
20 }

```

4.5 计算几何

4.5.1 计算几何

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  // 不要define double long double
4  // 如果有精度需要可以使用ld
5  using ld = long double;
6  //常用函数:fabsl(a),cosl(a).....
7  //即在末尾加上了字母l
8  const double eps = 1e-8; // 根据题目精度要求进行修改
9  const double PI = acos(-1.0); // pai, 3.1415926....
10 int sgn(double x) {
11     // 符号函数 返回1代表在精度范围内可认为是正数, 0代表为0, -1代表负数
12     return x < -eps ? -1 : x > eps;
13 }
14 /*
15     点与向量 一结构体两用
16     Point(x, y) 代表点
17     Vector(x, y) 代表向量
18 */
19 typedef struct Point {
20     double x, y;
21     Point(double x = 0, double y = 0) : x(x), y(y) {} // 构造函数, 初始值为 0
22     // 重载运算符// 点 - 点 = 向量(向量AB = B - A)
23     Point operator- (const Point &B) const { return Point(x - B.x, y - B.y); }
24     // 点 + 点 = 点, 点 + 向量 = 向量, 向量 + 向量 = 向量
25     Point operator+ (const Point &B) const { return Point(x + B.x, y + B.y); }
26     // 向量 × 向量 (叉积)
27     double operator^ (const Point &B) const { return x * B.y - y * B.x; }
28     // 向量 · 向量 (点积) Dot
29     double operator* (const Point &B) const { return x * B.x + y * B.y; }
30     // 点 * 数 = 点, 向量 * 数 = 向量
31     Point operator* (const double &B) const { return Point(x * B, y * B); }
32     // 点 / 数 = 点, 向量 / 数 = 向量
33     Point operator/ (const double &B) const { return Point(x / B, y / B); }
34     // 判断大小, 一般用于排序
35     bool operator< (const Point &B) const { return x < B.x || (x == B.x && y < B.y); }

```

```

36 // 判断相等, 点 == 点, 向量 == 向量, 一般用于判断和去重
37 bool operator==(const Point &B) const { return sgn(x - B.x) == 0 && sgn(y - B.y) == 0; }
38 // 判断不相等, 点 != 点, 向量 != 向量
39 bool operator!=(const Point &B) const { return sgn(x - B.x) || sgn(y - B.y); }
40 } Vector;
41 /*
42 夹角与点积的关系
43 1. 若夹角  $\theta = 0^\circ$ , 点积 ( $\text{vec}_a \cdot \text{vec}_b$ ) 等于  $|\text{vec}_a| \times |\text{vec}_b|$ 
44 2. 若  $\theta = 180^\circ$ , 点积 ( $\text{vec}_a \cdot \text{vec}_b$ ) 等于  $-|\text{vec}_a| \times |\text{vec}_b|$ 
45 3. 若  $\theta < 90^\circ$ , 点积 ( $\text{vec}_a \cdot \text{vec}_b$ )  $> 0$ 
46 4. 若  $\theta = 90^\circ$ , 点积 ( $\text{vec}_a \cdot \text{vec}_b$ )  $= 0$ 
47 5. 若  $\theta > 90^\circ$ , 点积 ( $\text{vec}_a \cdot \text{vec}_b$ )  $< 0$ 
48 */
49 // 两点距离 Need: (-, *)
50 double dist(Point a, Point b) { return sqrtl((a - b) * (a - b)); }
51 // 向量的模
52 double len(Vector A) { return sqrt(A * A); }
53 // 单位向量
54 Vector norm(Vector A) { return A / len(A); }
55 // 两向量夹角
56 double Angle(Vector A, Vector B) {
57     double t = acos((A * B) / len(A) / len(B));
58     return t; // 返回  $[0, \pi]$  弧度制
59     return t * (180 / PI); // 返回  $[0, 180]$  角度制
60 }
61 /*
62 -----
63 对于几乎所有的计算几何题目, 我们需要先把上面这二十多行(不含注释)
64 先打一遍 再根据需要抄下面的板子
65 -----
66 */
67 // 判断点与直线的关系
68 // 点在直线上, 返回 0 (三点共线)
69 // 点在直线的逆时针方向, 返回 1
70 // 点在直线的顺时针方向, 返回 -1
71 // 点 a, b (向量ab) 所在的直线和点 c
72 // 使用的时候要注意 a 和 b 的顺序, 有时顺序不同, 结果不同
73 int Cross(Point a, Point b, Point c) { return sgn((b - a) ^ (c - a)); }
74
75 // 叉乘的值 但是注意和上面这个重名 需要注释掉其中一个 建议使用 ^
76 //double Cross(Point a, Point b, Point c) { return (b - a) ^ (c - a); }
77
78 // 逆时针旋转向量
79 // 向量 A 和要逆时针转的角度  $[0, \pi]$  弧度制
80 Vector Rotate(Vector A, double b) {
81     Vector B(sin(b), cos(b));
82     return Vector(A ^ B, A * B);
83 }
84
85 /*
86 线: 实际上是由两点确定
87 # 直线表达式

```

```

88     一般式: (  $ax + by + c = 0$  )
89     点向式: 直线上一点 (x0, y0)和方向向量 (u, v) ->  $(x-x0)/u = (y-y0)/v$ 
90     斜截式: (  $y = kx + b$  )
91 */
92 struct Line {
93     Point s, e;
94     Line() {}
95     Line(Point x, Point y):s(x), e(y) {}
96 };
97 /*
98     由于线的实现和也可以用两个Point,
99     所以代码中一般不用Line类型,
100     下面的函数也是使用两个点模拟直线/线段
101 */
102 // 判断三点共线
103 // 向量叉乘 -> 两个向量构成的平行四边形面积 -> 面积为0 -> 共线
104 bool In_one_line(Point A, Point B, Point C) { return !sgn((B - A) ^ (C - B)); }
105 // 点到直线距离 P -> 直线AB
106 double Dist_point_to_line(Point P, Point A, Point B) {
107     Vector v1 = B - A, v2 = P - A;
108     return fabs((v1 ^ v2) / len(v1));
109 }
110 // 点到线段距离 P -> 线段AB
111 double Dist_point_to_seg(Point P, Point A, Point B) {
112     if (A == B) return len(P - A); // 如果重合, 那么就是两点的距离
113     Vector v1 = B - A, v2 = P - A, v3 = P - B;
114     if (sgn(v1 * v2) < 0) return len(v2); // AP 最短
115     if (sgn(v1 * v3) > 0) return len(v3); // BP 最短
116     return fabs((v1 ^ v2) / len(v1)); // 垂线
117 }
118 //判断点是否在线段上 P -> 线段AB
119 bool OnSegment(Point P, Point A, Point B) {
120     Vector PA = A - P, PB = B - P;
121     return sgn(PA ^ PB) == 0 && sgn(PA * PB) <= 0;
122     // <=, 包括端点; <, 不包括端点
123 }
124 // 判断直线与线段是否相交
125 // 直线 ab 与线段 cd
126 bool Intersect_line_seg(Point a, Point b, Point c, Point d) {
127     return Cross(a, b, c) * Cross(a, b, d) <= 0;
128 }
129 // 判断线段与线段是否相交
130 // 线段 ab 与线段 cd
131 bool Intersect_seg(Point a, Point b, Point c, Point d) {
132     if (OnSegment(a, c, d) || OnSegment(b, c, d) ||
133         OnSegment(c, a, b) || OnSegment(d, a, b)) return 1;
134     if (Cross(a, b, c) * Cross(a, b, d) >= 0) return 0;
135     if (Cross(c, d, a) * Cross(c, d, b) >= 0) return 0;
136     return 1;
137 }
138 // 判断两直线是否平行(包括重合)
139 // 返回true: 平行/重合, false: 相交
140 bool Line_parallel(Line A, Line B) { return sgn((A.s - A.e) ^ (B.s - B.e)) == 0; }

```

```

141 // 求两直线的交点
142 // 首先要判断两直线是否相交，即不平行(不重合)
143 // a, b 所在直线与 c, d 所在直线的交点
144 Point Intersection_line(Point a, Point b, Point c, Point d) {
145     Vector u = b - a, v = d - c;
146     double t = ((a - c) ^ v) / (v ^ u);
147     return a + u * t;
148 }
149 /*
150     多边形有关函数
151 */
152 // 三角形面积 海伦公式
153 double Triangle_area(Point A, Point B, Point C) {
154     double a = len(A - B), b = len(A - C), c = len(B - C);
155     double p = (a + b + c) / 2;
156     return sqrt(p * (p - a) * (p - b) * (p - c));
157 } // 直接用平行四边形面积除以2计算
158 double Triangle_area2(Point A, Point B, Point C) {
159     return fabs((B - A) ^ (C - A)) / 2;
160 }
161 /*
162     初高中三角形知识
163
164     三角形四心
165     外心：三边中垂线的交点，到三角形三个顶点的距离相同（外接圆圆心）
166     内心：角平分线的交点，到三角形三边的距离相同（内切圆圆心）
167     垂心：三条垂线的交点
168     重心：三条中线的交点，到三角形三顶点距离的平方和最小的点，
169         三角形内到三边距离之积最大的点。
170         三角形重心是三点各坐标的平均值  $((x_1+x_2+x_3)/3, (y_1+y_2+y_3)/3)$ 。
171
172     正弦定理 & 余弦定理
173     正：
174     在三角形ABC中，若角A,B,C所对应的边为a,b,c，则有
175      $a/\sin A = b/\sin B = c/\sin C = 2R$ 
176     其中，R为ABC的外接圆半径。
177     余：
178     在三角形ABC中，若角A,B,C所对应的边为a,b,c，则有
179      $a^2 = b^2 + c^2 - 2bc \cos A$ 
180      $b^2 = a^2 + c^2 - 2ac \cos A$ 
181      $c^2 = a^2 + b^2 - 2ab \cos A$ 
182 */
183 // 求多边形面积
184 // 因为叉积求得的三角形面积是有向的，在外面的面积可以正负抵消掉
185 // 所以能够求任意多边形面积(凸和凹都可以)
186 // p[]下标从 0 开始，长度为 n 如果使用vector可以省略一个参数
187 double Polygon_area(Point *p, int n) {
188     double area = 0;
189     for (int i = 1; i <= n - 2; i++)
190         area += (p[i] - p[0]) ^ (p[i + 1] - p[0]);
191     return fabs(area / 2); // 无向面积
192     return area / 2; // 有向面积
193 } // 鞋带定理，代码区别不大

```

```

194 double Polygon_area2(Point *p, int n) {
195     double area = 0;
196     for (int i = 0, j = n - 1; i < n; j = i++)
197         area += (p[j] ^ p[i]);
198     return fabs(area / 2); // 无向面积
199     return area / 2; // 有向面积
200 }
201 // 判断点和多边形关系 射线法
202 // 适用于任意多边形, 不用考虑精度误差和多边形的给出顺序
203 // 点在多边形边上, 返回 -1
204 // 点在多边形内, 返回 1
205 // 点在多边形外, 返回 0
206
207 // p[] 的下标从 0 开始, 长度为 n
208 int InPolygon(Point P, Point *p, int n) {
209     bool flag = false; // 相当于计数
210     for (int i = 0, j = n - 1; i < n; j = i++) {
211         Point p1 = p[i], p2 = p[j];
212         if (OnSegment(P, p1, p2)) return -1;
213         if (sgn(P.y - p1.y) > 0 == sgn(P.y - p2.y) > 0) continue;
214         if (sgn((P.y - p1.y) * (p1.x - p2.x) / (p1.y - p2.y) + p1.x - P.x) > 0)
215             flag = !flag;
216     }
217     return flag;
218 }
219 // 判断是否凸包
220 // 顶点必须按顺时针(或逆时针)给出, 允许共线边
221 // p[] 下标从 0 开始, 长度为 n
222 bool Is_context(Point *p, int n) {
223     bool s[3] = {0, 0, 0};
224     for (int i = 0, j = n - 1, k = n - 2; i < n; k = j, j = i++) {
225         int cnt = sgn((p[i] - p[j]) ^ (p[k] - p[j])) + 1;
226         s[cnt] = true;
227         if (s[0] && s[2]) return false;
228     }
229     return true;
230 }
231
232 /*
233     圆
234     由一个点和一个半径构成
235 */
236 struct Circle {
237     Point o;
238     double r;
239     Circle(Point _o = Point(), double _r = 0) : o(_o), r(_r) {}
240
241     // 圆的面积
242     double Circle_S() { return PI * r * r; }
243     // 圆的周长
244     double circle_C() { return 2 * PI * r; }
245
246     bool operator<(const Circle &B) const { return o < B.o || o == B.o && r < B.r; }

```

```

247     bool operator==(const Circle &B) const { return o == B.o && !sgn(r - B.r); }
248 };
249 // 求扇形面积
250 // 扇形的两交点A, B 和圆的半径 R
251 double SectorArea(Point A, Point B, double R) {
252     double angle = Angle(A, B);
253     if (sgn(A ^ B) < 0) angle = -angle;
254     return R * R * angle / 2;
255 }
256 // 判断点与圆的位置关系
257 // 点在圆上, 返回 0
258 // 点在圆外, 返回 -1
259 // 点在圆内, 返回 1
260 int Point_with_circle(Point p, Circle c) {
261     double d = dist(p, c.o);
262     if (sgn(d - c.r) == 0) return 0;
263     if (sgn(d - c.r) > 0) return -1;
264     return 1;
265 }
266 // 判断直线与圆的位置关系
267 // 相切, 返回 0
268 // 相交, 返回 1
269 // 相离, 返回 -1
270 int Line_with_circle(Point A, Point B, Circle c) {
271     double d = Dist_point_to_line(c.o, A, B);
272     if (sgn(d - c.r) == 0) return 0;
273     if (sgn(d - c.r) > 0) return -1;
274     return 1;
275 }
276 // 求圆与直线的交点
277 // 直线与圆相交, 返回两点
278 // 直线与圆相切, 返回两个一样的相切点
279 pair<Point, Point> Intersection_line_circle(Point A, Point B, Circle c) {
280     Vector AB = B - A;
281     Vector pr = A + AB * ((c.o - A) * AB / (AB * AB));
282     double base = sqrt(c.r * c.r - ((pr - c.o) * (pr - c.o)));
283
284     if (sgn(base) == 0) return make_pair(pr, pr);
285
286     Vector e = AB / sqrt(AB * AB);
287     return make_pair(pr + e * base, pr - e * base);
288 }
289 // 判断圆与圆的位置关系
290 // 相离, 返回 -1
291 // 外切, 返回 0
292 // 内切(A 包含 B), 返回 1
293 // 内切(B 包含 A), 返回 2
294 // 内含(A 包含 B), 返回 3
295 // 内含(B 包含 A), 返回 4
296 // 相交, 返回 5
297 int Circle_with_circle(Circle A, Circle B) {
298     double len1 = dist(A.o, B.o);
299     double len2 = A.r + B.r;

```

```

300     if (sgn(len1 - len2) > 0) return -1;
301     if (sgn(len1 - len2) == 0) return 0;
302     if (sgn(len1 + len2 - 2 * A.r) == 0) return 1;
303     if (sgn(len1 + len2 - 2 * B.r) == 0) return 2;
304     if (sgn(len1 + len2 - 2 * A.r) < 0) return 3;
305     if (sgn(len1 + len2 - 2 * B.r) < 0) return 4;
306     return 5;
307 }
308 // 求圆与圆的交点
309 // 相交, 返回两点坐标
310 // 相切, 返回两个一样的相切点
311 // 要先判断是否相交或相切再调用
312 pair<Point, Point> Intersection_circle_circle(Circle A, Circle B) {
313     Vector AB = B.o - A.o;
314     double d = len(AB);
315     double a = acos((A.r * A.r + d * d - B.r * B.r) / (2.0 * A.r * d));
316     double t = atan2(AB.y, AB.x);
317     Vector x(A.r * cos(t + a), A.r * sin(t + a));
318     Vector y(A.r * cos(t - a), A.r * sin(t - a));
319     return make_pair(A.o + x, A.o + y);
320 }
321 // 求圆外一点到圆的两条切线的切点
322 // 返回两个切点坐标
323 // 一定要先判断点在圆外, 然后再调用
324 pair<Point, Point> TangentPoint_point_circle(Point p, Circle c) {
325     double d = dist(p, c.o);
326     double l = sqrt(d * d - c.r * c.r);
327     Vector e = (c.o - p) / d;
328     double angle = asin(c.r / d);
329
330     Vector t1(sin(angle), cos(angle));
331     Vector t2(sin(-angle), cos(-angle));
332     Vector e1(e ^ t1, e * t1);
333     Vector e2(e ^ t2, e * t2);
334     e1 = e1 * l + p;
335     e2 = e2 * l + p;
336     return make_pair(e1, e2);
337 }
338
339 // 求三角形外接圆
340 Circle get_circumcircle(Point A, Point B, Point C) {
341     double Bx = B.x - A.x, By = B.y - A.y;
342     double Cx = C.x - A.x, Cy = C.y - A.y;
343     double D = 2 * (Bx * Cy - By * Cx);
344
345     double x = (Cy * (Bx * Bx + By * By) - By * (Cx * Cx + Cy * Cy)) / D + A.x;
346     double y = (Bx * (Cx * Cx + Cy * Cy) - Cx * (Bx * Bx + By * By)) / D + A.y;
347     Point P(x, y);
348     return Circle(P, dist(A, P));
349 }
350 // 求三角形内切圆
351 Circle get_incircle(Point A, Point B, Point C) {
352     double a = dist(B, C);

```

```

353     double b = dist(A, C);
354     double c = dist(A, B);
355     Point p = (A * a + B * b + C * c) / (a + b + c);
356     return Circle(p, Dist_point_to_line(p, A, B));
357 }
358
359 // -----网格计数类问题-----
360
361 // 求线段上整点个数
362 // 要保证传入的点是整点
363 int IntegerPoint_on_seg(Point A, Point B) {
364     int x = abs(A.x - B.x);
365     int y = abs(A.y - B.y);
366     if (x == 0 || y == 0) return 1;
367     return __gcd(x, y) + 1; // 包含端点
368     return __gcd(x, y) - 1; // 不包含端点
369 }
370
371 // 求多边形边上整点个数!!!是边上的点
372 // 点需要是顺时针(逆时针)给出
373 // p[] 下标从 0 开始, 长度为 n
374 // 要保证传入的点是整点
375 int IntegerPoint_on_polygon(Point *p, int n) {
376     int res = 0;
377     for (int i = 0, j = n - 1; i < n; j = i++) {
378         int x = abs(p[i].x - p[j].x);
379         int y = abs(p[i].y - p[j].y);
380         res += __gcd(x, y);
381     }
382     return res;
383 }
384 // 多边形内整点个数
385 // 返回不包括边界的, 多边形内整点个数
386 int IntegerPoint_in_polygon(Point *p, int n) {
387     double A = Polygon_area(p, n);
388     double B = IntegerPoint_on_polygon(p, n);
389     return A - B / 2 + 1;
390 }
391 // 极角排序, 按照逆时针排序
392 // 排序常数大, 但精度高
393 Point o(0, 0); // 极点自定义
394 // 获取象限 (0, 1, 2, 3)
395 int Quadrant(Vector p) { return sgn(p.y < 0) << 1 | sgn(p.x < 0) ^ sgn(p.y < 0); }
396 // 比较函数
397 bool cmp(Point a, Point b) {
398     Vector p = a - o, q = b - o; // 相对位置
399     int x = Quadrant(p), y = Quadrant(q);
400     if (x == y) {
401         if (sgn(p ^ q) == 0) return len(p) < len(q);
402         return sgn(p ^ q) > 0;
403     }
404     return x < y;
405 }

```



```

406
407 /*
408 -----
409 凸包 Andrew算法 O(nlogn)
410 给定平面上的点集, 凸包是最外层的点
411 连接起来构成的凸多边形, 它能包含点集中的所有点。
412 -----
413 */
414 const int N = 1e6+5;
415 Point s[N]; // 用来存凸包多边形的顶点
416 int top = 0;
417 // 点集 p[] 的下标从 1 开始, 长度为 n
418 void Andrew(Point *p, int n) {
419     sort(p + 1, p + n + 1);
420     for (int i = 1; i <= n; i++) { // 下凸包
421         while (top > 1 && Cross(s[top - 1], s[top], p[i]) <= 0) top--;
422         s[++top] = p[i];
423     }
424     int t = top;
425     for (int i = n - 1; i >= 1; i--) { // 上凸包
426         while (top > t && Cross(s[top - 1], s[top], p[i]) <= 0) top--;
427         s[++top] = p[i];
428     }
429     top--; // 因为首尾都会加一次第一个点, 所以去掉最后一个
430 }
431 // 凸圆: 给定点集, 求一个半径最小的圆, 能覆盖所有的点
432 // p[] 下标从 0 开始, 长度为 n
433 Circle get_min_circle(Point *p, int n) {
434     // 随机化, 降低复杂度
435     srand(time(NULL));
436     for (int i = 0; i < n; i++) swap(p[rand() % n], p[rand() % n]);
437     Point o = p[0];
438     double r = 0;
439     for (int i = 0; i < n; i++) {
440         if (sgn(dist(o, p[i]) - r) <= 0) continue;
441         o = (p[i] + p[0]) / 2;
442         r = dist(p[i], p[0]) / 2;
443         for (int j = 1; j < i; j++) {
444             if (sgn(dist(o, p[j]) - r) <= 0) continue;
445             o = (p[i] + p[j]) / 2;
446             r = dist(p[i], p[j]) / 2;
447             for (int k = 0; k < j; k++) {
448                 if (sgn(dist(o, p[k]) - r) <= 0) continue;
449                 o = get_circumcircle(p[i], p[j], p[k]).o;
450                 r = dist(o, p[i]);
451             }
452         }
453     }
454     return Circle(o, r);
455 }
456 // -----
457 // 求解多个圆面积的并
458 // 格林公式 O(n^2*logn)

```

```

459 Circle c[N];
460 pair<double, int> st[N << 1];
461 int n;
462 double cal(int x) {
463     int top = 0, cnt = 0;
464     for (int i = 1; i <= n; i++) {
465         if (i == x) continue;
466         double dis = dist(c[i].o, c[x].o);
467         double r1 = c[x].r, r2 = c[i].r;
468         if (sgn(r1 + dis - r2) <= 0) return 0.0;
469         if (sgn(r2 + dis - r1) <= 0 || sgn(r2 + r1 - dis) <= 0) continue;
470         double del = acos((r1 * r1 + dis * dis - r2 * r2) / (2 * r1 * dis));
471         double angle = atan2(c[i].o.y - c[x].o.y, c[i].o.x - c[x].o.x);
472         double l = angle - del, r = angle + del;
473         if (sgn(l + PI) < 0) l += 2 * PI;
474         if (sgn(r - PI) >= 0) r -= 2 * PI;
475         if (sgn(l - r) > 0) cnt++;
476         st[++top] = {l, 1}, st[++top] = {r, -1};
477     }
478     st[0] = {-PI, 0}, st[++top] = {PI, 0};
479     sort(st + 1, st + top + 1);
480     double res = 0;
481     for (int i = 1; i <= top; cnt += st[i++].second) {
482         if (cnt) continue;
483         Point o = c[x].o;
484         double r = c[x].r, t1 = st[i - 1].first, t2 = st[i].first;
485         res += r * (r * (t2 - t1) + o.x * (sin(t2) - sin(t1)) - o.y * (cos(t2) - cos(
            t1)));
486     }
487     return res;
488 }
489 double get_ans() {
490     sort(c + 1, c + n + 1);
491     n = unique(c + 1, c + n + 1) - c - 1;
492     double ans = 0;
493     for (int i = 1; i <= n; i++) ans += cal(i);
494     return ans / 2;
495 }
496 void s1(){
497     cin >> n;
498     for (int i = 1; i <= n; i++) scanf("%lf%lf%lf", &c[i].o.x, &c[i].o.y, &c[i].r);
499     printf("%.31f\n", get_ans());
500 }
501 // -----
502 // 给定一个多边形和一个圆,求面积交集
503 // 返回圆点到 ab 线段的距离,并带回圆与线段的交点 pa, pb
504 double getDP2(Point a, Point b, Circle c, Point &pa, Point &pb) {
505     Point o = c.o;
506     double R = c.r;
507     Point e = Intersection_line(a, b - a, o, Rotate(b - a, PI / 2)); // 垂足点
508     double d = dist(o, e);
509     if (!OnSegment(e, a, b)) d = min(dist(o, a), dist(o, b));
510     if (R <= d) return d;

```

```

511     double Len = sqrt(R * R - dist(o, e) * dist(o, e));
512     pa = e + norm(a - b) * Len;
513     pb = e + norm(b - a) * Len;
514     return d;
515 }
516
517 double getArea(Point a, Point b, Circle C) { // 面积的交
518     Point o = C.o;
519     double R = C.r;
520     if (sgn(a ^ b) == 0) return 0; // 共线
521     double da = dist(o, a), db = dist(o, b);
522     if (sgn(R - da) >= 0 && sgn(R - db) >= 0) return (a ^ b) / 2; // ab 在圆内
523     Point pa, pb;
524     double d = getDP2(a, b, C, pa, pb);
525     if (sgn(R - d) <= 0) return SectorArea(a, b, R); // ab 在圆外
526     if (sgn(R - da) >= 0) return (a ^ pb) / 2 + SectorArea(pb, b, R); // a 在圆内
527     if (sgn(R - db) >= 0) return SectorArea(a, pa, R) + (pa ^ b) / 2; // b 在圆内
528     return SectorArea(a, pa, R) + (pa ^ pb) / 2 + SectorArea(pb, b, R); // ab 是割线
529 }
530
531 // 返回所求的面积交
532 double Intersection_Area(Point *p, int n, Circle C) {
533     // 平移
534     for (int i = 0; i < n; i++) p[i] = p[i] - C.o;
535     C.o = Point(0.0, 0.0);
536
537     double area = 0;
538     for (int i = 0, j = n - 1; i < n; j = i++) area += getArea(p[j], p[i], C);
539     return fabs(area);
540 }
541 // 调用
542 Point p[N];
543 int n;
544 Circle C;
545 double area = Intersection_Area(p, n, C);
546 // -----
547 // 自适应辛普森积分法
548 /*
549 辛普森公式
550 对二次函数  $f(x)=ax^2+bx+c$  在区间  $[1,r]$  上求定积分 (即图像与  $x$  轴围成的面积)
551 原函数:  $F(x)= (a/3)x^3 + (b/2)x^2 + cx + d$  (13)
552  $\int (从1到r) f(x)dx = (a/3)(r^3-1^3) + (b/2)(r^2-1^2) + c(r-1)$  (14)
553  $= (r-1)[(a/3)(r^2-1^2) + (b/2)(r-1) + c]$  (15)
554  $= (r-1)/6 (2a1^2 + 2ar^2 + 2a1r + 3b1 + 3br + 6c)$  (16)
555  $= (r-1)/6 [(a1^2 + b1 + c) + (ar^2 + br + c) + 4(a((1+r)/2)^2 + b((1+r)/2) + c)]$ 
      (17)
556  $= (r-1)(f(1) + f(r) + 4f((1+r)/2))/6$  (18)
557 即 积分(从1到r) $f(x)dx = (r-1)(f(1)+f(r)+4f((1+r)/2))/6$  (即区间宽度 $\times$ 均值高度)。
558
559 自适应辛普森法:  $O(\log(n/eps))$ 
560 把任意可积函数分割成很多段, 对积分区间不断二分, 每次判断当前段和二次函数的相似程度,
561 如果足够相似就直接代入辛普森公式计算, 否则就继续分割递归求解。
562 */

```

```

563 // 积分函数，是啥填啥
564 double f(double x) {
565     // ...
566 }
567 // 辛普森公式
568 double simpson(double l, double r) {
569     return (r - l) * (f(l) + f(r) + 4 * f((l + r) / 2)) / 6;
570 }
571 // 自适应
572 double asr(double l, double r, double ans) {
573     double mid = (l + r) / 2, a = simpson(l, mid), b = simpson(mid, r);
574     if (sgn(a + b - ans) == 0) return ans;
575     return asr(l, mid, a) + asr(mid, r, b);
576 }
577 // 调用
578 int l, r; // 积分上下限
579 double ans = asr(l, r, 0);
580 // -----
581 // 求解点集中最近的点对的距离
582 // 归并排序 O(nlogn)
583 // Need: dist()
584
585 Point p[N], t[N]; // p[] 存点, t[] 是辅助数组
586
587 double divide(int l, int r) {
588     double d = 2e9;
589     if (l == r) return d;
590     int mid = l + r >> 1;
591     Point tmp = p[mid];
592     d = min(divide(l, mid), divide(mid + 1, r)); // 分治
593
594     // 归并排序部分
595     int k = 0, i = l, j = mid + 1, tt = 0;
596     while (i <= mid && j <= r)
597         if (p[i].y < p[j].y) t[k++] = p[i++];
598         else t[k++] = p[j++];
599     while (i <= mid) t[k++] = p[i++];
600     while (j <= r) t[k++] = p[j++];
601     for (i = l, j = 0; i <= r; i++, j++) p[i] = t[j];
602
603     for (int i = 0; i < k; i++)
604         if (fabs(tmp.x - t[i].x) < d) t[tt++] = t[i];
605
606     for (int i = 0; i < tt; i++)
607         for (int j = i + 1; j < tt && t[j].y - t[i].y < d; j++)
608             d = min(d, dist(t[i], t[j]));
609     return d;
610 }
611
612 // 调用
613 int n; // 点的个数
614 double dis = divide(1, n);
615

```

```

616 // -----
617 signed main() {
618     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
619     ld ans;
620
621     return 0;
622 }

```

4.5.2 三维几何必要初始化

点线面封装

其他函数

```

1  struct Point3 {
2      ld x, y, z;
3      Point3(ld x_ = 0, ld y_ = 0, ld z_ = 0) : x(x_), y(y_), z(z_) {}
4      Point3 &operator+=(Point3 p) & {
5          return x += p.x, y += p.y, z += p.z, *this;
6      }
7      Point3 &operator-=(Point3 p) & {
8          return x -= p.x, y -= p.y, z -= p.z, *this;
9      }
10     Point3 &operator*=(Point3 p) & {
11         return x *= p.x, y *= p.y, z *= p.z, *this;
12     }
13     Point3 &operator*=(ld t) & {
14         return x *= t, y *= t, z *= t, *this;
15     }
16     Point3 &operator/=(ld t) & {
17         return x /= t, y /= t, z /= t, *this;
18     }
19     friend Point3 operator+(Point3 a, Point3 b) { return a += b; }
20     friend Point3 operator-(Point3 a, Point3 b) { return a -= b; }
21     friend Point3 operator*(Point3 a, Point3 b) { return a *= b; }
22     friend Point3 operator*(Point3 a, ld b) { return a *= b; }
23     friend Point3 operator*(ld a, Point3 b) { return b *= a; }
24     friend Point3 operator/(Point3 a, ld b) { return a /= b; }
25     friend auto &operator>>(istream &is, Point3 &p) {
26         return is >> p.x >> p.y >> p.z;
27     }
28     friend auto &operator<<(ostream &os, Point3 p) {
29         return os << "(" << p.x << ", " << p.y << ", " << p.z << ")";
30     }
31 };
32 struct Line3 {
33     Point3 a, b;
34 };
35 struct Plane {
36     Point3 u, v, w;
37 };
38
39 ld len(P3 p) { // 原点到当前点的距离计算

```

```

40     return sqrt(p.x * p.x + p.y * p.y + p.z * p.z);
41 }
42 P3 crossEx(P3 a, P3 b) { // 叉乘
43     P3 ans;
44     ans.x = a.y * b.z - a.z * b.y;
45     ans.y = a.z * b.x - a.x * b.z;
46     ans.z = a.x * b.y - a.y * b.x;
47     return ans;
48 }
49 ld cross(P3 a, P3 b) {
50     return len(crossEx(a, b));
51 }
52 ld dot(P3 a, P3 b) { // 点乘
53     return a.x * b.x + a.y * b.y + a.z * b.z;
54 }
55 P3 getVec(Plane s) { // 获取平面法向量
56     return crossEx(s.u - s.v, s.v - s.w);
57 }
58 ld dis(P3 a, P3 b) { // 三维欧几里得距离公式
59     ld val = (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y) + (a.z - b.z) * (
60         a.z - b.z);
61     return sqrt(val);
62 }
63 P3 standardize(P3 vec) { // 将三维向量转换为单位向量
64     return vec / len(vec);
65 }

```

4.5.3 三维点线面相关

空间三点是否共线

其中第二个函数是专门用来判断给定的三个点能否构成平面的，因为不共线的三点才能构成平面。

四点是否共面

空间点是否在线段上

空间两点是否在线段同侧

当给定的两点与线段不共面、点在线段上时返回 *false*。

两点是否在平面同侧

点在平面上时返回 *false*。

空间两直线是否平行/垂直

两平面是否平行/垂直

空间两直线是否是同一条

两平面是否是同一个

直线是否与平面平行

空间两线段是否相交

空间两直线是否相交及交点

当两直线不共面、两直线平行时返回 *false*。

直线与平面是否相交及交点

当直线与平面平行、给定的点构不成平面时返回 *false* 。

两平面是否相交及交线

当两平面平行、两平面为同一个时返回 *false* 。

点到直线的最近点与最近距离**点到平面的最近点与最近距离****空间两直线的最近距离与最近点对**

```

1  bool onLine(P3 p1, P3 p2, P3 p3) { // 三点是否共线
2      return sign(cross(p1 - p2, p3 - p2)) == 0;
3  }
4  bool onLine(Plane s) {
5      return onLine(s.u, s.v, s.w);
6  }
7
8  bool onPlane(P3 p1, P3 p2, P3 p3, P3 p4) { // 四点是否共面
9      ld val = dot(getVec({p1, p2, p3}), p4 - p1);
10     return sign(val) == 0;
11 }
12
13 bool pointOnSegment(P3 p, L3 l) {
14     return sign(cross(p - l.a, p - l.b)) == 0 && min(l.a.x, l.b.x) <= p.x &&
15         p.x <= max(l.a.x, l.b.x) && min(l.a.y, l.b.y) <= p.y && p.y <= max(l.a.y, l
16         .b.y) &&
17         min(l.a.z, l.b.z) <= p.z && p.z <= max(l.a.z, l.b.z);
18 }
19 bool pointOnSegmentEx(P3 p, L3 l) { // pointOnSegment去除端点版
20     return sign(cross(p - l.a, p - l.b)) == 0 && min(l.a.x, l.b.x) < p.x &&
21         p.x < max(l.a.x, l.b.x) && min(l.a.y, l.b.y) < p.y && p.y < max(l.a.y, l.b
22         .y) &&
23         min(l.a.z, l.b.z) < p.z && p.z < max(l.a.z, l.b.z);
24 }
25 bool pointOnSegmentSide(P3 p1, P3 p2, L3 l) {
26     if (!onPlane(p1, p2, l.a, l.b)) { // 特判不共面
27         return 0;
28     }
29     ld val = dot(crossEx(l.a - l.b, p1 - l.b), crossEx(l.a - l.b, p2 - l.b));
30     return sign(val) == 1;
31 }
32 bool pointOnPlaneSide(P3 p1, P3 p2, Plane s) {
33     ld val = dot(getVec(s), p1 - s.u) * dot(getVec(s), p2 - s.u);
34     return sign(val) == 1;
35 }
36
37 bool lineParallel(L3 l1, L3 l2) {
38     return sign(cross(l1.a - l1.b, l2.a - l2.b)) == 0;
39 }
40 bool lineVertical(L3 l1, L3 l2) {
41     return sign(dot(l1.a - l1.b, l2.a - l2.b)) == 0;

```

```

42 }
43
44 bool planeParallel(Plane s1, Plane s2) {
45     ld val = cross(getVec(s1), getVec(s2));
46     return sign(val) == 0;
47 }
48 bool planeVertical(Plane s1, Plane s2) {
49     ld val = dot(getVec(s1), getVec(s2));
50     return sign(val) == 0;
51 }
52
53 bool same(L3 l1, L3 l2) {
54     return lineParallel(l1, l2) && lineParallel({l1.a, l2.b}, {l1.b, l2.a});
55 }
56
57 bool same(Plane s1, Plane s2) {
58     return onPlane(s1.u, s2.u, s2.v, s2.w) && onPlane(s1.v, s2.u, s2.v, s2.w) &&
59         onPlane(s1.w, s2.u, s2.v, s2.w);
60 }
61
62 bool linePlaneParallel(L3 l, Plane s) {
63     ld val = dot(l.a - l.b, getVec(s));
64     return sign(val) == 0;
65 }
66
67 bool segmentIntersection(L3 l1, L3 l2) { // 重叠、相交于端点均视为相交
68     if (!onPlane(l1.a, l1.b, l2.a, l2.b)) { // 特判不共面
69         return 0;
70     }
71     if (!onLine(l1.a, l1.b, l2.a) || !onLine(l1.a, l1.b, l2.b)) {
72         return !pointOnSegmentSide(l1.a, l1.b, l2) && !pointOnSegmentSide(l2.a, l2.b,
73             l1);
74     }
75     return pointOnSegment(l1.a, l2) || pointOnSegment(l1.b, l2) || pointOnSegment(l2
76         .a, l1) ||
77         pointOnSegment(l2.b, l2);
78 }
79 bool segmentIntersection1(L3 l1, L3 l2) { // 重叠、相交于端点不视为相交
80     return onPlane(l1.a, l1.b, l2.a, l2.b) && !pointOnSegmentSide(l1.a, l1.b, l2) &&
81         !pointOnSegmentSide(l2.a, l2.b, l1);
82 }
83
84 pair<bool, P3> lineIntersection(L3 l1, L3 l2) {
85     if (!onPlane(l1.a, l1.b, l2.a, l2.b) || lineParallel(l1, l2)) {
86         return {0, {}};
87     }
88     auto [s1, e1] = l1;
89     auto [s2, e2] = l2;
90     ld val = 0;
91     if (!onPlane(l1.a, l1.b, {0, 0, 0}, {0, 0, 1})) {
92         val = ((s1.x - s2.x) * (s2.y - e2.y) - (s1.y - s2.y) * (s2.x - e2.x)) /
93             ((s1.x - e1.x) * (s2.y - e2.y) - (s1.y - e1.y) * (s2.x - e2.x));
94     } else if (!onPlane(l1.a, l1.b, {0, 0, 0}, {0, 1, 0})) {

```



```

93     val = ((s1.x - s2.x) * (s2.z - e2.z) - (s1.z - s2.z) * (s2.x - e2.x)) /
94         ((s1.x - e1.x) * (s2.z - e2.z) - (s1.z - e1.z) * (s2.x - e2.x));
95 } else {
96     val = ((s1.y - s2.y) * (s2.z - e2.z) - (s1.z - s2.z) * (s2.y - e2.y)) /
97         ((s1.y - e1.y) * (s2.z - e2.z) - (s1.z - e1.z) * (s2.y - e2.y));
98 }
99 return {1, s1 + (e1 - s1) * val};
100 }
101
102 pair<bool, P3> linePlaneCross(L3 l, Plane s) {
103     if (linePlaneParallel(l, s)) {
104         return {0, {}};
105     }
106     P3 vec = getVec(s);
107     P3 U = vec * (s.u - l.a), V = vec * (l.b - l.a);
108     ld val = (U.x + U.y + U.z) / (V.x + V.y + V.z);
109     return {1, l.a + (l.b - l.a) * val};
110 }
111
112 pair<bool, L3> planeIntersection(Plane s1, Plane s2) {
113     if (planeParallel(s1, s2) || same(s1, s2)) {
114         return {0, {}};
115     }
116     P3 U = linePlaneParallel({s2.u, s2.v}, s1) ? linePlaneCross({s2.v, s2.w}, s1).
        second
117         : linePlaneCross({s2.u, s2.v}, s1).second;
118     P3 V = linePlaneParallel({s2.w, s2.u}, s1) ? linePlaneCross({s2.v, s2.w}, s1).
        second
119         : linePlaneCross({s2.w, s2.u}, s1).second;
120     return {1, {U, V}};
121 }
122
123 pair<ld, P3> pointToLine(P3 p, L3 l) {
124     ld val = cross(p - l.a, l.a - l.b) / dis(l.a, l.b); // 面积除以底边长
125     ld val1 = dot(p - l.a, l.a - l.b) / dis(l.a, l.b);
126     return {val, l.a + val1 * standardize(l.a - l.b)};
127 }
128
129 pair<ld, P3> pointToPlane(P3 p, Plane s) {
130     P3 vec = getVec(s);
131     ld val = dot(vec, p - s.u);
132     val = abs(val) / len(vec); // 面积除以底边长
133     return {val, p - val * standardize(vec)};
134 }
135
136 tuple<ld, P3, P3> lineToLine(L3 l1, L3 l2) {
137     P3 vec = crossEx(l1.a - l1.b, l2.a - l2.b); // 计算同时垂直于两直线的向量
138     ld val = abs(dot(l1.a - l2.a, vec)) / len(vec);
139     P3 U = l1.b - l1.a, V = l2.b - l2.a;
140     vec = crossEx(U, V);
141     ld p = dot(vec, vec);
142     ld t1 = dot(crossEx(l2.a - l1.a, V), vec) / p;
143     ld t2 = dot(crossEx(l2.a - l1.a, U), vec) / p;

```

```

144     return {val, l1.a + (l1.b - l1.a) * t1, l2.a + (l2.b - l2.a) * t2};
145 }

```

4.5.4 三维角度与弧度

空间两直线夹角的 $\cos$ 值

任意位置的空间两直线。

空间两平面夹角的 $\cos$ 值

直线与平面夹角的 $\sin$ 值

```

1  ld lineCos(L3 l1, L3 l2) {
2      return dot(l1.a - l1.b, l2.a - l2.b) / len(l1.a - l1.b) / len(l2.a - l2.b);
3  }
4
5  ld planeCos(Plane s1, Plane s2) {
6      P3 U = getVec(s1), V = getVec(s2);
7      return dot(U, V) / len(U) / len(V);
8  }
9
10 ld linePlaneSin(L3 l, Plane s) {
11     P3 vec = getVec(s);
12     return dot(l.a - l.b, vec) / len(l.a - l.b) / len(vec);
13 }

```

4.5.5 二维凸包

获取二维静态凸包 (Andrew 算法)

flag 用于判定凸包边上的点、重复的顶点是否要加入到凸包中，为 0 时表示加入凸包（不严格）；为 1 时不加入凸包（严格）。时间复杂度为 $\mathcal{O}(N \log N)$ 。

二维动态凸包

固定为 int 型，需要重新书写 Line 函数， cmp 用于判定边界情况。可以处理如下两个要求：

- 动态插入点 (x, y) 到当前凸包中；
- 判断点 (x, y) 是否在凸包上或是在内部（包括边界）。

点与凸包的位置关系

0 代表点在凸包外面；1 代表在凸壳上；2 代表在凸包内部。

闵可夫斯基和

计算两个凸包合成的大凸包。

半平面交

计算多条直线左边平面部分的交集。

```

1  template<class T> vector<Point<T>> staticConvexHull(vector<Point<T>> A, int flag =
    1) {
2      int n = A.size();
3      if (n <= 2) { // 特判
4          return A;

```

```

5     }
6     vector<Point<T>> ans(n * 2);
7     sort(A.begin(), A.end());
8     int now = -1;
9     for (int i = 0; i < n; i++) { // 维护下凸包
10         while (now > 0 && cross(A[i], ans[now], ans[now - 1]) <= 0) {
11             now--;
12         }
13         ans[++now] = A[i];
14     }
15     int pre = now;
16     for (int i = n - 2; i >= 0; i--) { // 维护上凸包
17         while (now > pre && cross(A[i], ans[now], ans[now - 1]) <= 0) {
18             now--;
19         }
20         ans[++now] = A[i];
21     }
22     ans.resize(now);
23     return ans;
24 }
25
26 template<class T> bool turnRight(Pt a, Pt b) {
27     return cross(a, b) < 0 || (cross(a, b) == 0 && dot(a, b) < 0);
28 }
29 struct Line {
30     static int cmp;
31     mutable Point<int> a, b;
32     friend bool operator<(Line x, Line y) {
33         return cmp ? x.a < y.a : turnRight(x.b, y.b);
34     }
35     friend auto &operator<<(ostream &os, Line l) {
36         return os << "<" << l.a << ", " << l.b << ">";
37     }
38 };
39
40 int Line::cmp = 1;
41 struct UpperConvexHull : set<Line> {
42     bool contains(const Point<int> &p) const {
43         auto it = lower_bound({p, 0});
44         if (it != end() && it->a == p) return true;
45         if (it != begin() && it != end() && cross(prev(it)->b, p - prev(it)->a) <= 0)
46             {
47                 return true;
48             }
49         return false;
50     }
51     void add(const Point<int> &p) {
52         if (contains(p)) return;
53         auto it = lower_bound({p, 0});
54         for (; it != end(); it = erase(it)) {
55             if (turnRight(it->a - p, it->b)) {
56                 break;
57             }
58         }
59     }
60 };

```

```

57     }
58     for (; it != begin() && prev(it) != begin(); erase(prev(it))) {
59         if (turnRight(prev(prev(it))->b, p - prev(prev(it))->a)) {
60             break;
61         }
62     }
63     if (it != begin()) {
64         prev(it)->b = p - prev(it)->a;
65     }
66     if (it == end()) {
67         insert({p, {0, -1}});
68     } else {
69         insert({p, it->a - p});
70     }
71 }
72 };
73 struct ConvexHull {
74     UpperConvexHull up, low;
75     bool empty() const {
76         return up.empty();
77     }
78     bool contains(const Point<int> &p) const {
79         Line::cmp = 1;
80         return up.contains(p) && low.contains(-p);
81     }
82     void add(const Point<int> &p) {
83         Line::cmp = 1;
84         up.add(p);
85         low.add(-p);
86     }
87     bool isIntersect(int A, int B, int C) const {
88         Line::cmp = 0;
89         if (empty()) return false;
90         Point<int> k = {-B, A};
91         if (k.x < 0) k = -k;
92         if (k.x == 0 && k.y < 0) k.y = -k.y;
93         Point<int> P = up.upper_bound({{0, 0}, k})->a;
94         Point<int> Q = -low.upper_bound({{0, 0}, k})->a;
95         return sign(A * P.x + B * P.y - C) * sign(A * Q.x + B * Q.y - C) > 0;
96     }
97     friend ostream &operator<<(ostream &out, const ConvexHull &ch) {
98         for (const auto &line : ch.up) out << "(" << line.a.x << "," << line.a.y << "
99             << ")";
100         cout << "/";
101         for (const auto &line : ch.low) out << "(" << -line.a.x << "," << -line.a.y
102             << ")";
103         return out;
104     }
105 };
106
107 template<class T> int contains(Point<T> p, vector<Point<T>> A) {
108     int n = A.size();
109     bool in = false;

```

```

108     for (int i = 0; i < n; i++) {
109         Point<T> a = A[i] - p, b = A[(i + 1) % n] - p;
110         if (a.y > b.y) {
111             swap(a, b);
112         }
113         if (a.y <= 0 && 0 < b.y && cross(a, b) < 0) {
114             in = !in;
115         }
116         if (cross(a, b) == 0 && dot(a, b) <= 0) {
117             return 1;
118         }
119     }
120     return in ? 2 : 0;
121 }
122
123 template<class T> vector<Point<T>> mincowski(vector<Point<T>> P1, vector<Point<T>>
    P2) {
124     int n = P1.size(), m = P2.size();
125     vector<Point<T>> V1(n), V2(m);
126     for (int i = 0; i < n; i++) {
127         V1[i] = P1[(i + 1) % n] - P1[i];
128     }
129     for (int i = 0; i < m; i++) {
130         V2[i] = P2[(i + 1) % m] - P2[i];
131     }
132     vector<Point<T>> ans = {P1.front() + P2.front()};
133     int t = 0, i = 0, j = 0;
134     while (i < n && j < m) {
135         Point<T> val = sign(cross(V1[i], V2[j])) > 0 ? V1[i++] : V2[j++];
136         ans.push_back(ans.back() + val);
137     }
138     while (i < n) ans.push_back(ans.back() + V1[i++]);
139     while (j < m) ans.push_back(ans.back() + V2[j++]);
140     return ans;
141 }
142
143 template<class T> vector<Point<T>> halfcut(vector<Line<T>> lines) {
144     sort(lines.begin(), lines.end(), [&](auto l1, auto l2) {
145         auto d1 = l1.b - l1.a;
146         auto d2 = l2.b - l2.a;
147         if (sign(d1) != sign(d2)) {
148             return sign(d1) == 1;
149         }
150         return cross(d1, d2) > 0;
151     });
152     deque<Line<T>> ls;
153     deque<Point<T>> ps;
154     for (auto l : lines) {
155         if (ls.empty()) {
156             ls.push_back(l);
157             continue;
158         }
159         while (!ps.empty() && !pointOnLineLeft(ps.back(), l)) {

```

```

160         ps.pop_back();
161         ls.pop_back();
162     }
163     while (!ps.empty() && !pointOnLineLeft(ps[0], l)) {
164         ps.pop_front();
165         ls.pop_front();
166     }
167     if (cross(l.b - l.a, ls.back().b - ls.back().a) == 0) {
168         if (dot(l.b - l.a, ls.back().b - ls.back().a) > 0) {
169             if (!pointOnLineLeft(ls.back().a, l)) {
170                 assert(ls.size() == 1);
171                 ls[0] = l;
172             }
173             continue;
174         }
175         return {};
176     }
177     ps.push_back(lineIntersection(ls.back(), l));
178     ls.push_back(l);
179 }
180 while (!ps.empty() && !pointOnLineLeft(ps.back(), ls[0])) {
181     ps.pop_back();
182     ls.pop_back();
183 }
184 if (ls.size() <= 2) {
185     return {};
186 }
187 ps.push_back(lineIntersection(ls[0], ls.back()));
188 return vector(ps.begin(), ps.end());
189 }

```

4.5.6 常用例题

将平面某点旋转任意角度

题意：给定平面上一点 (a, b) ，输出将其逆时针旋转 d 度之后的坐标。

平面最近点对 (set 解)

借助 `set`，在严格 $\mathcal{O}(N \log N)$ 复杂度内求解，比常见的分治法稍快。

平面若干点能构成的最大四边形的面积 (简单版, 暴力枚举)

题意：平面上存在若干个点，保证没有两点重合、没有三点共线，你需要从中选出四个点，使得它们构成的四边形面积是最大的，注意这里能组成的四边形可以不是凸四边形。

暴力枚举其中一条对角线后枚举剩余两个点， $\mathcal{O}(N^3)$ 。

平面若干点能构成的最大四边形的面积 (困难版, 分类讨论 + 旋转卡壳)

题意：平面上存在若干个点，可能存在多点重合、共线的情况，你需要从中选出四个点，使得它们构成的四边形面积是最大的，注意这里能组成的四边形可以不是凸四边形、可以是退化的四边形。

当凸包大小 ≤ 2 时，说明是退化的四边形，答案直接为 0；大小恰好为 3 时，说明是凹四边形，我们枚举不在凸包上的那一点，将两个三角形面积相减既可得到答案；大小恰好为 4 时，

说明是凸四边形，使用旋转卡壳求解。

线段将多边形切割为几个部分

题意：给定平面上一线段与一个任意多边形，求解线段将多边形切割为几个部分；保证线段的端点不在多边形内、多边形边上，多边形顶点不位于线段上，多边形的边不与线段重叠；多边形端点按逆时针顺序给出。下方的几个样例均合法，答案均为 3。

当线段切割多边形时，本质是与多边形的边交于两个点、或者说是与多边形的两条边相交，设交点数目为 x ，那么答案即为 $\frac{x}{2} + 1$ 。于是，我们只需要计算交点数量即可，先判断某一条边是否与线段相交，再判断边的两个端点是否位于线段两侧。

平面若干点能否构成凸包（暴力枚举）

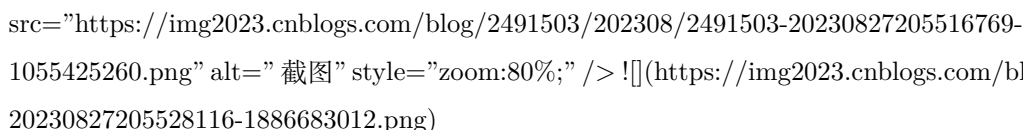
题意：给定平面上若干个点，判断其是否构成凸包 See。

可以直接使用凸包模板，但是代码较长；在这里我们使用暴力枚举试点，也能以 $\mathcal{O}(N)$ 的复杂度通过。当两个向量的叉乘 ≤ 0 时说明其夹角大于等于 180° ，使用这一点即可判定。

凸包上的点能构成的最大三角形（暴力枚举）

可以直接使用凸包模板，但是代码较长；在这里我们使用暴力枚举试点，也能以 $\mathcal{O}(N)$ 的复杂度通过。

另外补充一点性质：所求三角形的反互补三角形一定包含了凸包上的所有点（可以在边界）。通俗的说，构成的三角形是这个反互补三角形的中点三角形。如下图所示，点 A 不在 $\triangle BCE$ 的反互补三角形内部，故 $\triangle BCE$ 不是最大三角形； $\triangle ACE$ 才是。


<https://img2023.cnblogs.com/blog/2491503/202308/2491503-20230827205516769-1055425260.png> alt="截图" style="zoom:80%;"/>
<https://img2023.cnblogs.com/blog/2491503/202308/20230827205528116-1886683012.png>

凸包上的点能构成的最大四边形的面积（旋转卡壳）

由于是凸包上的点，所以保证了四边形一定是凸四边形，时间复杂度 $\mathcal{O}(N^2)$ 。

判断一个凸包是否完全在另一个凸包内

题意：给定一个凸多边形 A 和一个凸多边形 B ，询问 B 是否被 A 包含，分别判断严格/不严格包含。例题。

考虑严格包含，使用 A 点集计算出凸包 T_1 ，使用 A, B 两个点集计算出不严格凸包 T_2 ，如果包含，那么 T_1 应该与 T_2 完全相等；考虑不严格包含，在计算凸包 T_2 时严格即可。最终以 $\mathcal{O}(N)$ 复杂度求解，且代码不算很长。

多项式

```

1 signed main() {
2     int a, b, d;
3     cin >> a >> b >> d;
4
5     ld l = hypot(a, b); // 库函数，求直角三角形的斜边
6     ld alpha = atan2(b, a) + toArc(d);
7
8     cout << l * cos(alpha) << " " << l * sin(alpha) << endl;
9 }
10
```

```
11 template<class T> T sqr(T x) {
12     return x * x;
13 }
14
15 using V = Point<int>;
16 signed main() {
17     int n;
18     cin >> n;
19
20     vector<V> in(n);
21     for (auto &it : in) {
22         cin >> it;
23     }
24
25     int dis = disEx(in[0], in[1]); // 设定阈值
26     sort(in.begin(), in.end());
27
28     set<V> S;
29     for (int i = 0, h = 0; i < n; i++) {
30         V now = {in[i].y, in[i].x};
31         while (dis && dis <= sqr(in[i].x - in[h].x)) { // 删除超过阈值的点
32             S.erase({in[h].y, in[h].x});
33             h++;
34         }
35         auto it = S.lower_bound(now);
36         for (auto k = it; k != S.end() && sqr(k->x - now.x) < dis; k++) {
37             dis = min(dis, disEx(*k, now));
38         }
39         if (it != S.begin()) {
40             for (auto k = prev(it); sqr(k->x - now.x) < dis; k--) {
41                 dis = min(dis, disEx(*k, now));
42                 if (k == S.begin()) break;
43             }
44         }
45         S.insert(now);
46     }
47     cout << sqrt(dis) << endl;
48 }
49
50 signed main() {
51     int n;
52     cin >> n;
53     vector<Pi> in(n);
54     for (auto &it : in) {
55         cin >> it;
56     }
57     ld ans = 0;
58     for (int i = 0; i < n; i++) {
59         for (int j = i + 1; j < n; j++) { // 枚举对角线
60             ld l = 0, r = 0;
61             for (int k = 0; k < n; k++) { // 枚举第三点
62                 if (k == i || k == j) continue;
63                 if (pointOnLineLeft(in[k], {in[i], in[j]})) {
```



```

64         l = max(l, triangleS(in[k], in[j], in[i]));
65     } else {
66         r = max(r, triangleS(in[k], in[j], in[i]));
67     }
68 }
69 if (l * r != 0) { // 确保构成的是四边形
70     ans = max(ans, l + r);
71 }
72 }
73 }
74 cout << ans << endl;
75 }
76
77 using V = Point<int>;
78 signed main() {
79     int Task = 1;
80     for (cin >> Task; Task; Task--) {
81         int n;
82         cin >> n;
83
84         vector<V> in_(n);
85         for (auto &it : in_) {
86             cin >> it;
87         }
88         auto in = staticConvexHull(in_, 0);
89         n = in.size();
90
91         int ans = 0;
92         if (n > 3) {
93             ans = rotatingCalipers(in);
94         } else if (n == 3) {
95             int area = triangleAreaEx(in[0], in[1], in[2]);
96             for (auto it : in_) {
97                 if (it == in[0] || it == in[1] || it == in[2]) continue;
98                 int Min = min({triangleAreaEx(it, in[0], in[1]), triangleAreaEx(it, in
99                     [0], in[2]), triangleAreaEx(it, in[1], in[2])});
100                 ans = max(ans, area - Min);
101             }
102         }
103         cout << ans / 2;
104         if (ans % 2) {
105             cout << ".5";
106         }
107         cout << endl;
108     }
109 }
110
111 signed main() {
112     Pi s, e;
113     cin >> s >> e; // 读入线段
114
115     int n;

```

```

116     cin >> n;
117     vector<Pi> in(n);
118     for (auto &it : in) {
119         cin >> it; // 读入多边形端点
120     }
121
122     int cnt = 0;
123     for (int i = 0; i < n; i++) {
124         Pi x = in[i], y = in[(i + 1) % n];
125         cnt += (pointNotOnLineSide(x, y, {s, e}) && segmentIntersection(Line{x, y}, {
126             s, e}));
127     }
128     cout << cnt / 2 + 1 << endl;
129 }
130
131 signed main() {
132     int n;
133     cin >> n;
134
135     vector<Point<ld>> in(n);
136     for (auto &it : in) {
137         cin >> it;
138     }
139
140     for (int i = 0; i < n; i++) {
141         auto A = in[(i - 1 + n) % n];
142         auto B = in[i];
143         auto C = in[(i + 1) % n];
144         if (cross(A - B, C - B) > 0) {
145             cout << "No\n";
146             return 0;
147         }
148     }
149     cout << "Yes\n";
150 }
151
152 signed main() {
153     int n;
154     cin >> n;
155
156     vector<Point<int>> in(n);
157     for (auto &it : in) {
158         cin >> it;
159     }
160
161     #define S(x, y, z) triangleAreaEx(in[x], in[y], in[z])
162
163     int i = 0, j = 1, k = 2;
164     while (true) {
165         int val = S(i, j, k);
166         if (S((i + 1) % n, j, k) > val) {
167             i = (i + 1) % n;
168         } else if (S((i - 1 + n) % n, j, k) > val) {

```

```

168         i = (i - 1 + n) % n;
169     } else if (S(i, (j + 1) % n, k) > val) {
170         j = (j + 1) % n;
171     } else if (S(i, (j - 1 + n) % n, k) > val) {
172         j = (j - 1 + n) % n;
173     } else if (S(i, j, (k + 1) % n) > val) {
174         k = (k + 1) % n;
175     } else if (S(i, j, (k - 1 + n) % n) > val) {
176         k = (k - 1 + n) % n;
177     } else {
178         break;
179     }
180 }
181 cout << i + 1 << " " << j + 1 << " " << k + 1 << endl;
182 }
183
184 template<class T> T rotatingCalipers(vector<Point<T>> &p) {
185     #define S(x, y, z) triangleAreaEx(p[x], p[y], p[z])
186     int n = p.size();
187     T ans = 0;
188     auto nxt = [&](int i) -> int {
189         return i == n - 1 ? 0 : i + 1;
190     };
191     for (int i = 0; i < n; i++) {
192         int p1 = nxt(i), p2 = nxt(nxt(nxt(i)));
193         for (int j = nxt(nxt(i)); nxt(j) != i; j = nxt(j)) {
194             while (nxt(p1) != j && S(i, j, nxt(p1)) > S(i, j, p1)) {
195                 p1 = nxt(p1);
196             }
197             if (p2 == j) {
198                 p2 = nxt(p2);
199             }
200             while (nxt(p2) != i && S(i, j, nxt(p2)) > S(i, j, p2)) {
201                 p2 = nxt(p2);
202             }
203             ans = max(ans, S(i, j, p1) + S(i, j, p2));
204         }
205     }
206     return ans;
207     #undef S
208 }

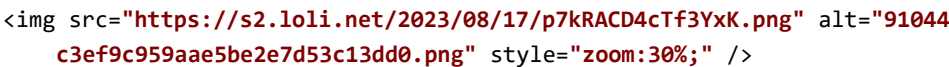
```

4.5.7 常用结论

- ```

1 ### 常用结论
2
3 #### 平面几何结论归档
4
5 - `hypot` 函数可以直接计算直角三角形的斜边长;
6 - **边心距**是指正多边形的外接圆圆心到正多边形某一边的距离, 边长为 s 的正 n 角形的边心距公式为 $\displaystyle a=\frac{t}{2}\cdot\tan\frac{\pi}{n}$, 外接圆半径为 R 的正 n 角形的边心距公式为 $a=R\cdot\cos\frac{\pi}{n}$;

```

- 7 - \*\*三角形外接圆半径\*\*为  $\frac{a}{2\sin A} = \frac{abc}{4S}$ ，其中  $S$  为三角形面积，  
内切圆半径为  $\frac{2S}{a+b+c}$ ；
- 8 - 由小正三角形拼成的大正三角形，耗费的小三角形数量即为构成一条边的小三角形数量的平方。如下图，总数量即为  $4^2$  [See](https://codeforces.com/problemset/problem/559/A)。
- 9
- 10    
 ``
- 11 - 正  $n$  边形圆心角为  $\frac{360^\circ}{n}$ ，圆周角为  $\frac{180^\circ}{n}$ 。定义正  $n$  边形上的三个顶点  $A, B$  和  $C$ （可以不相邻），使得  $\angle ABC = \theta$ ，当  $n \leq 360$  时， $\theta$  可以取  $1^\circ$  到  $179^\circ$  间的任何一个整数 [See](https://codeforces.com/problemset/problem/1096/C)。
- 12 - 某一点  $B$  到直线  $AC$  的距离公式为  $\frac{|\vec{BA} \times \vec{BC}|}{|\vec{AC}|}$ ，等价于  $\frac{|ax+by+c|}{\sqrt{a^2+b^2}}$ 。
- 13 -  $\text{atan}(y/x)$  函数仅用于计算第一、四象限的值，而  $\text{atan2}(y, x)$  则允许计算所有四个象限的正反切，在使用这个函数时，需要尽量保证  $x$  和  $y$  的类型为整数型，如果使用浮点数，实测会慢十倍。
- 14 - 在平面上有奇数个点  $A_0, A_1, \dots, A_n$  以及一个点  $X_0$ ，构造  $X_1$  使得  $X_0, X_1$  关于  $A_0$  对称、构造  $X_2$  使得  $X_1, X_2$  关于  $A_1$  对称、……、构造  $X_j$  使得  $X_{j-1}, X_j$  关于  $A_{(j-1) \bmod n}$  对称。那么周期为  $2n$ ，即  $A_0$  与  $A_{2n}$  共点、 $A_1$  与  $A_{2n+1}$  共点 [See](https://codeforces.com/contest/24/problem/C)。
- 15 - 已知  $A(x_A, y_A)$  和  $X(x_X, y_X)$  两点及这两点的坐标，构造  $Y$  使得  $X, Y$  关于  $A$  对称，那么  $Y$  的坐标为  $(2x_A - x_X, 2y_A - y_X)$ 。
- 16 - \*\*海伦公式\*\*：已知三角形三边长  $a, b$  和  $c$ ，定义  $p = \frac{a+b+c}{2}$ ，则  $S_{\triangle} = \sqrt{p(p-a)(p-b)(p-c)}$ ，在使用时需要注意越界问题，本质是铅锤定理，一般多使用叉乘计算三角形面积而不使用该公式。
- 17 - 棱台体积  $V = \frac{1}{3}(S_1 + S_2 + \sqrt{S_1 S_2}) \cdot h$ ，其中  $S_1, S_2$  为上下底面积。
- 18 - 正棱台侧面积  $\frac{1}{2}(C_1 + C_2) \cdot L$ ，其中  $C_1, C_2$  为上下底周长， $L$  为斜高（上下底对应的平行边的距离）。
- 19 - 球面积  $4\pi r^2$ ，体积  $\frac{4}{3}\pi r^3$ 。
- 20 - 正三角形面积  $\frac{\sqrt{3}}{4}a^2$ ，正四面体面积  $\frac{\sqrt{3}}{2}a^2$ 。
- 21 - 设扇形对应的圆心角弧度为  $\theta$ ，则面积为  $S = \frac{\theta}{2} \cdot R^2$ 。
- 22
- 23 #### 立体几何结论归档
- 24
- 25 - 已知向量  $\vec{r} = \{x, y, z\}$ ，则该向量的三个方向余弦为  $\cos \alpha = \frac{x}{|\vec{r}|}$ ， $\cos \beta = \frac{y}{|\vec{r}|}$ ， $\cos \gamma = \frac{z}{|\vec{r}|}$ 。其中  $\alpha, \beta, \gamma \in [0, \pi]$ ， $\cos^2 \alpha + \cos^2 \beta + \cos^2 \gamma = 1$ 。

#### 4.5.8 平面三角形相关（浮点数处理）

##### 三角形面积

###### 三角形外心

三角形外接圆的圆心，即三角形三边垂直平分线的交点。

###### 三角形内心

三角形内切圆的圆心，也是三角形三个内角的角平分线的交点。其到三角形三边的距离相等。

###### 三角形垂心

三角形的三条高线所在直线的交点。锐角三角形的垂心在三角形内；直角三角形的垂心在直

角顶点上；钝角三角形的垂心在三角形外。

```

1 ld area(Point<ld> a, Point<ld> b, Point<ld> c) {
2 return abs(cross(b, c, a)) / 2;
3 }
4
5 template<class T> Pt center1(Pt p1, Pt p2, Pt p3) { // 外心
6 return lineIntersection(midSegment({p1, p2}), midSegment({p2, p3}));
7 }
8
9 Pd center2(Pd p1, Pd p2, Pd p3) { // 内心
10 #define atan2(p) atan2(p.y, p.x) // 注意先后顺序
11 Line<ld> U = {p1, {}}, V = {p2, {}};
12 ld m, n, alpha;
13 m = atan2((p2 - p1));
14 n = atan2((p3 - p1));
15 alpha = (m + n) / 2;
16 U.b = {p1.x + cos(alpha), p1.y + sin(alpha)};
17 m = atan2((p1 - p2));
18 n = atan2((p3 - p2));
19 alpha = (m + n) / 2;
20 V.b = {p2.x + cos(alpha), p2.y + sin(alpha)};
21 return lineIntersection(U, V);
22 }
23
24 Pd center3(Pd p1, Pd p2, Pd p3) { // 垂心
25 Ld U = {p1, p1 + rotate(p2, p3)}; // 垂线
26 Ld V = {p2, p2 + rotate(p1, p3)};
27 return lineIntersection(U, V);
28 }

```

#### 4.5.9 平面几何必要初始化

##### 字符串读入浮点数

##### 预置函数

##### 点线封装

##### 叉乘

定义公式  $a \times b = |a||b|\sin\theta$ 。

##### 点乘

定义公式  $a \times b = |a||b|\cos\theta$ 。

##### 欧几里得距离公式

最常用的距离公式。**需要注意**，开根号会丢失精度，如无强制要求，先不要开根号，留到最后一步一起开。

##### 曼哈顿距离公式

##### 将向量转换为单位向量

##### 向量旋转

将当前向量移动至原点后顺时针旋转  $90^\circ$ ，即获取垂直于当前向量的、起点为原点的向量。在计算垂线时非常有用。例如，要想获取点  $a$  绕点  $o$  顺时针旋转  $90^\circ$  后的点，可以这样书写代

码: `auto ans = o + rotate(o, a);`; 如果是逆时针旋转, 那么只需更改符号即可: `auto ans = o - rotate(o, a);`。

```

1 const int Knum = 4;
2 int read(int k = Knum) {
3 string s;
4 cin >> s;
5
6 int num = 0;
7 int it = s.find('.');
8 if (it != -1) { // 存在小数点
9 num = s.size() - it - 1; // 计算小数位数
10 s.erase(s.begin() + it); // 删除小数点
11 }
12 for (int i = 1; i <= k - num; i++) { // 补全小数位数
13 s += '0';
14 }
15 return stoi(s);
16 }
17
18 using ld = long double;
19 const ld PI = acos(-1);
20 const ld EPS = 1e-7;
21 const ld INF = numeric_limits<ld>::max();
22 #define cc(x) cout << fixed << setprecision(x);
23
24 ld fgcd(ld x, ld y) { // 实数域gcd
25 return abs(y) < EPS ? abs(x) : fgcd(y, fmod(x, y));
26 }
27 template<class T, class S> bool equal(T x, S y) {
28 return -EPS < x - y && x - y < EPS;
29 }
30 template<class T> int sign(T x) {
31 if (-EPS < x && x < EPS) return 0;
32 return x < 0 ? -1 : 1;
33 }
34
35 template<class T> struct Point { // 在C++17下使用 emplace_back 绑定可能会导致CE!
36 T x, y;
37 Point(T x_ = 0, T y_ = 0) : x(x_), y(y_) {} // 初始化
38 template<class U> operator Point<U>() { // 自动类型匹配
39 return Point<U>(U(x), U(y));
40 }
41 Point &operator+=(Point p) & { return x += p.x, y += p.y, *this; }
42 Point &operator+=(T t) & { return x += t, y += t, *this; }
43 Point &operator--(Point p) & { return x -= p.x, y -= p.y, *this; }
44 Point &operator--(T t) & { return x -= t, y -= t, *this; }
45 Point &operator*=(T t) & { return x *= t, y *= t, *this; }
46 Point &operator/=(T t) & { return x /= t, y /= t, *this; }
47 Point operator-() const { return Point(-x, -y); }
48 friend Point operator+(Point a, Point b) { return a += b; }
49 friend Point operator+(Point a, T b) { return a += b; }
50 friend Point operator-(Point a, Point b) { return a -= b; }

```

```

51 friend Point operator-(Point a, T b) { return a -= b; }
52 friend Point operator*(Point a, T b) { return a *= b; }
53 friend Point operator*(T a, Point b) { return b *= a; }
54 friend Point operator/(Point a, T b) { return a /= b; }
55 friend bool operator<(Point a, Point b) {
56 return equal(a.x, b.x) ? a.y < b.y - EPS : a.x < b.x - EPS;
57 }
58 friend bool operator>(Point a, Point b) { return b < a; }
59 friend bool operator==(Point a, Point b) { return !(a < b) && !(b < a); }
60 friend bool operator!=(Point a, Point b) { return a < b || b < a; }
61 friend auto &operator>>(istream &is, Point &p) {
62 return is >> p.x >> p.y;
63 }
64 friend auto &operator<<(ostream &os, Point p) {
65 return os << "(" << p.x << ", " << p.y << ")";
66 }
67 };
68 template<class T> struct Line {
69 Point<T> a, b;
70 Line(Point<T> a_ = Point<T>(), Point<T> b_ = Point<T>()) : a(a_), b(b_) {}
71 template<class U> operator Line<U>() { // 自动类型匹配
72 return Line<U>(Point<U>(a), Point<U>(b));
73 }
74 friend auto &operator<<(ostream &os, Line l) {
75 return os << "<" << l.a << ", " << l.b << ">";
76 }
77 };
78
79 template<class T> T cross(Point<T> a, Point<T> b) { // 叉乘
80 return a.x * b.y - a.y * b.x;
81 }
82 template<class T> T cross(Point<T> p1, Point<T> p2, Point<T> p0) { // 叉乘 (p1 - p0)
83 x (p2 - p0);
84 return cross(p1 - p0, p2 - p0);
85 }
86
87 template<class T> T dot(Point<T> a, Point<T> b) { // 点乘
88 return a.x * b.x + a.y * b.y;
89 }
90 template<class T> T dot(Point<T> p1, Point<T> p2, Point<T> p0) { // 点乘 (p1 - p0) *
91 (p2 - p0);
92 return dot(p1 - p0, p2 - p0);
93 }
94
95 template<class T> ld dis(T x1, T y1, T x2, T y2) {
96 ld val = (x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2);
97 return sqrt(val);
98 }
99
100 template<class T> ld dis(Point<T> a, Point<T> b) {
101 return dis(a.x, a.y, b.x, b.y);
102 }
103
104 template<class T> T dis1(Point<T> p1, Point<T> p2) { // 曼哈顿距离公式

```

```

102 return abs(p1.x - p2.x) + abs(p1.y - p2.y);
103 }
104
105 Point<ld> standardize(Point<ld> vec) { // 转换为单位向量
106 return vec / sqrt(vec.x * vec.x + vec.y * vec.y);
107 }
108
109 template<class T> Point<T> rotate(Point<T> p1, Point<T> p2) { // 旋转
110 Point<T> vec = p1 - p2;
111 return {-vec.y, vec.x};
112 }

```

#### 4.5.10 平面圆相关（浮点数处理）

##### 点到圆的最近点

同时返回最近点与最近距离。需要注意，当点为圆心时，这样的点有无数个，此时我们视作输入错误，直接返回圆心。

##### 根据圆心角获取圆上某点

将圆上最右侧的点以圆心为旋转中心，逆时针旋转  $\text{rad}$  度。

##### 直线是否与圆相交及交点

0 代表不相交；1 代表相切；2 代表相交。

##### 线段是否与圆相交及交点

0 代表不相交；1 代表相切；2 代表相交于一个点；3 代表相交于两个点。

##### 两圆是否相交及交点

0 代表内含；1 代表相离；2 代表相切；3 代表相交。

##### 两圆相交面积

上述所言四种相交情况均可计算，之所以不使用三角形面积计算公式是因为在计算过程中会出现“负数”面积（扇形面积与三角形面积的符号关系会随圆的位置关系发生变化），故公式全部重新推导，这里采用的是扇形面积减去扇形内部的那个三角形的面积。

##### 三点确定一圆

##### 求解点到圆的切线数量与切点

##### 求解两圆的内公、外公切线数量与切点

同时返回公切线数量以及每个圆的切点。

```

1 pair<Pd, ld> pointToCircle(Pd p, Pd o, ld r) {
2 Pd U = o, V = o;
3 ld d = dis(p, o);
4 if (sign(d) == 0) { // p 为圆心时返回圆心本身
5 return {o, 0};
6 }
7 ld val1 = r * abs(o.x - p.x) / d;
8 ld val2 = r * abs(o.y - p.y) / d * ((o.x - p.x) * (o.y - p.y) < 0 ? -1 : 1);
9 U.x += val1, U.y += val2;
10 V.x -= val1, V.y -= val2;
11 if (dis(U, p) < dis(V, p)) {
12 return {U, dis(U, p)};

```



```

13 } else {
14 return {V, dis(V, p)};
15 }
16 }
17
18 Point<ld> getPoint(Point<ld> p, ld r, ld rad) {
19 return {p.x + cos(rad) * r, p.y + sin(rad) * r};
20 }
21
22 tuple<int, Pd, Pd> lineCircleCross(Ld l, Pd o, ld r) {
23 Pd P = project(o, l);
24 ld d = dis(P, o), tmp = r * r - d * d;
25 if (sign(tmp) == -1) {
26 return {0, {}, {}};
27 } else if (sign(tmp) == 0) {
28 return {1, P, {}};
29 }
30 Pd vec = standardize(l.b - l.a) * sqrt(tmp);
31 return {2, P + vec, P - vec};
32 }
33
34 tuple<int, Pd, Pd> segmentCircleCross(Ld l, Pd o, ld r) {
35 auto [type, U, V] = lineCircleCross(l, o, r);
36 bool f1 = pointOnSegment(U, l), f2 = pointOnSegment(V, l);
37 if (type == 1 && f1) {
38 return {1, U, {}};
39 } else if (type == 2 && f1 && f2) {
40 return {3, U, V};
41 } else if (type == 2 && f1) {
42 return {2, U, {}};
43 } else if (type == 2 && f2) {
44 return {2, V, {}};
45 } else {
46 return {0, {}, {}};
47 }
48 }
49
50 tuple<int, Pd, Pd> circleIntersection(Pd p1, ld r1, Pd p2, ld r2) {
51 ld x1 = p1.x, x2 = p2.x, y1 = p1.y, y2 = p2.y, d = dis(p1, p2);
52 if (sign(abs(r1 - r2) - d) == 1) {
53 return {0, {}, {}};
54 } else if (sign(r1 + r2 - d) == -1) {
55 return {1, {}, {}};
56 }
57 ld a = r1 * (x1 - x2) * 2, b = r1 * (y1 - y2) * 2, c = r2 * r2 - r1 * r1 - d * d;
58 ld p = a * a + b * b, q = -a * c * 2, r = c * c - b * b;
59 ld cosa, sina, cosb, sinb;
60 if (sign(d - (r1 + r2)) == 0 || sign(d - abs(r1 - r2)) == 0) {
61 cosa = -q / p / 2;
62 sina = sqrt(1 - cosa * cosa);
63 Point<ld> p0 = {x1 + r1 * cosa, y1 + r1 * sina};
64 if (sign(dis(p0, p2) - r2)) {

```

```

65 p0.y = y1 - r1 * sina;
66 }
67 return {2, p0, p0};
68 } else {
69 ld delta = sqrt(q * q - p * r * 4);
70 cosa = (delta - q) / p / 2;
71 cosb = (-delta - q) / p / 2;
72 sina = sqrt(1 - cosa * cosa);
73 sinb = sqrt(1 - cosb * cosb);
74 Pd ans1 = {x1 + r1 * cosa, y1 + r1 * sina};
75 Pd ans2 = {x1 + r1 * cosb, y1 + r1 * sinb};
76 if (sign(dis(ans1, p1) - r2)) ans1.y = y1 - r1 * sina;
77 if (sign(dis(ans2, p2) - r2)) ans2.y = y1 - r1 * sinb;
78 if (ans1 == ans2) ans1.y = y1 - r1 * sina;
79 return {3, ans1, ans2};
80 }
81 }
82
83 ld circleIntersectionArea(Pd p1, ld r1, Pd p2, ld r2) {
84 ld x1 = p1.x, x2 = p2.x, y1 = p1.y, y2 = p2.y, d = dis(p1, p2);
85 if (sign(abs(r1 - r2) - d) >= 0) {
86 return PI * min(r1 * r1, r2 * r2);
87 } else if (sign(r1 + r2 - d) == -1) {
88 return 0;
89 }
90 ld theta1 = angle(r1, dis(p1, p2), r2);
91 ld area1 = r1 * r1 * (theta1 - sin(theta1 * 2) / 2);
92 ld theta2 = angle(r2, dis(p1, p2), r1);
93 ld area2 = r2 * r2 * (theta2 - sin(theta2 * 2) / 2);
94 return area1 + area2;
95 }
96
97 tuple<int, Pd, ld> getCircle(Pd A, Pd B, Pd C) {
98 if (onLine(A, B, C)) { // 特判三点共线
99 return {0, {}, 0};
100 }
101 Ld l1 = midSegment(Line{A, B});
102 Ld l2 = midSegment(Line{A, C});
103 Pd O = lineIntersection(l1, l2);
104 return {1, O, dis(A, O)};
105 }
106
107 pair<int, vector<Point<ld>>> tangent(Point<ld> p, Point<ld> A, ld r) {
108 vector<Point<ld>> ans; // 储存切点
109 Point<ld> u = A - p;
110 ld d = sqrt(dot(u, u));
111 if (d < r) {
112 return {0, {}};
113 } else if (sign(d - r) == 0) { // 点在圆上
114 ans.push_back(u);
115 return {1, ans};
116 } else {
117 ld ang = asin(r / d);

```

```

118 ans.push_back(getPoint(A, r, -ang));
119 ans.push_back(getPoint(A, r, ang));
120 return {2, ans};
121 }
122 }
123
124 tuple<int, vector<Point<ld>>, vector<Point<ld>>> tangent(Point<ld> A, ld Ar, Point<
 ld> B, ld Br) {
125 vector<Point<ld>> a, b; // 储存切点
126 if (Ar < Br) {
127 swap(Ar, Br);
128 swap(A, B);
129 swap(a, b);
130 }
131 int d = disEx(A, B), dif = Ar - Br, sum = Ar + Br;
132 if (d < dif * dif) { // 内含, 无
133 return {0, {}, {}};
134 }
135 ld base = atan2(B.y - A.y, B.x - A.x);
136 if (d == 0 && Ar == Br) { // 完全重合, 无数条外公切线
137 return {-1, {}, {}};
138 }
139 if (d == dif * dif) { // 内切, 1条外公切线
140 a.push_back(getPoint(A, Ar, base));
141 b.push_back(getPoint(B, Br, base));
142 return {1, a, b};
143 }
144 ld ang = acos(dif / sqrt(d));
145 a.push_back(getPoint(A, Ar, base + ang)); // 保底2条外公切线
146 a.push_back(getPoint(A, Ar, base - ang));
147 b.push_back(getPoint(B, Br, base + ang));
148 b.push_back(getPoint(B, Br, base - ang));
149 if (d == sum * sum) { // 外切, 多1条内公切线
150 a.push_back(getPoint(A, Ar, base));
151 b.push_back(getPoint(B, Br, base + PI));
152 } else if (d > sum * sum) { // 相离, 多2条内公切线
153 ang = acos(sum / sqrt(d));
154 a.push_back(getPoint(A, Ar, base + ang));
155 a.push_back(getPoint(A, Ar, base - ang));
156 b.push_back(getPoint(B, Br, base + ang + PI));
157 b.push_back(getPoint(B, Br, base - ang + PI));
158 }
159 return {a.size(), a, b};
160 }

```

#### 4.5.11 平面多边形

##### 两向量构成的平面四边形有向面积

##### 判断四个点能否组成矩形/正方形

可以处理浮点数、共点的情况。返回分为三种情况：2 代表构成正方形；1 代表构成矩形；0 代表其他情况。

**点是否在任意多边形内**

射线法判定,  $t$  为穿越次数, 当其为奇数时即代表点在多边形内部; 返回 2 代表点在多边形边界上。

**线段是否在任意多边形内部****任意多边形的面积****皮克定理**

绘制在方格纸上的多边形面积公式可以表示为  $S = n + \frac{s}{2} - 1$ , 其中  $n$  表示多边形内部的点数、 $s$  表示多边形边界上的点数。一条线段上的点数为  $\gcd(|x_1 - x_2|, |y_1 - y_2|) + 1$ 。

**任意多边形上/内的网格点个数 (仅能处理整数)**

皮克定理用。

```

1 template<class T> T areaEx(Point<T> p1, Point<T> p2, Point<T> p3) {
2 return cross(b, c, a);
3 }
4
5 template<class T> int isSquare(vector<Pt> x) {
6 sort(x.begin(), x.end());
7 if (equal(dis(x[0], x[1]), dis(x[2], x[3])) && sign(dis(x[0], x[1])) &&
8 equal(dis(x[0], x[2]), dis(x[1], x[3])) && sign(dis(x[0], x[2])) &&
9 lineParallel(Lt{x[0], x[1]}, Lt{x[2], x[3]}) &&
10 lineParallel(Lt{x[0], x[2]}, Lt{x[1], x[3]}) &&
11 lineVertical(Lt{x[0], x[1]}, Lt{x[0], x[2]})) {
12 return equal(dis(x[0], x[1]), dis(x[0], x[2])) ? 2 : 1;
13 }
14 return 0;
15 }
16
17 template<class T> int pointInPolygon(Point<T> a, vector<Point<T>> p) {
18 int n = p.size();
19 for (int i = 0; i < n; i++) {
20 if (pointOnSegment(a, Line{p[i], p[(i + 1) % n]})) {
21 return 2;
22 }
23 }
24 int t = 0;
25 for (int i = 0; i < n; i++) {
26 auto u = p[i], v = p[(i + 1) % n];
27 if (u.x < a.x && v.x >= a.x && pointOnLineLeft(a, Line{v, u})) {
28 t ^= 1;
29 }
30 if (u.x >= a.x && v.x < a.x && pointOnLineLeft(a, Line{u, v})) {
31 t ^= 1;
32 }
33 }
34 return t == 1;
35 }
36
37 template<class T>
38 bool segmentInPolygon(Line<T> l, vector<Point<T>> p) {
39 // 线段与多边形边界不相交且两端点都在多边形内部

```

```

40 #define L(x, y) pointOnLineLeft(x, y)
41 int n = p.size();
42 if (!pointInPolygon(l.a, p)) return false;
43 if (!pointInPolygon(l.b, p)) return false;
44 for (int i = 0; i < n; i++) {
45 auto u = p[i];
46 auto v = p[(i + 1) % n];
47 auto w = p[(i + 2) % n];
48 auto [t, p1, p2] = segmentIntersection(l, Line(u, v));
49 if (t == 1) return false;
50 if (t == 0) continue;
51 if (t == 2) {
52 if (pointOnSegment(v, l) && v != l.a && v != l.b) {
53 if (cross(v - u, w - v) > 0) {
54 return false;
55 }
56 }
57 } else {
58 if (p1 != u && p1 != v) {
59 if (L(l.a, Line(v, u)) || L(l.b, Line(v, u))) {
60 return false;
61 }
62 } else if (p1 == v) {
63 if (l.a == v) {
64 if (L(u, l)) {
65 if (L(w, l) && L(w, Line(u, v))) {
66 return false;
67 }
68 } else {
69 if (L(w, l) || L(w, Line(u, v))) {
70 return false;
71 }
72 }
73 } else if (l.b == v) {
74 if (L(u, Line(l.b, l.a))) {
75 if (L(w, Line(l.b, l.a)) && L(w, Line(u, v))) {
76 return false;
77 }
78 } else {
79 if (L(w, Line(l.b, l.a)) || L(w, Line(u, v))) {
80 return false;
81 }
82 }
83 } else {
84 if (L(u, l)) {
85 if (L(w, Line(l.b, l.a)) || L(w, Line(u, v))) {
86 return false;
87 }
88 } else {
89 if (L(w, l) || L(w, Line(u, v))) {
90 return false;
91 }
92 }
93 }
94 }
95 }
96 }
97 }

```

```

93 }
94 }
95 }
96 }
97 return true;
98 }
99
100 template<class T> ld area(vector<Point<T>> P) {
101 int n = P.size();
102 ld ans = 0;
103 for (int i = 0; i < n; i++) {
104 ans += cross(P[i], P[(i + 1) % n]);
105 }
106 return ans / 2.0;
107 }
108
109 int onPolygonGrid(vector<Point<int>> p) { // 多边形上
110 int n = p.size(), ans = 0;
111 for (int i = 0; i < n; i++) {
112 auto a = p[i], b = p[(i + 1) % n];
113 ans += gcd(abs(a.x - b.x), abs(a.y - b.y));
114 }
115 return ans;
116 }
117 int inPolygonGrid(vector<Point<int>> p) { // 多边形内
118 int n = p.size(), ans = 0;
119 for (int i = 0; i < n; i++) {
120 auto a = p[i], b = p[(i + 1) % n], c = p[(i + 2) % n];
121 ans += b.y * (a.x - c.x);
122 }
123 ans = abs(ans);
124 return (ans - onPolygonGrid(p)) / 2 + 1;
125 }

```

#### 4.5.12 平面点线相关

点是否在直线上（三点是否共线）

点是否在向量（直线）左侧

需要注意，向量的方向会影响答案；点在向量上不视为在左侧。

两点是否在直线同侧/异侧

两直线相交交点

在使用前需要先判断直线是否平行。

两直线是否平行/垂直/相同

点到直线的最近距离与最近点

如果只需要计算最近距离，下方的写法可以减少书写的代码量，效果一致。

点是否在线段上

点到线段的最近距离与最近点

点在直线上的投影点（垂足）

## 线段的中垂线

### 两线段是否相交及交点

该扩展版可以同时返回相交状态和交点，分为四种情况：0 代表不相交；1 代表普通相交；2 代表重叠（交于两个点）；3 代表相交于端点。**需要注意**，部分运算可能会使用到直线求交点，此时务必保证变量类型为浮点数！

如果不需要求交点，那么使用快速排斥 + 跨立实验即可，其中重叠、相交于端点均视为相交。

```

1 template<class T> bool onLine(Point<T> a, Point<T> b, Point<T> c) {
2 return sign(cross(b, a, c)) == 0;
3 }
4 template<class T> bool onLine(Point<T> p, Line<T> l) {
5 return onLine(p, l.a, l.b);
6 }
7
8 template<class T> bool pointOnLineLeft(Pt p, Lt l) {
9 return cross(l.b, p, l.a) > 0;
10 }
11
12 template<class T> bool pointOnLineSide(Pt p1, Pt p2, Lt vec) {
13 T val = cross(p1, vec.a, vec.b) * cross(p2, vec.a, vec.b);
14 return sign(val) == 1;
15 }
16 template<class T> bool pointNotOnLineSide(Pt p1, Pt p2, Lt vec) {
17 T val = cross(p1, vec.a, vec.b) * cross(p2, vec.a, vec.b);
18 return sign(val) == -1;
19 }
20
21 Pd lineIntersection(Ld l1, Ld l2) {
22 Ld val = cross(l2.b - l2.a, l1.a - l2.a) / cross(l2.b - l2.a, l1.a - l1.b);
23 return l1.a + (l1.b - l1.a) * val;
24 }
25
26 template<class T> bool lineParallel(Lt p1, Lt p2) {
27 return sign(cross(p1.a - p1.b, p2.a - p2.b)) == 0;
28 }
29 template<class T> bool lineVertical(Lt p1, Lt p2) {
30 return sign(dot(p1.a - p1.b, p2.a - p2.b)) == 0;
31 }
32 template<class T> bool same(Line<T> l1, Line<T> l2) {
33 return lineParallel(Line{l1.a, l2.b}, {l1.b, l2.a}) &&
34 lineParallel(Line{l1.a, l2.a}, {l1.b, l2.b}) && lineParallel(l1, l2);
35 }
36
37 pair<Pd, Ld> pointToLine(Pd p, Ld l) {
38 Pd ans = lineIntersection({p, p + rotate(l.a, l.b)}, l);
39 return {ans, dis(p, ans)};
40 }
41
42 template<class T> Ld disPointToLine(Pt p, Lt l) {
43 Ld ans = cross(p, l.a, l.b);

```

```

44 return abs(ans) / dis(l.a, l.b); // 面积除以底边长
45 }
46
47 template<class T> bool pointOnSegment(Pt p, Lt l) { // 端点也算作在直线上
48 return sign(cross(p, l.a, l.b)) == 0 && min(l.a.x, l.b.x) <= p.x && p.x <= max(l
 .a.x, l.b.x) &&
49 min(l.a.y, l.b.y) <= p.y && p.y <= max(l.a.y, l.b.y);
50 }
51 template<class T> bool pointOnSegment(Pt p, Lt l) { // 端点不算
52 return pointOnSegment(p, l) && min(l.a.x, l.b.x) < p.x && p.x < max(l.a.x, l.b.x
) &&
53 min(l.a.y, l.b.y) < p.y && p.y < max(l.a.y, l.b.y);
54 }
55
56 pair<Pd, ld> pointToSegment(Pd p, Ld l) {
57 if (sign(dot(p, l.b, l.a)) == -1) { // 特判到两端点的距离
58 return {l.a, dis(p, l.a)};
59 } else if (sign(dot(p, l.a, l.b)) == -1) {
60 return {l.b, dis(p, l.b)};
61 }
62 return pointToLine(p, l);
63 }
64
65 Pd project(Pd p, Ld l) { // 投影
66 Pd vec = l.b - l.a;
67 ld r = dot(vec, p - l.a) / (vec.x * vec.x + vec.y * vec.y);
68 return l.a + vec * r;
69 }
70
71 template<class T> Lt midSegment(Lt l) {
72 Pt mid = (l.a + l.b) / 2; // 线段中点
73 return {mid, mid + rotate(l.a, l.b)};
74 }
75
76 template<class T> tuple<int, Pt, Pt> segmentIntersection(Lt l1, Lt l2) {
77 auto [s1, e1] = l1;
78 auto [s2, e2] = l2;
79 auto A = max(s1.x, e1.x), AA = min(s1.x, e1.x);
80 auto B = max(s1.y, e1.y), BB = min(s1.y, e1.y);
81 auto C = max(s2.x, e2.x), CC = min(s2.x, e2.x);
82 auto D = max(s2.y, e2.y), DD = min(s2.y, e2.y);
83 if (A < CC || C < AA || B < DD || D < BB) {
84 return {0, {}, {}};
85 }
86 if (sign(cross(e1 - s1, e2 - s2)) == 0) {
87 if (sign(cross(s2, e1, s1)) != 0) {
88 return {0, {}, {}};
89 }
90 Pt p1(max(AA, CC), max(BB, DD));
91 Pt p2(min(A, C), min(B, D));
92 if (!pointOnSegment(p1, l1)) {
93 swap(p1.y, p2.y);
94 }

```



```

95 if (p1 == p2) {
96 return {3, p1, p2};
97 } else {
98 return {2, p1, p2};
99 }
100 }
101 auto cp1 = cross(s2 - s1, e2 - s1);
102 auto cp2 = cross(s2 - e1, e2 - e1);
103 auto cp3 = cross(s1 - s2, e1 - s2);
104 auto cp4 = cross(s1 - e2, e1 - e2);
105 if (sign(cp1 * cp2) == 1 || sign(cp3 * cp4) == 1) {
106 return {0, {}, {}};
107 }
108 // 使用下方函数时请使用浮点数
109 Pd p = lineIntersection(l1, l2);
110 if (sign(cp1) != 0 && sign(cp2) != 0 && sign(cp3) != 0 && sign(cp4) != 0) {
111 return {1, p, p};
112 } else {
113 return {3, p, p};
114 }
115 }
116
117 template<class T> bool segmentIntersection(Lt l1, Lt l2) {
118 auto [s1, e1] = l1;
119 auto [s2, e2] = l2;
120 auto A = max(s1.x, e1.x), AA = min(s1.x, e1.x);
121 auto B = max(s1.y, e1.y), BB = min(s1.y, e1.y);
122 auto C = max(s2.x, e2.x), CC = min(s2.x, e2.x);
123 auto D = max(s2.y, e2.y), DD = min(s2.y, e2.y);
124 return A >= CC && B >= DD && C >= AA && D >= BB &&
125 sign(cross(s1, s2, e1) * cross(s1, e1, e2)) == 1 &&
126 sign(cross(s2, s1, e2) * cross(s2, e2, e1)) == 1;
127 }

```

#### 4.5.13 平面直线方程转换

##### 浮点数计算直线的斜率

一般很少使用到这个函数，因为斜率的取值不可控（例如接近平行于  $x, y$  轴时）。**需要注意**，当直线平行于  $y$  轴时斜率为 `inf`。

##### 分数精确计算直线的斜率

调用分数四则运算精确计算斜率，返回最简分数，只适用于整数计算。

##### 两点式转一般式

返回由三个整数构成的方程，在输入较大时可能找不到较小的满足题意的一组整数解。可以处理平行于  $x, y$  轴、两点共点的情况。

##### 一般式转两点式

由于整数点可能很大或者不存在，故直接采用浮点数；如果与  $x, y$  轴有交点则取交点。可以处理平行于  $x, y$  轴的情况。

##### 抛物线与 $x$ 轴是否相交及交点

0 代表没有交点；1 代表相切；2 代表有两个交点。

```

1 template <class T> ld slope(Pt p1, Pt p2) { // 斜率, 注意 inf 的情况
2 return (p1.y - p2.y) / (p1.x - p2.x);
3 }
4 template <class T> ld slope(Lt l) {
5 return slope(l.a, l.b);
6 }
7
8 template<class T> Frac<T> slopeEx(Pt p1, Pt p2) {
9 Frac<T> U = p1.y - p2.y;
10 Frac<T> V = p1.x - p2.x;
11 return U / V; // 调用分数精确计算
12 }
13
14 template<class T> tuple<T, T, T> getfun(Lt p) {
15 T A = p.a.y - p.b.y, B = p.b.x - p.a.x, C = p.a.x * A + p.a.y * B;
16 if (A < 0) { // 符号调整
17 A = -A, B = -B, C = -C;
18 } else if (A == 0) {
19 if (B < 0) {
20 B = -B, C = -C;
21 } else if (B == 0 && C < 0) {
22 C = -C;
23 }
24 }
25 if (A == 0) { // 数值计算
26 if (B == 0) {
27 C = 0; // 共点特判
28 } else {
29 T g = fgcd(abs(B), abs(C));
30 B /= g, C /= g;
31 }
32 } else if (B == 0) {
33 T g = fgcd(abs(A), abs(C));
34 A /= g, C /= g;
35 } else {
36 T g = fgcd(fgcd(abs(A), abs(B)), abs(C));
37 A /= g, B /= g, C /= g;
38 }
39 return tuple{A, B, C}; // Ax + By = C
40 }
41
42 Line<ld> getfun(int A, int B, int C) { // Ax + By = C
43 ld x1 = 0, y1 = 0, x2 = 0, y2 = 0;
44 if (A && B) { // 正常
45 if (C) {
46 x1 = 0, y1 = 1. * C / B;
47 y2 = 0, x2 = 1. * C / A;
48 } else { // 过原点
49 x1 = 1, y1 = 1. * -A / B;
50 x2 = 0, y2 = 0;
51 }

```

```

52 } else if (A && !B) { // 垂直
53 if (C) {
54 y1 = 0, x1 = 1. * C / A;
55 y2 = 1, x2 = x1;
56 } else {
57 x1 = 0, y1 = 1;
58 x2 = 0, y2 = 0;
59 }
60 } else if (!A && B) { // 水平
61 if (C) {
62 x1 = 0, y1 = 1. * C / B;
63 x2 = 1, y2 = y1;
64 } else {
65 x1 = 1, y1 = 0;
66 x2 = 0, y2 = 0;
67 }
68 } else { // 不合法, 请特判
69 assert(false);
70 }
71 return {{x1, y1}, {x2, y2}};
72 }
73
74 tuple<int, ld, ld> getAns(ld a, ld b, ld c) {
75 ld delta = b * b - a * c * 4;
76 if (delta < 0.) {
77 return {0, 0, 0};
78 }
79 delta = sqrt(delta);
80 ld ans1 = -(delta + b) / 2 / a;
81 ld ans2 = (delta - b) / 2 / a;
82 if (ans1 > ans2) {
83 swap(ans1, ans2);
84 }
85 if (sign(delta) == 0) {
86 return {1, ans2, 0};
87 }
88 return {2, ans1, ans2};
89 }

```

#### 4.5.14 平面角度与弧度

##### 弧度角度相互转换

##### 正弦定理

$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R$ , 其中  $R$  为三角形外接圆半径;

##### 余弦定理 (已知三角形三边, 求角)

$\cos C = \frac{a^2 + b^2 - c^2}{2ab}$ ,  $\cos B = \frac{a^2 + c^2 - b^2}{2ac}$ ,  $\cos A = \frac{b^2 + c^2 - a^2}{2bc}$ 。可以借此推导出三角形面积公式  $S_{\triangle ABC} = \frac{ab \cdot \sin C}{2} = \frac{bc \cdot \sin A}{2} = \frac{ac \cdot \sin B}{2}$ 。

注意, 计算格式是: 由  $b, c, a$  三边求  $\angle A$ ; 由  $a, c, b$  三边求  $\angle B$ ; 由  $a, b, c$  三边求  $\angle C$ 。

##### 求两向量的夹角

能够计算  $[0^\circ, 180^\circ]$  区间的角度。

#### 向量旋转任意角度

逆时针旋转, 转换公式: 
$$\begin{cases} x' = x \cos \theta - y \sin \theta \\ y' = x \sin \theta + y \cos \theta \end{cases}$$

#### 点绕点旋转任意角度

逆时针旋转, 转换公式: 
$$\begin{cases} x' = (x_0 - x_1) \cos \theta + (y_0 - y_1) \sin \theta + x_1 \\ y' = (x_1 - x_0) \sin \theta + (y_0 - y_1) \cos \theta + y_1 \end{cases}$$

```

1 ld toDeg(ld x) { // 弧度转角度
2 return x * 180 / PI;
3 }
4 ld toArc(ld x) { // 角度转弧度
5 return PI / 180 * x;
6 }
7
8 ld angle(ld a, ld b, ld c) { // 余弦定理
9 ld val = acos((a * a + b * b - c * c) / (2.0 * a * b)); // 计算弧度
10 return val;
11 }
12
13 ld angle(Point<ld> a, Point<ld> b) {
14 ld val = abs(cross(a, b));
15 return abs(atan2(val, a.x * b.x + a.y * b.y));
16 }
17
18 Point<ld> rotate(Point<ld> p, ld rad) {
19 return {p.x * cos(rad) - p.y * sin(rad), p.x * sin(rad) + p.y * cos(rad)};
20 }
21
22 Point<ld> rotate(Point<ld> a, Point<ld> b, ld rad) {
23 ld x = (a.x - b.x) * cos(rad) + (a.y - b.y) * sin(rad) + b.x;
24 ld y = (b.x - a.x) * sin(rad) + (a.y - b.y) * cos(rad) + b.y;
25 return {x, y};
26 }

```

#### 4.5.15 库实数类实现 (双精度)

```

1 using Real = int;
2 using Point = complex<Real>;
3
4 Real cross(const Point &a, const Point &b) {
5 return (conj(a) * b).imag();
6 }
7 Real dot(const Point &a, const Point &b) {
8 return (conj(a) * b).real();
9 }

```

#### 4.5.16 空间多边形

##### 正 N 棱锥体积公式

棱锥通用体积公式  $V = \frac{1}{3}Sh$ ，当其恰好是棱长为  $l$  的正  $n$  棱锥时，有公式  $V = \frac{l^3 \cdot n}{12 \tan \frac{\pi}{n}} \cdot \sqrt{1 - \frac{1}{4 \cdot \sin^2 \frac{\pi}{n}}}$ 。

#### 四面体体积

##### 点是否在空间三角形上

点位于边界上时返回 *false*。

##### 线段是否与空间三角形相交及交点

只有交点在空间三角形内部时才视作相交。

##### 空间三角形是否相交

相交线段在空间三角形内部时才视作相交。

```

1 ld V(ld l, int n) { // 正n棱锥体积公式
2 return l * l * l * n / (12 * tan(PI / n)) * sqrt(1 - 1 / (4 * sin(PI / n) * sin(
 PI / n)));
3 }
4
5 ld V(P3 a, P3 b, P3 c, P3 d) {
6 return abs(dot(d - a, crossEx(b - a, c - a))) / 6;
7 }
8
9 bool pointOnTriangle(P3 p, P3 p1, P3 p2, P3 p3) {
10 return pointOnSegmentSide(p, p1, {p2, p3}) && pointOnSegmentSide(p, p2, {p1, p3
 }) &&
11 pointOnSegmentSide(p, p3, {p1, p2});
12 }
13
14 pair<bool, P3> segmentOnTriangle(P3 l, P3 r, P3 p1, P3 p2, P3 p3) {
15 P3 x = crossEx(p2 - p1, p3 - p1);
16 if (sign(dot(x, r - l)) == 0) {
17 return {0, {}};
18 }
19 ld t = dot(x, p1 - l) / dot(x, r - l);
20 if (t < 0 || t - 1 > 0) { // 不在线段上
21 return {0, {}};
22 }
23 bool type = pointOnTriangle(l + (r - l) * t, p1, p2, p3);
24 if (type) {
25 return {1, l + (r - l) * t};
26 } else {
27 return {0, {}};
28 }
29 }
30
31 bool triangleIntersection(vector<P3> a, vector<P3> b) {
32 for (int i = 0; i < 3; i++) {
33 if (segmentOnTriangle(b[i], b[(i + 1) % 3], a[0], a[1], a[2]).first) {
34 return 1;
35 }
36 if (segmentOnTriangle(a[i], a[(i + 1) % 3], b[0], b[1], b[2]).first) {
37 return 1;

```

```

38 }
39 }
40 return 0;
41 }

```

## 4.6 frequently use

### 4.6.1 pbds

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 /*
4 使用pbds一定不要
5 开define int long long 否则会ce
6 */
7 // #include<bits/extc++.h> pbds万能头 一般来说只能在Ubuntu用
8 int n, m; string s;
9 #define pd __gnu_pbds
10 #include<ext/pb_ds/assoc_container.hpp> //前置头文件
11
12 // 1.比umap快三倍的map, 而且很难被hack
13 #include<ext/pb_ds/hash_policy.hpp>
14 pd::cc_hash_table<string, int> mp1;
15 pd::gp_hash_table<string, int> mp;
16 // 哈希方式不同, gp更快
17 // 这两个都没有 count() 函数 用 find(x) != end()
18 void pbds1(){
19 cin >> n >> m;
20 while(n--){
21 cin >> s;
22 mp[s]++;
23 }
24 while(m--){
25 cin >> s;
26 if(mp.find(s) != mp.end()){
27 cout << mp[s] << endl;
28 } else {
29 cout << "No find!\n";
30 }
31 }
32 }
33 // 2.平衡树
34 #include <ext/pb_ds/tree_policy.hpp>
35 pd::tree<pair<int, int>, // 相当于map<T1, T2>/set<T1>中的T1
36 pd::null_type, // T2类型, 如果写null_type 类似set 反之相当于map中的T2
37 less<pair<int, int>>, // less<T> 就是从小到大排, greater反之
38 pd::rb_tree_tag, pd::tree_order_statistics_node_update>
39 Tree, Tree2;
40 void pbds2(){
41 if(0){
42 int x, y;
43 Tree.insert({x, y}); // 插入;

```

```

44 Tree.erase({x,y}); //删除;
45 Tree.order_of_key({x,y}); //求排名,严格小于{x,y}的元素个数
46 Tree.find_by_order(x); //找x小值,返回迭代器
47 Tree.join(Tree2); //将b并入Tree2,两棵树必须没有重复元素,不建议用这个,不如遍历插入
48 Tree.split({x,y},Tree2); //分裂, key小于等于v的元素属于tr, 其余的属于b
49 Tree.lower_bound({x,y}); //返回第一个大于等于x的元素的迭代器
50 Tree.upper_bound({x,y}); //返回第一个大于x的元素的迭代器
51 }
52 //以上所有操作的时间复杂度均为O(logn)
53 /* 例题1
54 给你一个整数数组 nums , 按要求编写程序输出一个新数组 counts
55 数组 counts 有该性质: counts[i] 的值是 nums[i] 右侧小于 nums[i] 的元素的个数。
56 */
57 cin >> n;
58 vector<int>a(n),res(n);
59 for(auto &x:a) cin >> x;
60 int id = 0;
61 for(int i=n-1;~i;i--){
62 res[i]=Tree.order_of_key({a[i],-1});
63 Tree.insert({a[i],id++});
64 }
65 for(auto x:res) cout<<x<<" ";
66 /* 例题1 模板 普通平衡树 P6136
67 您需要动态地维护一个可重集合 M, 并且提供以下操作:
68 1.向 M 中插入一个数 x。
69 2.从 M 中删除一个数 x (若有多个相同的数,应只删除一个)。
70 3.查询 M 中有多少个数比 x 小,并且将得到的答案加一。
71 4.查询如果将 M 从小到大排列后,排名位于第 x 位的数。
72 5.查询 M 中 x 的前驱(前驱定义为小于 x,且最大的数)。
73 6.查询 M 中 x 的后继(后继定义为大于 x,且最小的数)。
74 本题强制在线,保证所有操作合法(操作 2 保证存在至少一个 x,操作 4,5,6 保证存在答案)。
75 输出一行一个整数,表示所有 3,4,5,6 操作的答案的异或和。
76 */
77 cin >> n >> m;
78 for(int i=1,a;i<=n;++i){
79 cin>>a;Tree.insert({a,-i});
80 }int last = 0,ans = 0;
81 for(int i=1,op,x;i<=m;++i){
82 cin>>op>>x; x^=last;
83 if(op==1)Tree.insert({x,i});
84 else if(op==2){
85 auto p = *Tree.lower_bound(make_pair(x,-n));
86 Tree.erase(p);
87 }else if(op==3)
88 last=Tree.order_of_key({x,-n})+1;
89 else if(op==4){
90 auto [u,v] = *Tree.find_by_order(x-1);
91 last=u;
92 }else if(op==5){
93 auto [u,v] = *(--Tree.lower_bound({x,-2*n}));
94 last = u;
95 }else{

```

```

96 auto [u,v] = *(Tree.upper_bound({x,i+1}));
97 last = u;
98 }
99 if(op>2)ans^=last;
100 }
101 cout<<ans<<'\n';
102 }
103 // .rope 可以平替主席树, 可持久化并查集等可持久化操作
104 // 严格来说rope并不属于pbds
105
106 #include <ext/rope>
107 using namespace __gnu_cxx;
108 void pbds3(){
109 rope<char>a;
110 crope b;//等价于rope<char>b
111 char s[10];
112 int p,x;
113 a.push_back(x);//在a的末尾添加字符 c;
114 a.insert(p,x);//在 a 的下标 p 后面添加 x;
115 a.insert(p,s,n);//将字符串 s 的前 n 位插入 a 的下标 p 处;
116 a.erase(p,x);//从 a 的下标 p 开始删除 x 个元素;
117 a.replace(p,s);//从 a 的下标 p 开始换成字符串 s, 若 a 的位数不够则补足;
118 a.copy(p,n,s);//从 a 的下标 p 开始的 n 个字符换成字符串 s, 若位数不够则补足;
119 a.substr(p,x);//从 a 的下标 p 开始截取 x 个元素;
120 //a[x]访问下标为 x 的元素;
121 a.append(s,p);/*
122 把字符串 s 中下标 p 后的所有字符连接到 a 的结尾, 如参数 p 也没有则把整个字符串 s 连接到
 a 的结尾;
123 以上, s 均为char[]类型, c 为字符(char)类型, n, p, x 均为整型。*/
124 // push_back 01 其余操作0n 但是常数较大 可以认为是n^1/2
125
126 /**例题, 洛谷P1383
127 早苗入手了最新的高级打字机。最新款自然有着与以往不同的功能, 那就是它具备撤销功能, 厉害
 吧。
128 请为这种高级打字机设计一个程序, 支持如下 3 种操作:
129 T x: Type 操作, 表示在文章末尾打下一个小写字母 x。
130 U x: Undo 操作, 表示撤销最后的 x 次修改操作。
131 Q x: Query 操作, 表示询问当前文章中第 x 个字母并输出。
132 */
133 cin >> n;
134 rope<char> *v[100005];//固定写法, 使用指针
135 p = 0;
136 v[0] = new rope<char>();
137 for(int i=0,x;i<n;i++){
138 char op,ch;
139 cin >> op;
140 if(op=='T'){
141 cin >> ch;
142 p++;
143 v[p] = new rope<char>(*v[p-1]);//01操作, 可持久化的核心
144 v[p]->push_back(ch);
145 }else if(op=='U'){
146 cin >> x;

```



```

147 p++;
148 v[p]=new rope<char>(*v[p-x-1]);
149 }else{
150 cin >> x;
151 cout << v[p]->at(x-1) << '\n';
152 }
153 }
154 }
155 signed main() {
156 pbds3();
157 return 0;
158 }

```

#### 4.6.2 重载小于号

```

1 bool operator<(const Data & x) const {
2 return b < x.b || (b == x.b && id > x.id);
3 }
4 //优先队列+结构体
5 #include<iostream>
6 #include<queue>
7 using namespace std;
8
9 struct Data {
10 int id, b;
11 bool operator<(const Data & x) const {
12 return b < x.b || (b == x.b && id > x.id);
13 }
14 };
15
16 int main() {
17 priority_queue<Data> que[4];
18 string event;
19 int n, a, b, i;
20 while (cin >> n) {
21 getchar(); // 读取换行符, 防止输入流中残留的换行符影响下一次输入
22 int id = 0;
23 for (i = 1; i <= 3; i++) que[i] = priority_queue<Data>(); // 清空所有队列
24 for (i = 1; i <= n; i++) {
25 cin >> event;
26 if (event == "IN") {
27 cin >> a >> b;
28 id++;
29 que[a].push({id, b}); // 根据事件类型将数据加入对应队列
30 } else {
31 cin >> a;
32 if (que[a].empty()) cout << "EMPTY" << endl;
33 else {
34 cout << que[a].top().id << endl; // 输出队首元素的id
35 que[a].pop(); // 移除队首元素
36 }
37 }

```

```

38 }
39 }
40 return 0;
41 }

```

#### 4.6.3 优先队列

```

1 priority_queue<int> q1; // 默认大根堆，即每次取出的元素是队列中的最大值
2
3 priority_queue<int, vector<int>, greater<int> > q3; // 小根堆，每次取出的元素是队列中的
 最小值
4
5 //结构体里定义排序规则
6 struct node {
7 int x, y;
8 bool operator < (const Point &a) const { //直接传入一个参数，不必要写friend
9 return x < a.x; //按x升序排列，x大的在堆顶
10 }
11 };

```

#### 4.6.4 set

```

1 //迭代器访问
2 for(set<int>::iterator it = s.begin(); it != s.end(); it++)
3 cout << *it << " ";
4
5 //智能指针
6 for(auto i : s) cout << i << endl;
7
8 set<int> s1; // 默认从小到大排序
9 set<int, greater<int> > s2; // 从大到小排序

```

#### 4.6.5 map

```

1 //迭代器（可双向）
2 map<int,int> mp;
3 mp[1] = 2;
4 mp[2] = 3;
5 mp[3] = 4;
6 auto it = mp.begin();
7 while(it != mp.end()) {
8 cout << it->first << " " << it->second << "\n";
9 it++;
10 }
11
12 //方式一：迭代器访问
13 map<string,string>::iterator it;
14 for(it = mp.begin(); it != mp.end(); it++) {
15 // 键 值
16 // it是结构体指针访问所以要用 -> 访问

```

```

17 cout << it->first << " " << it->second << "\n";
18 //*it是结构体变量 访问要用 . 访问
19 //cout<<(*it).first<<" "<<(*it).second;
20 }
21
22 //方式二：智能指针访问
23 for(auto i : mp)
24 cout << i.first << " " << i.second << endl;//键，值
25
26 //二分查找：返回迭代器
27 #include<bits/stdc++.h>
28 using namespace std;
29
30 int main() {
31 map<int, int> m{{1, 2}, {2, 2}, {1, 2}, {8, 2}, {6, 2}};//有序
32 map<int, int>::iterator it1 = m.lower_bound(2);
33 cout << it1->first << "\n";//it1->first=2
34 map<int, int>::iterator it2 = m.upper_bound(2);
35 cout << it2->first << "\n";//it2->first=6
36 return 0;
37 }

```

#### 4.6.6 others

```

1 //找到a,b,c,d的最大值和最小值
2 mx = max({a, b, c, d});
3 mn = min({a, b, c, d});
4
5 //1. 在a数组中查找第一个大于等于x的元素，返回该元素的地址
6 lower_bound(a, a + n, x);
7 //2. 在a数组中查找第一个大于x的元素，返回该元素的地址
8 upper_bound(a, a + n, x);
9 (!)如果未找到，返回尾地址的下一个位置的地址
10
11 bitset:
12 代码 含义
13 b.any() b中是否存在置为1的二进制位，有 返回true
14 b.none() b中是否没有1，没有 返回true
15 b.count() b中为1的个数
16 b.size() b中二进制位的个数
17 b[pos] 返回b在pos处的二进制位
18 b.set() 把b中所有位都置为1
19 b.set(pos) 把b中pos位置置为1
20 b.reset() 把b中所有位都置为0
21 b.reset(pos) 把b中pos位置置为0
22 b.flip() 把b中所有二进制位取反
23 b.flip(pos) 把b中pos位置取反
24 b.to_ulong() 用b中同样的二进制位返回一个unsigned long值
25 *b.find_first()/_Find_first() *
26 用于查找 bitset 中第一个设置为 1 的位的索引。
27 如果 bitset 中所有位都为 0，则返回 bitset 的大小
28 *b.find_next(pos) / _Find_next(pos)*

```

```

29 用于查找 bitset 中从指定位置 pos 开始 (不包含 pos 位置) 第一个设置为 1 的位的索引。
30 如果从指定位置 pos 开始之后的所有位都为 0, 则返回 bitset 的大小。
31
32 string:
33 s.replace (pos,n,str) 把当前字符串从索引pos开始的n个字符替换为str
34 s.replace (pos,n,n1,c) 把当前字符串从索引pos开始的n个字符替换为n1个字符c
35 s.replace (it1,it2,str) 把当前字符串 [it1,it2) 区间替换为str
36
37 to_string
38 将数字转化为字符串, 支持小数 (double)
39 int a = 12345678;
40 cout << to_string(a) << '\n';
41
42 __builtin_ 内置位运算函数
43 需要注意:
44 内置函数有相应的unsigned int和unsigned long long版本,
45 unsigned long long只需要在函数名后面加上ll就可以了,
46 比如__builtin_clzll(x), 默认是32位unsigned int
47
48 很多题目和 long long 数据类型有关, 如有需要注意添加 ll
49
50 popcount
51 1 的个数
52
53 ctz
54 count tail zero末尾0的个数
55
56 clz
57 count leading zero
58
59 ffs
60 最后一位1 的位数
61 4(100) 返回3
62
63 * next_permutation *
64 // 数组a不一定是字典序序列, 一定注意将它排序
65 sort(a, a + n);
66 do {
67 for(int i = 0; i < n; i++)
68 printf("%d ", a[i]);
69 } while(next_permutation(a, a + n));
70
71 * nth_element 无返回值 *
72 nth_element(beg, nth, end)
73
74 // 找到第 4 小的元素 (索引从 0 开始)
75 int* nth = arr + 3;
76 // 使用 nth_element 进行部分排序
77 std::nth_element(arr, nth, arr + size);
78
79 // 找到第 3 小的元素 (索引从 0 开始)
80 auto nth = numbers.begin() + 2;
81 std::nth_element(numbers.begin(), nth, numbers.end());

```

## 4.6.7 二分

```

1 // 检查x是否满足某种性质
2 bool check(int x) { /* ... */ }
3
4 // 二分查找模板1: 寻找第一个满足条件的元素 (左边界)
5 // 适用于将区间[l, r]划分为[l, mid]和[mid + 1, r]的情况
6 // 如果存在满足条件的元素, 返回最左边满足条件的元素下标 : XXXVVVV
7 // 如果所有元素都不满足条件, 返回初始右边界+1 (即越界位置)
8 int binary_search_left(int l, int r)
9 {
10 while (l < r)
11 {
12 int mid = l + r >> 1; // 向下取整
13 if (check(mid)) r = mid; // 检查mid是否满足条件, 满足则搜索左半部分
14 else l = mid + 1; // 不满足则搜索右半部分
15 }
16 return l; // 最终l和r相等, 返回任意一个
17 }
18
19 // 二分查找模板2: 寻找最后一个满足条件的元素 (右边界)
20 // 适用于将区间[l, r]划分为[l, mid - 1]和[mid, r]的情况
21 // 注意: 计算mid时需要加1防止死循环
22 // 如果存在满足条件的元素, 返回最右边满足条件的元素下标: : VVVVXXX
23 // 如果所有元素都不满足条件, 返回初始左边界-1 (即越界位置)
24 int binary_search_right(int l, int r)
25 {
26 while (l < r)
27 {
28 int mid = l + r + 1 >> 1; // 向上取整, 防止死循环
29 if (check(mid)) l = mid; // 检查mid是否满足条件, 满足则搜索右半部分
30 else r = mid - 1; // 不满足则搜索左半部分
31 }
32 return l; // 最终l和r相等, 返回任意一个
33 }

```

## 4.6.8 print 相关: 快读 and int128 读写等

```

1 int read() {
2 int x = 0, w = 1;
3 char ch = 0;
4 while (ch < '0' || ch > '9') {
5 if (ch == '-') w = -1;
6 ch = getchar();
7 }
8 while (ch >= '0' && ch <= '9') {
9 x = x * 10 + (ch - '0');
10 ch = getchar();
11 }
12 return x * w;
13 }
14

```

```

15 void read128(__int128 &x) {
16 x = 0;
17 bool f = 0;
18 char c = getchar();
19 while (!isdigit(c)) {
20 if (c == '-') f = 1;
21 c = getchar();
22 }
23 while (isdigit(c)) {
24 x = x * 10 + (c - '0');
25 c = getchar();
26 }
27 if (f) x = -x;
28 }
29
30 void print128(__int128 x) {
31 if (x < 0) {
32 putchar('-');
33 x = -x;
34 }
35 if (x > 9) print(x / 10);
36 putchar(x % 10 + '0');
37 }
38 // -----
39 // 去除前导 0 (至少留一个, 0不能输出空)
40 // -----
41 int p = 0;
42 while (p + 1 < (int)ans.size() && ans[p] == '0') p++;
43 cout << ans.substr(p);
44 // -----
45 // 输出二进制
46 // -----
47 for(int i=23;i>=0;--i){
48 putchar(48|(mask>>i&1));
49 }cout<<endl;

```

#### 4.6.9 all you need

```

1 // -----
2 // ccm逸闻
3 // -----
4 #include <bits/stdc++.h>
5 using namespace std;
6 #define int long long
7 typedef pair<int,int> pii;
8 typedef long long ll;
9 typedef unsigned long long ull;
10 #define endl '\n'
11
12 bool isValidDate(int x) {
13 int daysInMonth[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
14 int y = x / 10000; // 年份: 最后4位

```

```

15 int m = (x / 100) % 100; // 月份：中间两位
16 int d = x % 100; // 日：前两位
17
18 // 判断月份和天数基本合法性
19 if(m < 1 || m > 12) return false;
20 if(d < 1) return false;
21
22 // 判断闰年：满足(能被4整除且不能被100整除)或能被400整除
23 bool leap = (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
24
25 // 二月天数考虑闰年
26 if(m == 2) {
27 if(d > daysInMonth[m] + (leap ? 1 : 0)) return false;
28 }
29 else {
30 if(d > daysInMonth[m]) return false;
31 }
32 return true;
33 }
34
35 signed main(){
36 int ans = 0;
37 for (int i = 20250329; i <= 21160101; i++){
38 if(isValidDate(i)) ans++;
39 }
40 cout << ans-1;
41 return 0;
42 }
43
44
45 #include<bits/stdc++.h>
46 // #pragma GCC optimize("Ofast,no-stack-protector,fast-math",3)
47 using namespace std;
48 using ll=long long;
49 #define pii pair<int,int>
50 #define pct __builtin_popcountll
51 #define ctz __builtin_ctzll
52 #define pb push_back
53 #define all(x) x.begin(),x.end()
54 #define pli pair<ll,int>
55 #define endl '\n'
56 using ul=unsigned long long;
57 constexpr int qp(ll a,ll x=M-2){int res=1;for(;;x>>=1,a=a*a%M)(x&1)&&(res=a*res%M);
 return res;}
58 constexpr int __lg(unsigned long long x) {if (x == 0) return -1; int res = -1; while
 (x) { res++; x >>= 1;} return res;}
59 constexpr int bceil(int n) { return n <= 1 ? 1 : 1 << (__lg(n - 1) + 1); }
60
61 constexpr int N=(1<<21)+100,_g=3,
62 /* (σ`□`)σ */ M=998244353;
63 // /* (σ`□`)σ 「1e9」 */ M=1004535809;
64 // /* (σ`□`)σ 「4e18」 */ M=4179340454199820289;
65 // /* (σ`□`)σ */ M=1e9+7;

```

```

66 namespace fast_io{
67 char buf[N+10],*p1,*p2,c;
68 #define gc (p1==p2&&(p2=(p1=buf)+fread(buf,1,N,stdin),p1==p2))?EOF:*p1++
69 template<typename _Tp>
70 void read(_Tp &x){
71 int f=0;for(c=gc;c<48;c=gc)f^=c=='-';
72 for(x=0;c>47;x=(x<<1)+(x<<3)+(48^c),c=gc);
73 if(f)x=-x;
74 }
75 template<typename _Tp,typename..._tps>
76 void read(_Tp &x,_tps&...y){read(x),read(y...);}
77 char ob[N+100],stk[505];int tp,ot;
78 void fls(){fwrite(ob,1,ot,stdout),ot=0;}
79 int cntt;
80 template<typename _Tp>
81 static inline void write(_Tp x,char c='\0'){
82 if(!cntt)atexit(fls),cntt=1;
83 while(x>9)stk[++tp]=48^(x%10),x/=10;
84 for(ob[ot++]=48^x;tp;ob[ot++]=stk[tp--]);
85 if (c != '\0')ob[ot++]=c;if(ot>N)fls();
86 }
87 }
88 using fast_io::read;
89 using fast_io::write;
90
91 namespace DBG{
92 template <class T>
93 std::ostream& operator<< (std::ostream& os, const std::vector<T> &v);
94 template <class T>
95 void _dbg(const char *f,T t) { cerr<<f<<'\n'<<endl; }
96 template <class A,class... B>
97 void _dbg(const char *f,A a,B... b){
98 while (*f!=',') cerr<<*f++;
99 cerr<<'\n'<<a<<",";
100 _dbg(f+1,b...);
101 }
102 template <class T>
103 ostream& operator << (ostream& os,const vector<T> &v){
104 os<<"[";
105 for (const auto &x:v) os<<x<<",";
106 os<<"]";
107 return os;
108 }
109 #define dbg(...) _dbg(__VA_ARGS__, __VA_ARGS__)
110 }
111 using namespace DBG;
112
113 template<typename tp1,typename tp2>
114 void ckmx(tp1 &x,const tp2 &y){x<y?x=y:0;}
115 template<typename tp1,typename tp2>
116 void ckmn(tp1 &x,const tp2 &y){x>y?x=y:0;}
117
118 void add(int &x,int y){(x+=y)>=M?x-=M:0;}

```



```

119 void del(int &x,int y){(x-=y)<0?x+=M:0;}
120 void add(int &x,int y,int z){x=(y*z+x)%M;}
121 void del(int &x,int y,int z){(x-=y*z%M)<0&&(x+=M);}
122 void Mul_e(ll &a, ll b, int mod = M) { a=((__int128_t)a*b % mod); }
123 ll Mul(ll a, ll b, ll mod = M) {__int128_t res = a; res *= b; return (long long)(res
 % mod); }
124 constexpr int qp(ll a,ll x=M-2){
125 int res=1;for(;x>=1,a=a%M)
126 (x&1)&&(res=a*res%M);return res;
127 }
128
129 struct DET{
130 int a[3005][3005],n;
131 int run(){
132 if(!n)return 1;
133 int x,y,z,k,res=1;
134 for(x=1;x<=n;++x){
135 for(y=x;y<=n&&!a[y][x];++y);
136 if(y>n)return 0;
137 if(y>x){
138 for(k=1;k<=n;++k)swap(a[x][k],a[y][k]);
139 res&&(res=M-res);
140 }
141 k=qp(a[x][x]);
142 res=1ll*res*a[x][x]%M;
143 for(z=1;z<=n;++z)
144 a[x][z]=1ll*a[x][z]*k%M;
145 for(y=1;y<=n;++y)
146 if(x!=y){
147 k=a[y][x];
148 for(z=1;z<=n;++z)
149 del(a[y][z],a[x][z],k);
150 }
151 }
152 for(x=1;x<=n;++x)
153 res=1ll*res*a[x][x]%M;
154 return res;
155 }
156 }det;
157 namespace MATH{
158 vector<int>fac,ifac,inv;
159 int dv2(int x){return x&1? (x+M)>>1:x>>1;}
160 int C(int n,int m){
161 //assert(m<=n);
162 if(m<0||m>n)return 0;
163 return 1ll*fac[n]*ifac[m]%M*ifac[n-m]%M;
164 }
165 int P(int n,int m){
166 return 1ll*fac[n]*ifac[n-m]%M;
167 }
168 int D(int n,int m){
169 if(n<0||m<0)return 0;
170 if(!n)return 1;

```

```

171 if(!m)return 0;
172 return C(n+m-1,m-1);
173 }
174 int catalan(int n) { //1,1,2,5,14,42,132,429,1430,4862
175 return (C(2 * n, n) - C(2 * n, n + 1) + M) % M;
176 }
177 void init(int n){
178 int x;
179 fac.resize(n+2);
180 fac[0]=fac[1]=1;
181 ifac=inv=fac;
182 for(x=2;x<=n;++x){
183 fac[x]=1ll*x*fac[x-1]%M;
184 inv[x]=1l(M-M/x)*inv[M%x]%M;
185 ifac[x]=1ll*ifac[x-1]*inv[x]%M;
186 }
187 }
188 }
189
190 ll Gcd(ll x,ll y){if(!x||!y)return x|y;int k=min(ctz(x),ctz(y));ll d;y>>=ctz(y);
 while(x){x>>=ctz(x),d=x-y;if(x<y)y=x;if(d<0)x=-d;else x=d;}return y<<k;}
191 int gcd(int a,int b){return b?gcd(b,a%b):a;}
192 __int128_t lcm128(long long a, long long b) {return (__int128_t)(a / gcd(a, b)) * b
 ;}
193
194 namespace Prime {
195 vector<int> getDivisors(int n) {
196 vector<int> v; if (n <= 0) return v;
197 for (int i = 1; i <= n/i; i++) if (n % i == 0) {n/i == i ? v.push_back(i) : (
 v.push_back(i), v.push_back(n/i));}
198 sort(v.begin(), v.end());
199 return v;
200 }
201 vector<int> spf; vector<int> p;
202 vector<int> mu; vector<int> phi;
203 void initp(int n) {
204 spf.resize(n + 1, 0);
205 mu.resize(n + 1, 0);
206 phi.resize(n + 1, 0);
207 mu[1] = 1; phi[1] = 1; //1
208 for (int i = 2; i <= n; ++i) {
209 if (!spf[i]) {
210 spf[i] = i;
211 p.push_back(i);
212 mu[i] = -1; phi[i] = i - 1; //2
213 }
214 for(int j=0;j<(int)p.size() && p[j]<=spf[i] && i*p[j]<=n;++j){
215 int x = i * p[j];
216 spf[x] = p[j];
217 if (i % p[j] == 0) {
218 mu[x] = 0; phi[x] = phi[i] * p[j]; //3
219 break;
220 } else {

```

```

221 mu[x] = -mu[i]; phi[x] = phi[i] * (p[j] - 1); //4
222 }
223 }
224 }
225 }
226 }
227
228 mt19937_64 rg(random_device{}()); //rg()%100
229 using namespace MATH;
230 using namespace Prime;
231 using i128 = __int128_t;
232 using vt=vector<int>;
233 using ld=long double;
234 int T;
235 const int maxn = 2e5+10;
236 //int a[maxn];
237
238 void sol() {
239
240 }
241 signed main () {
242 ios::sync_with_stdio(false);
243 cin.tie(nullptr);
244 int T=1;
245 // cin>>T;
246 while(T--){ sol();}
247 return 0;
248 }

```

## 4.7 搜索

### 4.7.1 二维数组的一维化

```

1 //1.坐标->int : m*(x-1)+y
2 //2.从int获取x, y坐标:
3 // int xx = (e.second-1)/m+1;
4 // int yy = (e.second-1)%m+1;

```

### 4.7.2 八数码问题

```

1 #include<bits/stdc++.h>
2 const int MAX_LEN = 362888;
3 using namespace std;
4 struct node{
5 int state[9];
6 int dis;
7 };
8 int dir[4][2] = {{-1,0},{0,-1},{1,0},{0,1}}; //左、上、右、下顺时针方向。左上角坐标是
9 // (0,0)
10 int visited[MAX_LEN]={0}; //与每个状态对应的记录, Cantor函数对它置数, 并判重
11 int start[9];
12 int goal[9];

```

```

12 long int factory[] = {1,1,2,6,24,120,720,5040,40320,362880}; //Cantor用到的常数
13 bool Cantor(int a[],int n){ //用康托展开判重
14 long result = 0;
15 for(int i = 0;i < n;i++){
16 int counted = 0;
17 for(int j = i+1;j < n;j++){
18 if(a[i] > a[j])
19 ++counted;
20 }
21 result += counted*factory[n-i-1]; //注意数组下标: X=a[n]x(n-1)! + a[n-1]x(n-2)
 ! + ... + a[1]x 0!
22 }
23 if(!visited[result]){
24 visited[result] = 1;
25 return 1;
26 }
27 else
28 return 0;
29 }
30 void bfs(){
31 node head;
32 memcpy(head.state,start,sizeof(head.state)); //复制起点的状态
33 head.dis = 0;
34 queue<node> q;
35 Cantor(head.state,9); //用康托展开判重,目的是对起点的visited[]赋初值
36 q.push(head); //第一个进队列的是起点状态
37 while(!q.empty()){
38 head = q.front(); //可在此处置head.state,看弹出队列的情况
39 int z;
40 for(z=0;z < 9;z++){
41 if(head.state[z] == 0)
42 break; //找到了
43 }
44 //转化为二维,左上角是原点(0,0)。
45 int x = z%3; //横坐标
46 int y = z/3; //纵坐标
47 for(int i = 0;i < 4;i++){
48 int newx = x+dir[i][0];
49 int newy = y+dir[i][1];
50 //元素@转移后的新坐标
51 int newz = newx + 3*newy; //转化为一维
52 if(newz >= 0 && newz < 9 && newy >= 0 && newy < 3){ //未越界
53 node newnode;
54 memcpy(&newnode,&head,sizeof(struct node)); //复制新的状态
55 swap(newnode.state[z],newnode.state[newz]); //把@移动到新的位置
56 newnode.dis++;
57 if(memcmp(newnode.state,goal,sizeof(goal)) == 0){ //与目标状态对比
58 return; //到达目标状态,返回距离,结束
59 }
60 if(Cantor(newnode.state,9)) //用康托展开判重
61 q.push(newnode); //把新的状态放进队列
62 }
63 }

```

```

64 }
65 return -1; //没找到
66 }
67 int main(){
68 for(int i = 0; i < 9; i++) cin >> start[i]; //初始状态
69 for(int i = 0; i < 9; i++) cin >> goal[i]; //目标状态
70 int num = bfs();
71 if(num != -1)
72 cout << num << endl;
73 else
74 cout << "Impossible" << endl;
75 return 0;
76 }

```

#### 4.7.3 剪枝函数: 八皇后

```

1 #include <iostream>
2 using namespace std;
3
4 int n;
5 int ans=0;
6 vector<int> x;
7
8 bool check(int row, int col) {
9 for (int i = 0; i <= row - 1; i++)
10 if (col == x[i] || abs(row - i) == abs(col - x[i]))
11 return false;
12 return true;
13 }
14
15 void dfs(int k) {
16 if (k == n) {
17 ans++;
18 // for (int i = 0; i < n; i++) cout << x[i] << " ";
19 // cout << endl;
20 }
21 //int r = (k==0)? n/2:n; //要求去除“对称”的方案: 第一个皇后只能放前半
22 for (int i = 0; i < n; i++) { // 从0到n-1
23 if (check(k, i)) { // 检查能否把第k行的皇后放在第i行
24 x[k] = i;
25 dfs(k + 1);
26 }
27 }
28 }
29
30 int main() {
31 cin >> n;
32 x.resize(n); // x[i]
33
34 dfs(0);
35 cout<<ans<<endl;
36 return 0;

```

```
37 }
```

#### 4.7.4 迭代加深搜索 IDDFS

```
1 #include <iostream>
2
3 #define rei register int
4 #define LL long long
5 #define IOS ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
6 #define cvar int n, m, T;
7 #define rep(i, s, n, c) for (register int i = s; i <= n; i+=c)
8 #define repd(i, s, n, c) for (register int i = s; i >= n; i-=c)
9 #define CHECK cout<<"WALKED"<<endl;
10 inline int read(){int x=0,f=1;char ch=getchar();while(ch<'0' || ch>'9'){if(ch=='-')f
 =-1;ch=getchar();} while(ch>='0' && ch<='9')x=(x<<3)+(x<<1)+ch-'0',ch=getchar();
 return x*f;}
11 #define pb push_back
12 #define ls id<<1
13 #define rs id<<1|1
14 const int INF = INT_MAX;
15 long long binpow(long long a, long long b, LL mod){long long res = 1; while (b > 0){
 if (b & 1) res = res * a % mod;a = a * a % mod; b >>= 1; } return res;}
16 using namespace std;
17
18 int a, b, maxd;
19
20 LL gcd(LL a, LL b) { return b == 0 ? a : gcd(b, a % b); }
21
22 // 返回满足 $1/c \leq a/b$ 的最小 c 值
23 inline int get_first(LL a, LL b) { return b / a + 1; }
24 // $1/c \leq a/b$
25 // $b \leq ac$
26 // $b/a \leq c$
27 const int maxn = 105;
28
29 LL v[maxn], ans[maxn];
30
31 // 如果当前解 v 比目前最优解 ans 更优, 更新 ans
32 bool better(int d)
33 {
34 for (int i = d; i >= 0; i--)
35 if (v[i] != ans[i])
36 return ans[i] == -1 || v[i] < ans[i];
37 return false;
38 }
39
40 // 当前深度为 d , 分母不能小于 $from$, 分数之和恰好为 aa/bb
41 bool dfs(int d, int from, LL aa, LL bb)
42 {
43 if (d == maxd)
44 {
45 if (bb % aa)
```

```
46 return false; // aa/bb 必须是埃及分数
47 v[d] = bb / aa;
48 if (better(d)) // 估价
49 memcpy(ans, v, sizeof(LL) * (d + 1));
50 return true;
51 }
52 bool ok = false;
53 from = max(from, get_first(aa, bb)); // 枚举的起点
54 for (int i = from;; i++)
55 {
56 // 剪枝: 如果剩下的 maxd+1-d 个分数全部都是 1/i, 加起来仍然不超过
57 // aa/bb, 则无解
58 if (bb * (maxd + 1 - d) <= i * aa)
59 break;
60 v[d] = i;
61 // 计算 aa/bb - 1/i, 设结果为 a2/b2
62 LL b2 = bb * i;
63 LL a2 = aa * i - bb;
64 LL g = gcd(a2, b2); // 以便约分
65 if (dfs(d + 1, i + 1, a2 / g, b2 / g))
66 ok = true;
67 }
68 return ok;
69 }
70
71 int main()
72 {
73 cin >> a >> b;
74 for (maxd = 1; maxd <= 10; maxd++)
75 { // 枚举深度上限
76 memset(ans, -1, sizeof(ans));
77 if (dfs(0, get_first(a, b), a, b)) break; // 开始进行搜索
78 }
79 rep(i, 0, maxd, 1)
80 cout << ans[i] << " ";
81 return 0;
82 }
```